

latextool

DR. YIHSIANG LIOW (JUNE 26, 2026)

Contents

1 Tikz (<code>tikz.tex</code>)	9
2 Tikz node, point, small circles (<code>tikznode.tex</code>)	10
3 Notes and API and TODO (<code>notes.tex</code>)	14
4 Enumerate (<code>testmylist.tex</code>)	19
4.1 Basic enumerate	19
4.2 tightlist	20
4.3 axioms	21
5 Text color (<code>textcolor.tex</code>)	22
6 python (<code>python.tex</code>)	23
7 execute (<code>execute.tex</code>)	26
8 console (<code>console.tex</code>)	31
9 verbatim (<code>verbatim.tex</code>)	34
10 shell (<code>shell.tex</code>)	45
11 Plot (<code>plot.tex</code>)	48
12 Scaling (<code>scaling.tex</code>)	49
13 Grid (<code>grid.tex</code>)	50
14 Point and text placement (<code>point.tex</code>)	53
15 coordinate (<code>coordinate.tex</code>)	70

16	code (<code>code.tex</code>)	71
16.1	Position	72
16.2	Cell size	73
16.3	border linewidth	74
16.4	distance of border to rect	75
16.5	codirect	76
16.6	Substitution	77
16.7	Long division example	78
17	table (<code>table.tex</code>)	79
17.1	table2 (<code>table2.tex</code>)	90
17.1.1	Change row and column index values/names	91
17.1.2	No row and column indices/values	92
17.1.3	Change row/column names	93
17.1.4	No row and no column names	94
17.1.5	border_linewidth	95
17.1.6	Adding things beneath table2: background	96
17.1.7	Using cells	97
17.1.8	Rect for cells	99
17.1.9	Using rect to change the background	101
17.2	table3 (<code>table3.tex</code>)	102
17.2.1	2-by-2 case	103
17.2.2	2-by-2 case with top and left for labels	104
17.2.3	2-by-1 case	105
17.2.4	Specify topleft coordinates	106
17.2.5	2-by-1 case with title	107
17.2.6	1-by-2 case	108
17.2.7	Accessing cells	109
17.2.8	Coloring a cell under the table manually	110
17.2.9	Coloring a cell under the table using table3	111
17.2.10	Cells of different widths (table2)	112
17.2.11	Cells of different widths (table3)	113
17.2.12	Ragged arrays (error - to fix)	114
17.2.13	Justification	115
17.2.14	title	116
17.2.15	Title distance	117
17.3	table4 (<code>table4.tex</code>)	118
17.4	str_to_2darray (<code>helper-for-2d-arrays.tex</code>)	121
18	diamond (<code>diamond.tex</code>)	122

19	polygon (<code>polygon.tex</code>)	126
20	fillpolygon (<code>fillpolygon.tex</code>)	127
21	circle (<code>circle.tex</code>)	128
21.1	boundary	128
21.2	boundary	129
21.3	background	130
21.4	s: minipage text in circle	131
21.5	label anchor	132
21.6	foreground	133
21.7	Center and boundary	134
21.8	tikz name	135
22	ellipse (<code>ellipse.tex</code>)	137
23	arc (<code>arc.tex</code>)	138
24	Relative node placement (<code>relative-node-placement.tex</code>)	143
25	Bounding nodes (<code>boundingbox.tex</code>)	147
26	Axes (<code>axes.tex</code>)	150
27	Lines (<code>chap-line.tex</code>)	153
27.1	linecolor and linewidth (<code>line.tex</code>)	154
27.2	startstyle and endstyle	155
27.3	points parameter	158
27.4	boundary points	159
27.5	Midpoints	161
27.6	tikz names	165
27.7	bend	166
27.8	label and anchor	167
27.9	Using midpoint to place labels	170
27.10	Two control points	171
27.11	One control point	172
27.12	loop	173
27.13	bend (<code>bend.tex</code>)	175
27.14	Orthogonal lines; organization chart (<code>orthogonal-line.tex</code>)	178
28	Rect class (<code>rect.tex</code>)	181
28.1	No background	181

28.2	Background	181
28.3	border, background, foreground	182
28.4	border, background, foreground	183
28.5	align	184
28.6	innersep	185
28.7	rounded corners	186
28.8	linestyle	187
28.9	label	188
28.10	tikz name	189
29	RectContainer (<code>rectcontainer.tex</code>)	190
29.1	Using <code>\</code>	191
29.2	name	192
30	Rect2 and RectContainer ??? (<code>rect2.tex</code>)	205
31	Arrays (<code>chap-arrays.tex</code>)	207
32	Arrays	207
32.1	array (<code>array.tex</code>)	208
32.2	snipped array (<code>snipped-array.tex</code>)	214
32.3	chunkedarray (<code>chunkedarray.tex</code>)	221
32.4	2d array (<code>array2d.tex</code>)	229
33	position (<code>position.tex</code>)	233
34	vphantom (<code>vphantom.tex</code>)	235
35	Trees (<code>chap-tree.tex</code>)	236
35.1	Tree (<code>tree.tex</code>)	237
35.2	Parse tree example	239
35.3	Binary tree (<code>tree2.tex</code>)	242
35.4	bintree function	245
35.5	BinTree class	248
36	Graph (<code>chap-graph.tex</code>)	250
36.1	Graph (<code>graph.tex</code>)	251
36.2	graph function	251
36.3	graph function:nodes and labels	252
36.4	graph function: isocles triangle as subtree	254
36.5	The graph function: edge labels	255
36.6	graph2 function (<code>graph2.tex</code>)	256

36.7 Graph class (<code>some-graphs.tex</code>)	259
36.8 Edge label anchor	260
36.9 Cycle graphs	261
36.10 Name of nodes	262
36.11 Petersen graph	265
36.12 Complete graphs	266
36.13 Star graphs	268
36.14 Bipartite graphs	269
37 Frame (<code>frame.tex</code>)	271
38 Test Verbatim Spacing (<code>test-verbatim-spacing.tex</code>)	272
39 2D vector diagram (<code>vec2d.tex</code>)	273
40 Function line graph (<code>line-graph.tex</code>)	278
41 Function bar chart, line chart (<code>line-graph-nonnumeric.tex</code>)	288
42 Bar chart, line chart	288
43 Test isocles triangle (<code>isosceles-triangle.tex</code>)	290
44 ER diagrams (<code>er-diagram.tex</code>)	291
44.1 Attribute	291
44.2 Entity	292
44.3 Relation	304
44.4 Entity and relation with different arrows	307
44.5 Entity and relation with attributes	310
44.6 Weak entity sets and partial keys	311
44.7 Associative entity	312
44.8 Crow's foot	313
45 UML diagrams (<code>chap-uml.tex</code>)	315
45.1 UML: class diagram (<code>uml-classdiagram.tex</code>)	316
45.2 UML: sequence diagram (<code>uml-sequencediagram.tex</code>)	328
45.3 UML: sequence diagram	328
45.4 UML: use case diagram (<code>uml-usecasediagram.tex</code>)	330
46 Matrix (<code>matrix.tex</code>)	332
47 Board games (<code>boardgames.tex</code>)	337
47.1 Othello (<code>othello.tex</code>)	338

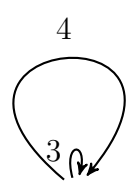
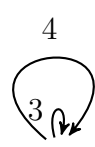
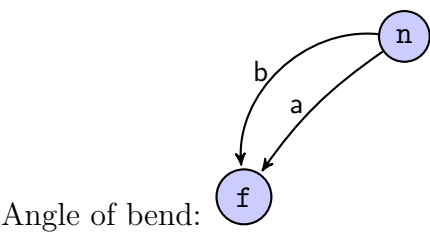
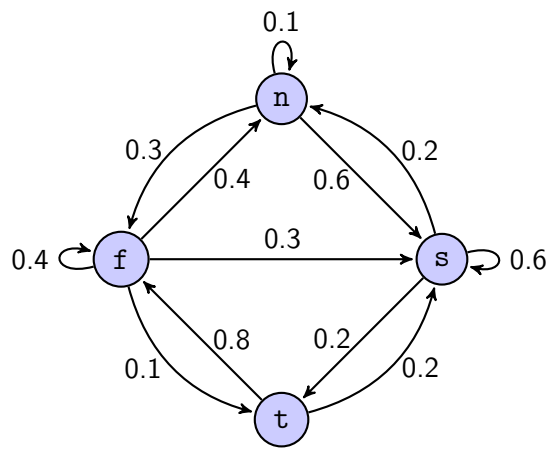
47.2 Tic-Tac-Toe (<code>ttt.tex</code>)	343
48 Proof trees (<code>prooftree.tex</code>)	349
49 K-maps (<code>kmap.tex</code>)	352
49.1 kmap: specify term(s) to circle	364
49.2 kmap: 2-by-4	366
49.3 kmap: force choice of prime implicant	371
49.4 kmap: style selector	373
49.5 kmap: change distance of implicant boundary to rect	374
49.6 kmap: decimal notation	376
49.7 kmap: decimal notation 2	377
49.8 kmap: using coordinates of rects in the kmap cells	378
50 Logic design (<code>logic-design.tex</code>)	380
50.1 AND gate	380
50.2 OR gate	381
50.3 NOT gate	382
50.4 NAND gate	383
50.5 NOR gate	384
50.6 XOR gate	385
50.7 Buffer gate	386
50.8 Orthogonal paths	388
50.9 Orthogonal path with gates	392
50.10 POINT	393
50.11 Linecolor for gate	394
50.12 Label for gate	395
50.13 Rotation	405
50.14 Example: SR latch	421
50.15 layout function	425
50.16 Logic blocks	431
50.17 Logic blocks: full adder	435
50.18 Two NOTs	437
51 Data structures (<code>chap-data-structures.tex</code>)	438
51.1 Singly linked list (<code>singlylinkedlist.tex</code>)	439
51.2 Doubly linked list (<code>doublylinkedlist.tex</code>)	440
51.3 Stack (<code>stack.tex</code>)	441
51.4 Adjacency list (<code>adjlist.tex</code>)	449
51.5 Heapfile (<code>heapfile.tex</code>)	457
51.6 B+ tree (<code>bptree.tex</code>)	464

51.7 Directed edge	466
51.8 Intervals (<code>intervals.tex</code>)	474
52 Automata (<code>chap-automata.tex</code>)	475
52.1 DFA (<code>dfa.tex</code>)	476
52.2 State	477
52.3 Tape and head	478
52.4 Label for tape	478
52.5 Position	479
52.6 Bounding rect	479
52.7 Dimensions of automata	481
52.8 Body-tape distance	483
52.9 No drawing	484
52.10 Sequence of automatas	485
52.11 PDA (<code>pda.tex</code>)	489
52.12 PDA: bounding rect	490
52.13 Sequence of PDA	494
52.14 CYK (<code>cyk.tex</code>)	496
52.14.1 Word being parsed	496
52.14.2 Background	497
52.14.3 Height and width	498
53 Automata (<code>automata.tex</code>)	500
54 Longtable of minipages (<code>longtable-minipage.tex</code>)	514
55 MDP – Markov decision process (<code>mdp.tex</code>)	515
56 Solutions (<code>solutions.tex</code>)	516

TODO: Shape location: Right now specify x0,y0,x1,y1 or center. Need to add “left= of a”, “left=1cm of a”, “below left=1cm and 3cm of a”,etc. Maybe `{pos = "below 1cm, left 3cm of a"}` or `{pos = {'direction':{'below':"1cm", "left":"1cm"}, 'node':'a'}}`

TODO: Point, POINT, coordinate are similar. Clean up.

1 Tikz (tikz.tex)



2 Tikz node, point, small circles (tikznode.tex)

Something is wrong with the library for drawing very small circles. (Probably error with inner sep: if circle is radius r , innersep should be $r/\sqrt{2}$ – correct and test circle class.) The following is a temporary manual fix using tikz strings.

```
from latextool_basic import *
p = Plot()

p += r"\node[draw,shape=circle,minimum size=0.1cm,fill=blue,inner sep=0](A)
at (0,0){};"

print(p)
```

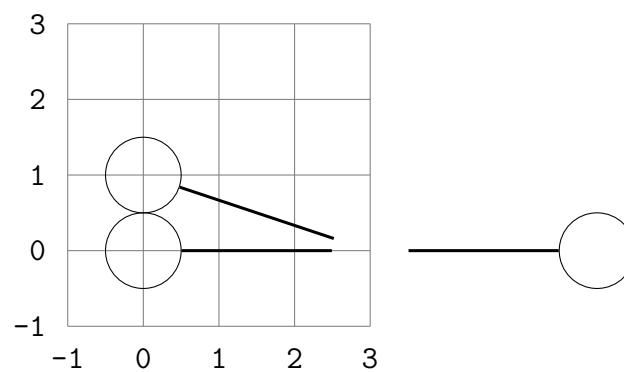


A node with the shape drawn or not drawn

```
from latextool_basic import *
p = Plot()

p += r"\node[draw,shape=circle,minimum size=1cm,inner sep=0](A) at (0,0){};
"
p += r"\node[draw,shape=circle,minimum size=1cm,inner sep=0](D) at (0,1){};
"
p += r"\node[draw=none,shape=circle,minimum size=1cm,inner sep=0](B) at (3,
0){};"
p += r"\node[draw,shape=circle,minimum size=1cm,inner sep=0](C) at (6,0){};
"

p += Line(names=['A', 'B'])
p += Line(names=['D', 'B'])
p += Line(names=['C', 'B'])
p += Grid(-1, -1, 3, 3)
print(p)
```

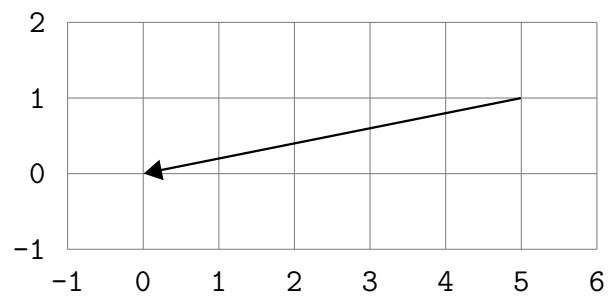


Create tikz node without shape (basically just to have a coordinate with name):

```
from latextool_basic import *
p = Plot()

p += Point(x=0, y=0, name='A')
p += Point(x=5, y=1, name="B")

p += Graph.arc(names=['B', 'A'])
p += Grid(x0=-1, y0=-1, x1=6, y1=2)
print(p)
```

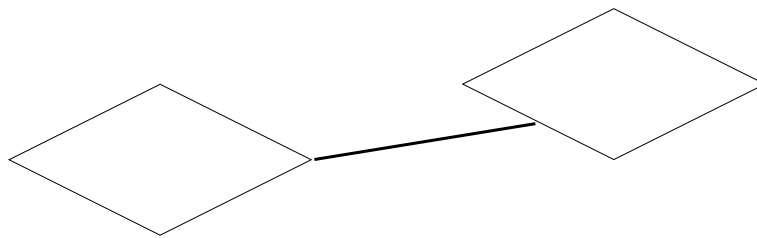


Creating node wrt to another:

```
from latextool_basic import *
p = Plot()
p += diamond(center=(0, 0), name='a')
p += diamond(center=(6, 1), name='b')

p += r'\node[right=0.0cm of a, inner sep=0.0cm, outer sep=0.0cm] (c) {};'
p += r'\node[below left=0.0cm of b, inner sep=0.0cm, outer sep=0.0cm] (d) {
};'

p += Line(names=['c','d'])
print(p)
```



[illegible]

ISSUE: The label in the rect is centered. The problem is the font for a character with greater height will appear lower than another. So for an array of rects of character, the base line of the labels are not the same.

ISSUE: The boundary line are drawn within the bounding box. So joining up rects would mean the common line will look thicker. DONE. See Rect2.

ISSUE:

- Joining rects in a uniform way to get hor and ver arrays and 2d arrays. Allows for containers with finite area, infinite horizontally, infinite vertically.
- Where to put first rect? What's the API? For hor array that grows to the right, only need top-left corner or bottom-left.or center-left. Maybe just specify the absolute corners of the first rect?
- Allow each element of array to be any shape. Use BaseNode of course.

CLEAN UP OF PYTHON-LATEX DRAWING LIBRARY

```
latextool.py
latextool_basic.py
```

```
automata
```

```
courses/book/graph.py -----> projects/latextool/latextool.py
courses/ciss362-automata/resources/dfa/dfa-backup.py
courses/ciss362-automata/resources/dfa/dfa.py
courses/ciss362-automata/resources/dfa/graph/graph.py
projects/automata/dfa/dfa.py
```

```
linked list
courses/math325/n/latex_ds.py
projects/test-latex/xxx/latex_ds.py
```

```
array sorting
```

```
2d graphs:
courses/math325/n/makegraph/plot.py
```

```
=====
automata
```

1d shapes: line or things made up of lines (broom?...) They have

- color
- style
- starting tip: dot, arrow (and arrow style)
- ending tip: dot, arrow (and arrow style)

2d shapes: rect, circle, They have

- boundary is a line. So it has boundary color and boundary style
- interior: color
- minipage

4 Enumerate (testmylist.tex)

4.1 Basic enumerate

For comparison, this is enumerate:

- Line 1.
- line 2.
- line 3.

After enumerate.

This is with paragraphing within items:

- Line 1. This is a test. This is a test. This is a test. This is a test. This is a test. This is a test. This is a test. This is a test.
Next paragraph. Lorem ipsum dolor sit amet, consectetur adipiscing elit. Ut purus elit, vestibulum ut, placerat ac, adipiscing vitae, felis. Curabitur dictum gravida mauris. Nam arcu libero, nonummy eget, consectetur id, vulputate a, magna. Donec vehicula augue eu neque. Pellentesque habitant morbi tristique senectus et netus et malesuada fames ac turpis egestas. Mauris ut leo. Cras viverra metus rhoncus sem. Nulla et lectus vestibulum urna fringilla ultrices. Phasellus eu tellus sit amet tortor gravida placerat. Integer sapien est, iaculis in, pretium quis, viverra ac, nunc. Praesent eget sem vel leo ultrices bibendum. Aenean faucibus. Morbi dolor nulla, malesuada eu, pulvinar at, mollis ac, nulla. Curabitur auctor semper nulla. Donec varius orci eget risus. Duis nibh mi, congue eu, accumsan eleifend, sagittis quis, diam. Duis eget orci sit amet orci dignissim rutrum.
- line 2.
- line 3.

A new paragraph after mylist.

4.2 tightlist

This is the same as mylist except that the vertical spacing before and after the environment is removed. This does not work quite right if the environment is in a paragraph of its own – there will be a little extra vertical spacing before the environment.

Testing tightlist:

- line 1
- line 2
- line 3

After environment.

This is with paragraphing within items:

- Line 1. This is a test. This is a test. This is a test. This is a test. This is a test. This is a test. This is a test. This is a test.

Next paragraph. Lorem ipsum dolor sit amet, consectetur adipiscing elit. Ut purus elit, vestibulum ut, placerat ac, adipiscing vitae, felis. Curabitur dictum gravida mauris. Nam arcu libero, nonummy eget, consectetur id, vulputate a, magna. Donec vehicula augue eu neque. Pellentesque habitant morbi tristique senectus et netus et malesuada fames ac turpis egestas. Mauris ut leo. Cras viverra metus rhoncus sem. Nulla et lectus vestibulum urna fringilla ultrices. Phasellus eu tellus sit amet tortor gravida placerat. Integer sapien est, iaculis in, pretium quis, viverra ac, nunc. Praesent eget sem vel leo ultrices bibendum. Aenean faucibus. Morbi dolor nulla, malesuada eu, pulvinar at, mollis ac, nulla. Curabitur auctor semper nulla. Donec varius orci eget risus. Duis nibh mi, congue eu, accumsan eleifend, sagittis quis, diam. Duis eget orci sit amet orci dignissim rutrum.

- line 2.
- line 3.

A new paragraph after above environment. The next environment is in a new paragraph.

- line 1
- line 2
- line 3

4.3 axioms

The label horizontal spacing is larger.

(GROUP-C) $\forall g, g' \in G, gg' \in G$

(GROUP-A) $\forall g, g', g'' \in G,$

5 Text color (textcolor.tex)

```

Testing \verb!\textred!:
\textred{hello world ... $x = 1$ ... $\int x \, dx = \frac{1}{2}x^2 + C$}
\[
\sin \theta = 0
\]
}
Black again.

Testing \verb!\textblue!:
\textblue{hello world ... $x = 1$ ... $\int x \, dx = \frac{1}{2}x^2 + C$}
\[
\sin \theta = 0
\]
}
Black again.

Testing \verb!\textwhite!:
\textwhite{hello world ... $x = 1$ ... $\int x \, dx = \frac{1}{2}x^2 + C$}
\[
\sin \theta = 0
\]
}
Black again.

Using underline/textbox with textwhite:
\begin{align*}
\int x \, dx &= \frac{1}{2} x^2 + C \\
\int x \, dx &= \underline{\textwhite{\frac{1}{2} x^2 + C}} \\
\int x \, dx &= \text{\textbox{\textwhite{$\frac{1}{2} x^2 + C$}}}
\end{align*}

```

6 python (python.tex)

There are vertical spacing issues with the python environment. Vertical spacing before and after the python environment has been removed from the python environment in `mypython.tex`.

TEST 1. Python environment with console function call. Line before python environment.

```
hello world
```

Line after the python environment.

TEST 2. One python environment with two console function calls. Line before python environment.

```
hello world 1
```

```
hello world 2
```

Line after the python environment. ISSUE: Note that the two frames are joined up. For the time being do not put two console environment immediately next to each other.

TEST 3. One python environment with two console function calls with one blank line between the console function calls. Line before python environment.

```
hello world 1
```

```
hello world 2
```

Line after the python environment. ISSUE: Note that the two frames are joined up. For the time being do not put two console environment immediately next to each other.

TEST 4. Two python environments with one console function call each. Line before python environment.

```
hello world 1
```

Line between the python environments.

```
hello world 2
```

Line after the second python environment.

TEST 5. This is the console environment in latex (no python).

```
hello world 1
```

Line after.

TEST 6. 1 console environment in latex (no python).

```
hello world 1
```

Line between.

```
hello world 2
```

Line after.

TEST 7. 2 console environments in latex next to each other.

```
hello world 1
```

```
hello world 2
```


Line after.

TEST 8. 1 verbatim environment in latex (no python).

```
hello world 1
```

Line after.

TEST 9. This is 2 Verbatim environments in latex (no python).

```
hello world 1
```

Line between.

```
hello world 2
```

Line after.

TEST 10. 2 Verbatim environments next to each.

```
hello world 1
```

```
hello world 2
```

Line after.

7 execute (execute.tex)

The `execute` (in `latexbasic_tool.py`) function executes a python string as a program and the stdout is inserted into the latex file:

```
\begin{python}
from latextool_basic import execute
execute(r"""
print("hello world")
""")
\end{python}
```

Here's the result: hello world

Note that there's some extra whitespace before the hello world. TODO: Need to figure a way to remove the whitespace.

The following does the same except that it first prints the python source in a framed verbatim box:

```
Result:
\begin{python}
from latextool_basic import execute
execute(r"""
print("hello world")
""", print_source=True)
\end{python}
```

If `debug` is set to `True`, there are three outputs: the python source, the stderr, and the stdout, all in framed verbatim boxes.

```
Result:
\begin{python}
from latextool_basic import execute
execute(r"""
print("hello world")
""", debug=True)
\end{python}
```

If there's an error in the python source, the output will be the same as if `debug` is set to `True`.

TEST: `debug=True`

```
from latextool_basic import execute
execute("print('hello world')", debug=True)
```

Result: PYTHON ERROR. See source, stderr, stdout below

```
print('hello world')
```

```
hello world
```

TEST: debug=False

```
from latextool_basic import execute
execute("print('hello world')", debug=False)
```

Result: hello world

TEST: print source=True, debug=False

```
from latextool_basic import execute
execute("print('hello world')", print_source=True, debug=False)
```

Result:

```
print('hello world')
```

hello world

TEST: Error in source

```
from latextool_basic import execute
execute("print(hello world)")
```

Result: PYTHON ERROR. See source, stderr, stdout below

```
print(hello world)
```

```
File "wzdfomep.tmp.py", line 1
    print(hello world)
          ^
SyntaxError: invalid syntax
```

8 console (console.tex)

NOTE: console function is now the same as the verbatim function.

[illegible]

There a console latex environment and a console function in `latexool_basic`.

TODO: Here's the console environment in latex:

```
hello world
```

Here's the console function call in the python environment:

```
hello world
```

Notice the the second case has a blank line before and after the console window. Fix it.

Here's a python environment that prints hello world, prints a console function return value, prints hello world: hello world

```
hello world
```

hello world

Note that the spacing is different from the above.

The console is just like verbatim:

```
from latextool_basic import console
print(console("hello world"))
```

```
hello world
```

However it wraps around:

```

from latextool_basic import console
print(console(r'''
line 1 ... hello world hello world hello world
line 2 ... hello world hello world hello world hello world hello world hell
o world hello world hello world hello world hello world hello world hello w
orld hello world hello world hello world hello world hello world hello worl
d hello world hello world hello world
'''.strip()))

```

```

line 1 ... hello world hello world hello world
line 2 ... hello world hello world hello world hello world hello world hell
o world hello world hello world hello world hello world hello world hello w
orld hello world hello world hello world hello world hello world hello worl
d hello world hello world hello world

```

You can control the window width:

```

from latextool_basic import console
print(console(r'''
line 1 ... hello world hello world hello world
line 2 ... hello world hello world hello world hello world hello world hell
o world hello world hello world hello world hello world hello world hello w
orld hello world hello world hello world hello world hello world hello worl
d hello world hello world hello world
'''.strip(), width=40))

```

```

line 1 ... hello world hello world hello
world
line 2 ... hello world hello world hello
world hello world hello world hello wor
ld hello world hello world hello world h
ello world hello world hello world hello
world hello world hello world hello wor
ld hello world hello world hello world h
ello world hello world

```

You can specify the `wrapmarker` that is used to indicate a line wrap:


```

from latextool_basic import console
print(console(r'''
line 1 ... hello world hello world hello world
line 2 ... hello world hello world hello world hello world hello world hell
o world hello world hello world hello world hello world hello world hello w
orld hello world hello world hello world hello world hello world hello worl
d hello world hello world hello world
'''.strip(), width=40, wrapmarker='---> '))

```

```

line 1 ... hello world hello world hello
---> world
line 2 ... hello world hello world hello
---> world hello world hello world hello wor
---> ld hello world hello world hello world h
---> ello world hello world hello world hello
---> world hello world hello world hello wor
---> ld hello world hello world hello world h
---> ello world hello world

```

TODO: Fix non-pagebreak.

How to insert → symbol into verbatim area?

9 verbatim (verbatim.tex)

The `verbatim` function allows you to create verbatim environments. Executing tex commands in the environment can be easier.

Example:

```
from latextool_basic import verbatim
print(verbatim("hello world\nlorem ipsum"))
```

```
hello world
lorem ipsum
```

No frame:

```
from latextool_basic import verbatim
print(verbatim("hello world\nlorem ipsum", frame=None))
```

```
hello world
lorem ipsum
```

Line numbers on the left:

```
from latextool_basic import verbatim
print(verbatim("hello world\nlorem ipsum", numbers='left'))
```

```
1 hello world
2 lorem ipsum
```

Line numbers on the right:

```
from latextool_basic import verbatim
print(verbatim("hello world\nlorem ipsum", numbers='right'))
```

```
hello world
lorem ipsum
```

```
1
2
```

Escape to a tex command:

```
t = r"""
#include <iostream>

int main()
{
    std::cout << "hello world" << std::endl;
    return 0;
}
"""
from latextool_basic import verbatim
print(verbatim(t.strip(), command=['redtext', 'iostream']))
```

```
#include <iostream>

int main()
{
    std::cout << "hello world" << std::endl;
    return 0;
}
```

Specify a list of words for each tex command:

```
t = r"""
#include <iostream>

int main()
{
    std::cout << "hello world" << std::endl;
    return 0;
}
"""
from latextool_basic import verbatim
print(verbatim(t.strip(), command=['redtext', ['iostream', 'std']]))
```

```
#include <iostream>

int main()
{
    std::cout << "hello world" << std::endl;
    return 0;
}
```

Use regular expressions:

```
t = r"""
#include <iostream>

int main()
{
    std::cout << "hello world" << std::endl;
    return 0;
}"""
from latextool_basic import verbatim
print(verbatim(t.strip(), command=['redtext', '[a-z ]*']))
```

```
#include <iostream>

int main()
{
    std::cout << "hello world" << std::endl;
    return 0;
}
```

WARNING. Since search is by regular expression, be careful to escape metacharacters:

```
from latextool_basic import verbatim
t = r"""
#include <iostream>

int main()
{
    std::cout << "hello world" << std::endl;
    return 0;
}
"""

print(verbatim(t.strip(), command=['textbox', r'main\(\)' ]))
```

```
#include <iostream>

int main()
{
    std::cout << "hello world" << std::endl;
    return 0;
}
```

You can also specify which part of the text should be escaped by specifying 3 numbers: the line number, the starting column number, and the ending column number.

```
t = r"""
#include <iostream>

int main()
{
    std::cout << "hello world" << std::endl;
    return 0;
}
"""
from latextool_basic import verbatim
print(verbatim(t.strip(), command=['underline', [4, 4, 13]]))
```

```
#include <iostream>

int main()
{
    std::cout << "hello world" << std::endl;
    return 0;
}
```

You can also specify a linenumber and a text:

```
t = r"""
#include <iostream>

int main()
{
    std::cout << "hello world" << std::endl;
    std::cout << "hello world" << std::endl;
    std::cout << "hello world" << std::endl;
    return 0;
}
"""
from latextool_basic import verbatim
print(verbatim(t.strip(), command=['textred', [5, 'std::cout']]))
```

```
#include <iostream>

int main()
{
    std::cout << "hello world" << std::endl;
    std::cout << "hello world" << std::endl;
    std::cout << "hello world" << std::endl;
    return 0;
}
```


You can have a list of tex commands:

```
from latextool_basic import verbatim
print(verbatim("the answer is 42",
               commands=[
                   ['underline', 'the'],
                   ['textbox', ['answer', '42']],
               ]
               ))
```

the is

You can also use the contents of a file like this:

```
from latextool_basic import verbatim
print(verbatim(filename='helloworld.cpp'))
```

`verbatim` is not exactly a verbatim function: there's line wrap. By default the width is set to 75. You can set the width to 40:

```
from latextool_basic import verbatim
print(verbatim("abcdeabcdeabcdeabcdeabcdeabcdeabcdeabcdeabcde", width=10))
```

```
abcdeabcde
abcdeabcde
abcdeabcde
abcdeabcde
abcde
```

You can also specify a string to indicate line wrap like this:

```
from latextool_basic import verbatim
print(verbatim("abcdeabcdeabcdeabcdeabcdeabcdeabcdeabcdeabcde", width=10,
               wrapmarker='---> '))
```

```
abcdeabcde
---> abcdeabcde
---> abcdeabcde
---> abcdeabcde
---> abcde
```

TEST 1. Test vertical spacing before and after verbatim(). Before.

```
hello world 1  
hello world 2
```

After.

TEST 2. Test vertical spacing between two verbatim() in 1 python environment. Before.

```
frame 1  
hello world 1  
hello world 2
```

```
frame 2  
hello world 1  
hello world 2
```

After.

TEST 3. Same as before but print a blank line between the 2 verbatim()s. Before.

```
frame 1  
hello world 1  
hello world 2
```

```
frame 2  
hello world 1  
hello world 2
```

After.

TEST 4. Test vertical spacing between 2 python environments, each containing 1 verbatim(), no blank line between the python environments. Before.

```
frame 1  
hello world 1  
hello world 2
```

```
frame 2
hello world 1
hello world 2
```

After.

TEST 5. For user input in verbatim environment. Before.

```
Please enter x: 42
Please enter y and z: 43 44
Please enter login: jdoe
```

After.

10 shell (shell.tex)

The following are 3 ways to draw a framed verbatim environment.

METHOD 1. The following is a console latex environment:

```
[student@localhost latextool] pwd  
home/student
```

Line after the console environment. Here are two such:

```
[student@localhost latextool] pwd  
home/student
```

```
[student@localhost latextool] pwd  
home/student
```

METHOD 2. The following is a python environment that prints a shell function call:

```
[student@localhost doc] pwd  
home/student
```

Line after python environment with shell function call. Here are two such:

```
[student@localhost doc] pwd  
home/student
```

```
[student@localhost doc] pwd  
home/student
```

METHOD 3. The following is a python environment that prints an execute function call:

```
[student@localhost doc] pwd  
home/student
```

Line after python environment with execute function call. Here are two such:

```
[student@localhost doc] pwd  
home/student
```

```
[student@localhost doc] pwd  
home/student
```

Notice the vertical spacing before and after the framed window is smaller for 1. The verbatim spacing before and after the framed window for 2 and 3 are the same.

METHOD 4. The following is a python environment that prints an execute function call:

```
from latextool_basic import shell  
print(shell('pwd'))
```

```
[student@localhost doc] pwd  
home/student
```

Line after python environment with execute function call.

Another example:

```
from latextool_basic import shell  
print(shell('ls -la c*.py'))
```

```
[student@localhost doc] ls -la c*.py  
-rwxrwxrwx. 1 root root 1807 Dec 25 16:17 crowfoot.py  
-rwxrwxrwx. 1 root root  206 Dec 21 19:16 cyhfunqd.tmp.py
```

Line after python environment with execute function call.

Execute command in another directory

```
from latextool_basic import shell  
print(shell('pwd; ls -la .b*', dir='/home/student'))
```

```
[student@localhost ~] pwd; ls -la .*
/home/student
-rw-----. 1 student student 7966 Dec 31 02:25 .bash_history
-rw-r--r--. 1 student student  18 Dec  6  2019 .bash_logout
-rw-r--r--. 1 student student 141 Dec  6  2019 .bash_profile
lrwxrwxrwx. 1 root    root    54 Dec 31 01:29 .bashrc -> /home/student/shares/
yliow/Documents/work/unix/.bashrc
-rwx-----. 1 student student 2813 Sep 22  2021 .bashrc~
-rwxrwxrwx. 1 student student 3029 Sep 28  2022 .bashrc.backup
lrwxrwxrwx. 1 root    root    54 Dec 30 02:20 .bashrc.old -> /home/student/sha
res/yliow/Documents/work/unix/.bashrc
```

no execution

```
from latextool_basic import shell
print(shell('ls -la', execute=False))
```

```
[student@localhost doc] ls -la
```

Executing with error: PYTHON ERROR. See source, stderr, stdout below

```
print(x)
```

```
Traceback (most recent call last):
  File "mmekgbld.tmp.py", line 1, in <module>
    print(x)
NameError: name 'x' is not defined
```

11 Plot (plot.tex)

All drawing go into a `Plot` object. The following should be in a python environment:

```
from latextool_basic import Plot, Circle
p = Plot()
p += Circle(x=1, y=1, r=1)
print(p)
```

or run in the `execute` function in a python environment:

```
s = r"""
from latextool_basic import Plot, Circle
p = Plot()
p += Circle(x=1, y=1, r=1)
print(p)
"""

from latextool_basic import execute
execute(s)
```

A centered `tikzpicture` environment is then inserted.

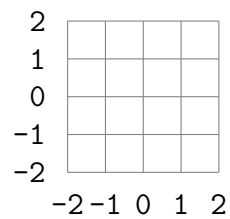
You can also add a `pgf/tikz` string to plot:

```
from latextool_basic import Plot, Circle
p = Plot()
p += Circle(x=1, y=1, r=1)
p.add("\draw ...")
print(p)
```

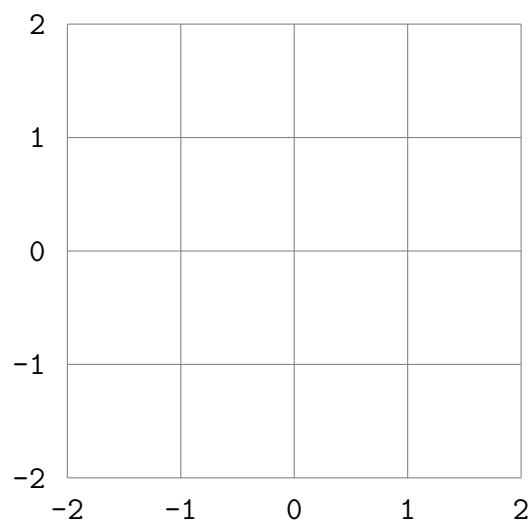
You can create and store a pdf image in the `tmp/` subdirectory. See next section.

12 Scaling (scaling.tex)

```
from latextool_basic import *  
p = Plot(scale=0.5)  
p += Grid(x0=-2, y0=-2, x1=2, y1=2)  
print(p)
```



```
from latextool_basic import *  
p = Plot(scale=1.5)  
p += Grid(x0=-2, y0=-2, x1=2, y1=2)  
print(p)
```



13 Grid (grid.tex)

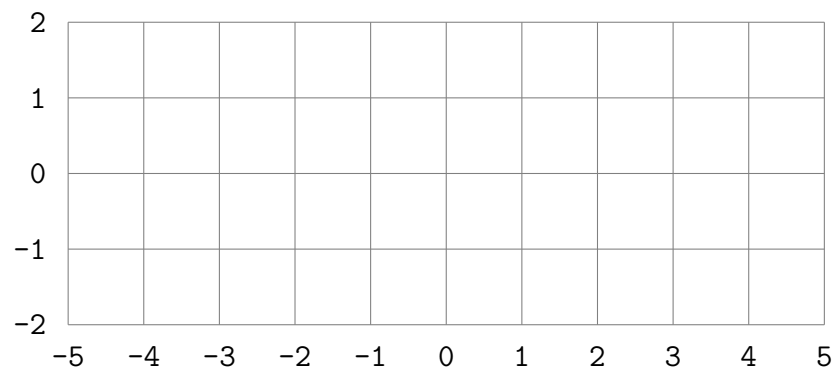
Plot with default grid.

```
from latextool_basic import *
p = Plot()
p += Grid()
print(p)
```

$$\begin{matrix} 0 & . \\ & 0 \end{matrix}$$

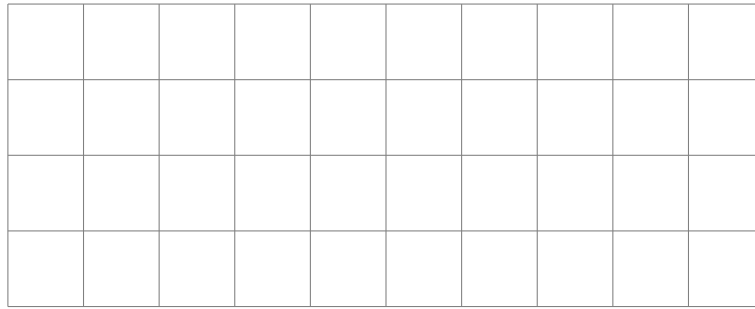
With object grid and bounding box:

```
from latextool_basic import *
p = Plot()
p += Grid(x0=-5, y0=-2, x1=5, y1=2)
print(p)
```



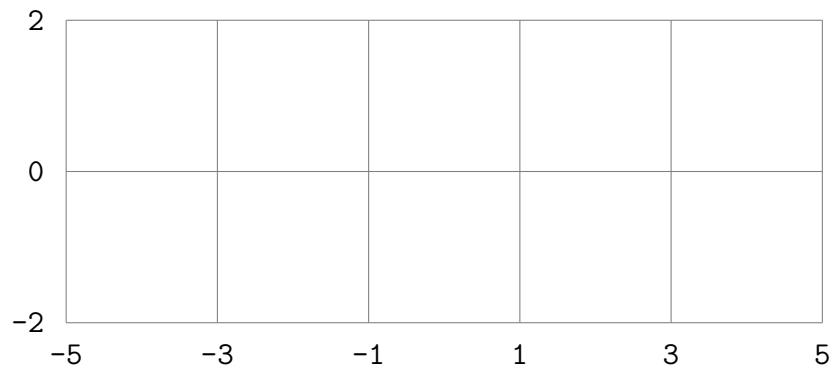
With label_axis set to false.

```
from latextool_basic import *
p = Plot()
p += Grid(x0=-5, y0=-2, x1=5, y1=2, label_axes=False)
print(p)
```



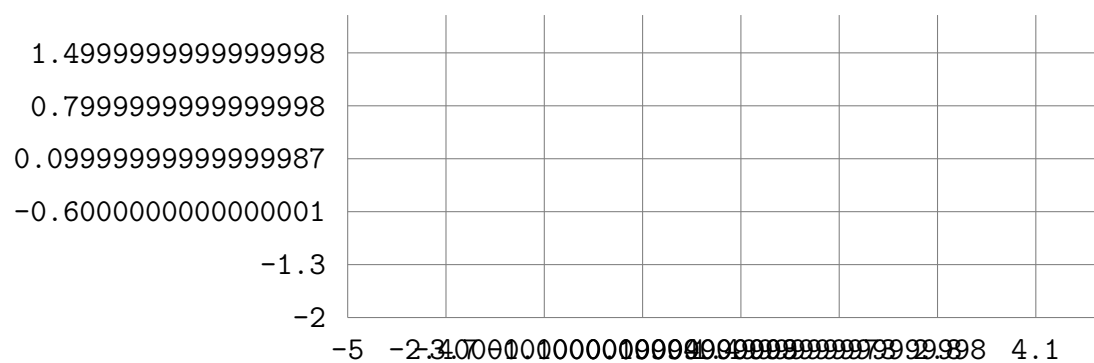
Example. $dx = 2$, $dy = 2$.

```
from latextool_basic import *  
p = Plot()  
p += Grid(x0=-5, y0=-2, x1=5, y1=2, dx=2, dy=2)  
print(p)
```



Example. $dx = 1.3$, $dy = 0.7$.

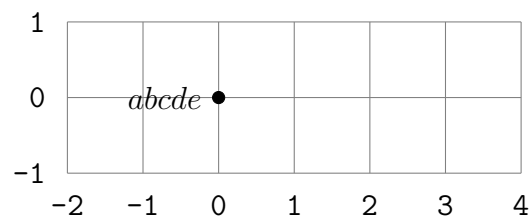
```
from latextool_basic import *  
p = Plot()  
p += Grid(x0=-5, y0=-2, x1=5, y1=2, dx=1.3, dy=0.7)  
print(p)
```



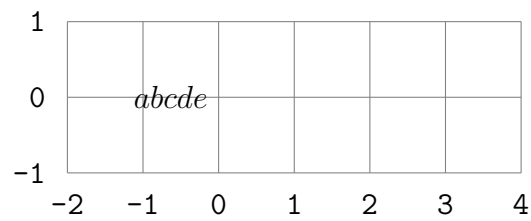
14 Point and text placement (point.tex)

[See section on logic design]

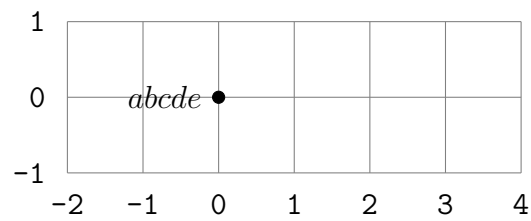
```
from latextool_basic import *
from latexcircuit import *
p = Plot()
X = POINT(x=0, y=0, label='$abcde$')
p += str(X)
p += Grid(x0=-2, y0=-1, x1=4, y1=1)
print(p)
```



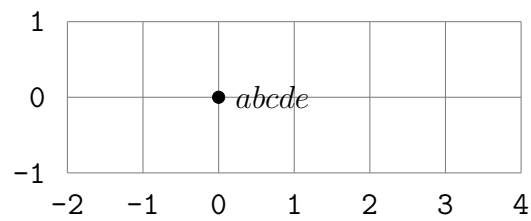
```
from latextool_basic import *
from latexcircuit import *
p = Plot()
X = POINT(x=0, y=0, r=0, label='$abcde$')
p += str(X)
p += Grid(x0=-2, y0=-1, x1=4, y1=1)
print(p)
```



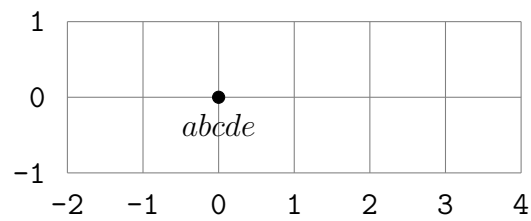
```
from latextool_basic import *
from latexcircuit import *
p = Plot()
X = POINT(x=0, y=0, label='$abcde$', anchor='east')
p += str(X)
p += Grid(x0=-2, y0=-1, x1=4, y1=1)
print(p)
```



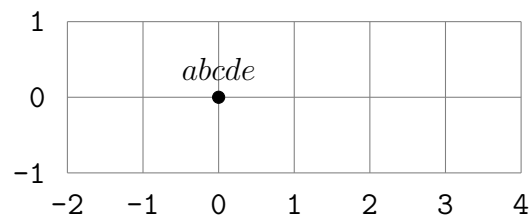
```
from latextool_basic import *
from latexcircuit import *
p = Plot()
X = POINT(x=0, y=0, label='$abcde$', anchor='west')
p += str(X)
p += Grid(x0=-2, y0=-1, x1=4, y1=1)
print(p)
```



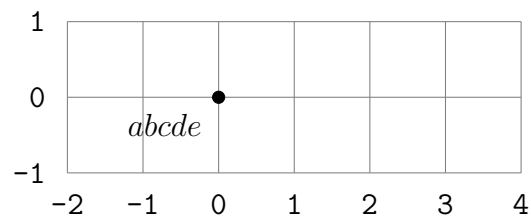

```
from latextool_basic import *
from latexcircuit import *
p = Plot()
X = POINT(x=0, y=0, label='$abcde$', anchor='north')
p += str(X)
p += Grid(x0=-2, y0=-1, x1=4, y1=1)
print(p)
```



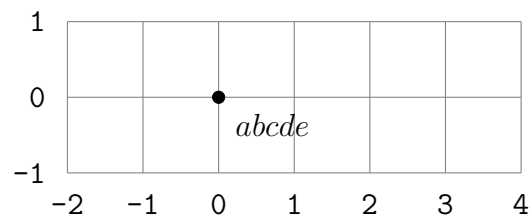
```
from latextool_basic import *
from latexcircuit import *
p = Plot()
X = POINT(x=0, y=0, label='$abcde$', anchor='south')
p += str(X)
p += Grid(x0=-2, y0=-1, x1=4, y1=1)
print(p)
```



```
from latextool_basic import *
from latexcircuit import *
p = Plot()
X = POINT(x=0, y=0, label='$abcde$', anchor='north east')
p += str(X)
p += Grid(x0=-2, y0=-1, x1=4, y1=1)
print(p)
```

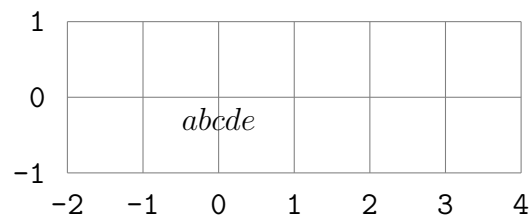


```
from latextool_basic import *
from latexcircuit import *
p = Plot()
X = POINT(x=0, y=0, label='$abcde$', anchor='north west')
p += str(X)
p += Grid(x0=-2, y0=-1, x1=4, y1=1)
print(p)
```

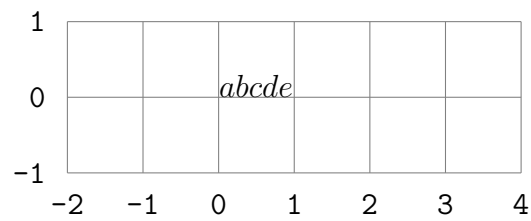


radius 0 cases:

```
from latextool_basic import *
from latexcircuit import *
p = Plot()
X = POINT(x=0, y=0, r=0, label='$abcde$', anchor='north')
p += str(X)
p += Grid(x0=-2, y0=-1, x1=4, y1=1)
print(p)
```



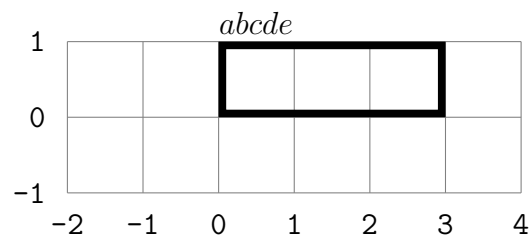
```
from latextool_basic import *
from latexcircuit import *
p = Plot()
X = POINT(x=0, y=0, r=0, label='$abcde$', anchor='flushtopleft')
p += str(X)
p += Grid(x0=-2, y0=-1, x1=4, y1=1)
print(p)
```



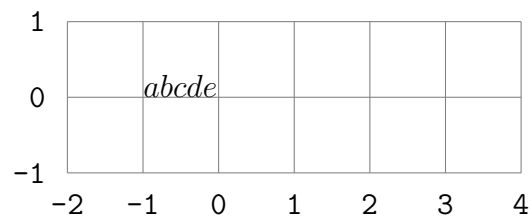
```

from latextool_basic import *
from latexcircuit import *
p = Plot()
p += Rect(x0=0, y0=0, x1=3, y1=1, linewidth=0.1)
X = POINT(x=0, y=1.1, r=0, label='$abcde$', anchor='flushtopleft')
p += str(X)
p += Grid(x0=-2, y0=-1, x1=4, y1=1)
print(p)

```



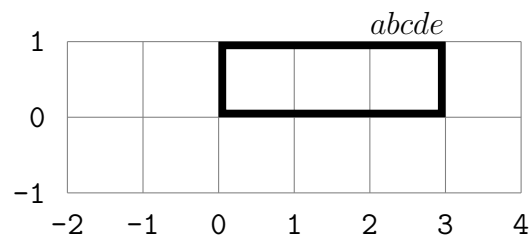
```
from latextool_basic import *
from latexcircuit import *
p = Plot()
X = POINT(x=0, y=0, r=0, label='$abcde$', anchor='flushtoprigh')
p += str(X)
p += Grid(x0=-2, y0=-1, x1=4, y1=1)
print(p)
```



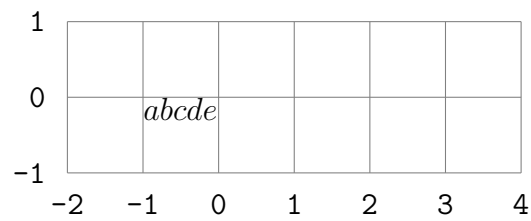

```

from latextool_basic import *
from latexcircuit import *
p = Plot()
p += Rect(x0=0, y0=0, x1=3, y1=1, linewidth=0.1)
X = POINT(x=3, y=1.1, r=0, label='$abcde$', anchor='flushtoprigh')
p += str(X)
p += Grid(x0=-2, y0=-1, x1=4, y1=1)
print(p)

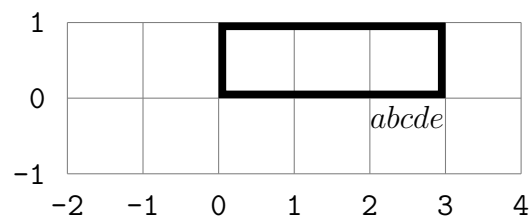
```



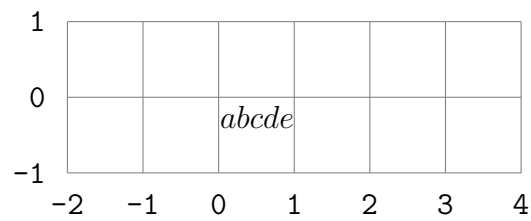
```
from latextool_basic import *
from latexcircuit import *
p = Plot()
X = POINT(x=0, y=0, r=0, label='$abcde$', anchor='flushbottomleft')
p += str(X)
p += Grid(x0=-2, y0=-1, x1=4, y1=1)
print(p)
```



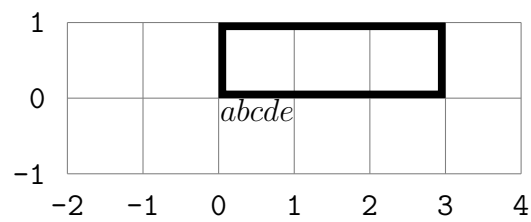
```
from latextool_basic import *
from latexcircuit import *
p = Plot()
p += Rect(x0=0, y0=0, x1=3, y1=1, linewidth=0.1)
X = POINT(x=3, y=-0.1, r=0, label='$abcde$', anchor='flushbottomleft')
p += str(X)
p += Grid(x0=-2, y0=-1, x1=4, y1=1)
print(p)
```



```
from latextool_basic import *
from latexcircuit import *
p = Plot()
X = POINT(x=0, y=-0.1, r=0, label='$abcde$', anchor='flushbottomright')
p += str(X)
p += Grid(x0=-2, y0=-1, x1=4, y1=1)
print(p)
```

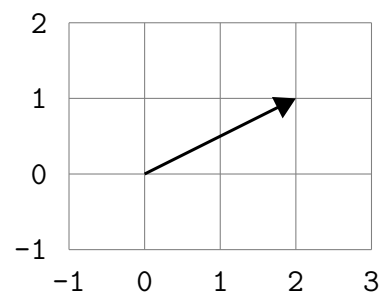


```
from latextool_basic import *
from latexcircuit import *
p = Plot()
p += Rect(x0=0, y0=0, x1=3, y1=1, linewidth=0.1)
X = POINT(x=0, y=0, r=0, label='$abcde$', anchor='flushbottomright')
p += str(X)
p += Grid(x0=-2, y0=-1, x1=4, y1=1)
print(p)
```



15 coordinate (coordinate.tex)

```
from latextool_basic import *
p = Plot()
p += coordinate(0, 0, 'a')
p += coordinate(2, 1, 'b')
p += Line(names=['a', 'b'], endstyle='>')
p += Grid(-1, -1, 3, 2)
print(p)
```



16 code (code.tex)

The `code` function creates a table based on the string passed in. The linewidths of the cells are set to 0.

```
from latextool_basic import *
p = Plot()

M = r"""
twice (fun x -> x * x) 42
(fun f -> (fun x -> f(f x))) (fun x -> x * x) 42
""".strip()

code(p, M)

print(p)
```

```
twice (fun x -> x * x) 42
(fun f -> (fun x -> f(f x))) (fun x -> x * x) 42
```

16.1 Position

```
from latextool_basic import *
p = Plot()

M = r"""
twice (fun x -> x * x) 42
(fun f -> (fun x -> f(f x))) (fun x -> x * x) 42
""".strip()

p += Circle(x=0, y=0, r=0.5)
code(p, M, x=0, y=0)

p += Circle(x=2, y=2, r=0.5, linecolor='red')
code(p, M, x=2, y=2)

print(p)
```

```
twice (fun x -> x * x) 42
(fun f -> (fun x -> f(f x))) (fun x -> x * x) 42
```

```
twice (fun x -> x * x) 42
(fun f -> (fun x -> f(f x))) (fun x -> x * x) 42
```


16.2 Cell size

```
from latextool_basic import *
p = Plot()

M = r"""
twice (fun x -> x * x) 42
(fun f -> (fun x -> f(f x))) (fun x -> x * x) 42
""".strip()

code(p, M, width=0.3, height=1)

print(p)
```

```
twice (fun x -> x * x) 42
```

```
(fun f -> (fun x -> f(f x))) (fun x -> x * x) 42
```

16.3 border linewidth

```
from latextool_basic import *
p = Plot()

M = r"""
twice (fun x -> x * x) 42
(fun f -> (fun x -> f(f x))) (fun x -> x * x) 42
""".strip()

code(p, M, border_linewidth=0.05)

print(p)
```

```
twice (fun x -> x * x) 42
(fun f -> (fun x -> f(f x))) (fun x -> x * x) 42
```

16.4 distance of border to rect

```
from latextool_basic import *
p = Plot()

M = r"""
twice (fun x -> x * x) 42
(fun f -> (fun x -> f(f x))) (fun x -> x * x) 42
""".strip()

code(p, M, border_linewidth=0.05, innersep=0.8)

print(p)
```

```
twice (fun x -> x * x) 42
(fun f -> (fun x -> f(f x))) (fun x -> x * x) 42
```

16.5 coderec

The return of `code` is just a rect. `coderec` takes in a `code` rect and starting row and column indices and ending row and column indices and returns a rect. The linecolor of this rect is red.

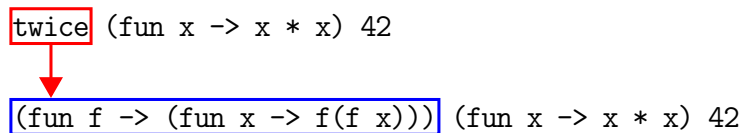
```
from latextool_basic import *
p = Plot()

M = r"""
twice (fun x -> x * x) 42

(fun f -> (fun x -> f(f x))) (fun x -> x * x) 42
""".strip()

N = code(p, M)
r0 = coderec(N, 0, 0, 0, 4)
p += r0
r1 = coderec(N, 3, 0, 3, 27)
r1.linecolor='blue'
p += r1

p0 = r0.bottom()
p1 = r1.top(); p1 = (p0[0], p1[1])
p += Line(points=[p0,p1], endstyle='>', linecolor='red')
print(p)
```



```
twice (fun x -> x * x) 42
↓
(fun f -> (fun x -> f(f x))) (fun x -> x * x) 42
```

16.6 Substitution

If you want to print ρ which takes more than one character, you can use an unused character in the string and then pass in a dictionary to replace the character to the actual string that you need.

```
from latextool_basic import *
p = Plot()

M = r"""
twice (fun x -> x * x) 42      #

(fun f -> (fun x -> f(f x))) (fun x -> x * x) 42
""".strip()

code(p, M, d={'#':r'\textred{$\rho$}'})
print(p)
```

```
twice (fun x -> x * x) 42       $\rho$ 
```

```
(fun f -> (fun x -> f(f x))) (fun x -> x * x) 42
```

16.7 Long division example

`linebelow` is used to draw lines in code. `divlinebelow` is used for long division line.

```
from latextool_basic import *
p = Plot()

M = r"""
.    01
1101 101010101010
    -0000
      10101
      -1101
        10000
          1101
        """.strip()

N = code(p, M)
divlinebelow(p, N, 0, 5, 16)
linebelow(p, N, 2, 4, 8)
linebelow(p, N, 4, 5, 9)
print(p)
```

```

      01
1101)101010101010
    -0000
      10101
      -1101
        10000
          1101
```

17 table (table.tex)

```
from latextool_basic import table
print(table({0:0, 1:1, 2:4, 3:9, 4:16}))
```

0	0
1	1
2	4
3	9
4	16

```
from latextool_basic import table
print(table([(1, 2, 3),
              (2, 2, 5),
              (5, 2),
              (-1, 2, 6),
              (1, 2, 3),
              ]))
```

1	2	3
2	2	5
5	2	
-1	2	6
1	2	3

```
from latextool_basic import table
print(table([(1, 2, 3),
              (2, 2, 5),
              (5, 2),
              (-1, 2, 6),
              (1, 2, 3),
              ],
            col_headings = ['$x$', '$y$', '$z$']))
```

x	y	z
-----	-----	-----

1	2	3
2	2	5
5	2	
-1	2	6
1	2	3

```

from latextool_basic import table
print(table([(' ', 2, 5, 8),
            (' ', ' ', 3, 6),
            (' ', ' ', ' ', 3),
            (' ', ' ', ' ', ' ')]
            ],
            col_headings = ['0', '2', '5', '8'],
            row_headings = ['0', '2', '5', '8']))

```

	0	2	5	8
0		2	5	8
2			3	6
5				3
8				

```

from latextool_basic import table
print(table([(' ', 2, 5, 8),
            (' ', ' ', 3, 6),
            (' ', ' ', ' ', 3),
            (' ', ' ', ' ', ' ')]
            ],
            col_headings = ['0', '2', '5', '8'],
            row_headings = ['0', '2', '5', '8'],
            topleft_heading = r'$\Delta X$',
            ))

```

ΔX	0	2	5	8
0		2	5	8
2			3	6
5				3

8				
---	--	--	--	--

Too few row headings:

```
from latextool_basic import table
print(table([(' ', 2, 5, 8),
            (' ', ' ', 3, 6),
            (' ', ' ', ' ', 3),
            (' ', ' ', ' ', ' ')
            ],
            col_headings = ['0', '2', '5', '8'],
            row_headings = ['0', '2'],
            topleft_heading = r'$\Delta X$',
            ))
```

ΔX	0	2	5	8
0		2	5	8
2			3	6
				3

Too few col headings

```
from latextool_basic import table
print(table([(' ', 2, 5, 8),
            (' ', ' ', 3, 6),
            (' ', ' ', ' ', 3),
            (' ', ' ', ' ', ' ')
            ],
            col_headings = ['0', '2'],
            row_headings = ['0', '2', '5', '8'],
            topleft_heading = r'$\Delta X$',
            ))
```

ΔX	0	2		
0		2	5	8
2			3	6
5				3
8				

Too few rows of data

```
from latextool_basic import table
print(table([('0', 2, 5, 8),
            ('2', 3, 6),
            ],
            col_headings = ['0', '2', '5', '8'],
            row_headings = ['0', '2', '5', '8'],
            topleft_heading = r'$\Delta X$',
            ))
```

ΔX	0	2	5	8
0		2	5	8
2			3	6
5				
8				

Too few rows of data

```
from latextool_basic import table
print(table([('0', 2, 5, 8),
            ('2', 3, 6),
            ],
            col_headings = ['0', '2', '5', '8'],
            row_headings = ['0', '2', '5', '8'],
            topleft_heading = r'$\Delta X$',
            ))
```

ΔX	0	2	5	8
0		2	5	8
2			3	6
5				
8				

Column headings only (no row headings). Note that topleft heading does not show.

```
from latextool_basic import table
print(table([(' ', 2, 5, ),
            (' ', ' ', 3, ),
            (' ', ' ', ' ', ),
            (' ', ' ', ' ', )
            ],
            col_headings = ['0', '2', '5', '8'],
            topleft_heading = r'$\Delta X$',
            ))
```

0	2	5	8
	2	5	
		3	

Row headings only (no column headings). Note that topleft heading does not show.

```
from latextool_basic import table
print(table([(' ', 2, 5, ),
            (' ', ' ', 3, ),
            (' ', ' ', ' ', ),
            (' ', ' ', ' ', )
            ],
            row_headings = ['0', '2', '5', '8'],
            topleft_heading = r'$\Delta X$',
            ))
```

0		2	5
2			3
5			
8			

Setting column justification.

```
from latextool_basic import table
print(table([('AAAAAAAAAA', 'BBBBBBBBB', 'CCCCCCCCC'),
            ('1', '2', 3),
            ('', '', ''),
            ('', '', ''),
            ],
            row_headings = ['0', '2', '5', '8'],
            col_headings = ['a', 'b', 'c'],
            topleft_heading = r'$\Delta X$',
            style='|c|l|r|c|',
            ))
```

ΔX	a	b	c
0	AAAAAAAAAA	BBBBBBBBB	CCCCCCCCC
2	1	2	3
5			
8			

Right row headings

```

from latextool_basic import table
print(table([(' ', 2, 5, ),
            (' ', ' ', 3, ),
            (' ', ' ', ' ', ),
            (' ', ' ', ' ', )
            ],
            right_row_headings = ['0', '2', '5', '8'],
            col_headings = ['a', 'b', 'c'],
            ))

```

a	b	c
	2	5
		3

17.1 table2 (table2.tex)

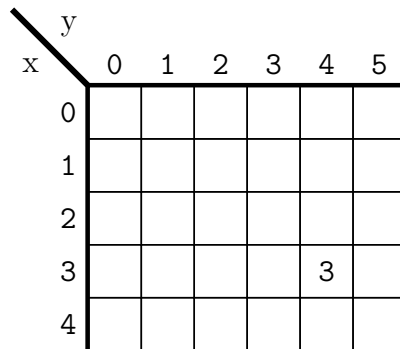
```

from latextool_basic import *
p = Plot()

m = [
    ['', '', '', '', '', ''],
    ['', '', '', '', '', ''],
    ['', '', '', '', '', ''],
    ['', '', '', '', '3', ''],
    ['', '', '', '', '', ''],
    ['', '', '', '', '', ''],
]

table2(p, m, width=0.7, height=0.7, rowlabel='x', collabel='y')
print(p)

```



	y	0	1	2	3	4	5
x	0						
	1						
	2						
	3				3		
	4						

17.1.1 Change row and column index values/names

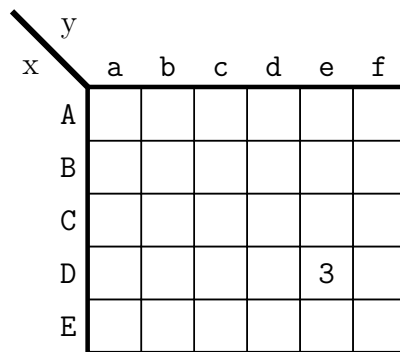
```

from latextool_basic import *
p = Plot()

m = [
    ['', '', '', '', '', ''],
    ['', '', '', '', '', ''],
    ['', '', '', '', '', ''],
    ['', '', '', '', '3', ''],
    ['', '', '', '', '', ''],
    ['', '', '', '', '', ''],
]

table2(p, m, width=0.7, height=0.7, rowlabel='x', collabel='y',
        rownames=[r'\texttt{%s}' % _ for _ in "ABCDE"],
        colnames=[r'\texttt{%s}' % _ for _ in "abcdef"])
print(p)

```



	y					
x	a	b	c	d	e	f
A						
B						
C						
D					3	
E						

17.1.2 No row and column indices/values

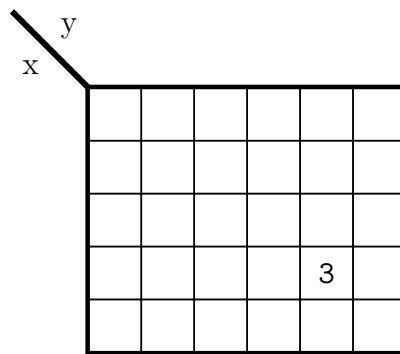
```

from latextool_basic import *
p = Plot()

m = [['', '', '', '', '', ''],
      ['', '', '', '', '', ''],
      ['', '', '', '', '', ''],
      ['', '', '', '', '3', ''],
      ['', '', '', '', '', ''],
      []]

table2(p, m, width=0.7, height=0.7, rowlabel='x', collabel='y',
        rownames=[],
        colnames=[])
print(p)

```



17.1.3 Change row/column names

```

from latextool_basic import *

p = Plot()

m = [[0,1,1,0],
      [0,1,0,1],
      [1,0,1,1],
      [0,1,1,1],
      ]

table2(p, m, width=0.7, height=0.7,
        rownames=['00','01','11','10'],
        colnames=['00','01','11','10'],
        rowlabel='$WX$', collabel='$YZ$')
print(p)

```

WX \ YZ	YZ			
	00	01	11	10
00	0	1	1	0
01	0	1	0	1
11	1	0	1	1
10	0	1	1	1

17.1.4 No row and no column names

```

from latextool_basic import *
p = Plot()

m = [['', '', '', '', '', ''],
      ['', '', '', '', '', ''],
      ['', '', '', '', '', ''],
      ['', '', '', '', '3', ''],
      ['', '', '', '', '', ''],
      []]

table2(p, m, width=0.7, height=0.7,
        rownames=[],
        colnames=[],
        rowlabel=None, collabel=None)
print(p)

```

				3	

17.1.5 border_linewidth

```

from latextool_basic import *
p = Plot()

m = [['', '', '', '', '', ''],
      ['', '', '', '', '', ''],
      ['', '', '', '', '', ''],
      ['', '', '', '', '3', ''],
      ['', '', '', '', '', ''],
      []]

table2(p, m, x=0, y=0, width=0.7, height=0.7,
        rownames=[],
        colnames=[],
        rowlabel=None, collabel=None, border_linewidth=0)
table2(p, m, x=8, y=0, width=0.7, height=0.7,
        rownames=[],
        colnames=[],
        rowlabel=None, collabel=None, border_linewidth=0.5)
print(p)

```

				3	

				3	

17.1.6 Adding things beneath table2: background

The `do_not_plot` can be used to compute the coordinate of the cells. `table2` will then return the `RectContainer` object but the table is not drawn. So one way to change the background is to do `do_not_plot` to get the coordinates, draw a rect with background color, then call `table2` to draw the contents of the table. (Also, see `rect` parameter later.)

```
from latextool_basic import *

m = [['', '', '', '', '', ''],
      ['', '', '', '', '', ''],
      ['', '', '', '', '', ''],
      ['', '', '', '', '3', ''],
      ['0', '', '', ''],          # OK to have jagged arrays
      ]

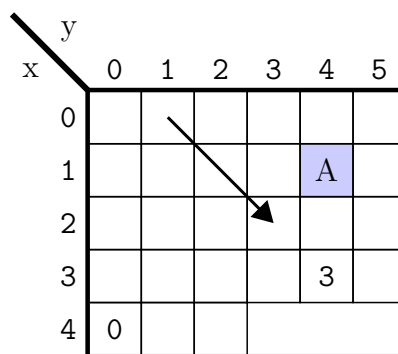
p = Plot()
C = table2(p, m, rowlabel='x', collabel='y', do_not_plot=True)

# Draw over C[1][4]
x0,y0 = C[1][4].bottomleft()
x1,y1 = C[1][4].topright()
p += Rect(x0=x0, y0=y0, x1=x1, y1=y1,
          background='blue!20', label='A', linewidth=0)

# Draw table one more time to get border of C[1][4] correct
table2(p, m, rowlabel='x', collabel='y')

p += Line(points=[C[0][1].center(), C[2][3].center()], endstyle='>')

print(p)
```



17.1.7 Using cells

Drawing arrows between cells using `shorten` function for arrows using a shortening factor:

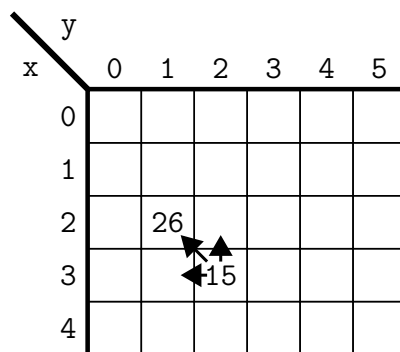
```
x = "      "
y = "      "
m = [['' for i in range(len(y))] for j in range(len(x))]
m[3][2] = 15
m[2][1] = 26
from latextool_basic import *
p = Plot()
C = table2(p, m, width=0.7, height=0.7,
           rowlabel=r'\texttt{x}', collabel=r'\texttt{y}')

p0 = C[3][2].center()
p1 = C[2][1].center()
p0, p1 = shorten(p0, p1, factor=0.5)
p += Line(points=[p0, p1], endstyle='>')

p0 = C[3][2].center()
p1 = C[2][2].center()
p0, p1 = shorten(p0, p1, factor=0.5)
p += Line(points=[p0, p1], endstyle='>')

p0 = C[3][2].center()
p1 = C[3][1].center()
p0, p1 = shorten(p0, p1, factor=0.5)
p += Line(points=[p0, p1], endstyle='>')

print(p)
```



Drawing arrows between cells using `shorten` function for arrows using a shortening amount:

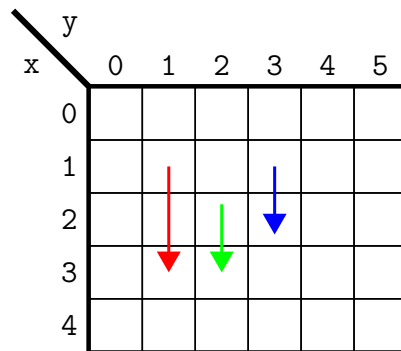
```
x = "      "
y = "      "
m = [['' for i in range(len(y))] for j in range(len(x))]
from latextool_basic import *
p = Plot()
C = table2(p, m, width=0.7, height=0.7,
           rowlabel=r'\texttt{x}', collabel=r'\texttt{y}')

p0 = C[1][1].center()
p1 = C[3][1].center()
p += Line(points=[p0, p1], endstyle='>', linecolor='red')

p0 = C[1][2].center()
p1 = C[3][2].center()
p0, p1 = shorten(p0, p1, start_by=0.5)
p += Line(points=[p0, p1], endstyle='>', linecolor='green')

p0 = C[1][3].center()
p1 = C[3][3].center()
p0, p1 = shorten(p0, p1, end_by=0.5)
p += Line(points=[p0, p1], endstyle='>', linecolor='blue')

print(p)
```



17.1.8 Rect for cells

You can specify a rect function of your own:

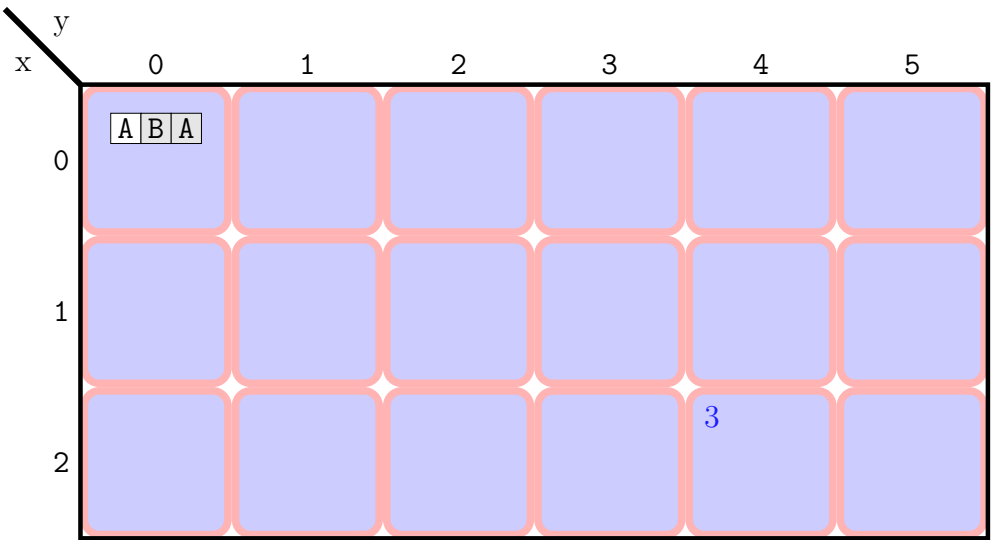
```
from latextool_basic import *

c = RectContainer(x=0, y=0)
xs = 'ABA'; shadedbackground='black!10'
for i,x in enumerate(xs):
    if i in [1,2]:
        c += Rect2(x0=0, y0=0, x1=0.4, y1=0.4,
                    background=shadedbackground,
                    linewidth=0.01, label=r'\texttt{%s}' % x)
    else:
        c += Rect2(x0=0, y0=0, x1=0.4, y1=0.4,
                    background='white',
                    linewidth=0.01, label=r'\texttt{%s}' % x)
q = Plot(); q += c
m = [[str(q),'', '', '', '', ''],
      ['', '', '', '', '', ''],
      ['', '', '', '', '3', ''],
      []]

p = Plot()

def rect(x):
    return Rect(x0=0, y0=0, x1=2, y1=2, radius=0.2, innersep=0.2,
                linecolor='red!30!white', linewidth=0.1,
                background='blue!20!white!', foreground='blue!90!white!',
                s='%s' % x, align='t')

table2(p, m, width=0.7, height=0.7, rowlabel='x', collabel='y', rect=rect)
print(p)
```



17.1.9 Using rect to change the background

You can use the rect to for instance change the background. In the following, the rects at column 2 are given larger width and a different background, and the contents are placed in a minipage with tiny fontsize.

```
from latextool_basic import *
m = [['1','primes: 2, 3, 5, 7, 11, 13, 19, ...',' ',' ',' ',' '],
      [' ',' ',' ',' ',' ',' '],
      [' ',' ',' ',' ',' ',' '],
      [' ',' ',' ',' ','3',' '],
      ]
def get_rect():
    i = [0]
    def rect(x):
        if i[0] % 6 == 1:
            i[0] += 1
            return Rect(x0=0, y0=0, x1=2, y1=1,
                        background='blue!20!white!',
                        innersep=0.1,
                        s=r'\tiny %s' % x, align='t')
        else:
            i[0] += 1
            return Rect(x0=0, y0=0, x1=1, y1=1, label=x)
    return rect
p = Plot()
table2(p, m, rowlabel='x', collabel='y', rect=get_rect())
print(p)
```

	0	1	2	3	4	5
0	1	primes: 2, 3, 5, 7, 11, 13, 19, ...				
1						
2						
3					3	

17.2 **table3** (`table3.tex`)

table3 is for drawing 2d arrays of 2d arrays.

This can be used for truth tables.

17.2.1 2-by-2 case

```
from latextool_basic import *
p = Plot()

m00 = [['$w$', '$x$', '$y$', '$z$']]
m10 = [['$0$', '', '', ''],
        ['', '', '', ''],
        ['', '', '', ''],
        ['', '', '', ''],
        ['', '', '', '']]
m01 = [['$f$', '$g$'],
        ]
m11 = [['$1$', ''],
        ['', ''],
        ['', ''],
        ['', ''],
        ['', '']]
M = [[m00, m01],
      [m10, m11]]
N = table3(p, M, width=1, height=0.8)
print(p)
```

<i>w</i>	<i>x</i>	<i>y</i>	<i>z</i>	<i>f</i>	<i>g</i>
0				1	

17.2.2 2-by-2 case with top and left for labels

```

from latextool_basic import *
p = Plot()

rowlabels = ['A', 'B', 'C', 'D', 'E']
collabels = ['a', 'b', 'c']
data = [['00', '01', '02'],
        ['10', '11', '12'],
        ['20', '21', '22'],
        ['30', '31', '32'],
        ['40', '41', '42']]

m00 = [['']]
m10 = [[x] for x in rowlabels]
m01 = [collabels]
m11 = data
M = [[m00, m01],
      [m10, m11]]
N = table3(p, M, width=1, height=0.8)
print(p)

```

	a	b	c
A	00	01	02
B	10	11	12
C	20	21	22
D	30	31	32
E	40	41	42

17.2.3 2-by-1 case

```
from latextool_basic import *
p = Plot()

m00 = [['$w$', '$x$', '$y$', '$z$'],
       ]
m10 = [['$0$', '', '', ''],
       ['', '', '', ''],
       ['', '', '', ''],
       ['', '', '', ''],
       ['', '', '', ''],
       ]
M = [[m00],
     [m10]]
N = table3(p, M, width=1, height=0.8)
print(p)
```

w	x	y	z
0			

17.2.4 Specify topleft coordinates

```
from latextool_basic import *
p = Plot()

m00 = [['$w$', '$x$', '$y$', '$z$'],
       ]
m10 = [['$0$', '', '', ''],
       ['', '', '', ''],
       ['', '', '', ''],
       ['', '', '', ''],
       ['', '', '', ''],
       ]
M = [[m00],
     [m10]]
table3(p, x0=0, y0=0, M=M, width=1, height=0.8)
table3(p, x0=8, y0=1, M=M, width=1, height=0.8)
print(p)
```

w	x	y	z
0			

w	x	y	z
0			

17.2.5 2-by-1 case with title

```
from latextool_basic import *
p = Plot()

m00 = [['$w$', '$x$', '$y$', '$z$'],
       ]
m10 = [['$0$', '', '', ''],
       ['', '', '', ''],
       ['', '', '', ''],
       ['', '', '', ''],
       ['', '', '', ''],
       ]
M = [[m00],
     [m10]]
table3(p, M, width=1, height=0.8, title='This is the title')
print(p)
```

This is the title

w	x	y	z
0			

17.2.6 1-by-2 case

```
from latextool_basic import *
p = Plot()

m00 = [['$w$'], ['$x$'], ['$y$'], ['$z$'],
        ]
m01 = [['0'], ['1'], ['2'], ['3'],
        ]
M = [[m00, m01],
        ]
N = table3(p, M, width=1, height=0.8)
print(p)
```

w	0
x	1
y	2
z	3

17.2.7 Accessing cells

```

from latextool_basic import *
p = Plot()

m00 = [['$w$', '$x$', '$y$', '$z$'],
        ]
m10 = [['$0$', '', '', ''],
        [' ', ' ', ' ', ' '],
        [' ', ' ', ' ', ' '],
        [' ', ' ', ' ', ' '],
        [' ', ' ', ' ', ' '],
        ]
m01 = [['$f$', '$g$'],
        ]
m11 = [['$1$', ''],
        [' ', ''],
        [' ', ''],
        [' ', ''],
        [' ', ''],
        ]
M = [[m00, m01],
      [m10, m11]]
N = table3(p, M, width=1, height=0.8)
p0 = x0, y0 = N[1][0][3][2].center()
p1 = x1, y1 = x0 + 5, y0 + 1
p += Line(points=[p1, p0], endstyle='>', linecolor='red', linewidth=0.1)
x2, y2 = x1+0.5, y1+0.5
p += Rect(x0=x2, y0=y0, x1=x2, y1=y2, s='look', linewidth=0)
print(p)

```

w	x	y	z	f	g
0				1	

look

17.2.8 Coloring a cell under the table manually

```

from latextool_basic import *
p = Plot()

m00 = [['$w$', '$x$', '$y$', '$z$'],
        ]
m10 = [['$0$', '', '', ''],
        ['', '', '', ''],
        ['', '', '', ''],
        ['', '', '', ''],
        ['', '', '', ''],
        ]
M = [[m00],
      [m10]]

N = table3(p, M, width=1, height=0.8)

r = N[1][0][2][3]
x0,y0 = r.bottomleft()
x1,y1 = r.topright()
p += Rect(x0=x0, y0=y0, x1=x1, y1=y1, background='blue!20', label='42',
          linewidth=0)

N = table3(p, M, width=1, height=0.8)

print(p)

```

w	x	y	z
0			
			42

17.2.9 Coloring a cell under the table using table3

```
from latextool_basic import *
p = Plot()

m00 = [['$w$', '$x$', '$y$', '$z$'],
        ]
m10 = [['$0$', '', '', ''],
        ['', '', '', ''],
        ['', '', '', '42'],
        ['', '', '', ''],
        ['', '', '', ''],
        ]
M = [[m00],
      [m10]]

N = table3(p, M, width=2, height=0.5,
            background={(1,0,2,3): 'blue!20'})

print(p)
```

w	x	y	z
0			
			42

17.2.10 Cells of different widths (table2)

```

from latextool_basic import *
p = Plot()

i = 0
def rect(x):
    global i
    if i % 2 == 0:
        i += 1
        return Rect(x0=0, y0=0, x1=1, y1=0.7, s=x)
    else:
        i += 1
        return Rect(x0=0, y0=0, x1=3, y1=0.7, s=x)

m = [[0,1,2,3],
      [4,5,6,7],
      [8,9,10,11],
      [12,13,14,15],
      ]

table2(p, m, rect=rect)
print(p)

```

	0	1	2	3
0	0	1	2	3
1	4	5	6	7
2	8	9	10	11
3	12	13	14	15

17.2.11 Cells of different widths (table3)

```

from latextool_basic import *
p = Plot()

i = 0
def rect(x):
    global i
    if i in [0,3,6,8,10,13,16,19,21,23]:
        i += 1; return Rect(x0=0, y0=0, x1=1, y1=0.7, label=x)
    elif i in [1,4,7,9,11,14,17,20,22,24]:
        i += 1; return Rect(x0=0, y0=0, x1=3, y1=0.7, label=x)
    else:
        i += 1; return Rect(x0=0, y0=0, x1=5, y1=0.7, label=x)

m00 = [[0,1,2],
        [3,4,5]]
m01 = [[6,7],
        [8,9]]
m10 = [[10,11,12],
        [13,14,15],
        [16,17,18]]
m11 = [[19,20],
        [21,22],
        [23,24]]
M = [[m00, m01],
      [m10, m11]]
table3(p, M, rect=rect)
print(p)

```

0	1	2	6	7
3	4	5	8	9
10	11	12	19	20
13	14	15	21	22
16	17	18	23	24

XXX

17.2.12 Ragged arrays (error - to fix)

```

from latextool_basic import *
p = Plot()

i = 0
def rect(x):
    global i
    if i in [0,3,6,8,10,13,16,19,21,23]:
        i += 1; return Rect(x0=0, y0=0, x1=1, y1=0.7, label=x)
    elif i in [1,4,7,9,11,14,17,20,22,24]:
        i += 1; return Rect(x0=0, y0=0, x1=3, y1=0.7, label=x)
    else:
        i += 1; return Rect(x0=0, y0=0, x1=5, y1=0.7, label=x)

m00 = [[0,1,2],
        [3,4,5]]
m01 = [[6,7],
        [8,9]]
m10 = [[10,11,12],
        [13,14,15],
        [16,17,18]]
m11 = [[19,20],
        [21,22],
        [23,24]]
M = [[m00, m01],
      [m10, m11]]
table3(p, M, rect=rect)
print(p)

```

0	1	2	6	7
3	4	5	8	9
10	11	12	19	20
13	14	15	21	22
16	17	18	23	24

17.2.13 Justification

```

from latextool_basic import *
p = Plot()

i = 0
def rect(x):
    global i
    if i in [0,3,6,8,10,13,16,19,21,23]:
        i += 1; return Rect(x0=0, y0=0, x1=1, y1=0.7, label=x)
    elif i in [1,4,7,9,11,14,17,20,22,24]:
        i += 1; return Rect(x0=0, y0=0, x1=3, y1=0.7, label=x)
    else:
        i += 1; return Rect(x0=0, y0=0, x1=5, y1=0.7, label=x)

m00 = [[0,1,2],
        [3,4,5]]
m01 = [[6,7],
        [8,9]]
m10 = [[10,11,12],
        [13,14,15],
        [16,17,18]]
m11 = [[19,20],
        [21,22],
        [23,24]]
M = [[m00, m01],
      [m10, m11]]
table3(p, M, rect=rect)
print(p)

```

0	1	2	6	7
3	4	5	8	9
10	11	12	19	20
13	14	15	21	22
16	17	18	23	24

17.2.14 title

```

from latextool_basic import *
p = Plot()

i = 0
def rect(x):
    global i
    i += 1
    return Rect(x0=0, y0=0, x1=1.8, y1=0.8, label=x)

m00 = [['A','B','C']]

s = r"""
1 2 3
4 5 6
7 8 9
"""
X = str_to_2darray(s)
M = [[[X[0]]],
      [X[1:]],
      ]
table3(p, M, rect=rect, title='This is a title')
print(p)

```

This is a title

1	2	3
4	5	6
7	8	9

17.2.15 Title distance

```

from latextool_basic import *
p = Plot()

i = 0
def rect(x):
    global i
    i += 1
    return Rect(x0=0, y0=0, x1=1.8, y1=0.8, label=x)

m00 = [['A','B','C']]

s = r"""
1 2 3
4 5 6
7 8 9
"""
X = str_to_2darray(s)
M = [[[X[0]]],
      [X[1:]],
      ]
table3(p, M, rect=rect, title='This is a title', title_distance=0.8)
print(p)

```

This is a title

1	2	3
4	5	6
7	8	9

17.3 table4 (table4.tex)

```
from latextool_basic import *
p = Plot()
M = [['a', 'b', 'c'],
      [0, 1, 2],
      [3, 4, 5],
      [6, 7, 8]]
widths=[0.8, 1.5, 3]
N = table4(p=p, M=M, widths=widths)
print(p)
```

a	b	c
0	1	2
3	4	5
6	7	8

title distance:

```
from latextool_basic import *
p = Plot()
M = [['a', 'b', 'c'],
      [0, 1, 2],
      [3, 4, 5],
      [6, 7, 8]]
widths=[0.8, 1.5, 3]
N = table4(p=p, M=M, widths=widths, title='This is the title')
print(p)
```

This is the title

a	b	c
0	1	2
3	4	5
6	7	8

make caption:

```

from latextool_basic import *
from latexcircuit import *
p = Plot()
M = [['a', 'b', 'c'],
      [0, 1, 2],
      [3, 4, 5],
      [6, 7, 8]]
widths=[0.8, 1.5, 3]
N = table4(p=p, M=M, widths=widths, title='This is the title')
x0,y0 = N[-1][0].bottomleft()
x1,y1 = N[-1][-1].bottomright()
y = y0
x = (x0 + x1) / 2
p += POINT(x=x, y=y, r=0, label='This is the caption', anchor='north')
print(p)

```

This is the title

a	b	c
0	1	2
3	4	5
6	7	8

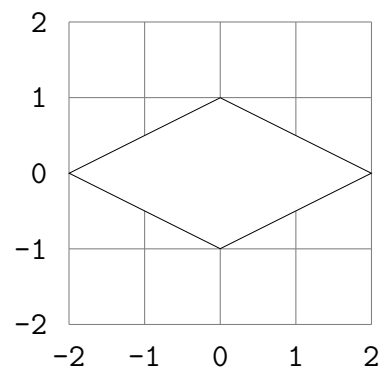
17.4 str_to_2darray (helper-for-2d-arrays.tex)

```
from latextool_basic import *
p = Plot()
M = r"""
a b c
0 1 2
3 4 5
6 7 8
"""
M = str_to_2darray(M)
widths=[0.8, 1.5, 3]
N = table4(p=p, M=M, widths=widths)
print(p)
```

a	b	c
0	1	2
3	4	5
6	7	8

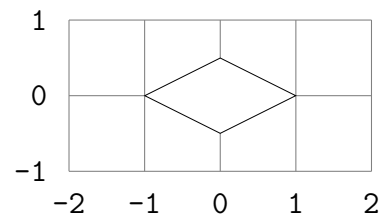
18 diamond (diamond.tex)

```
from latextool_basic import *  
p = Plot()  
p += Grid(-2,-2,2,2)  
p += diamond(center=(0,0))  
print(p)
```

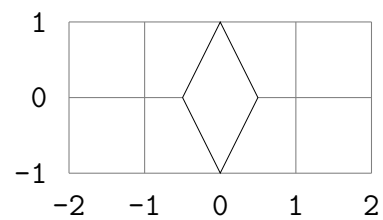


Width, height:

```
from latextool_basic import *
p = Plot()
p += Grid(-2,-1,2,1)
p += diamond(center=(0,0), width=2, height=1)
print(p)
```



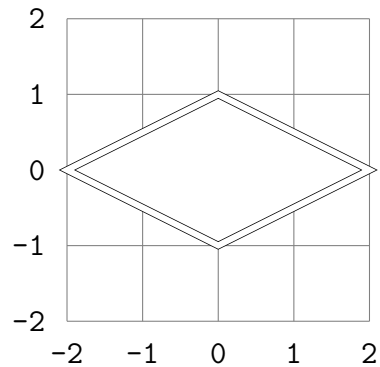
```
from latextool_basic import *
p = Plot()
p += Grid(-2,-1,2,1)
p += diamond(center=(0,0), width=1, height=2)
print(p)
```



double line:

```
from latextool_basic import *
p = Plot()

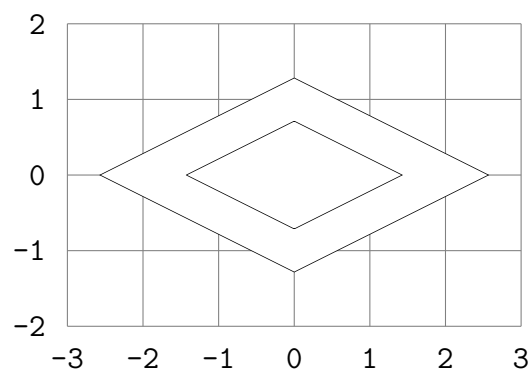
p += Grid(-2,-2,2,2)
p += diamond(center=(0,0), double=True)
print(p)
```



double line with double distance:

```
from latextool_basic import *
p = Plot()

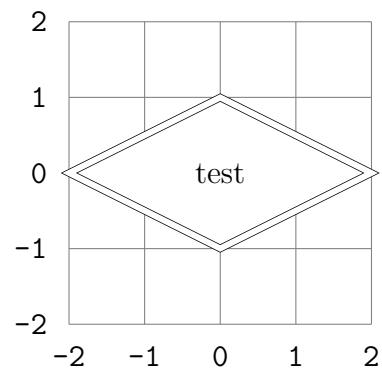
p += Grid(-3,-2,3,2)
p += diamond(center=(0,0), double=True, double_distance=0.5)
print(p)
```



double and label

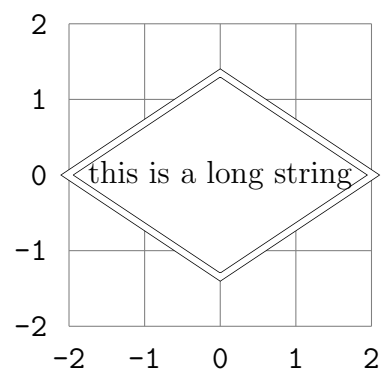
```
from latextool_basic import *
p = Plot()

p += Grid(-2,-2,2,2)
p += diamond(center=(0,0), double=True, label='test')
print(p)
```



```
from latextool_basic import *
p = Plot()

p += Grid(-2,-2,2,2)
p += diamond(center=(0,0), double=True, label='this is a long string')
print(p)
```



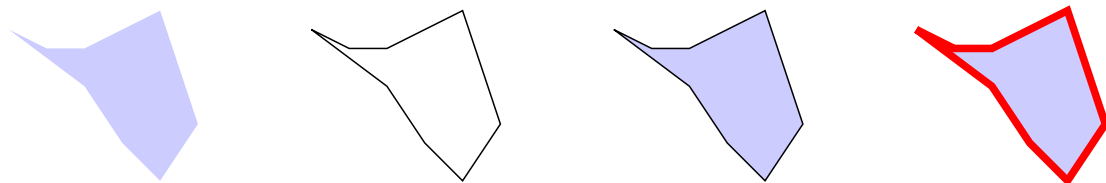
19 polygon (polygon.tex)

```
import random; random.seed()
from latextool_basic import *

def IFELSE(b, x, y):
    if b: return x
    else: return y

def polygon(points,
            background=None, # if background is None, no fill
            linewidth=None,
            linecolor=None,
            ):
    if points[-1] != points[0]: points.append(points[0])
    points_str = " -- ".join("(%s,%s)" % (x,y) for (x,y) in points)
    interior = IFELSE(background, "fill=%s" % background, '')
    boundary = ''
    if linewidth or linecolor:
        if not linecolor: linecolor='black'
        if not linewidth: linewidth=0.02
        boundary = 'draw=%s, line width=%scm' % (linecolor, linewidth)
    return r"\path [%s, %s] %s;" % (interior, boundary, points_str)

p = Plot()
points = [(0,2)] + \
    [(i/2.0, random.randrange(3*2, 5*2)/4.0) for i in range(1, 5)] + \
    [(i/2.0, random.randrange(0, 3*2)/4.0) for i in range(5, 10)]
p += polygon(points=points, background='blue!20')
p += polygon(points=[(x+4,y) for (x,y) in points], linewidth=0.02)
p += polygon(points=[(x+8,y) for (x,y) in points], linewidth=0.02,
            background='blue!20')
p += polygon(points=[(x+12,y) for (x,y) in points], linewidth=0.1,
            background='blue!20', linecolor='red')
print(p)
```



20 fillpolygon (fillpolygon.tex)

```

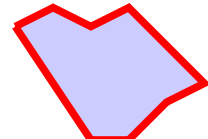
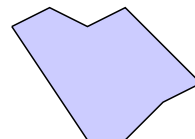
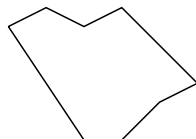
import random; random.seed()
from latextool_basic import *

def IFELSE(b, x, y):
    if b: return x
    else: return y

def polygon(points,
            background=None, # if background is None, no fill
            linewidth=None,
            linecolor=None,
            ):
    if points[-1] != points[0]: points.append(points[0])
    points_str = " -- ".join("(%s,%s)" % (x,y) for (x,y) in points)
    interior = IFELSE(background, "fill=%s" % background, '')
    boundary = ''
    if linewidth or linecolor:
        if not linecolor: linecolor='black'
        if not linewidth: linewidth=0.02
        boundary = 'draw=%s, line width=%scm' % (linecolor, linewidth)
    return r"\path [%s, %s] %s;" % (interior, boundary, points_str)

p = Plot()
points = [(0,2)] + \
    [(i/2.0, random.randrange(3*2, 5*2)/4.0) for i in range(1, 5)] + \
    [(i/2.0, random.randrange(0, 3*2)/4.0) for i in range(5, 1, -1)]
p += polygon(points=points, background='blue!20')
p += polygon(points=[(x+4,y) for (x,y) in points], linewidth=0.02)
p += polygon(points=[(x+8,y) for (x,y) in points], linewidth=0.02,
            background='blue!20')
p += polygon(points=[(x+12,y) for (x,y) in points], linewidth=0.1,
            background='blue!20', linecolor='red')
print(p)

```



21 circle (circle.tex)

21.1 boundary

```

from latextool_basic import *
from latexcircuit import *

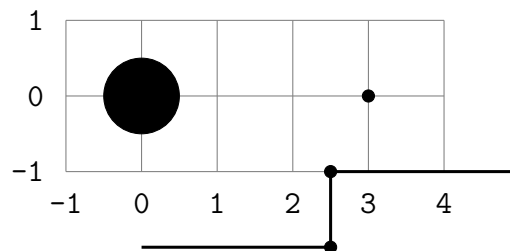
p = Plot()
p += Circle(x=0, y=0, r=0.5, background='black')
p += Circle(x=3, y=0, r=0.08, background='black', name=None)

p += str(OrthogonalPath([(0,-2), (5,-1)]))

p += POINT(x=0, y=-3)
p += '%s' % Circle(x=0, y=-5, r=0.08, background='black', name=None)

p += Grid()
print(p)

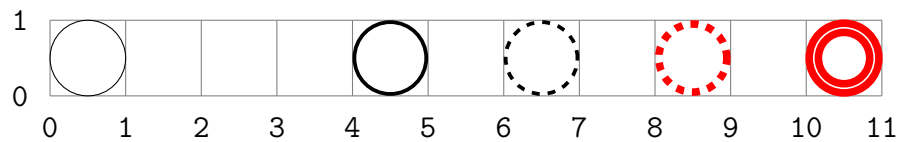
```



•

21.2 boundary

```
from latextool_basic import *
p = Plot()
p += Circle(x=0.5, y=0.5, r=0.5)
p += Circle(x=2.5, y=0.5, r=0.5, linewidth=0)
p += Circle(x=4.5, y=0.5, r=0.5, linewidth=0.05)
p += Circle(x=6.5, y=0.5, r=0.5, linewidth=0.05, linestyle='dashed')
p += Circle(x=8.5, y=0.5, r=0.5, linewidth=0.1, linestyle='dashed',
            linecolor='red')
p += Circle(x=10.5, y=0.5, r=0.5, linewidth=0.1, linestyle='double',
            linecolor='red')
p += Grid()
print(p)
```

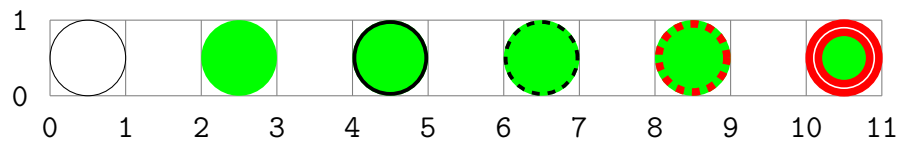


21.3 background

```

from latextool_basic import *
p = Plot()
p += Circle(x=0.5, y=0.5, r=0.5)
p += Circle(x=2.5, y=0.5, r=0.5, linewidth=0, background='green')
p += Circle(x=4.5, y=0.5, r=0.5, linewidth=0.05, background='green')
p += Circle(x=6.5, y=0.5, r=0.5, linewidth=0.05, linestyle='dashed',
            background='green')
p += Circle(x=8.5, y=0.5, r=0.5, linewidth=0.1, linestyle='dashed',
            linecolor='red', background='green')
p += Circle(x=10.5, y=0.5, r=0.5, linewidth=0.1, linestyle='double',
            linecolor='red', background='green')
p += Grid()
print(p)

```

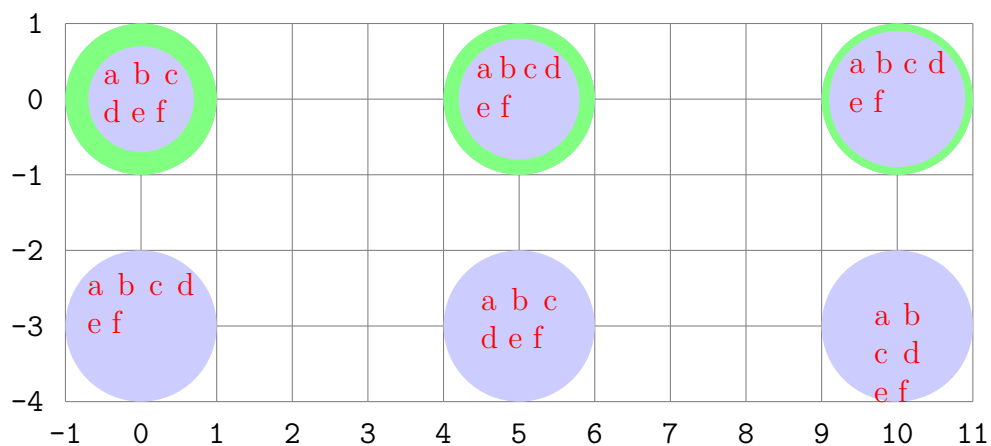


21.4 s: minipage text in circle

```

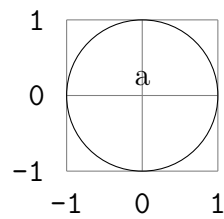
from latextool_basic import *
p = Plot()
s = 'a b c d e f'
p += Circle(x=0, y=0, r=1, linecolor='green!50!white', linewidth=0.3,
            background='blue!20!white', foreground='red', innersep=0, s=s)
p += Circle(x=5, y=0, r=1, linecolor='green!50!white', linewidth=0.2,
            background='blue!20!white', foreground='red', innersep=0, s=s)
p += Circle(x=10, y=0, r=1, linecolor='green!50!white', linewidth=0.1,
            background='blue!20!white', foreground='red', innersep=0, s=s)
p += Circle(x=0, y=-3, r=1, linecolor='green!50!white', linewidth=0,
            background='blue!20!white', foreground='red', innersep=0, s=s)
p += Circle(x=5, y=-3, r=1, linecolor='green!50!white', linewidth=0,
            background='blue!20!white', foreground='red',
            innersep=0.2, s=s)
p += Circle(x=10, y=-3, r=1, linecolor='green!50!white', linewidth=0,
            background='blue!20!white', foreground='red',
            innersep=0.4, s=s)
p += Grid()
print(p)

```



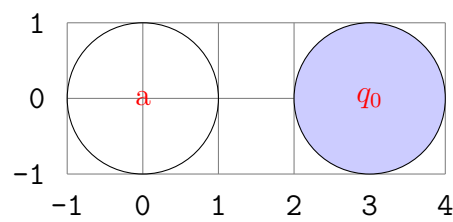
21.5 label anchor

```
from latextool_basic import *  
p = Plot()  
label = 'a'  
p += Circle(x=0, y=0, r=1, label=label, anchor='above')  
p += Grid()  
print(p)
```



21.6 foreground

```
from latextool_basic import *
p = Plot()
p += Circle(x=0, y=0, r=1, foreground='red', label='a')
p += Circle(x=3, y=0, r=1, background='blue!20!white',
            foreground='red', label='$q_0$')
p += Grid()
print(p)
```



21.7 Center and boundary

```
s='1 2 3 4 5 6 7 8 9 10 11'
b1 = 'blue!50!white'
b2 = 'red!50!white'

from latextool_basic import *
p = Plot()

c1 = Circle(x=0, y=0, r=1, background=b1, s=s)
c2 = Circle(center=c1.center(), r=0.5, background=b2)
p += c1; p += c2

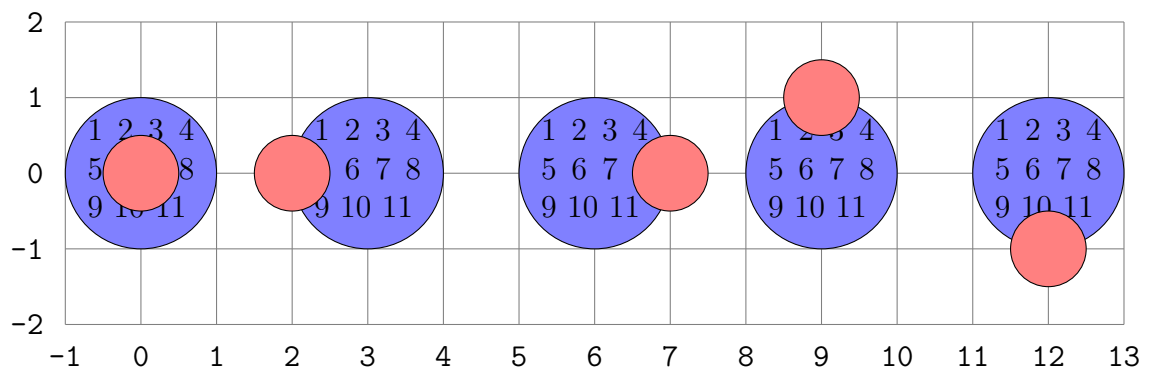
c1 = Circle(x=3, y=0, r=1, background=b1, s=s)
c2 = Circle(center=c1.left(), r=0.5, background=b2)
p += c1; p += c2

c1 = Circle(x=6, y=0, r=1, background=b1, s=s)
c2 = Circle(center=c1.right(), r=0.5, background=b2)
p += c1; p += c2

c1 = Circle(x=9, y=0, r=1, background=b1, s=s)
c2 = Circle(center=c1.top(), r=0.5, background=b2)
p += c1; p += c2

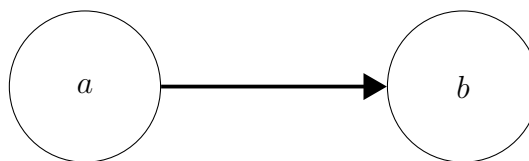
c1 = Circle(x=12, y=0, r=1, background=b1, s=s)
c2 = Circle(center=c1.bottom(), r=0.5, background=b2)
p += c1; p += c2

p += Grid()
print(p)
```



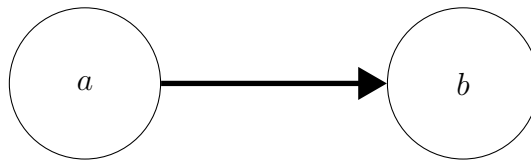
21.8 tikz name

```
from latextool_basic import *  
p = Plot()  
  
p += Circle(center=(0,0), r = 1, label=r'$a$', name='a')  
p += Circle(center=(5,0), r = 1, label=r'$b$', name='b')  
  
p += Line(names=['a', 'b'], linewidth=0.05, endstyle='>')  
  
print(p)
```



Test with and without name:

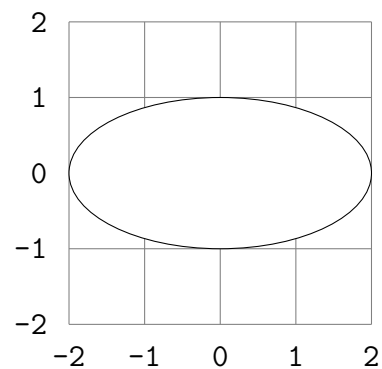
```
from latextool_basic import *  
p = Plot()  
  
p += Circle(center=(0,0), r = 1, label=r'$a$', name='a')  
p += Circle(center=(5,0), r = 1, label=r'$b$')  
  
p += r"\draw[->,line width=2] (a) -- (b);"  
  
print(p)
```



22 ellipse (ellipse.tex)

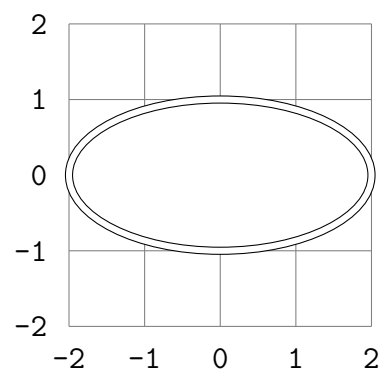
```
from latextool_basic import *
p = Plot()

p += Grid(-2,-2,2,2)
p += ellipse(-2,-1,2,1)
print(p)
```



```
from latextool_basic import *
p = Plot()

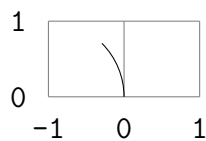
p += Grid(-2,-2,2,2)
p += ellipse(-2,-1,2,1,double=True)
print(p)
```



23 `arc` (`arc.tex`)

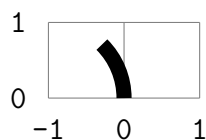
Draw part of a circle. The x, y is the starting point, NOT the center.

```
from latextool_basic import Plot, Grid, arc
p = Plot()
p += arc(x=0, y=0, r=1, angle0=0, angle1=45)
p += Grid(x0=-1, y0=0, x1=1, y1=1)
print(p)
```



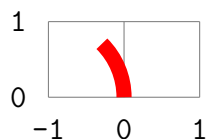
Linewidth:

```
from latextool_basic import Plot, Grid, arc
p = Plot()
p += arc(x=0, y=0, r=1, angle0=0, angle1=45, linewidth=0.2)
p += Grid(x0=-1, y0=0, x1=1, y1=1)
print(p)
```



Color:

```
from latextool_basic import Plot, Grid, arc
p = Plot()
p += arc(x=0, y=0, r=1, angle0=0, angle1=45,
         linewidth=0.2, linecolor='red')
p += Grid(x0=-1, y0=0, x1=1, y1=1)
print(p)
```

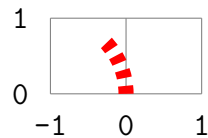


Linestyle:

```

from latextool_basic import Plot, Grid, arc
p = Plot()
p += arc(x=0, y=0, r=1, angle0=0, angle1=45,
         linewidth=0.2, linecolor='red', linestyle='dashed')
p += Grid(x0=-1, y0=0, x1=1, y1=1)
print(p)

```

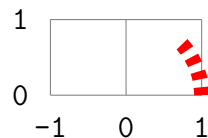


Change starting point of arc:

```

from latextool_basic import Plot, Grid, arc
p = Plot()
p += arc(x=1, y=0, r=1, angle0=0, angle1=45,
         linewidth=0.2, linecolor='red', linestyle='dashed')
p += Grid(x0=-1, y0=0, x1=1, y1=1)
print(p)

```



Arrow:

```

from latextool_basic import Plot, Grid, arc
p = Plot()
p += arc(x=1, y=0, r=1, angle0=0, angle1=45,
         linewidth=0.2, endstyle='>')
p += Grid(x0=-1, y0=0, x1=1, y1=1)
print(p)

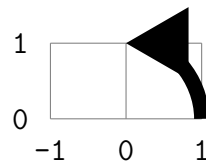
```



```

from latextool_basic import Plot, Grid, arc
p = Plot()
p += arc(x=1, y=0, r=1, angle0=0, angle1=90,
         linewidth=0.2, endstyle='>')
p += Grid(x0=-1, y0=0, x1=1, y1=1)
print(p)

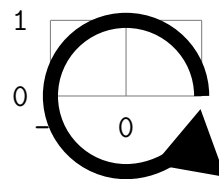
```



```

from latextool_basic import Plot, Grid, arc
p = Plot()
p += arc(x=1, y=0, r=1, angle0=0, angle1=350,
         linewidth=0.2, endstyle='>')
p += Grid(x0=-1, y0=0, x1=1, y1=1)
print(p)

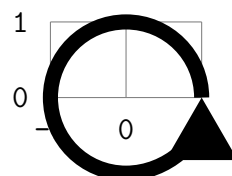
```



```

from latextool_basic import Plot, Grid, arc
p = Plot()
p += arc(x=1, y=0, r=1, angle0=0, angle1=360,
         linewidth=0.2, endstyle='>')
p += Grid(x0=-1, y0=0, x1=1, y1=1)
print(p)

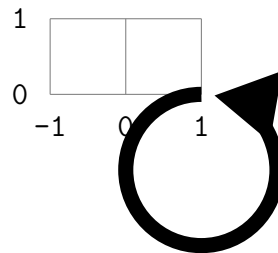
```



```

from latextool_basic import Plot, Grid, arc
p = Plot()
p += arc(x=1, y=0, r=1, angle0=90, angle1=360+80,
        linewidth=0.2, endstyle='>')
p += Grid(x0=-1, y0=0, x1=1, y1=1)
print(p)

```

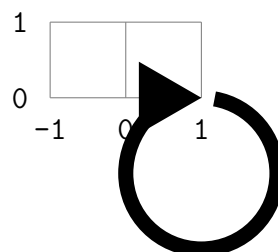


Start style

```

from latextool_basic import Plot, Grid, arc
p = Plot()
p += arc(x=1, y=0, r=1, angle0=90, angle1=360+80,
        linewidth=0.2, startstyle='>')
p += Grid(x0=-1, y0=0, x1=1, y1=1)
print(p)

```

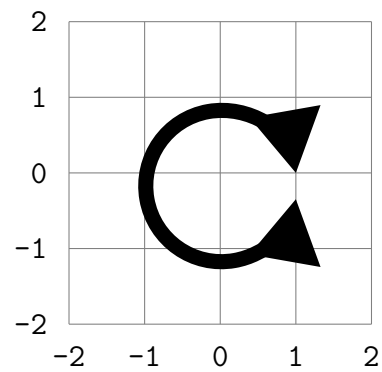


Start and end style

```

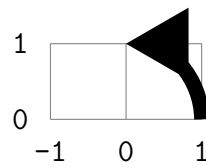
from latextool_basic import Plot, Grid, arc
p = Plot()
p += arc(x=1, y=0, r=1, angle0=10, angle1=350,
        linewidth=0.2, startstyle='>', endstyle='>')
p += Grid(x0=-2, y0=-2, x1=2, y1=2)
print(p)

```



With center:

```
from latextool_basic import Plot, Grid, arc
p = Plot()
p += arc(center=(0,0), r=1, angle0=0, angle1=90,
         linewidth=0.2, endstyle='>')
p += Grid(x0=-1, y0=0, x1=1, y1=1)
print(p)
```



24 Relative node placement (relative-node-placement.tex)

Temporary fix to relative node placement: use the `next_to` function.

<http://tex.stackexchange.com/questions/69439/how-can-i-achieve-relative-positioning>

```
from latextool_basic import *
p = Plot()
p += ellipse(1, 1, 4, 2, name='a', label='a')
b = str(ellipse(5, 2, 7, 3, name='b', label='b'))
p += b
print(p)
```



```
from latextool_basic import *
p = Plot()
p += ellipse(1, 1, 4, 2, name='a', label='a')
b = str(ellipse(5, 2, 7, 3, name='b', label='b'))
b = next_to(b, name='a', directions=('right', 0))
p += b
print(p)
```



```
from latextool_basic import *
p = Plot()
p += ellipse(1, 1, 4, 2, name='a', label='a')
b = str(ellipse(5, 2, 7, 3, name='b', label='b'))
b = next_to(b, name='a', directions=('left', 1))
p += b
print(p)
```



```
from latextool_basic import *
p = Plot()
p += ellipse(1, 1, 4, 2, name='a', label='a')

x = str(ellipse(5, 2, 7, 3, name='b', label='b'))
x = next_to(x, name='a', directions=('right', 1))
p += x

x = str(ellipse(5, 2, 7, 3, name='c', label='c'))
x = next_to(x, name='b', directions=('right', 1))
p += x

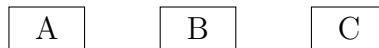
x = str(ellipse(5, 2, 7, 3, name='d', label='d'))
x = next_to(x, name='c', directions=('right', 1))
p += x

print(p)
```



Example that uses both =of and anchor:

```
from latextool_basic import *
p = Plot()
p += r'\node[minimum width=5cm, draw, rectangle] (basis) {};'
p += r'\node[above=of basis.north west, anchor=south west, draw, rectangle,
  minimum width=1cm] (a) {A};'
p += r'\node[above=of basis, draw, rectangle,minimum width=1cm] (b) {B};'
p += r'\node[above=of basis.north east, anchor=south east, draw, rectangle,
  minimum width=1cm] (c) {C};'
print(p)
```



Example that uses both =of and anchor and labeling a rect. Basically flushing left on two nodes.

```
from latextool_basic import *
p = Plot()
p += r'\node[minimum width=5cm, draw, rectangle] (basis) {};'
p += r'\node[above=0.2cm of basis.north west, anchor=south west, rectangle,
  minimum width=1cm, inner sep=0cm] (a) {This is the title};'
print(p)
```

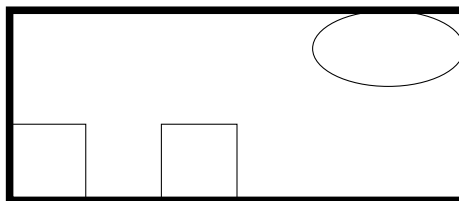
This is the title



25 Bounding nodes (boundingbox.tex)

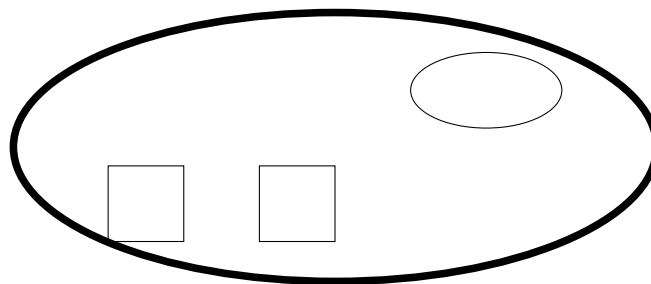
Bounding box (rectangle) for a group of tikz nodes:

```
from latextool_basic import *
p = Plot()
p += Rect(0,0,1,1,name='a')
p += Rect(2,0,3,1,name='b')
p += ellipse(center=(5,2), width=2, height=1, name='c')
p += r'\node [draw=black, fit=(a) (b) (c), line width=0.1cm, inner sep=0.0cm] {};'
print(p)
```



Bounding ellipse for a group of tikz nodes:

```
from latextool_basic import *
p = Plot()
p += Rect(0,0,1,1,name='a')
p += Rect(2,0,3,1,name='b')
p += ellipse(center=(5,2), width=2, height=1, name='c')
p += r'\node [ellipse, draw=black, fit=(a) (b) (c), line width=0.1cm, inner sep=0.0cm] {};'
print(p)
```

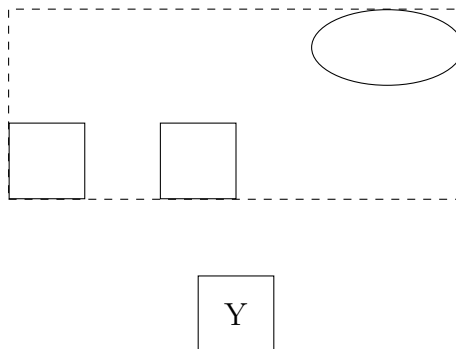


Put a node below a group of nodes:

```

from latextool_basic import *
p = Plot()
p += Rect(0,0,1,1,name='a')
p += Rect(2,0,3,1,name='b')
p += ellipse(center=(5,2), width=2, height=1, name='c')
p += r'\node [draw=black, fit=(a) (b) (c), inner sep=0.0cm, dashed] (X) {};
'
p += r'\node [draw=black, rectangle, minimum width=1cm, below=1cm of X, minimum height=1cm, distance=2cm] (Y) {Y};'
print(p)

```

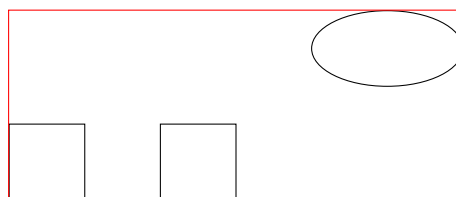


fit function:

```

from latextool_basic import *
p = Plot()
p += Rect(0,0,1,1,name='a')
p += Rect(2,0,3,1,name='b')
p += ellipse(center=(5,2), width=2, height=1, name='c')
p += fit(name='X', names=['a', 'b', 'c'], linecolor='red')
print(p)

```

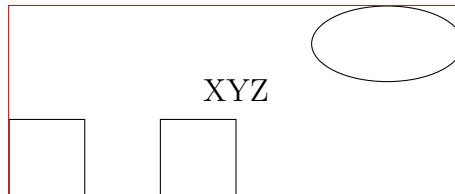


fit function with label:

```

from latextool_basic import *
p = Plot()
p += Rect(0,0,1,1,name='a')
p += Rect(2,0,3,1,name='b')
p += ellipse(center=(5,2), width=2, height=1, name='c')
p += fit(name='X', names=['a', 'b', 'c'], linecolor='red', label='XYZ')
print(p)

```

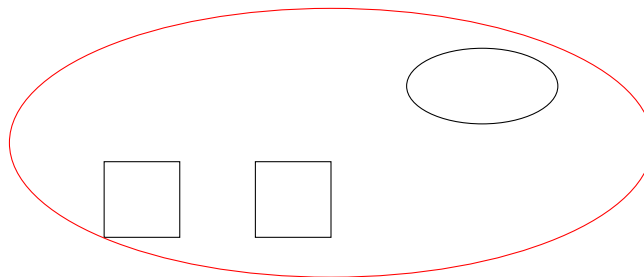


fit function with shape:

```

from latextool_basic import *
p = Plot()
p += Rect(0,0,1,1,name='a')
p += Rect(2,0,3,1,name='b')
p += ellipse(center=(5,2), width=2, height=1, name='c')
p += fit(name='X', names=['a', 'b', 'c'], shape='ellipse', linecolor='red')
print(p)

```

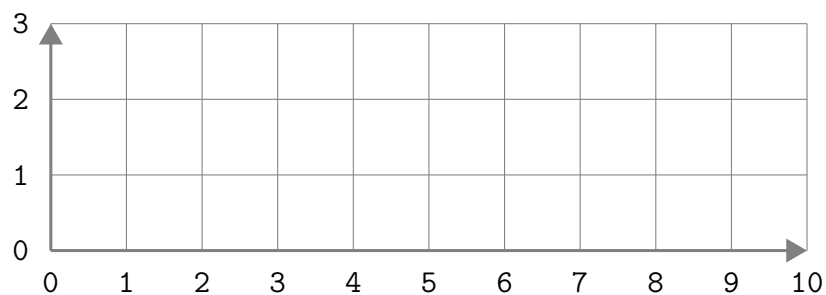


26 Axes (axes.tex)

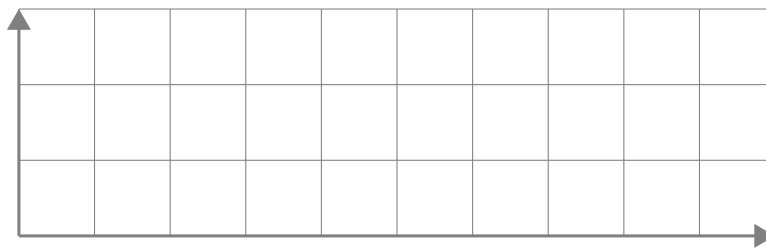
```
from latextool_basic import *
p = Plot()
axes(p, x0=0, y0=0, x1=10, y1=3)
print(p)
```



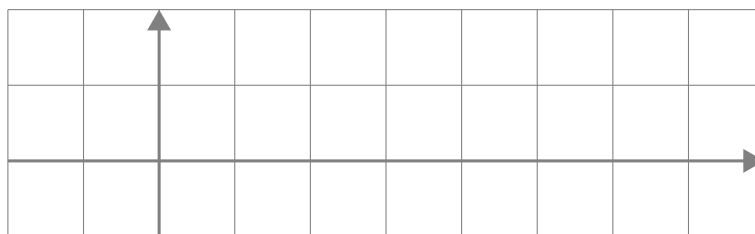
```
from latextool_basic import *
p = Plot()
p += Grid(x0=0, y0=0, x1=10, y1=3)
axes(p, x0=0, y0=0, x1=10, y1=3)
print(p)
```



```
from latextool_basic import *
p = Plot()
p += Grid(x0=0, y0=0, x1=10, y1=3, label_axes=False)
axes(p, x0=0, y0=0, x1=10, y1=3)
print(p)
```



```
from latextool_basic import *  
p = Plot()  
p += Grid(x0=-2, y0=-1, x1=8, y1=2, label_axes=False)  
axes(p, x0=-2, y0=-1, x1=8, y1=2)  
print(p)
```



NEW VERSION

```
from latextool_basic import *
p = Plot()
xaxis = XAxis(x0=0, y0=0,
              start=5.0, end=20.0,
              xscale=1.0,
              start_tick=0.2, end_tick=12.25,
              tick_gap=0.25,
              tick_len=0.05,
              start_label=6.0, end_label=18.0,
              label_gap=1,
              label_anchor_gap=0,
              label_fontsize=r'\small',
              arrowhead=None)

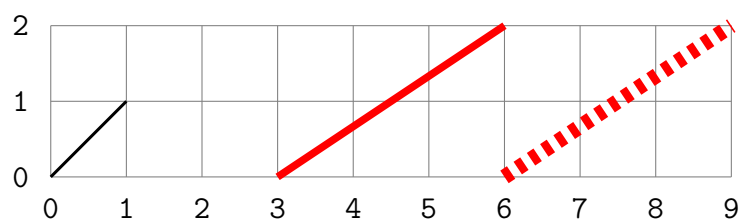
p += Circle(x=0, y=0, r=0.1, background='red')
p += Circle(x=1, y=0, r=0.1, background='blue')
p += str(xaxis)
print(p)
```



27 Lines (chap-line.tex)

27.1 linecolor and linewidth (line.tex)

```
from latextool_basic import *
p = Plot()
p += Line(x0=0, y0=0, x1=1, y1=1)
p += Line(x0=3, y0=0, x1=6, y1=2, linecolor='red', linewidth=0.1)
p += Line(x0=6, y0=0, x1=9, y1=2, linecolor='red', linewidth=0.2,
          linestyle='dashed')
p += Grid()
print(p)
```

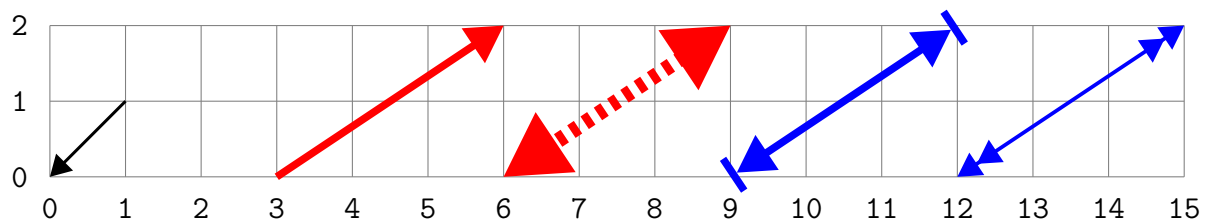


27.2 startstyle and endstyle

```

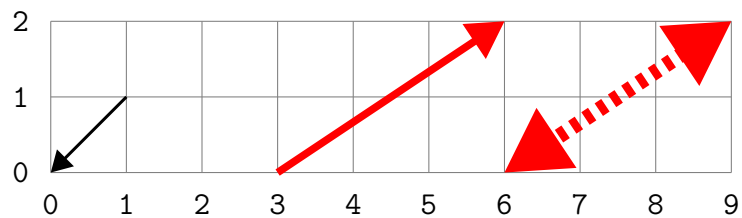
from latextool_basic import *
p = Plot()
p += Line(x0=0, y0=0, x1=1, y1=1, startstyle='>')
p += Line(x0=3, y0=0, x1=6, y1=2, linecolor='red', linewidth=0.1,
          endstyle='>')
p += Line(x0=6, y0=0, x1=9, y1=2, linecolor='red', linewidth=0.2,
          linestyle='dashed',
          startstyle='>', endstyle='>')
p += Line(x0=9, y0=0, x1=12, y1=2, linecolor='blue', linewidth=0.1,
          startstyle='>|', endstyle='>|')
p += Line(x0=12, y0=0, x1=15, y1=2, linecolor='blue', linewidth=0.05,
          startstyle='>>', endstyle='>>')
p += Grid()
print(p)

```



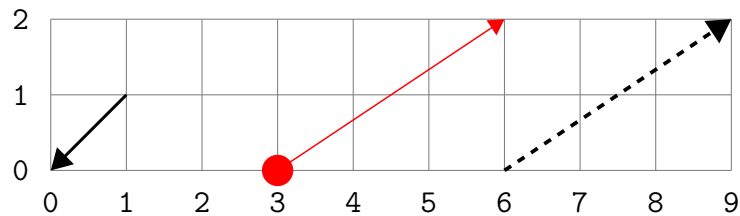
arrowstyle=triangle.

```
from latextool_basic import *
p = Plot()
p += Line(x0=0, y0=0, x1=1, y1=1, startstyle='->', arrowstyle='triangle')
p += Line(x0=3, y0=0, x1=6, y1=2, linecolor='red', linewidth=0.1,
          endstyle='->', arrowstyle='triangle')
p += Line(x0=6, y0=0, x1=9, y1=2, linecolor='red', linewidth=0.2,
          linestyle='dashed',
          startstyle='->', endstyle='->',
          arrowstyle='triangle')
p += Grid()
print(p)
```



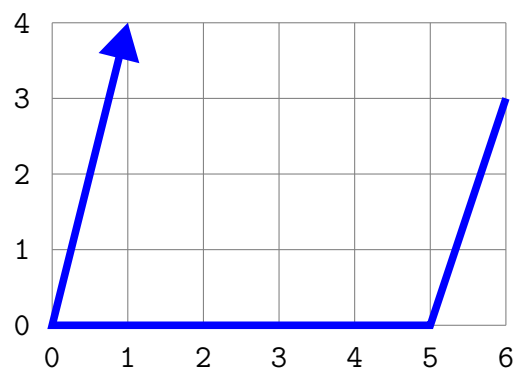
startstyle='dot'.

```
from latextool_basic import *
p = Plot()
p += Line(x0=0, y0=0, x1=1, y1=1, startstyle='->',
          arrowstyle='triangle', endstyle='dot')
p += Line(x0=3, y0=0, x1=6, y1=2, linecolor='red', linewidth=0.02,
          endstyle='->', arrowstyle='triangle',
          startstyle='dot', r=0.2)
p += Line(x0=6, y0=0, x1=9, y1=2, linecolor='black', linewidth=0.05,
          linestyle='dashed', startstyle='dot',
          endstyle='->', arrowstyle='triangle')
p += Grid()
print(p)
```



27.3 points parameter

```
from latextool_basic import *
p = Plot()
p += Line(points=[(1,4), (0,0), (5,0), (6,3)],
          linecolor='blue', linewidth=0.1,
          startstyle='->', arrowstyle='triangle', endstyle='dot')
p += Grid()
print(p)
```



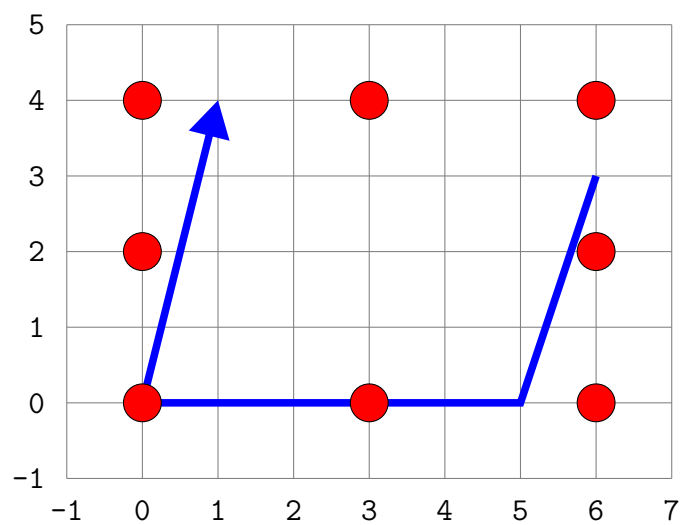
27.4 boundary points

```

from latextool_basic import *
p = Plot()

aline = Line(points=[(1,4), (0,0), (5,0), (6,3)],
              linecolor='blue', linewidth=0.1,
              startstyle='->', arrowstyle='triangle', endstyle='dot')
p += aline
p += Circle(center=aline.top(), r=0.25, background='red')
p += Circle(center=aline.bottom(), r=0.25, background='red')
p += Circle(center=aline.left(), r=0.25, background='red')
p += Circle(center=aline.right(), r=0.25, background='red')
p += Circle(center=aline.topleft(), r=0.25, background='red')
p += Circle(center=aline.topright(), r=0.25, background='red')
p += Circle(center=aline.bottomleft(), r=0.25, background='red')
p += Circle(center=aline.bottomright(), r=0.25, background='red')
p += Grid()
print(p)

```

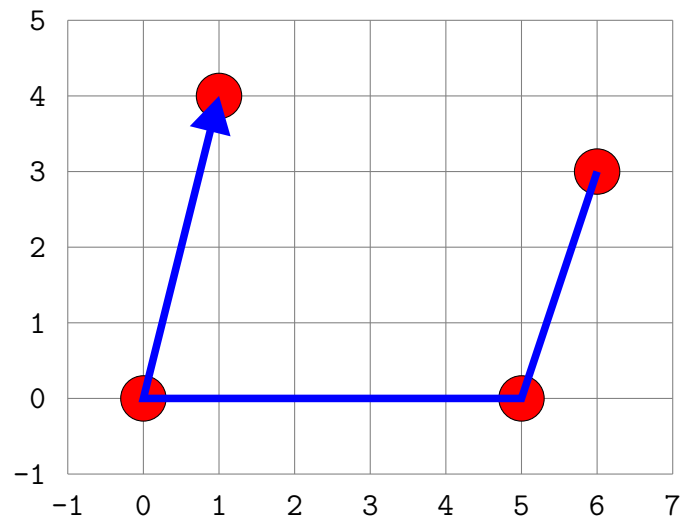


Test points

```
from latextool_basic import *
p = Plot()

aline = Line(points=[(1,4), (0,0), (5,0), (6,3)],
              linecolor='blue', linewidth=0.1,
              startstyle='->', arrowstyle='triangle', endstyle='dot')
for point in aline.points:
    p += Circle(center=point, r=0.3, background='red')

p += aline
p += Grid()
print(p)
```

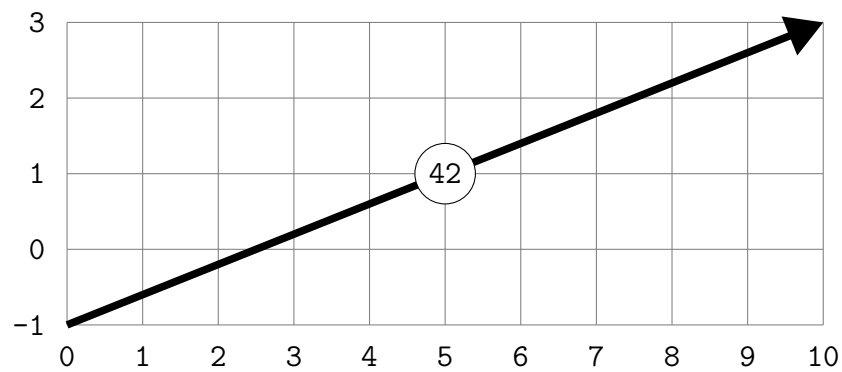


27.5 Midpoints

Test midpoint (can be useful for weighted graphs and network flows.

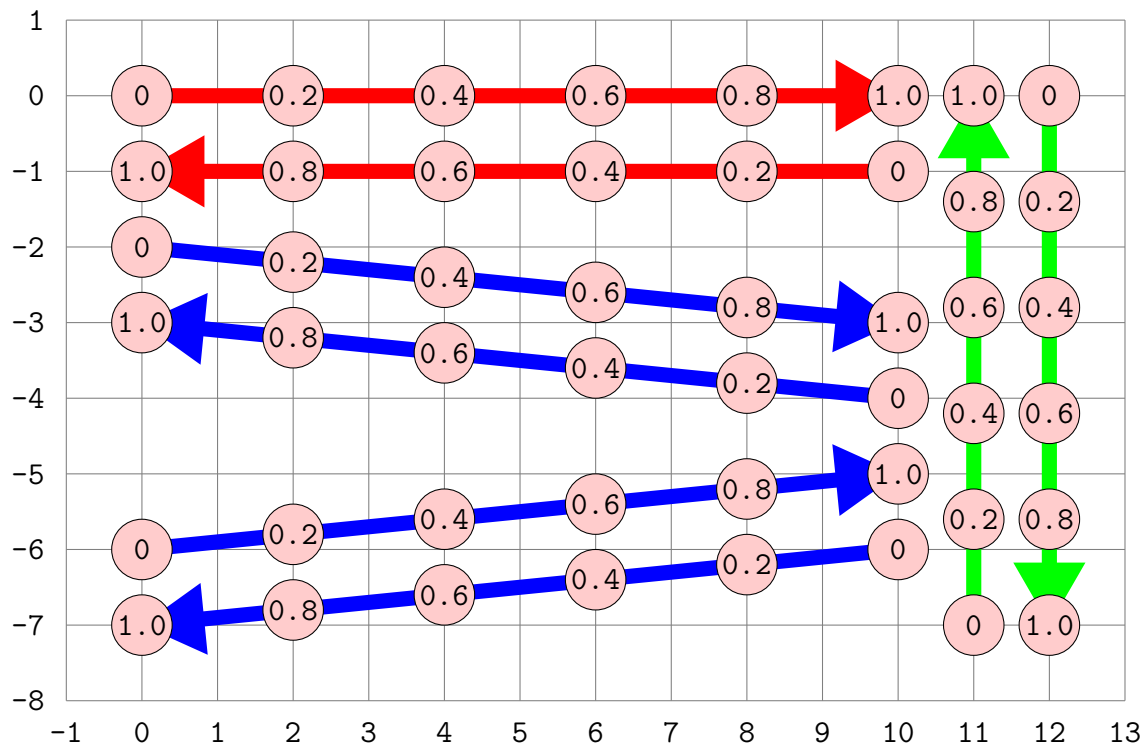
```
from latextool_basic import *
p = Plot()
points = [(0,-1), (10,3)]
aline = Line(points=points, endstyle='>', linewidth=0.1)
p += aline

p += Circle(center=aline.midpoint(), r=0.4,
            background='white', label=r'\texttt{%s}' % 42)
p += Grid()
print(p)
```



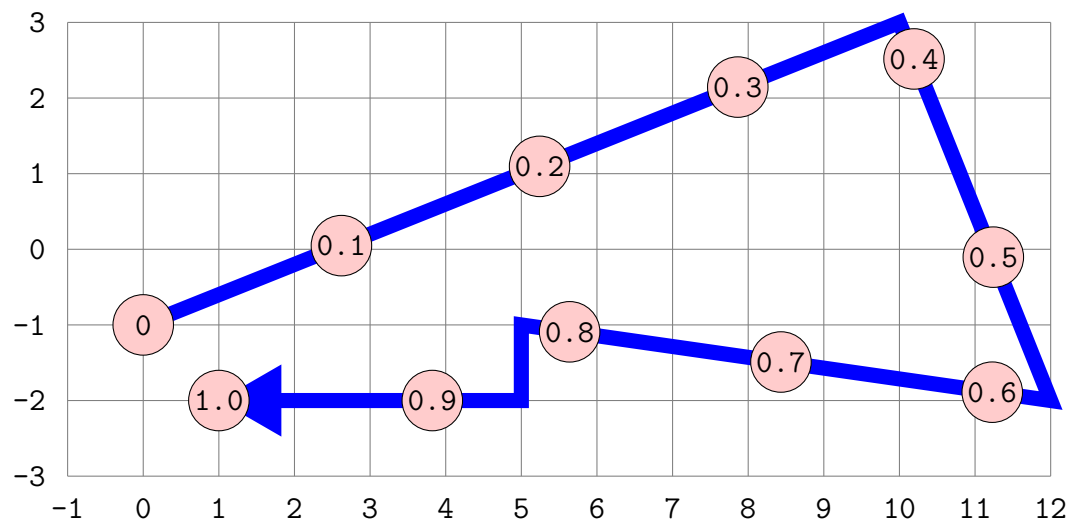
Test midpoint with different ratios.

```
from latextool_basic import *
p = Plot()
def linewidthpoints(p, points, color1, color2):
    aline = Line(points=points, linecolor=color1, linewidth=0.2,
                 startstyle='dot', arrowstyle='triangle', endstyle='->')
    p += aline
    for i in [0, 0.2, 0.4, 0.6, 0.8, 1.0]:
        p += Circle(center=aline.midpoint(ratio=i), r=0.4,
                    background=color2, label=r'\texttt{%s}' % i)
linewidthpoints(p, [(0, 0), (10, 0)], 'red', 'red!20')
linewidthpoints(p, [(10, -1), (0, -1)], 'red', 'red!20')
linewidthpoints(p, [(0, -2), (10, -3)], 'blue', 'red!20')
linewidthpoints(p, [(10, -4), (0, -3)], 'blue', 'red!20')
linewidthpoints(p, [(0, -6), (10, -5)], 'blue', 'red!20')
linewidthpoints(p, [(10, -6), (0, -7)], 'blue', 'red!20')
linewidthpoints(p, [(11, -7), (11, 0)], 'green', 'red!20')
linewidthpoints(p, [(12, 0), (12, -7)], 'green', 'red!20')
p += Grid()
print(p)
```



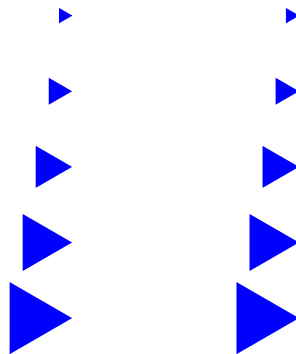
Test midpoint for 1 segment.

```
from latextool_basic import *
p = Plot()
points = [(0,-1), (10,3), (12,-2), (5,-1), (5,-2), (1,-2)]
aline = Line(points=points, linecolor='blue', linewidth=0.2,
             startstyle='dot', arrowstyle='triangle', endstyle='->')
p += aline
for i in [0, 0.1, 0.2, 0.3, 0.4, 0.5, 0.6, 0.7, 0.8, 0.9, 1.0]:
    p += Circle(center=aline.midpoint(ratio=i), r=0.4,
               background='red!20', label=r'\texttt{%s}' % i)
p += Grid()
print(p)
```



Test short line with arrow tip (basically to test arrow tip).

```
from latextool_basic import *
p = Plot()
for i, x in enumerate([0.2, 0.15, 0.1, 0.05, 0.01]):
    p += Line(points=[(1-x, i), (1, i)], linecolor='blue', linewidth=x,
               endstyle='->')
    p += Line(points=[(4-x, i), (4, i)], linecolor='blue', linewidth=x,
               endstyle='->', arrowstyle='triangle')
#p += Grid()
print(p)
```



For arrows with line width x , it seems enough to have a length of x .

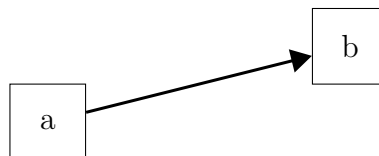
27.6 tikz names

line from tikz name to tikz name:

```
from latextool_basic import *
p = Plot()
p += Rect(x0=1, y0=0, x1=2, y1=1, label='a', name='a')
p += Rect(x0=5, y0=1, x1=6, y1=2, label='b', name='b')

p += Line(names=['a', 'b'], endstyle='>')

print(p)
```



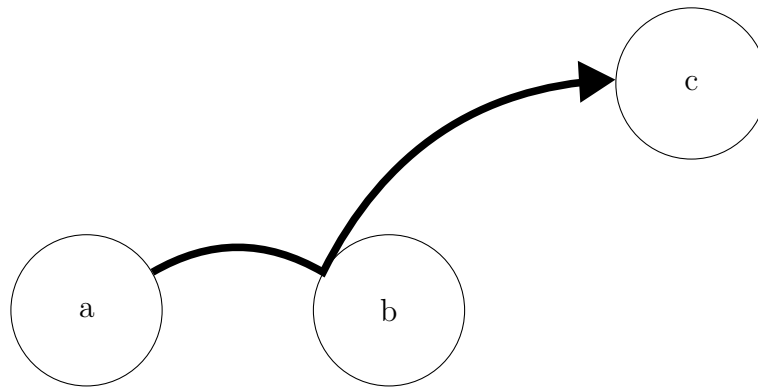
27.7 bend

```
from latextool_basic import *
p = Plot()

p += Circle(x=0, y=0, r=1, name='a', label='a')
p += Circle(x=4, y=0, r=1, name='b', label='b')
p += Circle(x=8, y=3, r=1, name='c', label='c')

p += Line(names=['a','b','c'], bend_left=30, endstyle='>', linewidth=0.1)

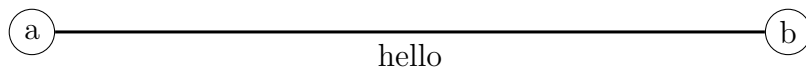
print(p)
```



27.8 label and anchor

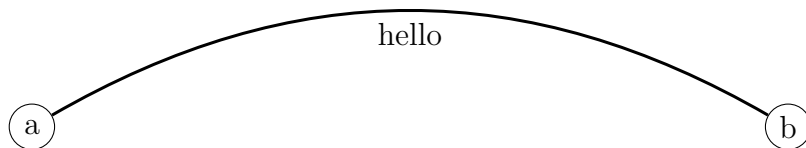
anchor = below

```
from latextool_basic import *
p = Plot()
p += Circle(x=0, y=0, r=0.3, name='a', label='a')
p += Circle(x=10, y=0, r=0.3, name='b', label='b')
p += Line(names=['a', 'b'], anchor='below', label='hello')
print(p)
```



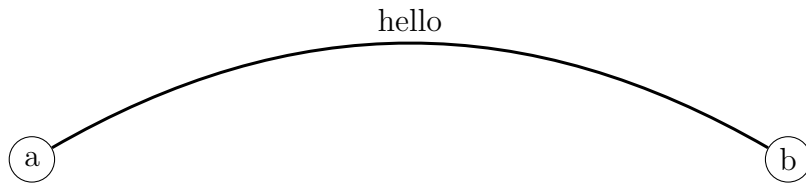
anchor = below with bend

```
from latextool_basic import *
p = Plot()
p += Circle(x=0, y=0, r=0.3, name='a', label='a')
p += Circle(x=10, y=0, r=0.3, name='b', label='b')
p += Line(names=['a', 'b'], bend_left=30, anchor='below', label='hello')
print(p)
```



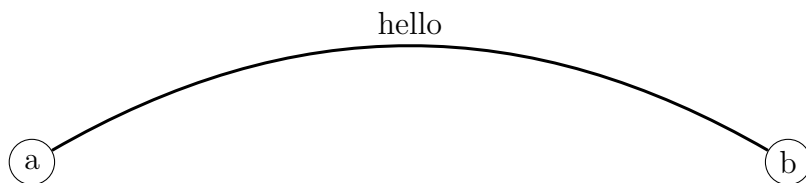
anchor = above

```
from latextool_basic import *
p = Plot()
p += Circle(x=0, y=0, r=0.3, name='a', label='a')
p += Circle(x=10, y=0, r=0.3, name='b', label='b')
p += Line(names=['a', 'b'], bend_left=30, anchor='above', label='hello')
print(p)
```



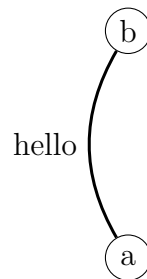
anchor = above with bend

```
from latextool_basic import *
p = Plot()
p += Circle(x=0, y=0, r=0.3, name='a', label='a')
p += Circle(x=10, y=0, r=0.3, name='b', label='b')
p += Line(names=['a', 'b'], bend_left=30, anchor='above', label='hello')
print(p)
```



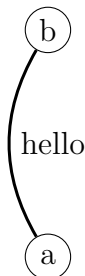
anchor = left with bend

```
from latextool_basic import *
p = Plot()
p += Circle(x=0, y=0, r=0.3, name='a', label='a')
p += Circle(x=0, y=3, r=0.3, name='b', label='b')
p += Line(names=['a', 'b'], bend_left=30, anchor='left', label='hello')
print(p)
```



anchor = right with bend

```
from latextool_basic import *
p = Plot()
p += Circle(x=0, y=0, r=0.3, name='a', label='a')
p += Circle(x=0, y=3, r=0.3, name='b', label='b')
p += Line(names=['a', 'b'], bend_left=30, anchor='right', label='hello')
print(p)
```

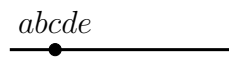


27.9 Using midpoint to place labels

```
from latextool_basic import *
p = Plot()
aline = Line(points=[(0,0), (3,0)])
p += aline
x,y = aline.midpoint(ratio=0.2)

from latexcircuit import *
p += Circle(x=x, y=y, r=0.05, background='red')
X = POINT(x=x, y=y, label='$abcde$', anchor='south')
p += str(X)

print(p)
```

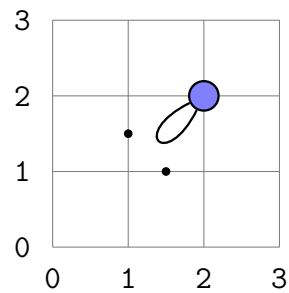


27.10 Two control points

```
from latextool_basic import *
p = Plot()
L = 0.03
C = 'blue!50'

p += Circle(x=2, y=2, r=0.2, background=C, name='a', linewidth=L)
p += Circle(x=1, y=1.5, r=0.05, background='black')
p += Circle(x=1.5, y=1, r=0.05, background='black')

p += Line(names=['a','a'], controls=[(1,1.5),(1.5,1)], linewidth=L)
p += Grid(0, 0, 3, 3)
print(p)
```



27.11 One control point

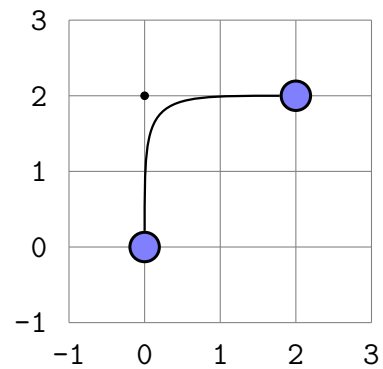
```
from latextool_basic import *
p = Plot()
C = 'blue!50'

p += Circle(x=2, y=2, r=0.2, background=C, name='22', linewidth=L)
p += Circle(x=0, y=0, r=0.2, background=C, name='00', linewidth=L)

p += Circle(x=0, y=2, r=0.05, background='black')

p += Line(names=['22','00'], controls=[(0,2)], linewidth=0.03)
p += Grid(x0=-1,y0=-1,x1=3,y1=3)

print(p)
```



1 control point and label

```

from latextool_basic import *
p = Plot()
L = 0.03
C = 'blue!50'

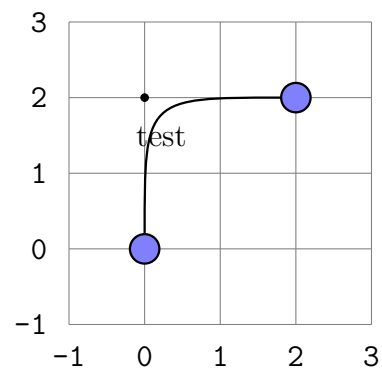
p += Circle(x=2, y=2, r=0.2, background=C, name='22', linewidth=L)
p += Circle(x=0, y=0, r=0.2, background=C, name='00', linewidth=L)

p += Circle(x=0, y=2, r=0.05, background='black')

p += Line(names=['22','00'], controls=[(0,2)], linewidth=L, label='test', a
nchor='below')

p += Grid(x0=-1,y0=-1,x1=3,y1=3)
print(p)

```

**27.12 loop**

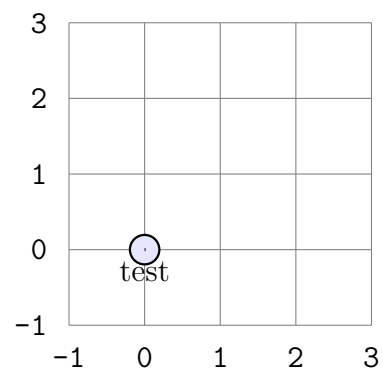
Example uses Graph.

xxx

```
from latextool_basic import *
p = Plot()
L = 0.03
C = 'blue!50'

p += Graph.node(x=0, y=0, name='a')
p += Line(names=['a','a'], label='test', anchor='below', loop='loop above')

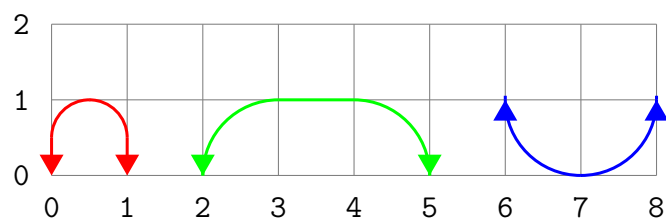
p += Grid(x0=-1,y0=-1,x1=3,y1=3)
print(p)
```



27.13 bend (bend.tex)

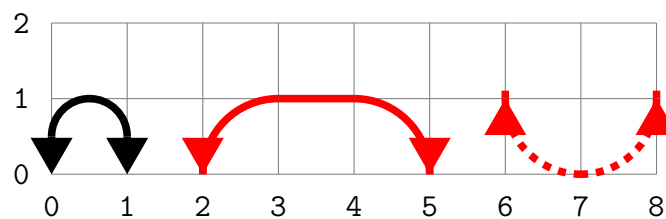
Basically used for drawing swaps in array sorting.

```
from latextool_basic import Plot, Grid, bend
p = Plot()
p += bend([0,0], [1, 0], dy=1, linecolor='red')
p += bend([2,0], [5, 0], dy=1, linecolor='green')
p += bend([6,1], [8, 1], dy=-1, linecolor='blue')
p += Grid(x0=0, y0=0, x1=8, y1=2)
print(p)
```



You can specify linewidth, linecolor, linestyle:

```
from latextool_basic import Plot, Grid, bend
p = Plot()
p += bend([0,0], [1, 0], dy=1, linewidth=0.1)
p += bend([2,0], [5, 0], dy=1, linewidth=0.1, linecolor='red')
p += bend([6,1], [8, 1], dy=-1, linewidth=0.1, linecolor='red',
          linestyle='dashed')
p += Grid(x0=0, y0=0, x1=8, y1=2)
print(p)
```

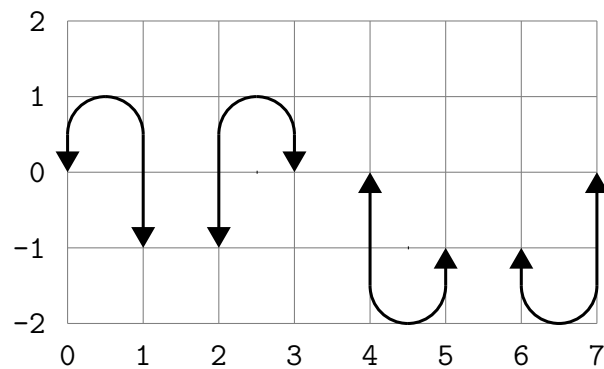


The y values can be different:

```

from latextool_basic import Plot, Grid, bend
p = Plot()
p += bend([0,0], [1, -1], dy=1)
p += bend([2,-1], [3, 0], dy=1)
p += bend([4,0], [5, -1], dy=-1)
p += bend([6,-1], [7, 0], dy=-1)
p += Grid(x0=0, y0=-2, x1=7, y1=2)
print(p)

```

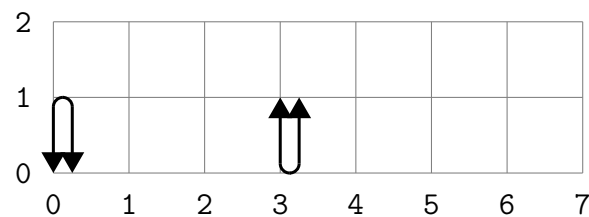


default radius (dy too large):

```

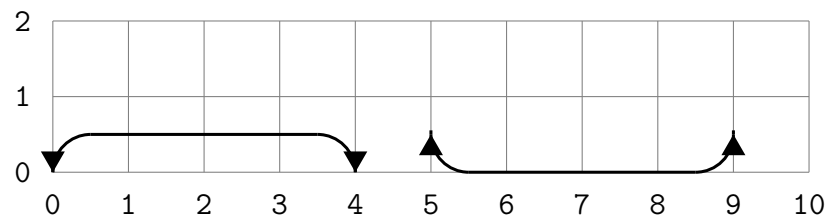
from latextool_basic import Plot, Grid, bend
p = Plot()
s0 = bend([0,0], [0.25,0], dy=1)
s1 = bend([3,1], [3.25,1], dy=-1)
p.add(s0)
p.add(s1)
p += Grid(x0=0, y0=0, x1=7, y1=2)
print(p)

```



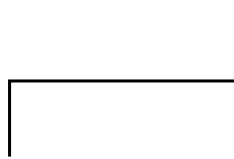
default radius (dx too large):


```
from latextool_basic import Plot, Grid, bend
p = Plot()
s0 = bend([0,0], [4,0], dy=0.5)
s1 = bend([5,0.5], [9,0.5], dy=-0.5)
p.add(s0)
p.add(s1)
p += Grid(x0=0, y0=0, x1=10, y1=2)
print(p)
```



27.14 Orthogonal lines; organization chart (orthogonal-line.tex)

```
from latextool_basic import *
p = Plot()
points = get_points(0, 0, 3, 2, 'vbroom')
p += Line(points = points)
print(p)
```



With adjustment:

```
from latextool_basic import *
p = Plot()

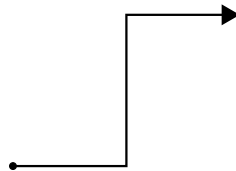
x = 0
points = get_points(x, 0, x+3, 2, 'vbroom', delta=0.5)
p += Line(points = points)

x = 5
points = get_points(x, 0, x+3, 2, 'vbroom', delta=-0.5)
p += Line(points = points)

print(p)
```



```
from latextool_basic import *
p = Plot()
points = get_points(0, 0, 3, 2, 'hbroom')
p += Pointer(points = points)
print(p)
```



With adjustment:

```
from latextool_basic import *
p = Plot()

x = 0
points = get_points(x, 0, x+3, 2, 'hbroom', delta=-0.5)
p += Pointer(points = points)

x = 5
points = get_points(x, 0, x+3, 2, 'hbroom', delta=+0.5)
p += Pointer(points = points)

print(p)
```



```

from latextool_basic import *
p = Plot()
d = positions(r'''
    A

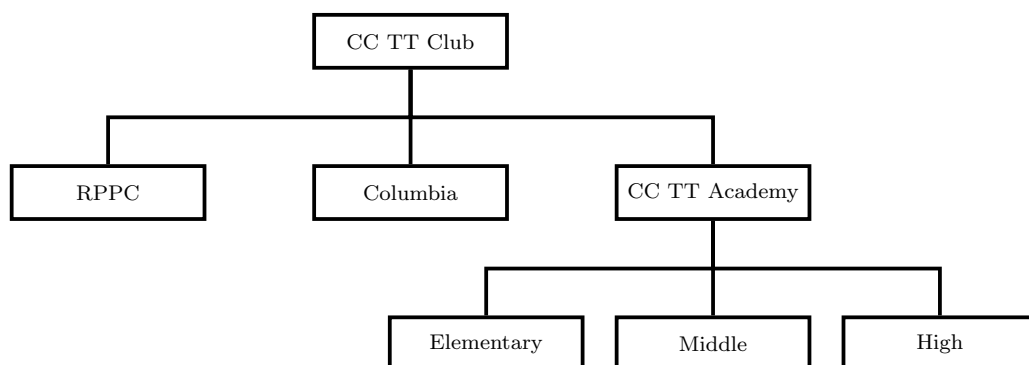
B   C   D

    E   F   G
''')
label = {'A':'CC TT Club',      'B':'RPPC',      'C':'Columbia',
         'D':'CC TT Academy', 'E':'Elementary', 'F':'Middle', 'G':'High'}
WIDTH = 2.6; HEIGHT = 0.5
rect = {}
for lab, pos in d.items():
    x, y = pos
    rect[lab] = Rect(x0=x-WIDTH/2, y0=y-HEIGHT/2,
                    x1=x+WIDTH/2, y1=y+HEIGHT, label=r'\scriptsize{%s}' % la
bel[lab],
                    linewidth=0.05)
    p += rect[lab]

for lab0, lab1s in [(('A', ('B','C','D'))),
                   (('D', ('E','F','G'))),
                   []]:
    x0,y0 = rect[lab0].bottom()
    for lab1 in lab1s:
        x1,y1 = rect[lab1].top()
        points = get_points(x0,y0,x1,y1,'vbroom')
        p += Line(points=points, linewidth=0.05)

print(p)

```

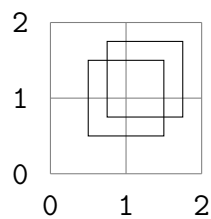


28 Rect class (rect.tex)

28.1 No background

TEST: Background is '', i.e., totally transparent.

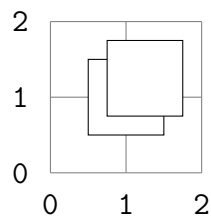
```
from latextool_basic import *
p = Plot()
p += Rect(x0=0.5, y0=0.5, x1=1.5, y1=1.5)
p += Rect(x0=0.75, y0=0.75, x1=1.75, y1=1.75)
p += Grid()
print(p)
```



28.2 Background

TEST: Background is white

```
from latextool_basic import *
p = Plot()
p += Rect(x0=0.5, y0=0.5, x1=1.5, y1=1.5, background='white')
p += Rect(x0=0.75, y0=0.75, x1=1.75, y1=1.75, background='white')
p += Grid()
print(p)
```

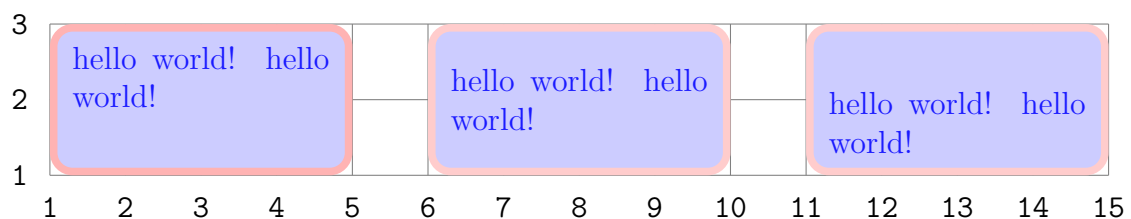


28.3 border, background, foreground

```

from latextool_basic import *
p = Plot()
p += Rect(x0=1, y0=1, x1=5, y1=3,
          radius=0.25, innersep=0.25,
          linecolor='red!30!white', linewidth=0.1,
          background='blue!20!white!', foreground='blue!90!white',
          s='hello world! hello world!', align='t')
p += Rect(x0=6, y0=1, x1=10, y1=3,
          radius=0.25, innersep=0.25,
          linecolor='red!20!white', linewidth=0.1,
          background='blue!20!white!', foreground='blue!90!white',
          s='hello world! hello world!', align='c')
p += Rect(x0=11, y0=1, x1=15, y1=3,
          radius=0.25, innersep=0.25,
          linecolor='red!20!white', linewidth=0.1,
          background='blue!20!white!', foreground='blue!90!white',
          s='hello world! hello world!', align='b')
p += Grid()
print(p)

```



28.4 border, background, foreground

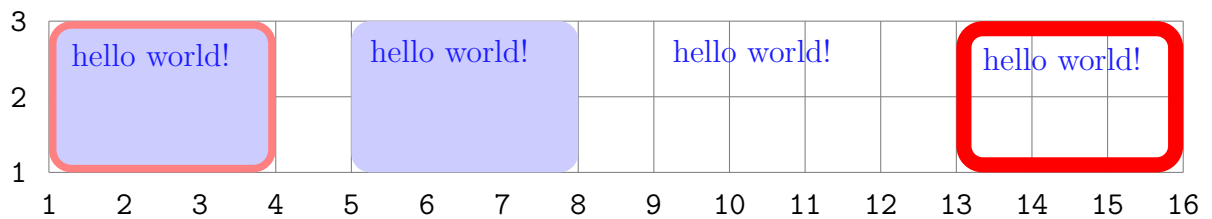
```

from latextool_basic import *
p = Plot(radius=0.25, innersep=0.25,
          linecolor='red!50!white', linewidth=0.1,
          background='blue!20!white', foreground='blue!90!white',
          align='t')
p += RectAdaptor(env=p.env, x0= 1, y0=1, x1= 4, y1=3, s='hello world!')
p.env['linewidth'] = 0
p += RectAdaptor(env=p.env, x0= 5, y0=1, x1= 8, y1=3, s='hello world!')
p.env['background'] = ''
p += RectAdaptor(env=p.env, x0= 9, y0=1, x1= 12, y1=3, s='hello world!')

p.env['background'] = ''
p.env['linecolor'] = 'red'
p.env['linewidth'] = 0.2
p += RectAdaptor(env=p.env, x0= 13, y0=1, x1= 16, y1=3, s='hello world!')

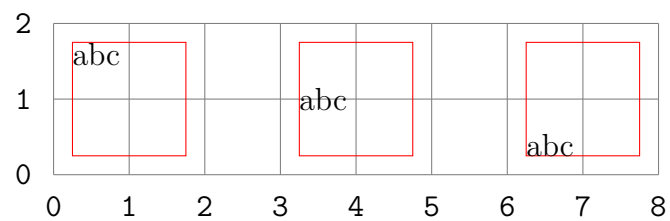
p += Grid()
print(p)

```



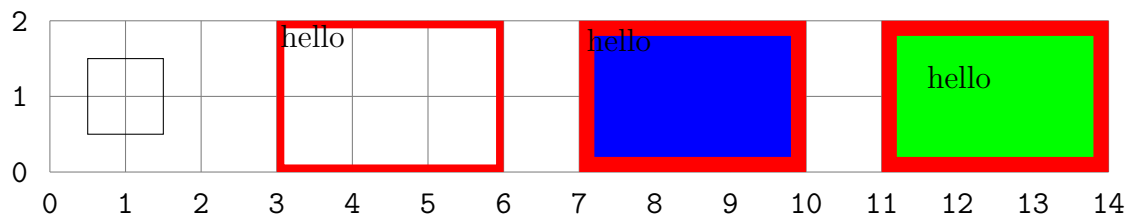
28.5 align

```
from latextool_basic import *
p = Plot()
p += Rect(x0=0.25, y0=0.25, x1=1.75, y1=1.75, s='abc', align='t',
          linecolor='red', foreground='black')
p += Rect(x0=3.25, y0=0.25, x1=4.75, y1=1.75, s='abc', align='c',
          linecolor='red')
p += Rect(x0=6.25, y0=0.25, x1=7.75, y1=1.75, s='abc', align='b',
          linecolor='red')
p += Grid()
print(p)
```



28.6 innersep

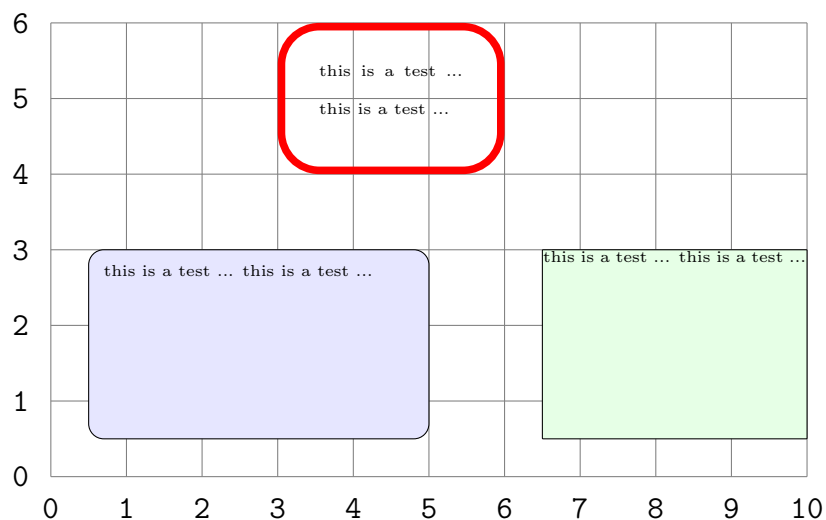
```
from latextool_basic import *
p = Plot()
p += Rect(x0=0.5, y0=0.5, x1=1.5, y1=1.5)
p += Rect(x0=3, y0=0, x1=6, y1=2, linecolor='red', linewidth=0.1, s='hello'
)
p += Rect(x0=7, y0=0, x1=10, y1=2, linecolor='red', background='blue',
          linewidth=0.2, s='hello')
p += Rect(x0=11, y0=0, x1=14, y1=2, linecolor='red', linewidth=0.2,
          background='green',
          innersep=0.5, s='hello')
p += Grid()
print(p)
```



28.7 rounded corners

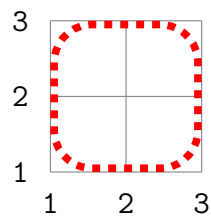
(radius of corner is set to innersep by default):

```
from latextool_basic import *
p = Plot()
p += Rect(x0=3, y0=4, x1=6, y1=6, linecolor='red', linewidth=0.1,
          innersep=0.5, radius=0.5,
          s=r'\tiny this is a test ... this is a test ...')
p += Rect(x0=0.5, y0=0.5, x1=5, y1=3,
          background='blue!10!white',
          innersep=0.2, radius=0.2,
          s=r'\tiny this is a test ... this is a test ...')
p += Rect(x0=6.5, y0=0.5, x1=10, y1=3,
          background='green!10!white',
          innersep=0.01, radius=0.01,
          s=r'\tiny this is a test ... this is a test ...')
p += Grid()
print(p)
```



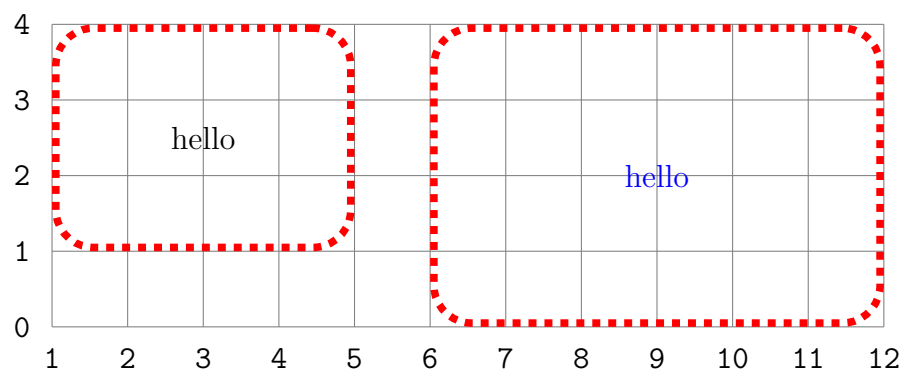
28.8 linestyle

```
from latextool_basic import *
p = Plot()
p += Rect(x0=1, y0=1, x1=3, y1=3,
          linewidth=0.1, linecolor='red',
          radius=0.5, linestyle='dashed')
p += Grid()
print(p)
```



28.9 label

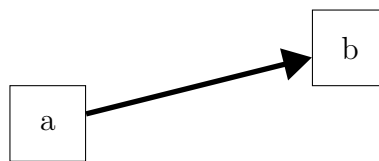
```
from latextool_basic import *
p = Plot()
p += Rect(x0=1, y0=1, x1=5, y1=4,
          linewidth=0.1, linecolor='red',
          radius=0.5, linestyle='dashed', label='hello')
p += Rect(x0=6, y0=0, x1=12, y1=4,
          linewidth=0.1, linecolor='red',
          foreground='blue',
          radius=0.5, linestyle='dashed', label='hello')
p += Grid()
print(p)
```



28.10 tikz name

Rect with tikz name:

```
from latextool_basic import *  
p = Plot()  
p += Rect(x0=1, y0=0, x1=2, y1=1, label='a', name='a')  
p += Rect(x0=5, y0=1, x1=6, y1=2, label='b', name='b')  
  
p += r"\draw[->,line width=2] (a) -- (b);"  
  
print(p)
```

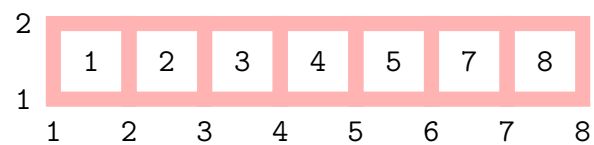


29 RectContainer (rectcontainer.tex)

Horizontal array:

```
from latextool_basic import *
p = Plot()

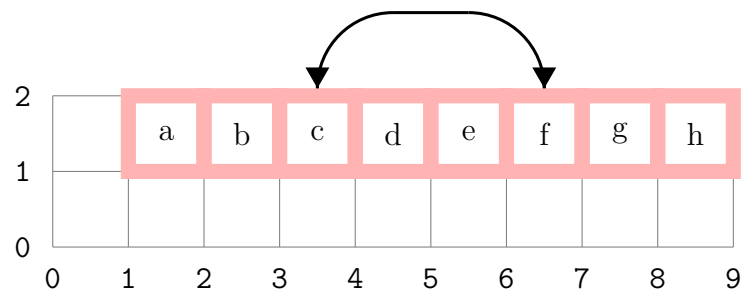
c = RectContainer(x=1, y=1)
for x in '1234578':
    c += Rect2(x0=0, y0=0, x1=1, y1=1,
               linewidth=0.2, linecolor='red!30!white',
               label=r'\texttt{%s}' % x)
p += c
p += Grid()
print(p)
```



29.1 Using []

```
from latextool_basic import *
p = Plot()

c = RectContainer(x=1, y=1)
for x in 'abcdefgh':
    c += Rect2(x0=0, y0=0, x1=1, y1=1,
               linewidth=0.2, linecolor='red!30!white',
               label=x)
p += c
p += bend(c[2].top(), c[5].top())
p += Grid()
print(p)
```



29.2 name

```

from latextool_basic import *
p = Plot()

c = RectContainer(x=1, y=1, name='A')
for x in '12':
    c += Rect2(x0=0, y0=0, x1=1, y1=1,
               linewidth=0.2, linecolor='red!30!white',
               label=r'\texttt{%s}' % x)

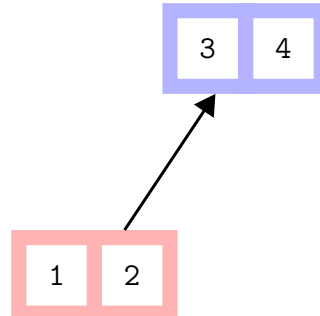
p += c

d = RectContainer(x=3, y=4, name='B')
for x in '34':
    d += Rect2(x0=0, y0=0, x1=1, y1=1,
               linewidth=0.2, linecolor='blue!30!white',
               label=r'\texttt{%s}' % x)

p += d

p += Line(names=['A', 'B'], endstyle='>')
print(p)

```



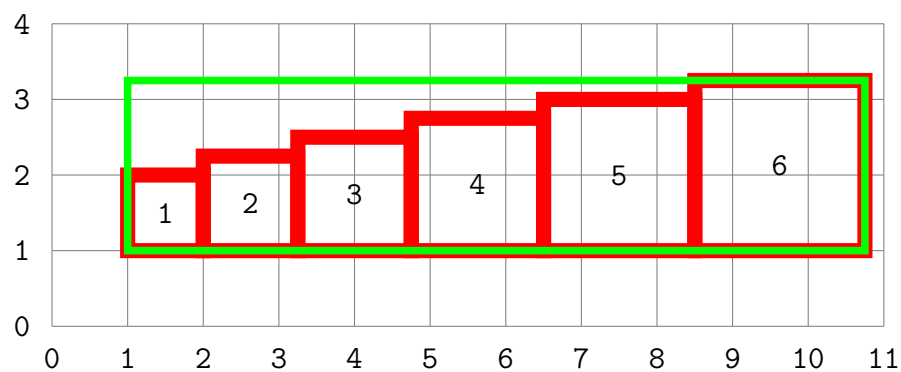
Left-to-right, align bottom:

```
from latextool_basic import *
p = Plot()

c = RectContainer(x=1, y=1)
for i, x in enumerate([1,2,3,4,5,6]):
    c += Rect2(x0=0, y0=0, x1=1+i/4.0, y1=1+i/4.0, linecolor='red', linewidth
th=0.2,
               label=r'\texttt{%s}' % x)
c.layout()
p += c

# boundary of container
p += Rect2(x0=c.x0, y0=c.y0, x1=c.x1, y1=c.y1, linewidth=0.1, linecolor='gr
een')

p += Grid()
print(p)
```



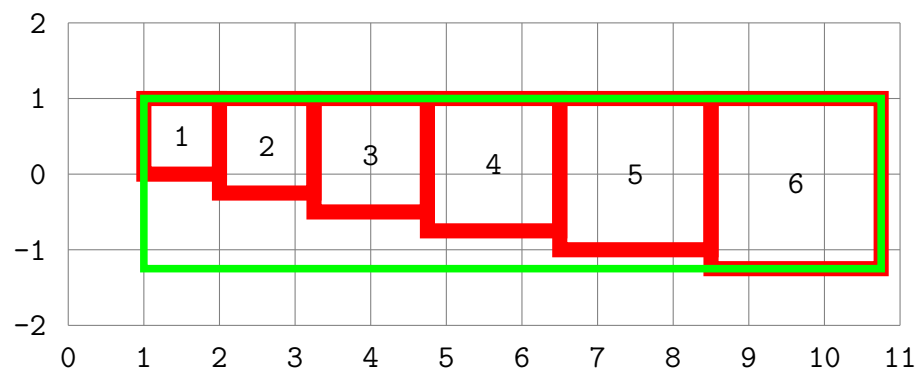
Align top, left-to-right:

```
from latextool_basic import *
p = Plot()

c = RectContainer(x=1, y=1, align='top')
for i, x in enumerate([1,2,3,4,5,6]):
    c += Rect2(x0=0, y0=0, x1=1+i/4.0, y1=1+i/4.0, linecolor='red', linewidth
th=0.2,
               label=r'\textttt{%s}}' % x)
p += c

# boundary of container
p += Rect2(x0=c.x0, y0=c.y0, x1=c.x1, y1=c.y1, linewidth=0.1, linecolor='gr
een')

p += Grid()
print(p)
```



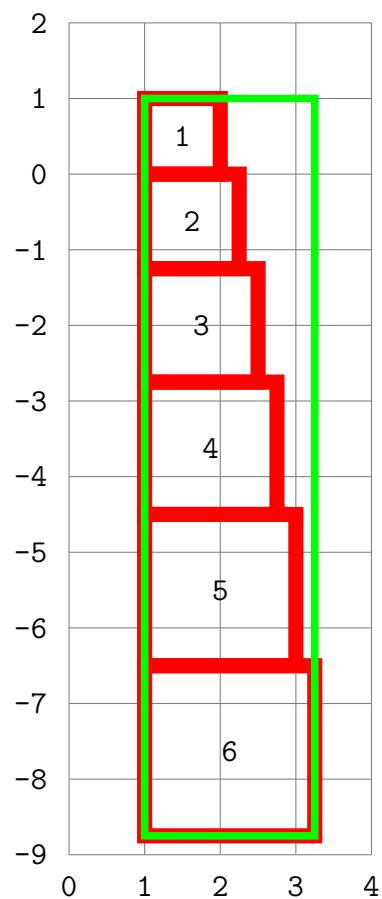
top-to-bottom, align left:

```
from latextool_basic import *
p = Plot()

c = RectContainer(x=1, y=1, align='left', direction='top-to-bottom')
for i, x in enumerate([1,2,3,4,5,6]):
    c += Rect2(x0=0, y0=0, x1=1+i/4.0, y1=1+i/4.0, linecolor='red', linewidth
th=0.2,
                label=r'\texttt{\%s}' % x)
p += c

# boundary of container
p += Rect2(x0=c.x0, y0=c.y0, x1=c.x1, y1=c.y1, linewidth=0.1, linecolor='gr
een')

p += Grid()
print(p)
```



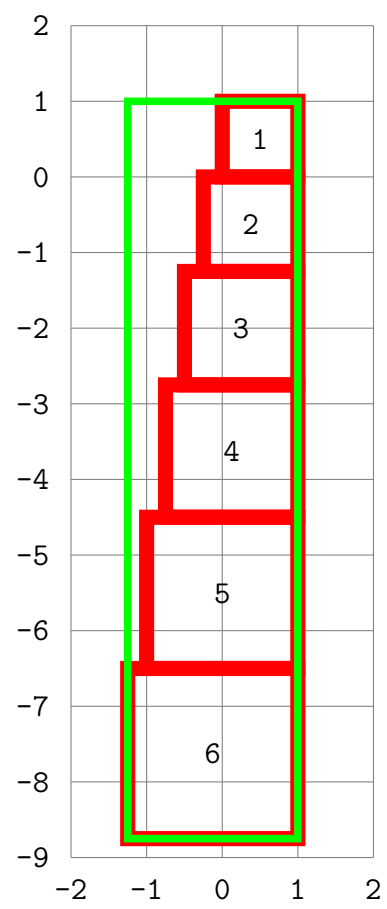
top-to-bottom, align right:

```
from latextool_basic import *
p = Plot()

c = RectContainer(x=1, y=1, align='right', direction='top-to-bottom')
for i, x in enumerate([1,2,3,4,5,6]):
    c += Rect2(x0=0, y0=0, x1=1+i/4.0, y1=1+i/4.0, linecolor='red', linewidth=0.2,
               label=r'\textttt{s}' % x)
p += c

# boundary of container
p += Rect2(x0=c.x0, y0=c.y0, x1=c.x1, y1=c.y1, linewidth=0.1, linecolor='green')

p += Grid()
print(p)
```



2-d stack

```

from latextool_basic import *
p = Plot()

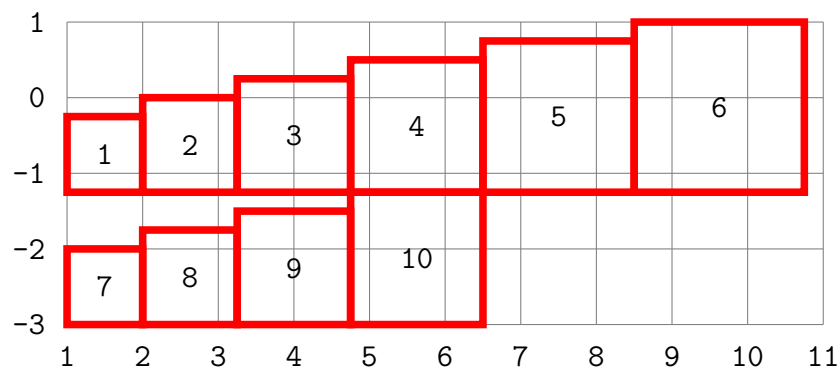
c0 = RectContainer(x=1, y=1, align='bottom', direction='left-to-right')
for i, x in enumerate([1,2,3,4,5,6]):
    c0 += Rect2(x0=0, y0=0, x1=1+i/4.0, y1=1+i/4.0, linecolor='red', linewidth=0.1,
                label=r'\texttt{%s}' % x)

c1 = RectContainer(x=1, y=1, align='bottom', direction='left-to-right')
for i, x in enumerate([7,8,9,10]):
    c1 += Rect2(x0=0, y0=0, x1=1+i/4.0, y1=1+i/4.0, linecolor='red', linewidth=0.1,
                label=r'\texttt{%s}' % x)

C = RectContainer(x=1, y=1, align='left', direction='top-to-bottom')
C += c0; c0.x = c0.x0; c0.y = c0.y0; c0.layout()
C += c1; c1.x = c1.x0; c1.y = c1.y0; c1.layout()

p += C;
p += Grid()
print(p)

```



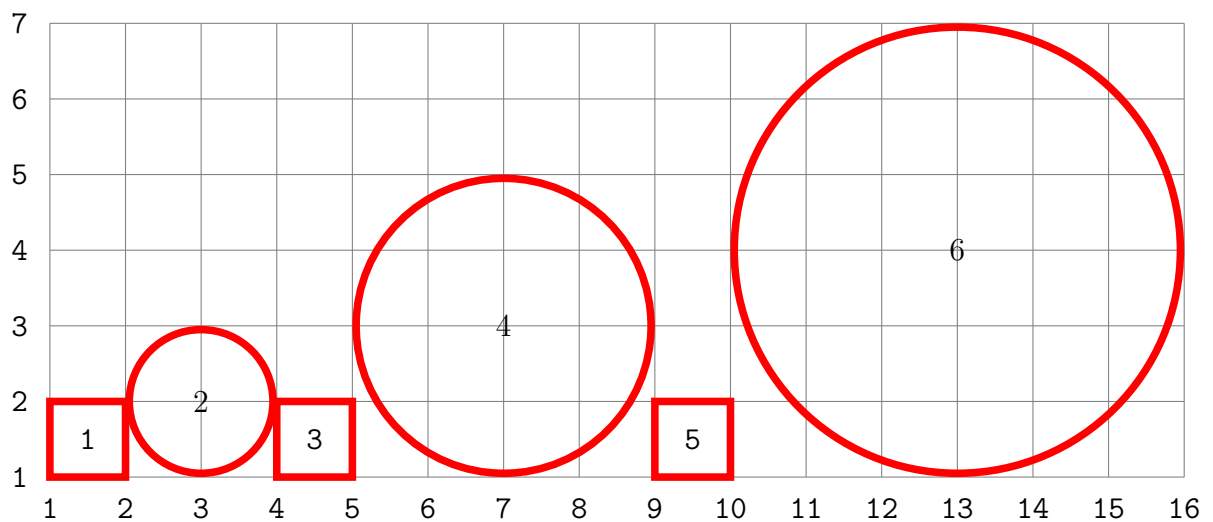
```

from latextool_basic import *
p = Plot()

c = RectContainer(x=1, y=1)
for i, x in enumerate([1,2,3,4,5,6]):
    if i % 2 == 0:
        c += Rect2(x0=0, y0=0, x1=1, y1=1, linecolor='red', linewidth=0.1,
                    label=r'\texttt{%s}}' % x)
    else:
        c += Circle(x=0.5, y=0.5, r=0.5 + i/2, label=x, linecolor='red', li
newidth=0.1)

p += c
p += Grid()
print(p)

```



Vertical array and pointers

```

L = 0.1 # line width
from latextool_basic import *
p = Plot()

v = RectContainer(x=0, y=0, align='left', direction='top-to-bottom')
for i in ['', '', '', '']:
    v += Rect2(0,0,1,1,s=i, linewidth=0.1, linecolor='red')

p += v

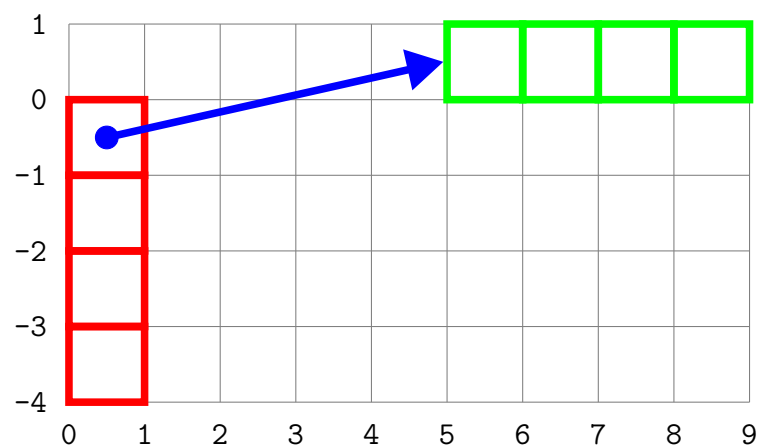
h = RectContainer(x=5, y=0, align='bottom', direction='left-to-right')
for i in ['', '', '', '']:
    h += Rect2(0,0,1,1,s=i, linewidth=0.1, linecolor='green')

p += h

p += Line(x0=v[0].centerx(), y0=v[0].centery(),
          x1=h[0].leftx(), y1=h[0].centery(),
          linewidth=0.1, linecolor='blue',
          endstyle='->', startstyle='dot', r=0.01)

p += Grid()
print(p)

```



DFA/NFA:

```

from latextool_basic import *
p = Plot()

h = RectContainer(x=0, y=0, align='bottom', direction='left-to-right')
for i in ['a', 'b', 'a', 'a', '$\sqcup$', '$\sqcup$', '$\sqcup$', ]:
    h += Rect2(0,0,1,1,
               label= r'\texttt{%s}' % i,
               linewidth=0.1, linecolor='red')
p += h

p += Line(x0=h[-1].x1, y0=h[-1].y1, x1=h[-1].x1+0.5, y1=h[-1].y1,
          linewidth=0.1, linecolor='red')

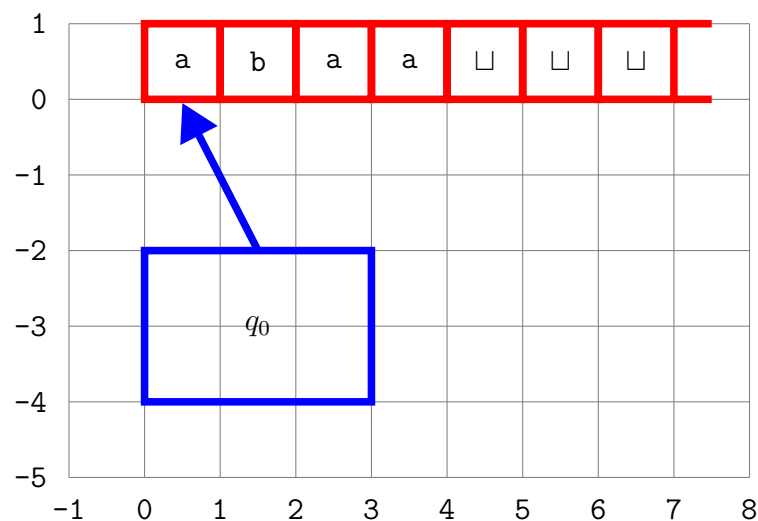
p += Line(x0=h[-1].x1, y0=h[-1].y0, x1=h[-1].x1+0.5, y1=h[-1].y0,
          linewidth=0.1, linecolor='red')

dfa = Rect2(0,-4,3,-2, linewidth=0.1, linecolor='blue', label='$q_0$')
p += dfa

p += Line(x0=dfa.centerx(), y0=dfa.y1,
          x1=h[0].centerx(), y1=h[0].bottomy(),
          linewidth=0.1, linecolor='blue', endstyle='->')

p += Grid()
print(p)

```



PDA:

```

from latextool_basic import *
p = Plot()
h = RectContainer(x=0, y=0, align='bottom', direction='left-to-right')
for i in ['a', 'b', 'a', 'a', '$\sqcup$', '$\sqcup$', '$\sqcup$', ]:
    h += Rect2(0,0,1,1,
               label= r'\texttt{%s}' % i,
               linewidth=0.1, linecolor='red')
p += h

p += Line(x0=h[-1].x1, y0=h[-1].y1, x1=h[-1].x1+0.5, y1=h[-1].y1,
          linewidth=0.1, linecolor='red')
p += Line(x0=h[-1].x1, y0=h[-1].y0, x1=h[-1].x1+0.5, y1=h[-1].y0,
          linewidth=0.1, linecolor='red')

pda = Rect2(0,-4,3,-2, linewidth=0.1, linecolor='blue', label='$q_0$')
p += pda

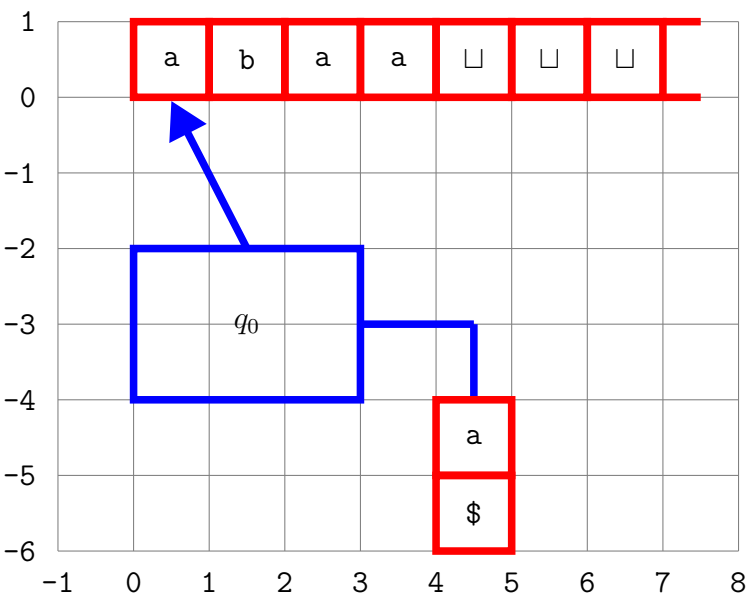
p += Line(x0=pda.centerx(), y0=pda.y1,
          x1=h[0].centerx(), y1=h[0].bottomy(),
          linewidth=0.1, linecolor='blue', endstyle='->')

v = RectContainer(x=4, y=-4, align='left', direction='top-to-bottom')
for i in ['a', '$', ]:
    v += Rect2(0,0,1,1,
               label= r'\texttt{%s}' % i, linewidth=0.1, linecolor='red')
p += v

p += Line(x0=pda.x1,y0=pda.centery(), x1=v[0].centerx(), y1=pda.centery(),
          linewidth=0.1, linecolor='blue')
p += Line(x0=v[0].centerx(),y0=pda.centery(),x1=v[0].centerx(),y1=v[0].topy
          (),
          linewidth=0.1, linecolor='blue')

p += Grid()
print(p)

```

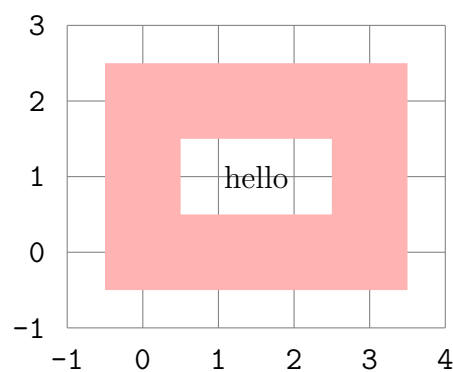


30 Rect2 and RectContainer ??? (rect2.tex)

Rect2 is the same as Rect except that the border is draw with half of the width in the rect and half of it outside. Use this for arrays – adjacent cells with have their boundaries overlap correctly.

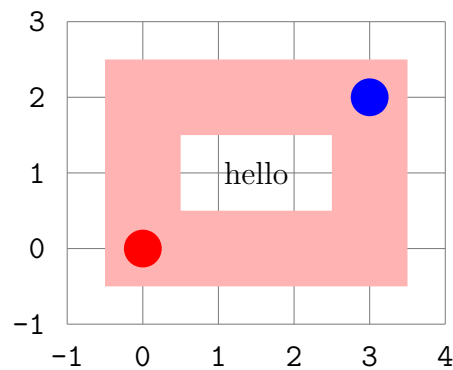
Rect2:

```
from latextool_basic import *
p = Plot()
p += Rect2(x0=0, y0=0, x1=3, y1=2, linecolor='red!30!white',
           linewidth=1.0, label='hello')
p += Grid()
print(p)
```



Note that x0, y0, x1, y1 is what was specified:

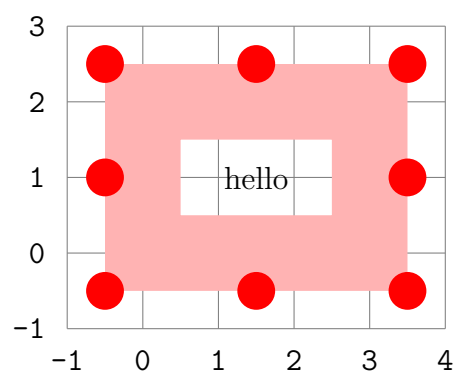
```
from latextool_basic import *
p = Plot()
r = Rect2(x0=0, y0=0, x1=3, y1=2, linecolor='red!30!white',
           linewidth=1.0, label='hello')
p += r
p += Circle(x=r.x0, y=r.y0, r=0.25, background='red', linewidth=0)
p += Circle(x=r.x1, y=r.y1, r=0.25, background='blue', linewidth=0)
p += Grid()
print(p)
```



However the top, bottom, left, right functions return points on the outer boundary:

```
from latextool_basic import *
p = Plot()
r = Rect2(x0=0, y0=0, x1=3, y1=2, linecolor='red!30!white',
          linewidth=1.0, label='hello')
p += r
p += Circle(center=r.top(), r=0.25, background='red', linewidth=0)
p += Circle(center=r.bottom(), r=0.25, background='red', linewidth=0)
p += Circle(center=r.left(), r=0.25, background='red', linewidth=0)
p += Circle(center=r.right(), r=0.25, background='red', linewidth=0)
p += Circle(center=r.topleft(), r=0.25, background='red', linewidth=0)
p += Circle(center=r.topright(), r=0.25, background='red', linewidth=0)
p += Circle(center=r.bottomleft(), r=0.25, background='red', linewidth=0)
p += Circle(center=r.bottomright(), r=0.25, background='red', linewidth=0)

p += Grid()
print(p)
```



31 Arrays (chap-arrays.tex)

32 Arrays

32.1 array (array.tex)

```
from latextool_basic import Plot, grid, array
p = Plot()
s = array(x0=0.5, y0=0.5, width=1, height=1, xs=[1,2,3,4,5,6])
p.add(s)
p.add(grid, x0=0, y0=0, x1=7, y1=2)
print(p)
```

1	2	3	4	5	6
---	---	---	---	---	---


```
from latextool_basic import Plot, array
p = Plot()
s = array(x0=0.5, y0=0.5, width=1, height=0.6, xs=[1,2,3,4,5,6])
p.add(s)
print(p)
```

1	2	3	4	5	6
---	---	---	---	---	---

Array of characters:

```
from latextool_basic import Plot, array
p = Plot()
s = array(x0=0.5, y0=0.5, width=1, height=0.6,
          xs=['h','e','l','l','o'])
p.add(s)
print(p)
```

'h'	'e'	'l'	'l'	'o'
-----	-----	-----	-----	-----

array with arraylinewidth

```
from latextool_basic import Plot, array
p = Plot()
s = array(x0=0.5, y0=0.5, width=1, height=0.6,
          xs=[1,2,3,4,5,6],
          arraylinewidth=0.2)
p.add(s)
print(p)
```



array with celllinewidth

```
from latextool_basic import Plot, array
p = Plot()
s = array(x0=0.5, y0=0.5, width=1, height=0.6,
          xs=[1,2,3,4,5,6],
          celllinewidth=0.1)
p.add(s)
print(p)
```

1	2	3	4	5	6
---	---	---	---	---	---

new array2

```
from latextool_basic import Plot, array2
p = Plot()
s = array2(x0=0.5, y0=0.5, width=1, height=0.6,
           vs=[1,2,3,4,5,6],
           celllinewidth=0.04)
p.add(s)
print(p)
```

1	2	3	4	5	6
---	---	---	---	---	---

TODO: Array variable name to left of array? Variables in general? Align variables and values?

32.2 snipped array (snipped-array.tex)

Snipped array (snip in the end):

```
from latextool_basic import *
p = Plot()

c = SnippedArray(x=1, y=1, xs=[1, 2, 3, '...'])
p += c
print(p)
```

1	2	3	...
---	---	---	-----

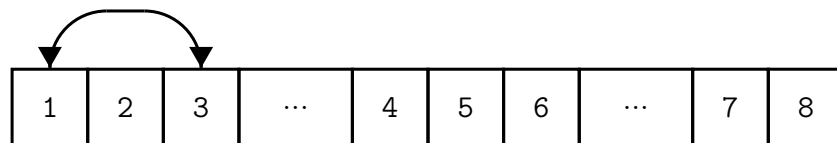
Snipped array (snip in the end):

```
from latextool_basic import *  
p = Plot()  
  
c = SnippedArray(x=1, y=1, xs=[1, 2, 3, '...', 4, 5, 6, '...', 7, 8])  
p += c  
print(p)
```

1	2	3	...	4	5	6	...	7	8
---	---	---	-----	---	---	---	-----	---	---

Snipped array (multiple snips):

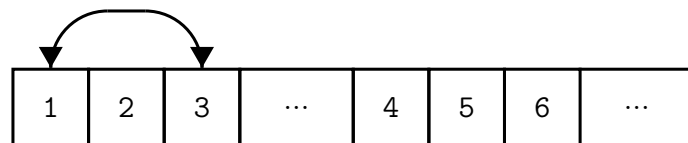
```
from latextool_basic import *  
p = Plot()  
  
c = SnippedArray(x=1, y=1, xs=[1, 2, 3, '...', 4, 5, 6, '...', 7, 8])  
p += c  
p += bend(c[0].top(), c[2].top(), dy=0.75)  
print(p)
```



Snipped array (snip at the end):

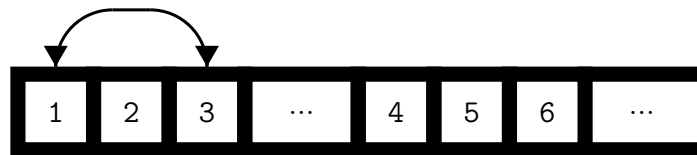
```
from latextool_basic import *
p = Plot()

c = SnippedArray(x=1, y=1, xs=[1, 2, 3, '...', 4, 5, 6, '...'])
p += c
p += bend(c[0].top(), c[2].top(), dy=0.75)
print(p)
```



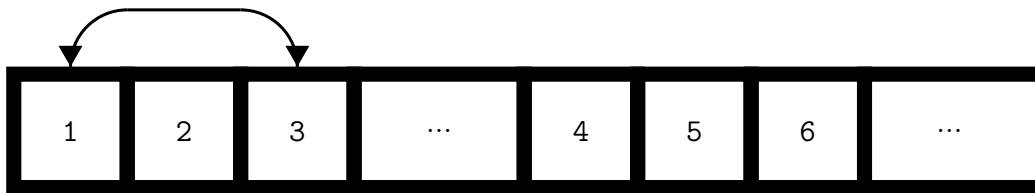
Snipped array (snip at the end, with linewidth):

```
from latextool_basic import *  
p = Plot()  
  
c = SnippedArray(x=1, y=1, xs=[1, 2, 3, '...', 4, 5, 6, '...'], linewidth=0  
.2)  
p += c  
p += bend(c[0].top(), c[2].top(), dy=0.75)  
print(p)
```



Snipped array (width and height):

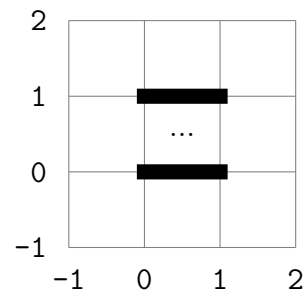
```
from latextool_basic import *  
p = Plot()  
  
c = SnippedArray(x=1, y=1, xs=[1, 2, 3, '...', 4, 5, 6, '...'],  
                 width=1.5, height=1.5, linewidth=0.2)  
p += c  
p += bend(c[0].top(), c[2].top(), dy=0.75)  
print(p)
```



Snipped array (snip at the end, with linewidth):

```
from latextool_basic import *
p = Plot()

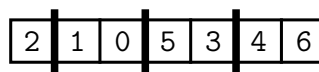
c = Rect2NoLeftRight(x0=0, y0=0, x1=1, y1=1, linewidth=0.2, label='...')
p += c
p += Grid(-1, -1, 2, 2)
print(p)
```



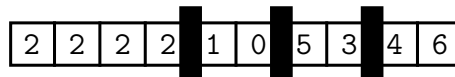
32.3 chunkedarray (chunkedarray.tex)

This is for list of lists.

```
from latextool_basic import Plot, grid, chunkedarray
p = Plot()
s = chunkedarray(cellwidth=0.6,
                  cellheight=0.6,
                  arr=[[2],[1,0],[5,3],[4,6]])
p.add(s)
p.add(grid, x0=0, y0=0, x1=7, y1=2)
print(p)
```

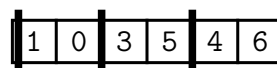


```
from latextool_basic import Plot, chunkedarray
p = Plot()
s = chunkedarray(cellwidth=0.6,
                  cellheight=0.6,
                  pipewidth=0.3,
                  pipeheight=1,
                  arr=[[2,2,2,2],[1,0],[5,3],[4,6]])
p.add(s)
print(p)
```



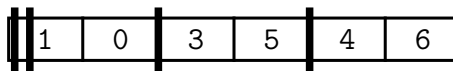
One empty list in front:

```
from latextool_basic import Plot, chunkedarray
p = Plot()
s = chunkedarray(cellwidth=0.6,
                  cellheight=0.6,
                  arr=[[ ],[1,0],[3,5],[4,6]])
p.add(s)
print(p)
```



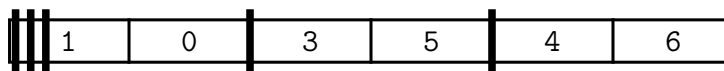
2 empty list in front:

```
from latextool_basic import Plot, chunkedarray
p = Plot()
s = chunkedarray(cellwidth=1,
                  cellheight=0.6,
                  arr=[[ ], [ ], [1,0], [3,5], [4,6]])
p.add(s)
print(p)
```



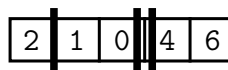
3 empty list in front:

```
from latextool_basic import Plot, chunkedarray
p = Plot()
s = chunkedarray(cellwidth=1.6,
                  cellheight=0.6,
                  arr=[[ ], [ ], [ ], [1,0], [3,5], [4,6]])
p.add(s)
print(p)
```



One empty list in the middle:

```
from latextool_basic import Plot, chunkedarray
p = Plot()
s = chunkedarray(cellwidth=0.6,
                  cellheight=0.6,
                  arr=[[2], [1,0], [ ], [4,6]])
p.add(s)
print(p)
```



1 empty list at the end:

```

from latextool_basic import Plot, grid, chunkedarray
p = Plot()
s = chunkedarray(cellwidth=0.6,
                  cellheight=0.6,
                  arr=[[2,3],[1,0],[4,5],[ ]])
p.add(s)
print(p)

```

2	3	1	0	4	5
---	---	---	---	---	---

2 empty lists at the end:

```

from latextool_basic import Plot, grid, chunkedarray
p = Plot()
s = chunkedarray(cellwidth=0.6,
                  cellheight=0.6,
                  arr=[[2,3],[1,0],[4,5],[ ], [ ]])
p.add(s)
print(p)

```

2	3	1	0	4	5
---	---	---	---	---	---

2 empty lists at the end and in front:

```

from latextool_basic import Plot, grid, chunkedarray
p = Plot()
s = chunkedarray(cellwidth=0.6,
                  cellheight=0.6,
                  arr=[ [ ], [ ], [2,3],[1,0],[4,5],[ ], [ ] ])
p.add(s)
print(p)

```

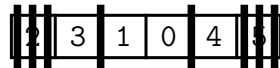
2	3	1	0	4	5
---	---	---	---	---	---

3 empty lists in front:

```

from latextool_basic import Plot, grid, chunkedarray
p = Plot()
s = chunkedarray(cellwidth=0.6,
                  cellheight=0.6,
                  arr=[[ ], [ ], [ ], [2,3], [1,0], [4,5], [ ], [ ], [ ]])
p.add(s)
print(p)

```



Two empty lists in the middle:

```

from latextool_basic import Plot, grid, chunkedarray
p = Plot()
s = chunkedarray(cellwidth=1.2,
                  cellheight=0.6,
                  arr=[[2,3], [6,7], [ ], [ ], [1,0], [4,5]])
p.add(s)
print(p)

```



3 empty lists in the middle:

```

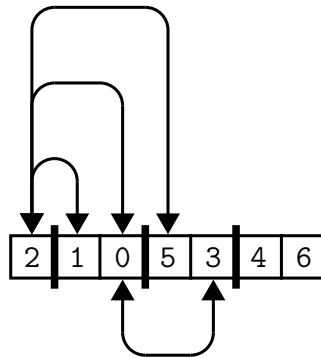
from latextool_basic import Plot, grid, chunkedarray
p = Plot()
s = chunkedarray(cellwidth=1.2,
                  cellheight=0.6,
                  arr=[[2,3], [6,7], [ ], [ ], [ ], [1,0], [4,5]])
p.add(s)
print(p)

```



Swaps:


```
from latextool_basic import Plot, grid, chunkedarray
p = Plot()
s = chunkedarray(cellwidth=0.6,
                 cellheight=0.6,
                 arr=[[2],[1,0],[5,3],[4,6]],
                 swaps=[(0,1,1),(0,2,2),(0,3,3),(2,4,-1)])
p.add(s)
print(p)
```

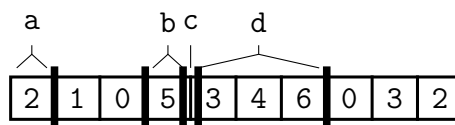


Labels for chunks

```

from latextool_basic import Plot, grid, chunkedarray
p = Plot()
s = chunkedarray(cellwidth=0.6,
                 cellheight=0.6,
                 arr=[[2],[1,0],[5],[],[3,4,6],[0,3,2]],
                 chunklabels=['a', '', 'b', 'c','d'])
p.add(s)
print(p)

```

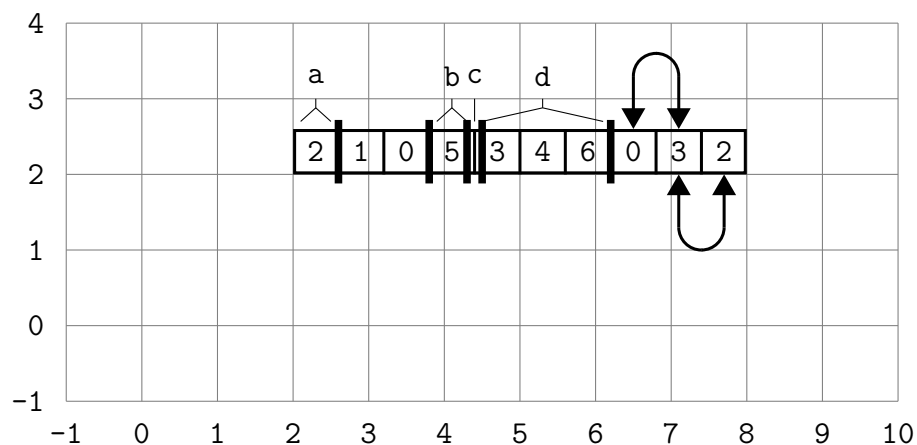


Labels for chunks with x,y:

```

from latextool_basic import Plot, chunkedarray, Grid
p = Plot()
s = chunkedarray(x=2, y=2,
                 cellwidth=0.6,
                 cellheight=0.6,
                 arr=[[2],[1,0],[5],[],[3,4,6],[0,3,2]],
                 swaps=[(7,8,1),(8,9,-1)],
                 chunklabels=['a', '', 'b', 'c','d'])
p.add(s)
p += Grid(x0=-1,y0=-1,x1=10,y1=4)
print(p)

```

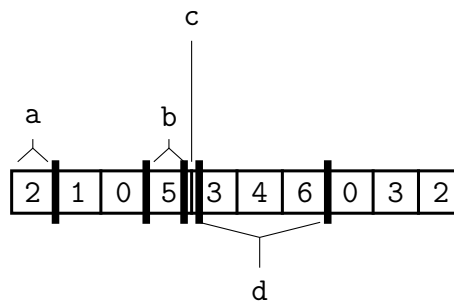


Labels with heights:

```

from latextool_basic import Plot, grid, chunkedarray
p = Plot()
s = chunkedarray(cellwidth=0.6,
                 cellheight=0.6,
                 arr=[[2],[1,0],[5],[],[3,4,6],[0,3,2]],
                 chunklabels=['a', '', 'b', ('c',2),('d',-1)])
p.add(s)
print(p)

```

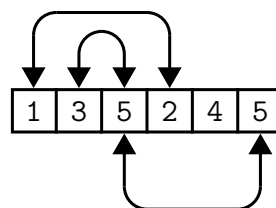


Using chunkedarray as array with swaps:

```

from latextool_basic import Plot, grid, chunkedarray
p = Plot()
s = chunkedarray(cellwidth=0.6,
                 cellheight=0.6,
                 arr=[[1,3,5,2,4,5]],
                 swaps=[(0,3,1), (1,2,0.75), (2,5,-1)])
)
p.add(s)
print(p)

```



TODO: unify API for chunked and regulars.

TODO: avoid collision of arrow tips and edge crossing.

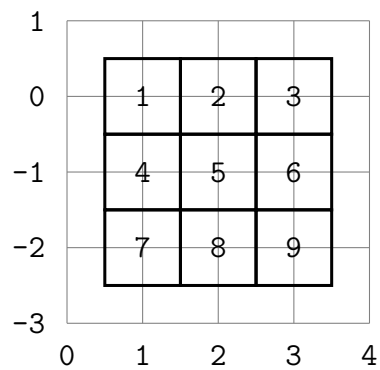
TODO: label arrow

TODO: label array

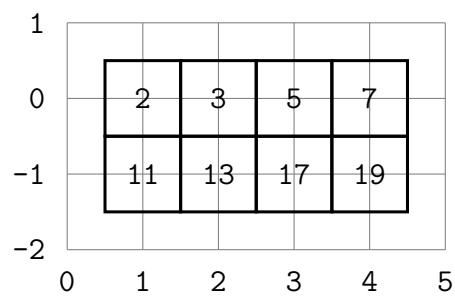
32.4 2d array (array2d.tex)

```
from latextool_basic import *
p = Plot()

c = Array2d(x=0.5, y=0.5, xs=[[1,2,3],[4,5,6],[7,8,9]])
p += c
p += Grid()
print(p)
```



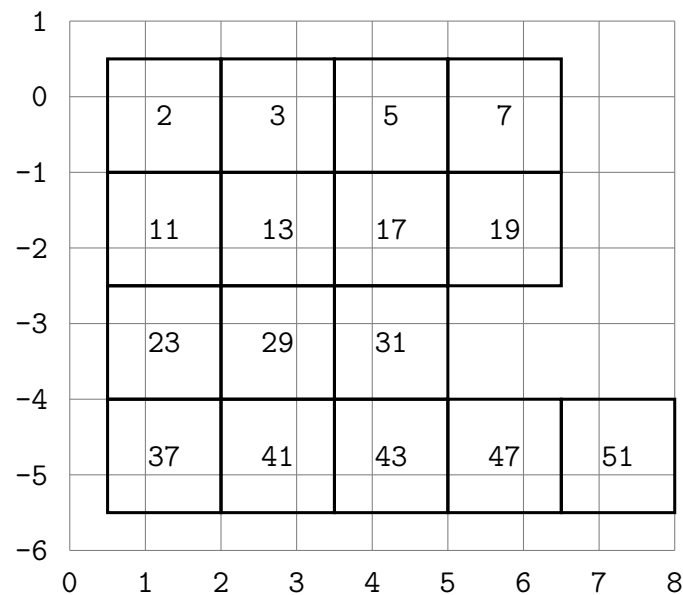
```
from latextool_basic import *
p = Plot()
a = Array2d(0.5, 0.5, [[2, 3, 5, 7], [11,13,17,19]])
p += a
p += Grid()
print(p)
```



```

from latextool_basic import *
p = Plot()
a = Array2d(0.5, 0.5, width=1.5, height=1.5, xs=[[2, 3, 5, 7], [11,13,17,19]
], [23, 29, 31], [37,41,43,47,51]])
p += a
p += Grid()
print(p)

```



PYTHON ERROR. See source, stderr, stdout below

```

from latextool_basic import *
p = Plot()
a = Array2d(0, 0, width=1.5, height=1.5, xs=[[2, 3, 5, 7], [11,13,17,19], [
23, 29, 31], [37,41,43,47,51]])

def l((r0,c0), (r1,c1)):
    return Line(a[r0][c0].centerx(), a[r0][c0].centery(),
                a[r1][c1].centerx(), a[r1][c1].centery(),
                linecolor='red', linewidth=0.25, endstyle='->')

p += a
p += l((0,0),(1,1))
p += l((3,0),(2,2))

print(p)

```

```
File "gdwmfjti.tmp.py", line 5
    def l((r0,c0), (r1,c1)):
        ^
SyntaxError: invalid syntax
```

Test Array2d drawing 1d array:

```
from latextool_basic import *
p = Plot()
p += Array2d(0, 0, width=1, height=1, xs=[[2, 3, 5, 7]])
print(p)
```

2	3	5	7
---	---	---	---

The split operation in mergesort: PYTHON ERROR. See source, stderr, stdout below

```

from latextool_basic import *
p = Plot()

def arr(x, y, xs):
    return Array2d(x, y, width=1, height=1, xs=xs)

def string(x, y, s):
    return Rect2(x0=x, y0=y, x1=x, y1=y, linewidth=0, label = s)

def _line(x0, y0, x1, y1):
    gap = 0.1
    y0 -= gap
    y1 += gap
    return Line(x0, y0, x1, y1, endstyle='->', linewidth=0.05)

xs = [2, 5, 3, 7]; left = xs[:len(xs)/2]; right = xs[len(xs)/2:]
common_len = max(len(left), len(right))

x = 1; y = 3
a = arr(x, y, xs=xs)
midx = a.bottom()[0]

b = arr(midx - 1 - common_len, y - 2, xs=left)
c = arr(midx + 1, y - 2, xs=right)
d = string(midx, 1.5, "split")
p += a; p += b; p += c; p += d

x0, y0 = a.bottom()
x1, y1 = midx - 1 - common_len/2.0, y0 - 1
p += _line(x0, y0, x1, y1)
x1, y1 = midx + 1 + common_len/2.0, y0 - 1
p += _line(x0, y0, x1, y1)

p += Grid()
print(p)

```

```

Traceback (most recent call last):
  File "hvielasw.tmp.py", line 16, in <module>
    xs = [2, 5, 3, 7]; left = xs[:len(xs)/2]; right = xs[len(xs)/2:]
TypeError: slice indices must be integers or None or have an __index__ meth
od

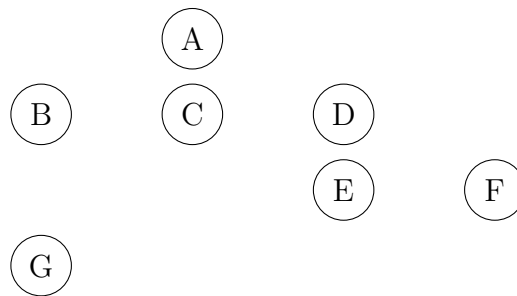
```


33 position (position.tex)

The `positions` function computes positions.

```
from latextool_basic import positions
d = positions(r'''
  A
B C D
  E F
G
''')

from latextool_basic import Plot, Circle
p = Plot()
for label, pos in d.items():
    x, y = pos
    p += Circle(x=x, y=y, r=0.4, label=label)
print(p)
```



You can scale the x and y axes:

```
from latextool_basic import positions
d = positions(r'''
  A
B C D
  E F
G
''', xscale=2.0, yscale=0.5)

from latextool_basic import *
p = Plot()
for label, pos in d.items():
    x, y = pos
    p += Circle(x=x, y=y, r=0.25, label=label)
print(p)
```



34 vphantom (vphantom.tex)

no vphantom:

```
from latextool_basic import *
p = Plot()
p += Rect(x0=0, y0=0, x1=1, y1=1,
          label=r'\textsf{%s}' % "b")
p += Rect(x0=1, y0=0, x1=2, y1=1,
          label=r'\textsf{%s}' % "g")
print(p)
```

b	g
---	---

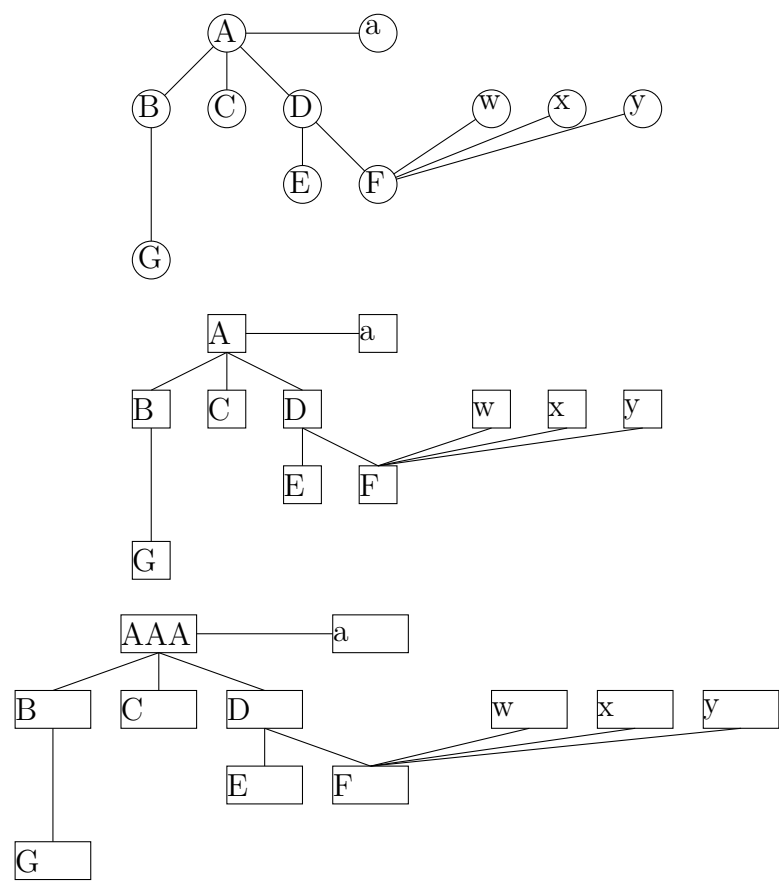
with vphantom:

```
from latextool_basic import *
p = Plot()
p += Rect(x0=0, y0=0, x1=1, y1=1,
          label=r'\vphantom{bg}\textsf{%s}' % "b")
p += Rect(x0=1, y0=0, x1=2, y1=1,
          label=r'\vphantom{bg}\textsf{%s}' % "g")
print(p)
```

b	g
---	---

35 Trees (chap-tree.tex)

35.1 Tree (tree.tex)

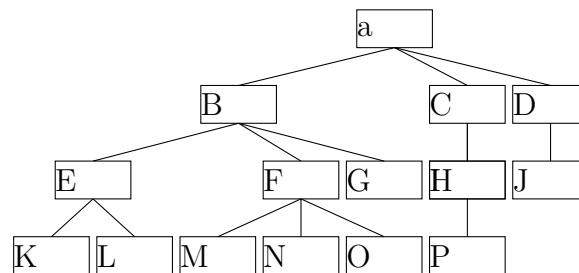


Auto adjust node position

```

from latextool_basic import positions
d = positions(r"""
    A
  B  C  D
E F G H  I J
K L M N O P
""", xscale=0.7)
edges = {'A':['B','C','D'], 'B':['E', 'F', 'G'],
        'C':['H','I'],      'D':['J'],
        'E':['K', 'L'],     'F':['M', 'N', 'O'],
        'I':['P'],
        }
}
from latextool_basic import Plot, tree
p = Plot()
p.add(tree(d, hor_sep=0.1,
          width=1, height=0.5,
          node_label={'A':'a'}, node_shape='rect',
          edges=edges,
          autoadjust=True))
print(p)

```



35.2 Parse tree example

```

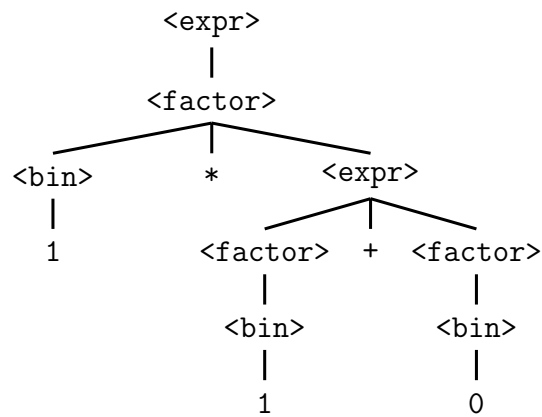
<expr> => <factor>
      => <bin> * <expr>
      => 1 * <expr>
      => 1 * <factor> + <factor>
      => 1 * <bin> + <factor>
      => 1 * 1 + <factor>
      => 1 * 1 + <bin>
      => 1 * 1 + 0

```

```

from latextool_basic import *
d = positions(r"""
  A
  B
C D E
F  G H I
   J  L
   K  M
""", xscale=0.7)
edges = {'A':['B'],          'B':['C', 'D', 'E'], 'C':['F'],
        'E':['G','H','I'],  'G':['J'],          'J':['K'],
        'I':['L'],          'L':['M'],
        }
p = Plot()
labels = {'A':r'\texttt{<expr>}',  'B':r'\texttt{<factor>}',
          'C':r'\texttt{<bin>}',    'D':r'\texttt{*}',
          'E':r'\texttt{<expr>}',  'F':r'\texttt{1}',
          'G':r'\texttt{<factor>}', 'H':r'\texttt{+}',
          'I':r'\texttt{<factor>}', 'J':r'\texttt{<bin>}',
          'K':r'\texttt{1}',        'L':r'\texttt{<bin>}',
          'M':r'\texttt{0}',
          }
rects = {}
for k,(x,y) in d.items():
    rects[k] = Rect(x0=x, y0=y-0.3, x1=x, y1=y+0.3, label=labels[k],
                    name=k, linecolor='white')
    p += rects[k]
for k,v in edges.items():
    for _ in v:
        p += Line(points=[rects[k].bottom(), rects[_].top()])
print(p)

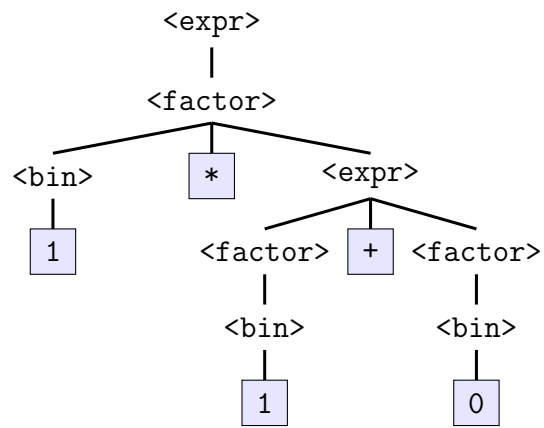
```



```

from latextool_basic import *
d = positions(r"""
  A
  B
C D E
F  G H I
  J  L
  K  M
""", xscale=0.7)
edges = {'A':['B'],          'B':['C', 'D', 'E'], 'C':['F'],
        'E':['G','H','I'], 'G':['J'],          'J':['K'],
        'I':['L'],          'L':['M']}
p = Plot()
labels = {'A':r'\texttt{<expr>}',  'B':r'\texttt{<factor>}',
        'C':r'\texttt{<bin>}',    'D':r'\texttt{*}',
        'E':r'\texttt{<expr>}',  'F':r'\texttt{1}',
        'G':r'\texttt{<factor>}', 'H':r'\texttt{+}',
        'I':r'\texttt{<factor>}', 'J':r'\texttt{<bin>}',
        'K':r'\texttt{1}',        'L':r'\texttt{<bin>}',
        'M':r'\texttt{0}'}
rects = parse_tree(p=p, d=d, edges=edges, labels=labels)
for v in 'DFHKM':
    r = rects[v]
    x1,y1 = r.top(); x1 += 0.3
    x0,y0 = r.bottom(); x0 -= 0.3
    p += Rect(x0=x0, y0=y0, x1=x1, y1=y1, background='blue!10',
              label=labels[v])
print(p)

```

35.3 Binary tree (tree2.tex)

```

from latextool_basic import *

p = Plot()
edges={'A':['B','C'],
      'B':['E','F'],
      'E':['G','H'],
      'F':['I'],
      'C':['K','L'],
      'K':['M','N'],
      'L':['','P']}

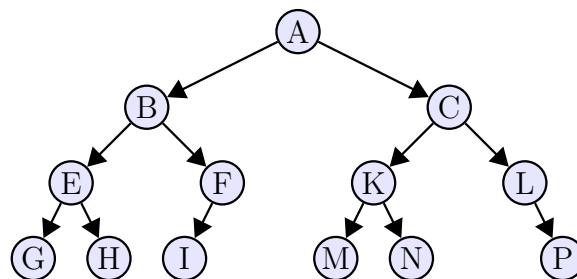
pos = bintreepositions(edges=edges)

Graph.r = 0.3
for k,(x,y) in pos.items():
    p += Graph.node(x=x, y=y, name=k, label=k)

for k,v in edges.items():
    for x in v:
        if x in [None,'']: continue
        p += Graph.arc(names=[k,x])

print(p)

```



Changing separation between nodes:

```
from latextool_basic import *

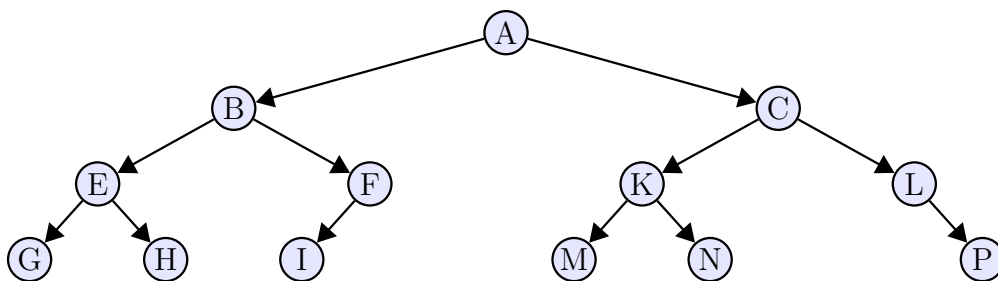
p = Plot()
edges={'A':['B','C'],
      'B':['E','F'],
      'E':['G','H'],
      'F':['I'],
      'C':['K','L'],
      'K':['M','N'],
      'L':['','P']}

pos = bintreepositions(edges=edges, node_hsep=1, node_vsep=1)

Graph.r = 0.3
for k,(x,y) in pos.items():
    p += Graph.node(x=x, y=y, name=k, label=k)

for k,v in edges.items():
    for x in v:
        if x in [None,'']: continue
        p += Graph.arc(names=[k,x])

print(p)
```



Changing separation between nodes:

```
from latextool_basic import *

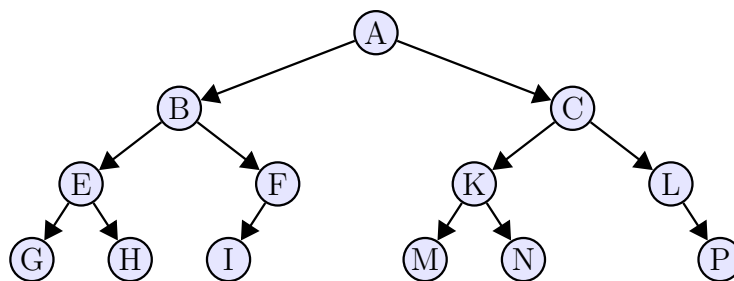
p = Plot()
edges={'A':['B','C'],
      'B':['E','F'],
      'E':['G','H'],
      'F':['I'],
      'C':['K','L'],
      'K':['M','N'],
      'L':['','P']}

pos = bintreepositions(edges=edges, node_hsep=0.5, node_vsep=1)

Graph.r = 0.3
for k,(x,y) in pos.items():
    p += Graph.node(x=x, y=y, name=k, label=k)

for k,v in edges.items():
    for x in v:
        if x in [None,'']: continue
        p += Graph.arc(names=[k,x])

print(p)
```

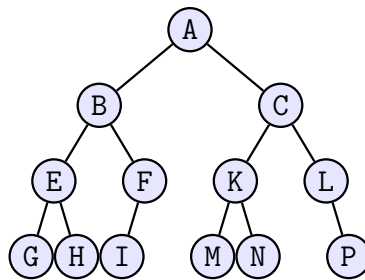


35.4 bintree function

```
from latextool_basic import *
p = Plot()
edges={'A':['B','C'],
      'B':['E','F'],
      'E':['G','H'],
      'F':['I'],
      'C':['K','L'],
      'K':['M','N'],
      'L':['','P']}

bintree(p, edges=edges)

print(p)
```



bintree function with edge function

```

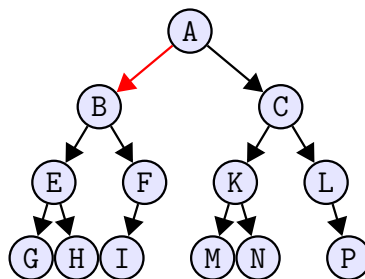
from latextool_basic import *

p = Plot()
edges={'A':['B','C'],
      'B':['E','F'],
      'E':['G','H'],
      'F':['I'],
      'C':['K','L'],
      'K':['M','N'],
      'L':['','P']}

def edge(names):
    if names==['A','B']:
        return Graph.arc(names=names, linecolor='red')
    else:
        return Graph.arc(names=names)
bintree(p, edges=edges, edge=edge)

print(p)

```



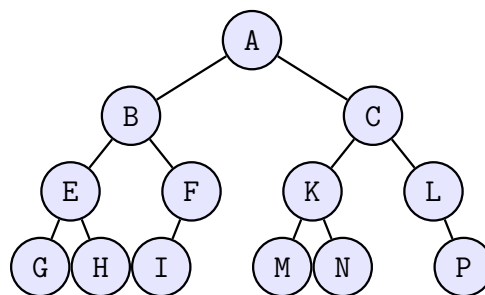
35.5 BinTree class

```
from latextool_basic import *

p = Plot()
edges={'A':['B','C'],
      'B':['E','F'],
      'E':['G','H'],
      'F':['I'],
      'C':['K','L'],
      'K':['M','N'],
      'L':['','P']}

BinTree.run(p, edges=edges)

print(p)
```

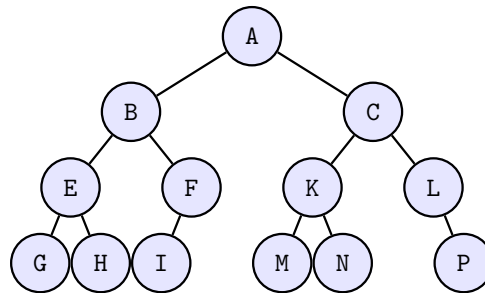


Using the BinTree class with label function

```
from latextool_basic import *

p = Plot()
edges={'A':['B','C'],
      'B':['E','F'],
      'E':['G','H'],
      'F':['I'],
      'C':['K','L'],
      'K':['M','N'],
      'L':['','P']}
def label(x): return r'\footnotesize \texttt{%s}' % x
BinTree.run(p, edges=edges, label=label)

print(p)
```

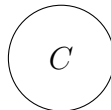
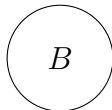
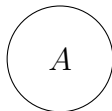


36 Graph (chap-graph.tex)

36.1 Graph (graph.tex)

36.2 graph function

```
from latextool_basic import graph
print(graph(layout="""
A
B C
"""))
```

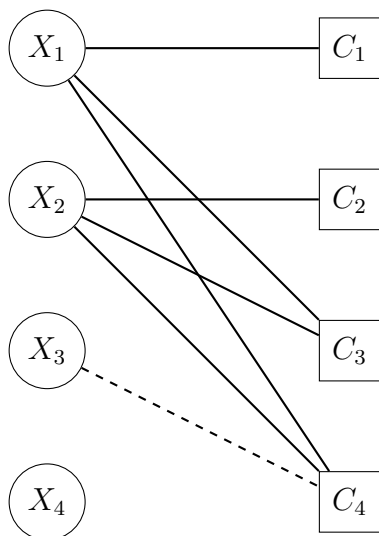


36.3 graph function:nodes and labels

```

from latextool_basic import graph
print(graph(layout='''
    X1  C1
    X2  C2
    X3  C3
    X4  C4
''',
minimum_size='8mm',
edges='X1-C1,X2-C2,X1-C3,X2-C3,X1-C4,X2-C4,X3dashedC4',
X1='label=$X_1$',
X2='label=$X_2$',
X3='label=$X_3$',
X4='label=$X_4$',
C1='shape=rectangle, label=$C_1$',
C2='shape=rectangle, label=$C_2$',
C3='shape=rectangle, label=$C_3$',
C4='shape=rectangle, label=$C_4$',
))

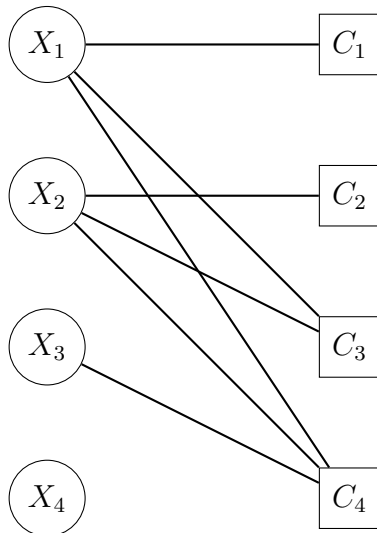
```



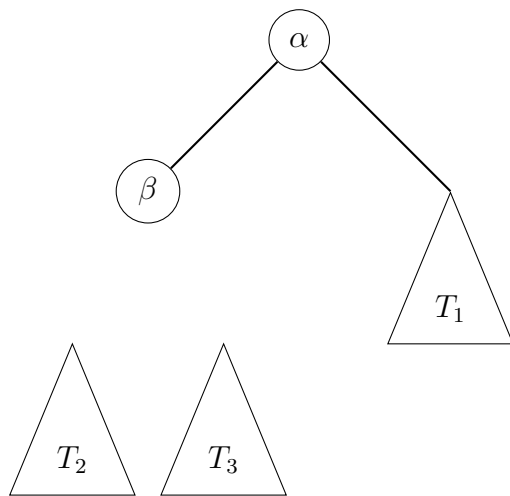
```

from latextool_basic import graph
print(graph(layout=''
    X1  C1
    X2  C2
    X3  C3
    X4  C4
'',
    minimum_size='8mm',
    edges='X1-C1,X2-C2,X1-C3,X2-C3,X1-C4,X2-C4,X3-C4',
    X1='label=$X_1$',
    X2='label=$X_2$',
    X3='label=$X_3$',
    X4='label=$X_4$',
    C1='shape=rectangle, label=$C_1$',
    C2='shape=rectangle, label=$C_2$',
    C3='shape=rectangle, label=$C_3$',
    C4='shape=rectangle, label=$C_4$',
))

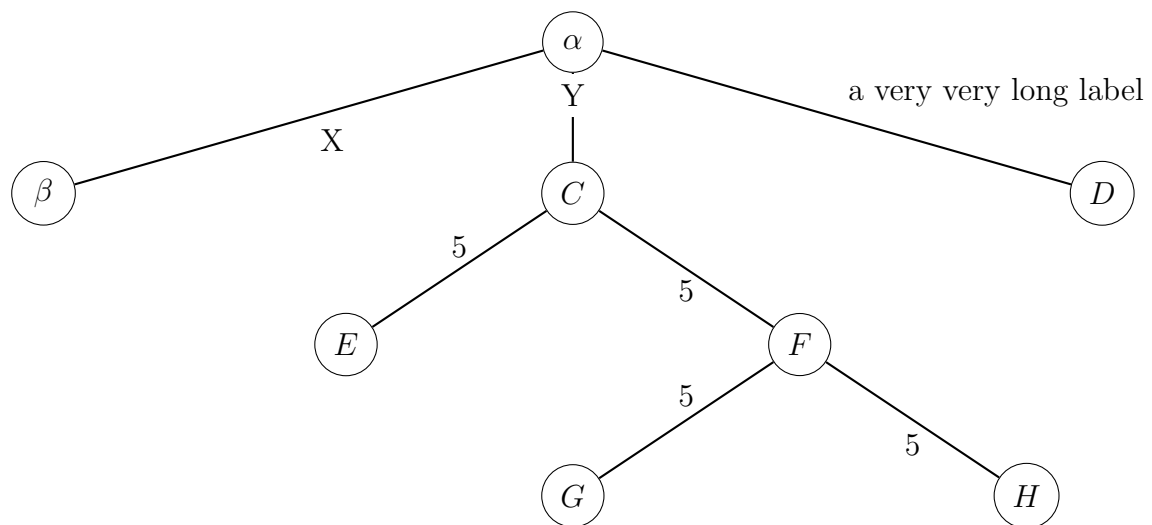
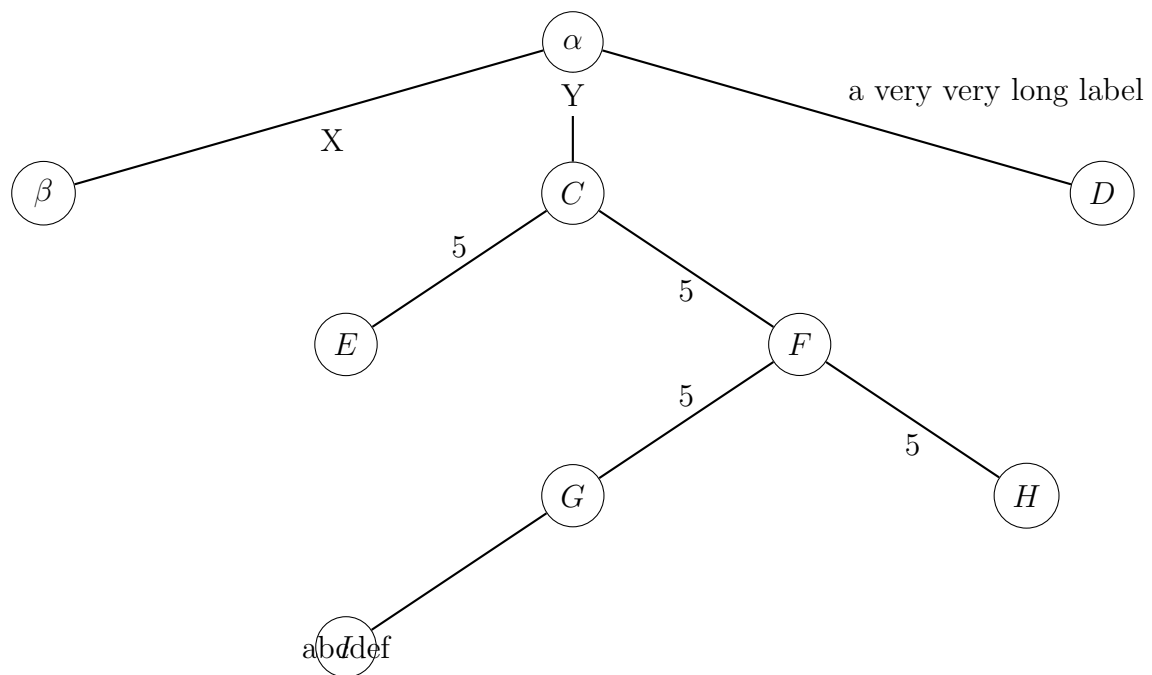
```



36.4 graph function: isoceles triangle as subtree



36.5 The graph function: edge labels



<http://tex.stackexchange.com/questions/86386/edge-nodes-auto-and-node-distance>

36.6 graph2 function (graph2.tex)

graph2 is an improvement and extension of graph.

```
from latextool_basic import *
p = Plot()

s = graph2(yscale=1, xscale=1,
layout="""
    A
  B   C
D E F G
H I
""",
minimum_size='8mm',
edges='A-B,A-C,B-D,B-E,C-G,C-F,D-H,D-I',
A=r'label=\texttt{12}',
B=r'label=\texttt{9}',
C=r'label=\texttt{8}',
D=r'label=\texttt{5}',
E=r'label=\texttt{3}',
F=r'label=\texttt{0}',
G=r'label=\texttt{7}',
H=r'label=\texttt{2}',
I=r'label=\texttt{1}',
fill_dict={'A': 'blue!10'},
)

p += s

p += r'\path[<->,dashed] (A) edge [bend left] (B);

print(p)
```

XXX

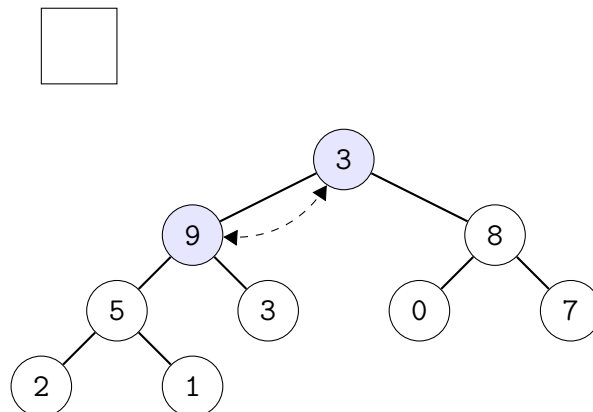

```

from latextool_basic import *
p = Plot()

s = graph2(yscale=1, xscale=1,
layout="""
    A
  B   C
D E F G
H I
""",
minimum_size='8mm',
edges='A-B,A-C,B-D,B-E,C-G,C-F,D-H,D-I',
A=r'label=\texttt{3}',
B=r'label=\texttt{9}',
C=r'label=\texttt{8}',
D=r'label=\texttt{5}',
E=r'label=\texttt{3}',
F=r'label=\texttt{0}',
G=r'label=\texttt{7}',
H=r'label=\texttt{2}',
I=r'label=\texttt{1}',
fill_dict={'A':'blue!10', 'B':'blue!10'},
)

p += s
p += r'\path[triangle 60-triangle 60,dashed] (A) edge [bend left] (B);'
p += Rect(0,0,1,1)
print(p)

```



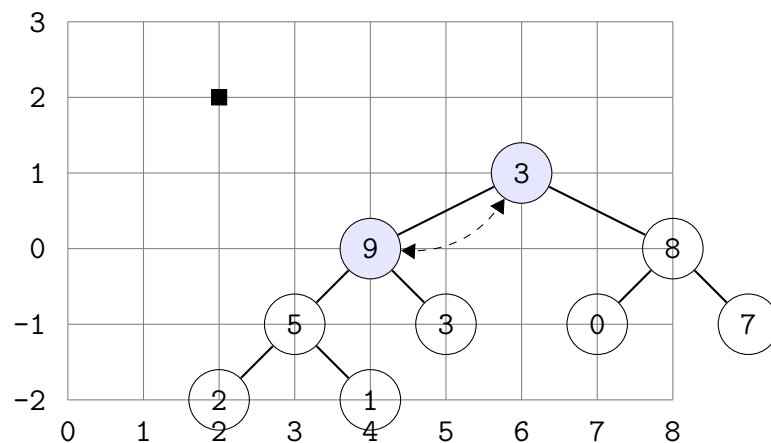
```

from latextool_basic import *
p = Plot()

s = graph2(xoffset=2, yoffset=2, yscale=1, xscale=1,
layout="""
    A
    B  C
    D E F G
    H I
    """,
minimum_size='8mm',
edges='A-B,A-C,B-D,B-E,C-G,C-F,D-H,D-I',
A=r'label=\texttt{3}',
B=r'label=\texttt{9}',
C=r'label=\texttt{8}',
D=r'label=\texttt{5}',
E=r'label=\texttt{3}',
F=r'label=\texttt{0}',
G=r'label=\texttt{7}',
H=r'label=\texttt{2}',
I=r'label=\texttt{1}',
fill_dict={'A':'blue!10', 'B':'blue!10'},
)

p += s
p += r'\path[triangle 60-triangle 60,dashed] (A) edge [bend left] (B);'
p += Rect(2-0.1,2-0.1,2+0.1,2+0.1,background='black')
p += Grid(x0=0,y0=-2,x1=8,y1=3)
print(p)

```



36.7 Graph class (some-graphs.tex)

```

from latextool_basic import *
p = Plot()
for i in [1,2,3]:
    p += Graph.node(p, x=i, y=1, name='a%s' % i)
    p += Graph.node(p, x=i, y=-1, name='c%s' % i)

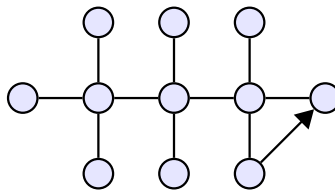
for i in range(5):
    p += Graph.node(p, x=i, y=0, name='b%s' % i)

p += Graph.edge(names=['b%s' % i for i in range(5)])
for i in [1,2,3]:
    p += Graph.edge(names=['a%s' % i, 'b%s' % i])
    p += Graph.edge(names=['c%s' % i, 'b%s' % i])

p += Graph.arc(names=['c3', 'b4'])

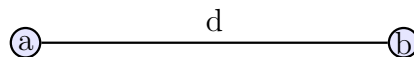
print(p)

```



36.8 Edge label anchor

```
from latextool_basic import *  
p = Plot()  
p += Graph.node(x=0, y=0, label='a', name='0')  
p += Graph.node(x=5, y=0, label='b', name='1')  
p += Graph.edge(names=['0','1'], label='d', anchor='above')  
  
print(p)
```



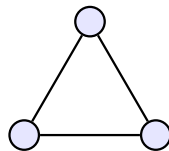
36.9 Cycle graphs

Odd Cycle.

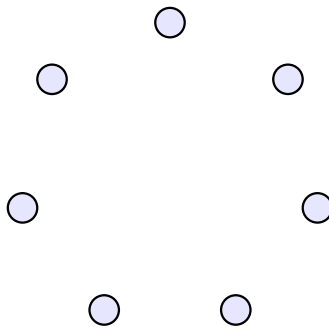
The node name is of the form `[radius]_[i]` where `i` is the index of node going in an anticlockwise direction, starting at degree `startdegree`.

By default, the first node is at `startdegree=90` degrees.

```
from latextool_basic import *
p = Plot()
cyclegraph(p, num=3, radius=1)
print(p)
```



```
from latextool_basic import *
p = Plot()
cyclegraph(p, num=7, radius=2, drawline=False)
print(p)
```

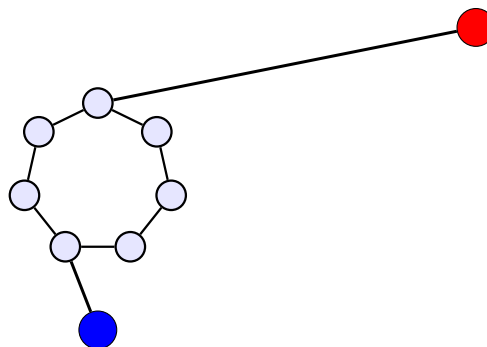


36.10 Name of nodes

Name of the nodes:

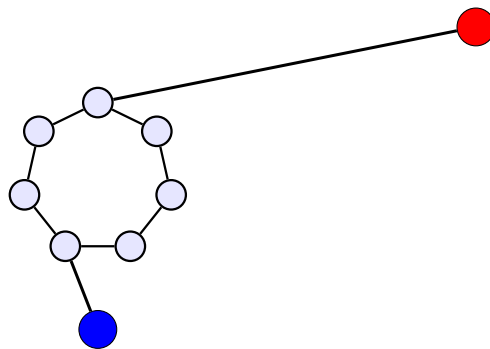
The nodes are named '0', '1', etc. in the anticlockwise direction where '0' is the node with angle `startdegree`. You can specify the `startdegree` in the `cyclegraph`. By default, when the number of nodes is even, the `startdegree` is 0 and when the number of nodes is odd, the `startdegree` is 90.

```
from latextool_basic import *
p = Plot()
cyclegraph(p, num=7, radius=1, drawline=True)
p += Circle(x=5, y=2, r=0.25, background='red', name='A')
p += Circle(x=0, y=-2, r=0.25, background='blue', name='B')
p += Line(names=['A', '0'])
p += Line(names=['B', '3'])
print(p)
```



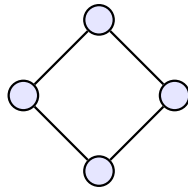
You can change the pgf/tikz name of the nodes by setting `names` dictionary in the `cyclegraph`. For instance if you want the first node to be named 'a' instead of 0, then 0:'a' should be a key-value pair in `names`.

```
from latextool_basic import *
p = Plot()
cyclegraph(p, num=7, radius=1, drawline=True,
           names={0:'a',
                  3:'d',
                  })
p += Circle(x=5, y=2, r=0.25, background='red', name='A')
p += Circle(x=0, y=-2, r=0.25, background='blue', name='B')
p += Line(names=['A', 'a'])
p += Line(names=['B', 'd'])
print(p)
```

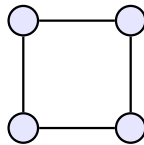


Even Cycle. By default, the starting degree is 0.

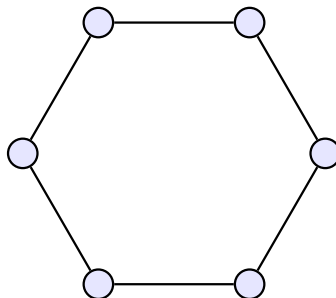
```
from latextool_basic import *  
p = Plot()  
cyclegraph(p, num=4, radius=1)  
print(p)
```



```
from latextool_basic import *  
p = Plot()  
cyclegraph(p, num=4, radius=1, startdegree=45)  
print(p)
```

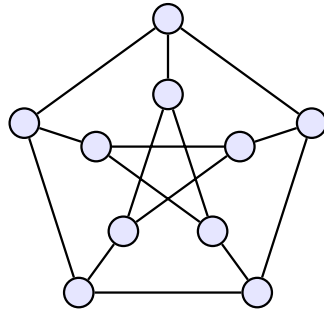


```
from latextool_basic import *  
p = Plot()  
cyclegraph(p, num=6, radius=2)  
print(p)
```



36.11 Petersen graph

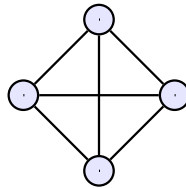
```
from latextool_basic import *  
p = Plot()  
petersen(p)  
print(p)
```



36.12 Complete graphs

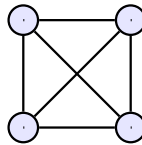
K_4 :

```
from latextool_basic import *
p = Plot()
completegraph(p, num=4)
print(p)
```



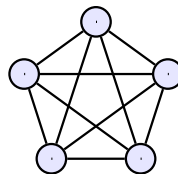
K_4 :

```
from latextool_basic import *
p = Plot()
completegraph(p, num=4, startdegree=45)
print(p)
```



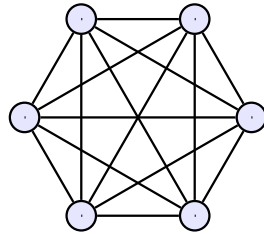
K_5 :

```
from latextool_basic import *
p = Plot()
completegraph(p, num=5)
print(p)
```



K_6 :

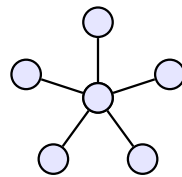
```
from latextool_basic import *  
p = Plot()  
completegraph(p, num=6, radius=1.5)  
print(p)
```



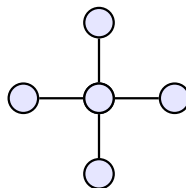
36.13 Star graphs

The name of the center node is 'center'.

```
from latextool_basic import *
p = Plot()
stargraph(p, num=5)
print(p)
```

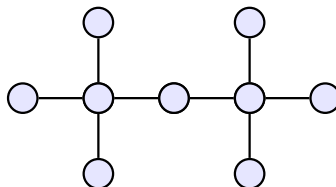


```
from latextool_basic import *
p = Plot()
stargraph(p, num=4)
print(p)
```



When there are two stars, you need to give the centers different names:

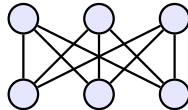
```
from latextool_basic import *
p = Plot()
stargraph(p, x=0, y=0, num=4, radius=1, names={'center':'c0'})
stargraph(p, x=2, y=0, num=4, radius=1, names={'center':'c1'})
print(p)
```



36.14 Bipartite graphs

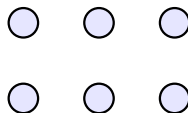
$K_{3,3}$:

```
from latextool_basic import *
p = Plot()
completebipartite(p=p, num1=3, num2=3)
print(p)
```



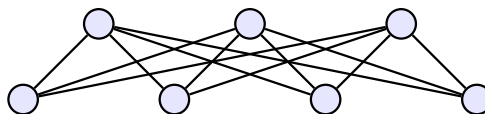
$K_{3,3}$ without edges:

```
from latextool_basic import *
p = Plot()
completebipartite(p=p, num1=3, num2=3, drawline=False)
print(p)
```



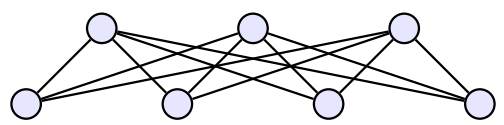
$K_{3,4}$:

```
from latextool_basic import *
p = Plot()
completebipartite(p=p, num1=3, num2=4, horsep=2)
print(p)
```



$K_{3,4}$:

```
from latextool_basic import *
p = Plot()
completebipartite(p=p, num1=3, num2=4, horsep=2)
print(p)
```



37 Frame (frame.tex)

```
from latextool_basic import *
p = Plot()
env = [('x', 1), ('y', 2), ('z', 3.14), ('w', 'a')]
p.add(frame(env, top="main()"))
print(p)
```

main()

x	1
y	2
z	3.14
w	'a'

38 Test Verbatim Spacing (test-verbatim-spacing.tex)

First here's the vertical spacing between paragraphs. Nam dui ligula, fringilla a, euismod sodales, sollicitudin vel, wisi. Morbi auctor lorem non justo. Nam lacus libero, pretium at, lobortis vitae, ultricies et, tellus. Donec aliquet, tortor sed accumsan bibendum, erat ligula aliquet magna, vitae ornare odio metus a mi. Morbi ac orci et nisl hendrerit mollis. Suspendisse ut massa. Cras nec ante. Pellentesque a nulla. Cum sociis natoque penatibus et magnis dis parturient montes, nascetur ridiculus mus. Aliquam tincidunt urna. Nulla ullamcorper vestibulum turpis. Pellentesque cursus luctus mauris.

Nam dui ligula, fringilla a, euismod sodales, sollicitudin vel, wisi. Morbi auctor lorem non justo. Nam lacus libero, pretium at, lobortis vitae, ultricies et, tellus. Donec aliquet, tortor sed accumsan bibendum, erat ligula aliquet magna, vitae ornare odio metus a mi. Morbi ac orci et nisl hendrerit mollis. Suspendisse ut massa. Cras nec ante. Pellentesque a nulla. Cum sociis natoque penatibus et magnis dis parturient montes, nascetur ridiculus mus. Aliquam tincidunt urna. Nulla ullamcorper vestibulum turpis. Pellentesque cursus luctus mauris.

TEST 1. Text, console environment, text:

```
hello world
```

Line after.

TEST 2. Text, console environment, console environment, text:

```
hello world
```

```
hello world
```

Line after. Probably won't be used. Probably too much spacing between the two environments.

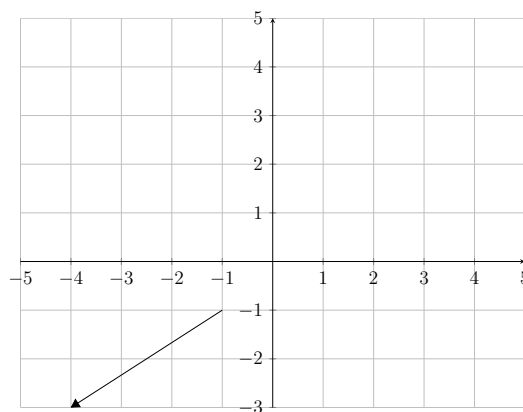
TEST 3. Text, python environment with console function, text:

```
hello world
```

Line after.

39 2D vector diagram (vec2d.tex)

```
from latextool_basic import *
p, q = (-1,-1), (-4,-3)
print(plot(vector=[p,q]))
#v = vec(p, q)
#print(answer(answer=''))
```



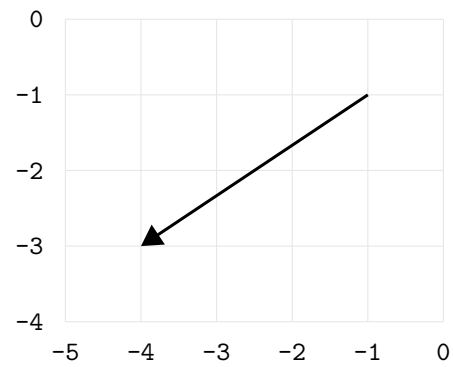
PYTHON ERROR. See source, stderr, stdout below

```
from latextool_basic import *
p = Plot()
p, q = (-1,-1), (-4,-3); vector = [p, q]; vectors = [vector]
vec2dplot(p, vectors=vectors)
print(p)
```

```
Traceback (most recent call last):
  File "fedidzee.tmp.py", line 4, in <module>
    vec2dplot(p, vectors=vectors)
  File "/usr/lib64/python3.7/site-packages/latextool_basic.py", line 3857,
in vec2dplot
    plot += Grid(x0=x0, y0=y0, x1=x1, y1=y1, linecolor='black!10', label_ax
es=True, fontsize='footnotesize')
TypeError: can only concatenate tuple (not "Grid") to tuple
```

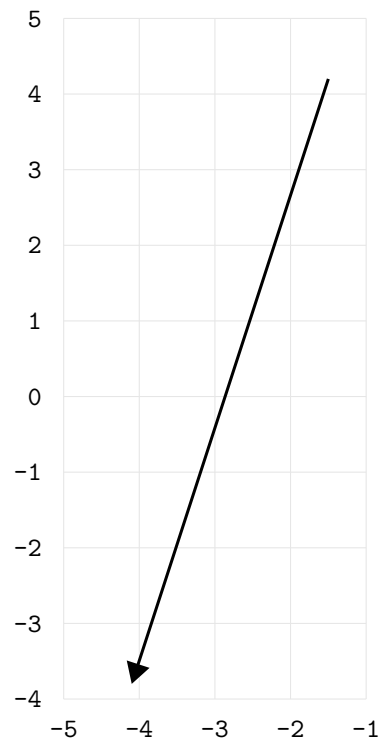
Without axes:

```
from latextool_basic import *  
plot = Plot()  
p, q = (-1,-1), (-4,-3); vector = [p, q]; vectors = [vector]  
vec2dplot(plot, vectors=[vector], draw_axes=False)  
print(plot)
```



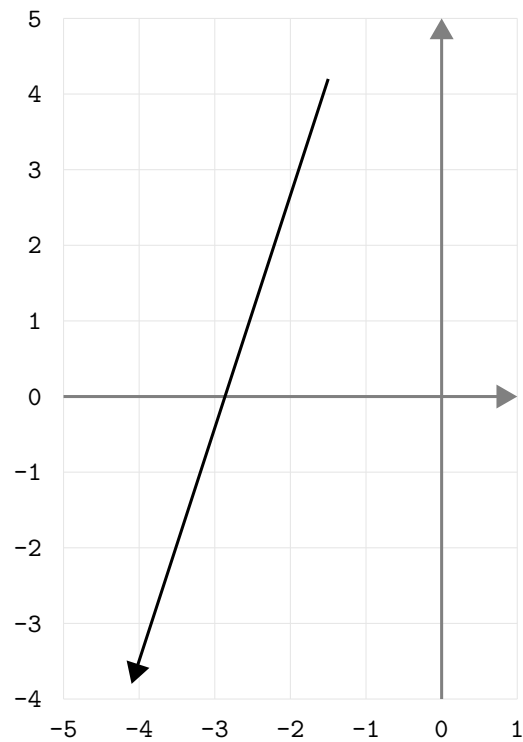
Without axes and vector with non integer endpoints:

```
from latextool_basic import *  
plot = Plot()  
p, q = (-1.5, 4.2), (-4.1, -3.8); vector = [p, q]; vectors = [vector]  
vec2dplot(plot, vectors=[vector], draw_axes=False)  
print(plot)
```



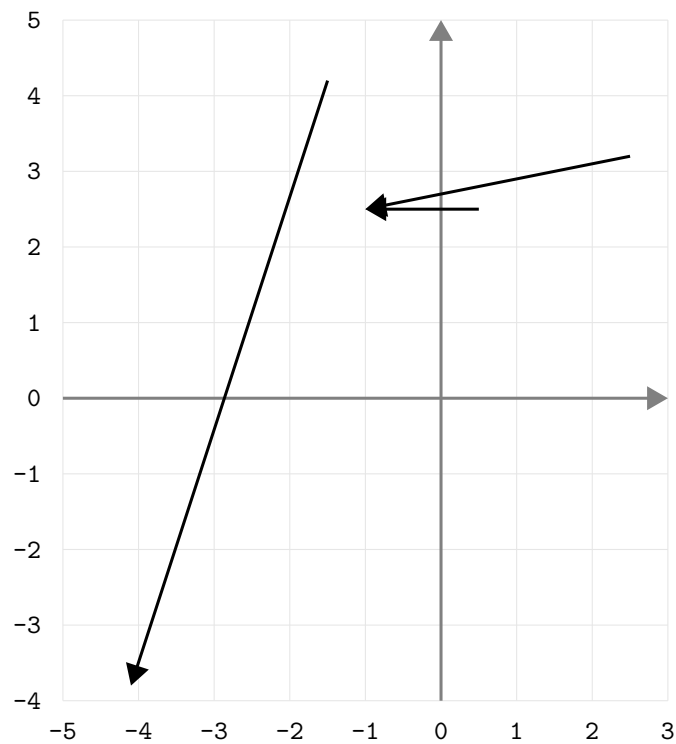
With axes and vector with non integer endpoints:

```
from latextool_basic import *  
plot = Plot()  
p, q = (-1.5, 4.2), (-4.1, -3.8); vector = [p, q]; vectors = [vector]  
vec2dplot(plot, vectors=[vector])  
print(plot)
```



With axes and multiple vectors with non integer endpoints:

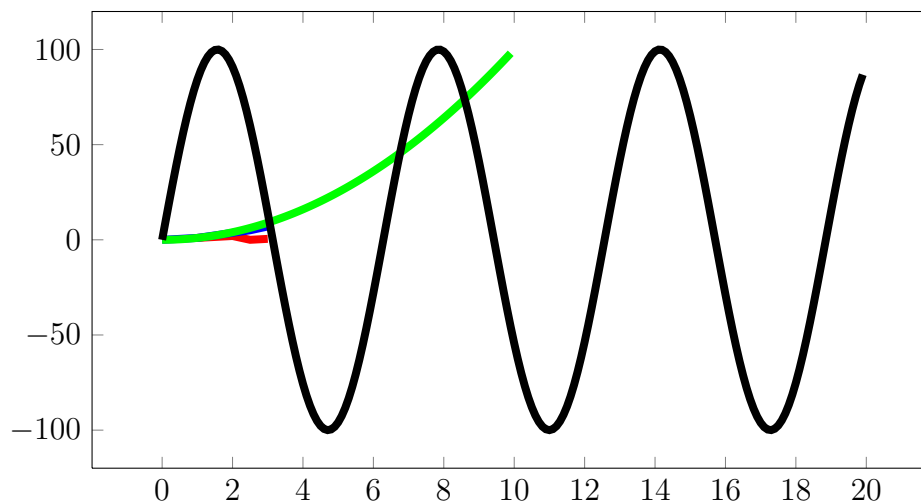
```
from latextool_basic import *
plot = Plot()
p, q = (-1.5, 4.2), (-4.1, -3.8); v0 = [p, q];
p, q = (2.5, 3.2), (-1, 2.5); v1 = [p, q];
p, q = (0.5, 2.5), (-1, 2.5); v2 = [p, q];
vectors = [v0, v1, v2]
vec2dplot(plot, vectors=vectors)
print(plot)
```



40 Function line graph (line-graph.tex)

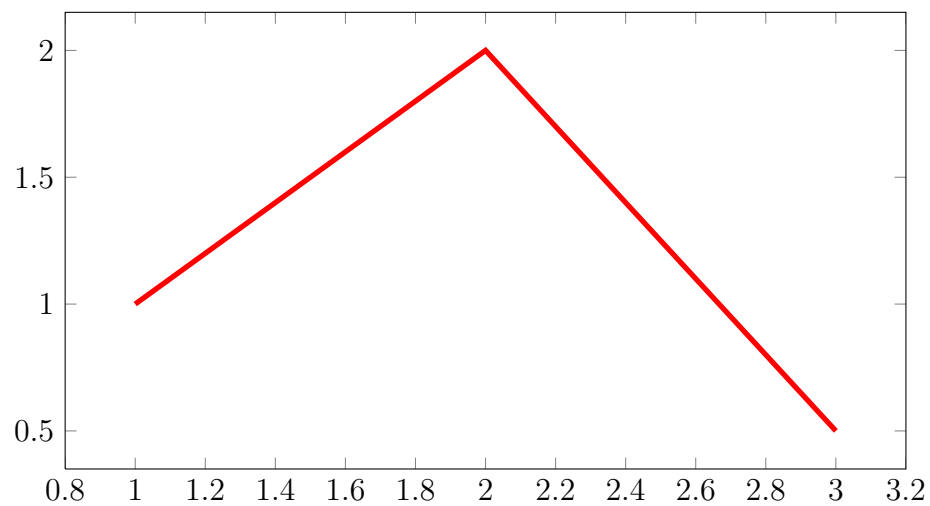
```
from math import sin
from latextool_basic import *
plot = FunctionPlot()

data = ((1, 1), (2, 2), (2.5, 0), (3, 0.5))
data2 = ((0, 0), (1, 1), (2.5, 5), (3, 7))
data3 = [(x/10.0, (x/10.0)**2) for x in range(0, 100)]
data4 = [(x/10.0, 100 * sin(x/10.0)) for x in range(0, 200)]
plot.add(data, line_width='3', color='red')
plot.add(data2, line_width='3', color='blue')
plot.add(data3, line_width='3', color='green')
plot.add(data4, line_width='3', color='black')
print(plot)
```



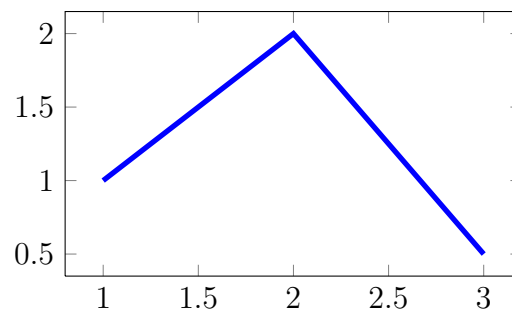
Line width and color:

```
from latextool_basic import *  
plot = FunctionPlot()  
plot.add(((1, 1), (2, 2), (3, 0.5))), line_width='2', color='red')  
print(plot)
```



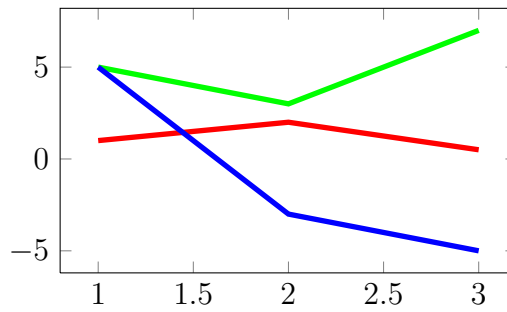
Width and height:

```
from latextool_basic import *
plot = FunctionPlot(width="3in", height="2in")
plot.add(((1, 1), (2, 2), (3, 0.5)), line_width='2', color='blue')
print(plot)
```



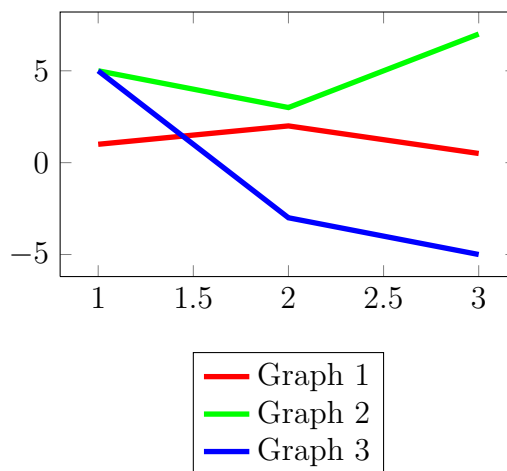
Multiple lines:

```
from latextool_basic import *
plot = FunctionPlot(width="3in", height="2in")
plot.add(((1, 1), (2, 2), (3, 0.5)), line_width='2', color='red')
plot.add(((1, 5), (2, 3), (3, 7)), line_width='2', color='green')
plot.add(((1, 5), (2, -3), (3, -5)), line_width='2', color='blue')
print(plot)
```



Legend:

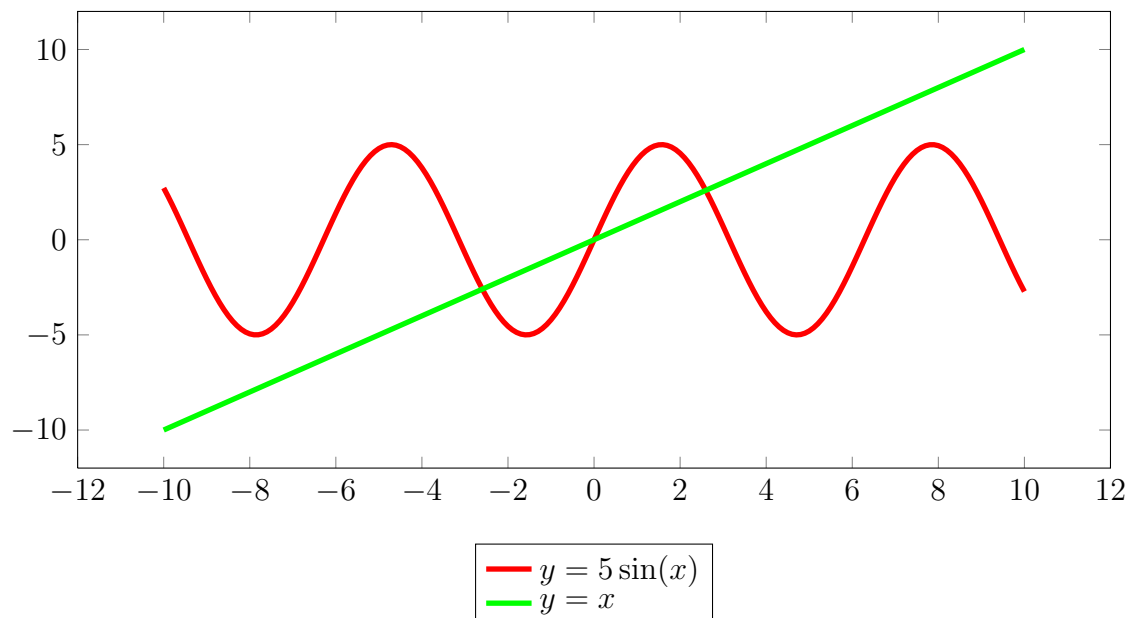
```
from latextool_basic import *
plot = FunctionPlot(width="3in", height="2in")
plot.add(((1, 1), (2, 2), (3, 0.5)), line_width='2', color='red', legend='G
raph 1')
plot.add(((1, 5), (2, 3), (3, 7)), line_width='2', color='green', legend='G
raph 2')
plot.add(((1, 5), (2, -3), (3, -5)), line_width='2', color='blue', legend='
Graph 3')
print(plot)
```



Legend:

```
from math import sin
domain = [x / 10.0 for x in range(-100, 101)]
y_sinx = [(x, 5 * sin(x)) for x in domain]
y_x = [(x, x) for x in domain]

from latextool_basic import *
plot = FunctionPlot(width="6in", height="3in")
plot.add(y_sinx, line_width='2', color='red', legend=r'$y = 5 \sin(x)$')
plot.add(y_x, line_width='2', color='green', legend='$y = x$')
print(plot)
```



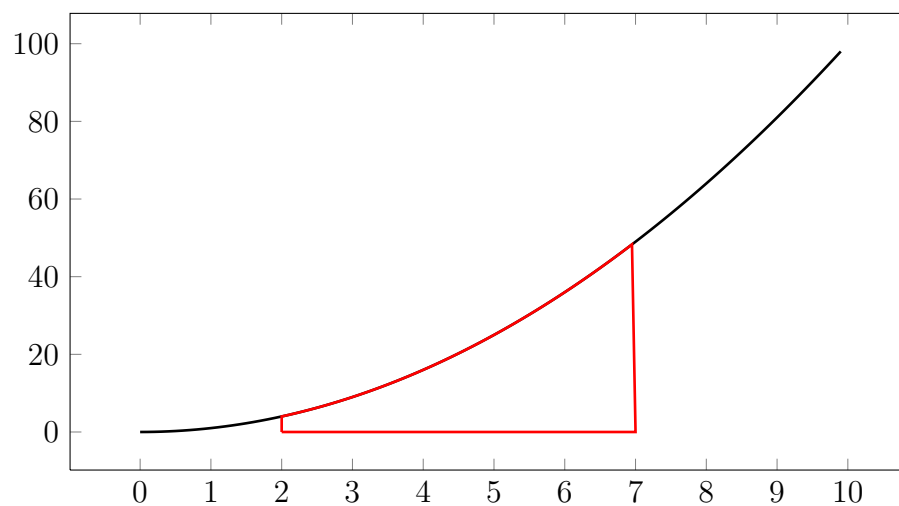
```

from latextool_basic import *

from math import sin
from latextool_basic import *
plot = FunctionPlot()

data = [(x/10.0, (x/10.0)**2) for x in range(100)]
data2 = [(2,0),(2,4)]
data2 += [(2 + x/100.0*5, (2 + x/100.0*5)**2) for x in range(100)]
data2 += [(7, 0), (2, 0)]
plot.add(data)
plot.add(data2, color='red')
print(plot)

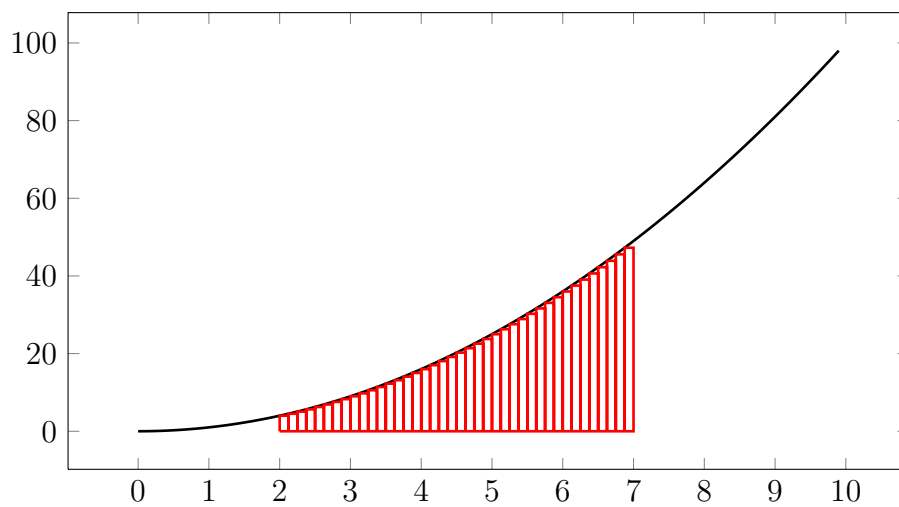
```



```
from latextool_basic import *

from math import sin
from latextool_basic import *
plot = FunctionPlot()

data = [(x/10.0, (x/10.0)**2) for x in range(100)]
data2 = []
x = 2
dx = 0.125
while x < 7:
    data2.append((x, 0))
    data2.append((x, x*x))
    data2.append((x + dx, x*x))
    data2.append((x + dx, 0))
    x += dx
data2.append((2,0))
plot.add(data)
plot.add(data2, color='red')
print(plot)
```



```

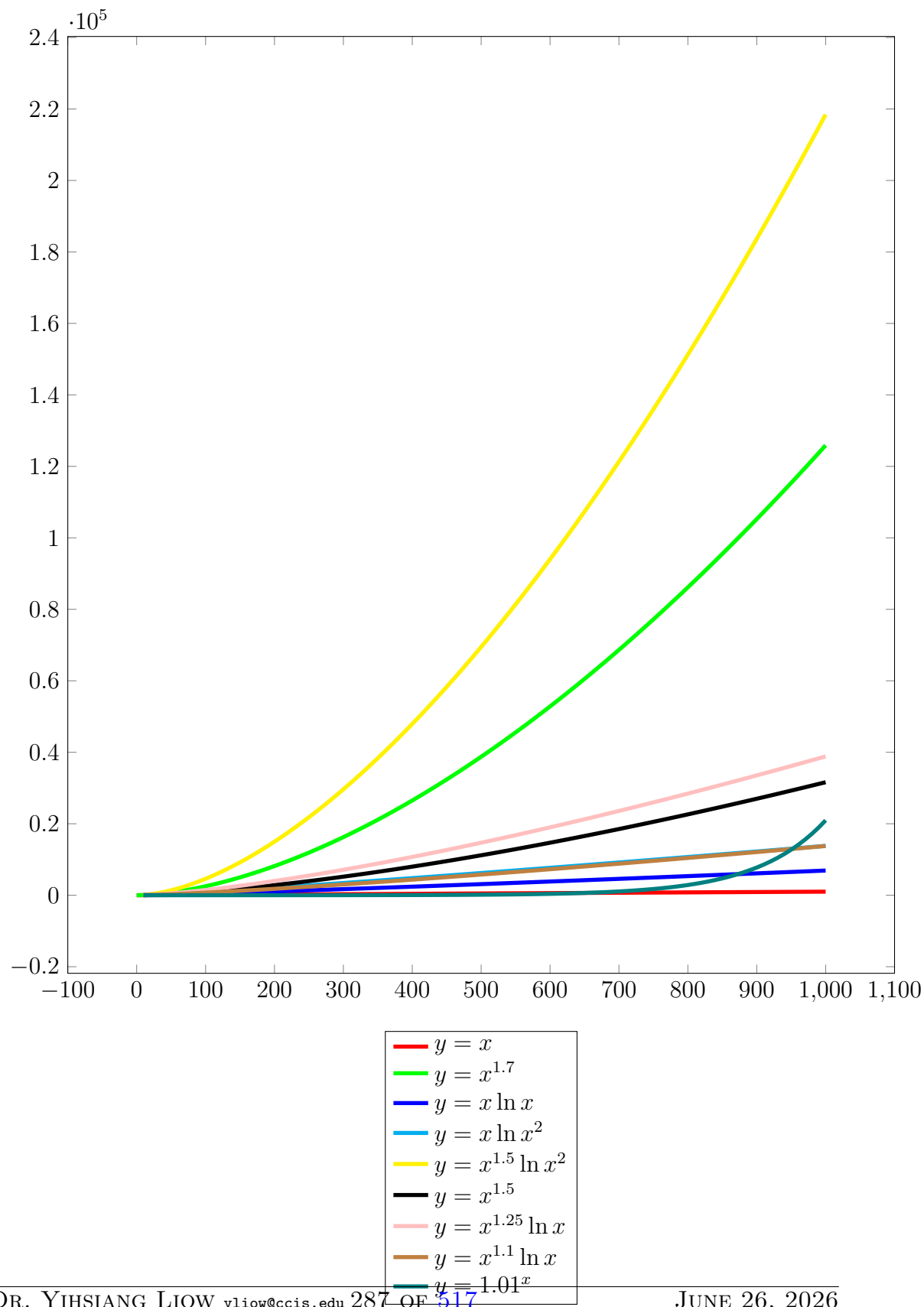
from latextool_basic import *
from math import log

domain = [float(x) for x in range(0, 1001, 10)]
f1 = [(x, x) for x in domain]
f2 = [(x, x**1.7) for x in domain]
f3 = [(x, log(x)) for x in [_ for _ in domain if _ > 0]]
f3 = [(x, x * log(x)) for x in [_ for _ in domain if _ > 0]]
f4 = [(x, x * log(x**2)) for x in [_ for _ in domain if _ > 0]]
f5 = [(x, x**1.5 * log(x)) for x in [_ for _ in domain if _ > 0]]
f6 = [(x, x**1.5) for x in [_ for _ in domain if _ > 0]]
f7 = [(x, x**1.25 * log(x)) for x in [_ for _ in domain if _ > 0]]
f8 = [(x, x**1.1 * log(x)) for x in [_ for _ in domain if _ > 0]]
f9 = [(x, 1.01**x) for x in [_ for _ in domain if _ > 0]]

plot = FunctionPlot(width="6.25in", height="7in")
plot.add(f1, line_width='2', color='red', legend=r'$y = x$')
plot.add(f2, line_width='2', color='green', legend=r'$y = x^{\{1.7\}}$')
plot.add(f3, line_width='2', color='blue', legend=r'$y = x \ln x$')
plot.add(f4, line_width='2', color='cyan', legend=r'$y = x \ln x^2$')
plot.add(f5, line_width='2', color='yellow', legend=r'$y = x^{\{1.5\}} \ln x^2$')
plot.add(f6, line_width='2', color='black', legend=r'$y = x^{\{1.5\}}$')
plot.add(f7, line_width='2', color='pink', legend=r'$y = x^{\{1.25\}} \ln x$')
plot.add(f8, line_width='2', color='brown', legend=r'$y = x^{\{1.1\}} \ln x$')
plot.add(f9, line_width='2', color='teal', legend=r'$y = 1.01^x$')

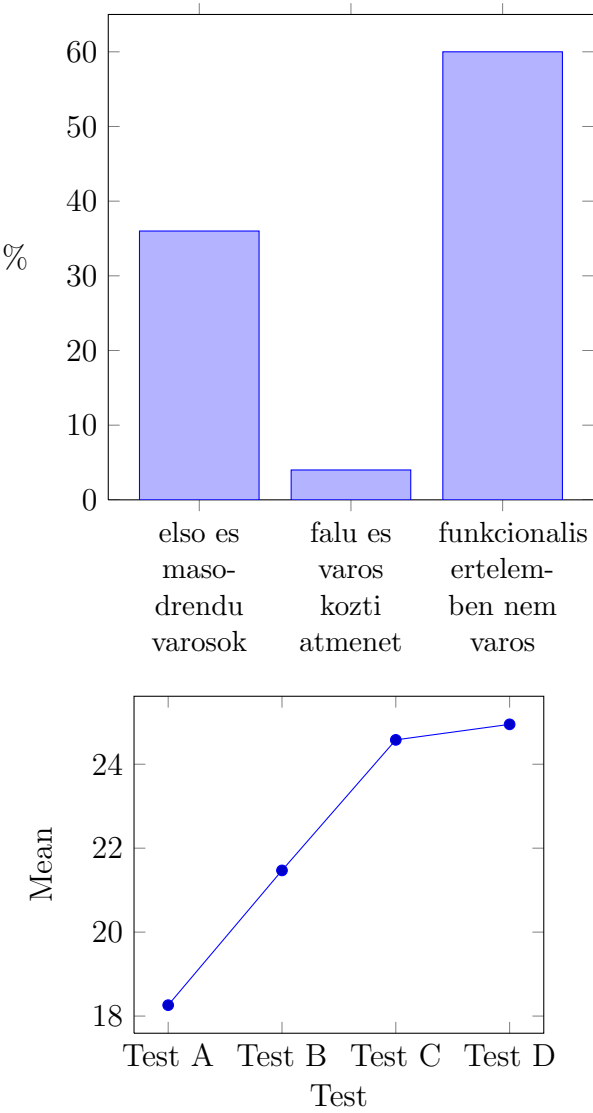
print(plot)

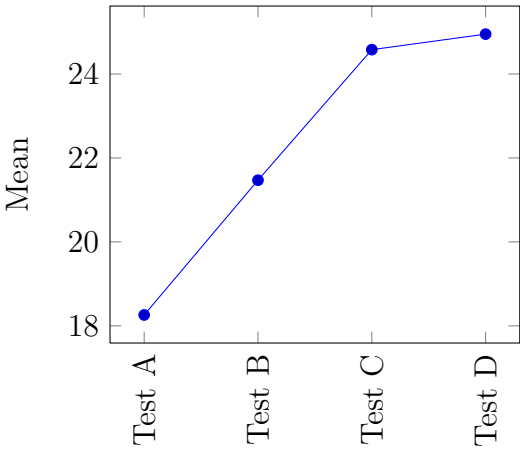
```



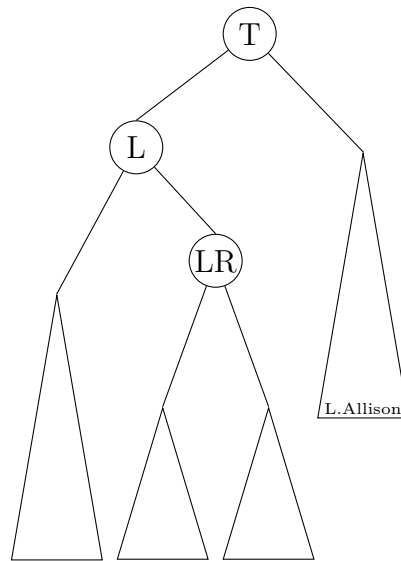
41 Function bar chart, line chart (line-graph-nonnumeric.tex)

42 Bar chart, line chart





43 Test isoceles triangle (isosceles-triangle.tex)



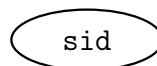
<http://tex.stackexchange.com/questions/7862/triangle-node-with-adjustable-height>

<http://tex.stackexchange.com/questions/37462/placing-a-triangle-around-nodes-in-a-t>

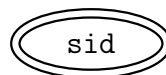
44 ER diagrams (er-diagram.tex)

44.1 Attribute

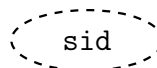
```
from latextool_basic import *; p = Plot()
p += ER.attrib(center=(0,0), name='sid', label=r'\texttt{sid}')
print(p)
```



```
from latextool_basic import *; p = Plot()
p += ER.attrib(center=(0,0), name='sid', label=r'\texttt{sid}',
               double=True)
print(p)
```

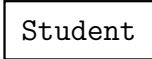


```
from latextool_basic import *; p = Plot()
p += ER.attrib(center=(0,0), name='sid', label=r'\texttt{sid}', linestyle='
dashed')
print(p)
```



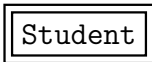
44.2 Entity

```
from latextool_basic import *; p = Plot()
p += ER.entity(center=(6,0), name='Student')
print(p)
```



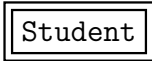
Entity with double lines:

```
from latextool_basic import *; p = Plot()
p += ER.entity(center=(6,0), name='Student', double=True)
print(p)
```



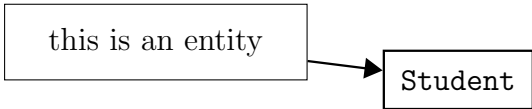
Entity with double lines:

```
from latextool_basic import *; p = Plot()
p += ER.entity(center=(6,0), name='Student', double=['Student'])
print(p)
```



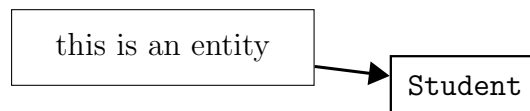
Entity name is a tikz node name:

```
from latextool_basic import *; p = Plot()
p += ER.entity(center=(6,0), name='Student')
p += Rect(0,0,4,1,name='r',label='this is an entity')
p += ER.line(names=['r', 'Student'], endstyle='>')
print(p)
```



Entity name can be difference from label:

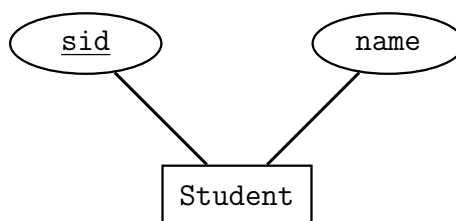
```
from latextool_basic import *; p = Plot()
p += ER.entity(center=(6,0), name='s', label='Student')
p += Rect(0,0,4,1,name='r',label='this is an entity')
p += Line(names=['r', 's'], endstyle='>')
print(p)
```



```
from latextool_basic import *
p = Plot()

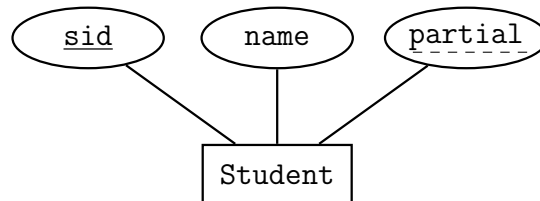
p += ER.attrib(center=(4,2), name='sid', label=r'\texttt{\underline{sid}}')
p += ER.attrib(center=(8,2), name='name', label=r'\texttt{name}')
p += ER.entity(center=(6,0), name='Student', label=r'\texttt{Student}')

p += Line(names=['sid', 'Student'])
p += Line(names=['name', 'Student'])
print(p)
```

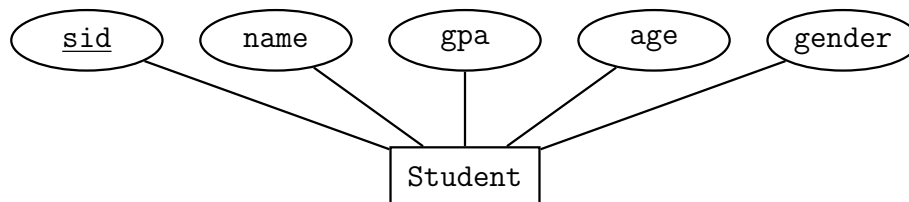


Entity with attributes (primary key and partial key):

```
from latextool_basic import *; p = Plot()
p += ER.entity(center=(0,0),
               name='Student',
               attrs=['sid', 'name', 'partial'], keys=['sid'], dasheds=['partial'])
print(p)
```

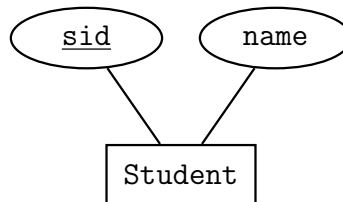


```
from latextool_basic import *; p = Plot()
p += ER.entity(center=(0,0),
               name='Student',
               attrs=['sid', 'name', 'gpa', 'age', 'gender'], keys=['sid'])
print(p)
```

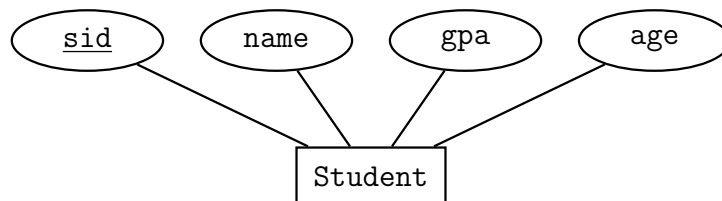


Entity with attribs on north (even number):

```
from latextool_basic import *; p = Plot()
p += ER.entity(center=(0,0),
               name='Student',
               attribs=['sid', 'name'], keys=['sid'])
print(p)
```

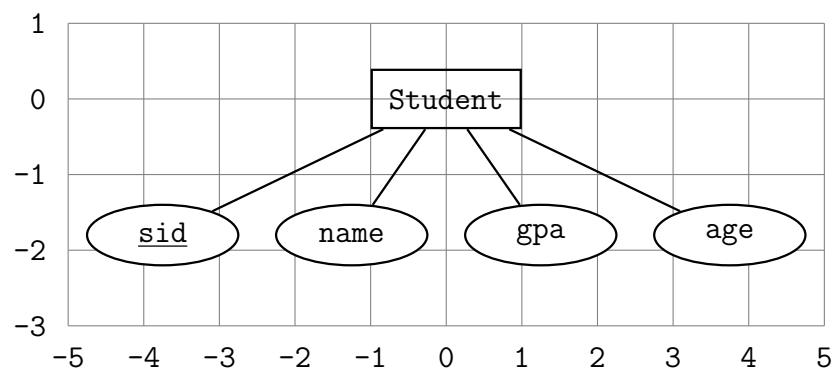


```
from latextool_basic import *; p = Plot()
p += ER.entity(center=(0,0),
               name='Student',
               attribs=['sid', 'name', 'gpa', 'age'], keys=['sid'])
print(p)
```



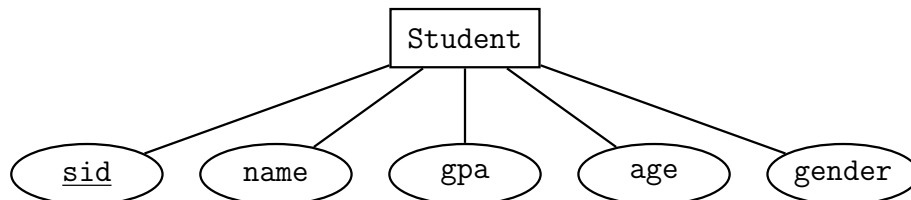
Even number of attributes on south:

```
from latextool_basic import *
p = Plot()
p += ER.entity(center=(0,0),
               name='Student',
               attrs=['sid', 'name', 'gpa', 'age'],
               keys=['sid'],
               anchor={'sid':'south', 'name':'south', 'gpa':'south', 'age':'south'
})
p += Grid(-5,-3,5,1)
print(p)
```

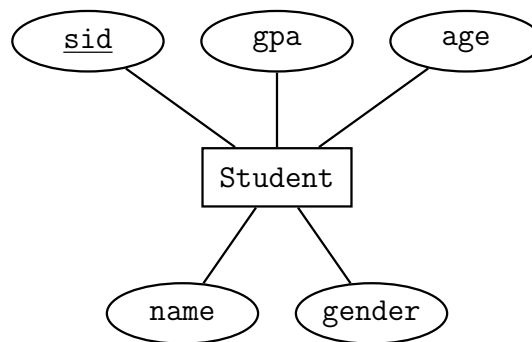


Odd number of attributes on south:

```
from latextool_basic import *
p = Plot()
p += ER.entity(center=(0,0),
               name='Student',
               attrs=['sid', 'name', 'gpa', 'age', 'gender'],
               keys=['sid'],
               anchor={'sid':'south', 'name':'south', 'gpa':'south', 'age':'south'
, 'gender':'south'})
print(p)
```

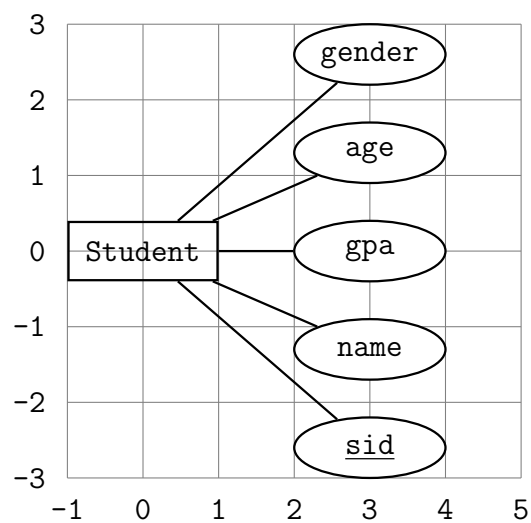


```
from latextool_basic import *
p = Plot()
p += ER.entity(center=(0,0),
               name='Student',
               attribs=['sid', 'name', 'gpa', 'age', 'gender'],
               keys=['sid'],
               anchor={'name':'south', 'gender':'south'})
print(p)
```



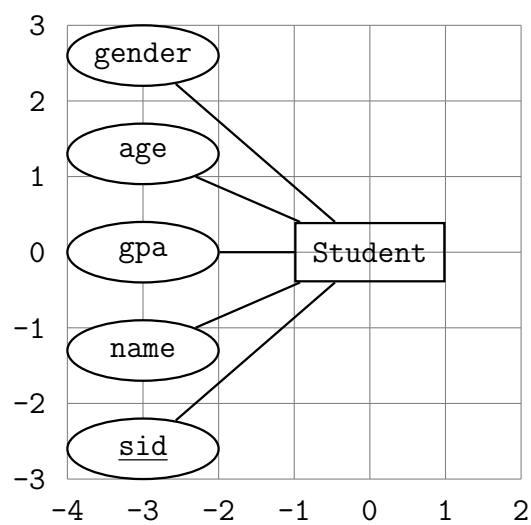
Attributes on east:

```
from latextool_basic import *
p = Plot()
p += ER.entity(center=(0,0),
               name='Student',
               attribs=['sid', 'name', 'gpa', 'age', 'gender'],
               keys=['sid'],
               anchor={'sid':'east', 'name':'east', 'gpa':'east',
                     'age':'east',
                     'gender':'east'})
p += Grid(-1, -3, 5, 3)
print(p)
```



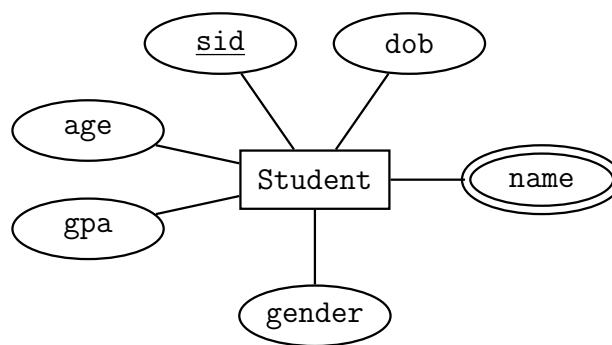
Attributes on west:

```
from latextool_basic import *
p = Plot()
p += ER.entity(center=(0,0),
               name='Student',
               attribs=['sid', 'name', 'gpa', 'age', 'gender'],
               keys=['sid'],
               anchor={'sid':'west', 'name':'west', 'gpa':'west',
                     'age':'west', 'gender':'west'})
p += Grid(-4, -3, 2, 3)
print(p)
```



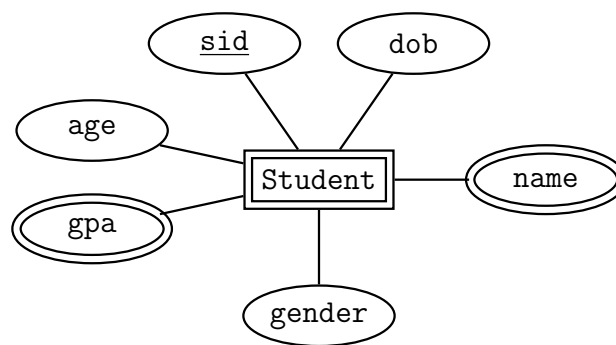
All directions:

```
from latextool_basic import *
p = Plot()
p += ER.entity(center=(0,0),
               name='Student',
               attribs=['sid', 'name', 'gpa', 'age', 'gender', 'dob'],
               keys=['sid'],
               double=['name'],
               anchor={'name':'east', 'gpa':'west',
                     'age':'west', 'gender':'south'})
print(p)
```



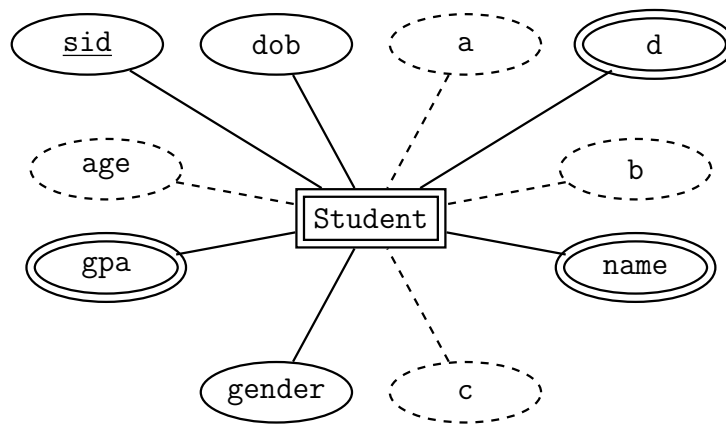
All directions and double:

```
from latextool_basic import *
p = Plot()
p += ER.entity(center=(0,0),
               name='Student',
               attribs=['sid', 'name', 'gpa', 'age', 'gender', 'dob'],
               keys=['sid'],
               double=['name', 'gpa', 'Student'],
               anchor={'name':'east', 'gpa':'west',
                      'age':'west', 'gender':'south'})
print(p)
```



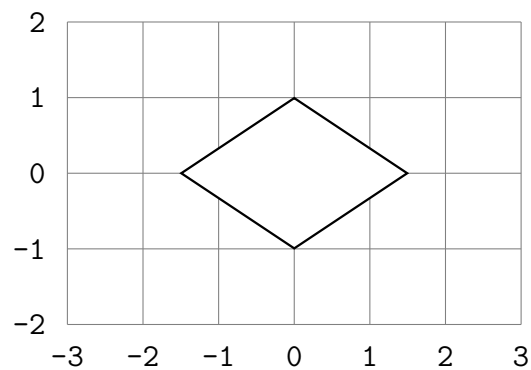
All directions and double and derived

```
from latextool_basic import *
p = Plot()
ER.attrib_dist = 1.5
p += ER.entity(center=(0,0),
               name='Student',
               attrs=['sid', 'name', 'gpa', 'age', 'gender', 'dob', 'a', 'b',
                    'c', 'd'],
               keys=['sid'],
               derived=['age', 'a', 'b', 'c'],
               double=['name', 'gpa', 'Student', 'd'],
               anchor={'name':'east', 'gpa':'west', 'b':'east', 'c':'south',
                      'age':'west', 'gender':'south'})
print(p)
```

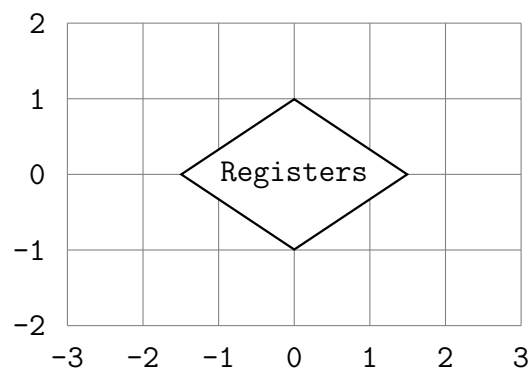


44.3 Relation

```
from latextool_basic import *
p = Plot()
p += ER.relation(center=(0,0), name='Registers', label='')
p += Grid(-3, -2, 3, 2)
print(p)
```



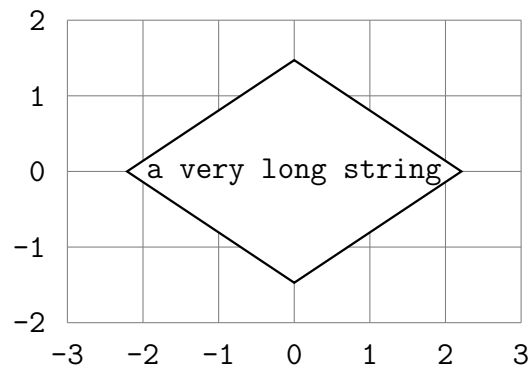
```
from latextool_basic import *
p = Plot()
p += ER.relation(center=(0,0), name='Registers', label='Registers')
p += Grid(-3, -2, 3, 2)
print(p)
```




```

from latextool_basic import *
p = Plot()
p += ER.relation(center=(0,0), name='Registers', label='a very long string'
)
p += Grid(-3, -2, 3, 2)
print(p)

```



```

from latextool_basic import *
p = Plot()
p += ER.relation(center=(4,0), double=True,
                  name='Registers', label='Registers')
print(p)

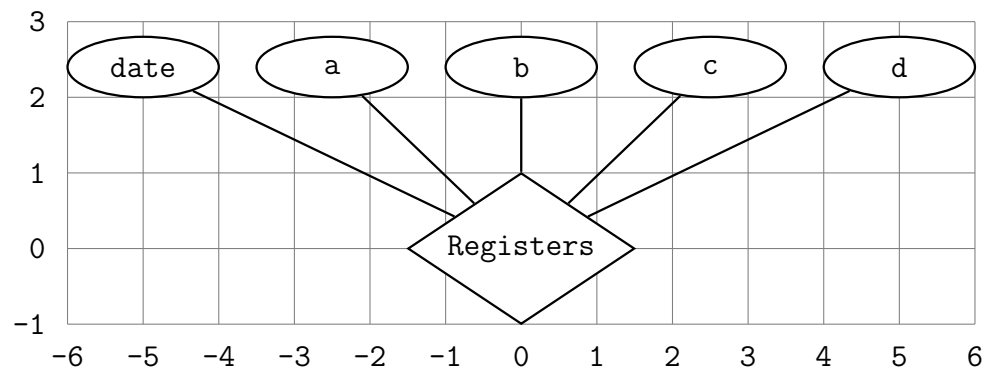
```



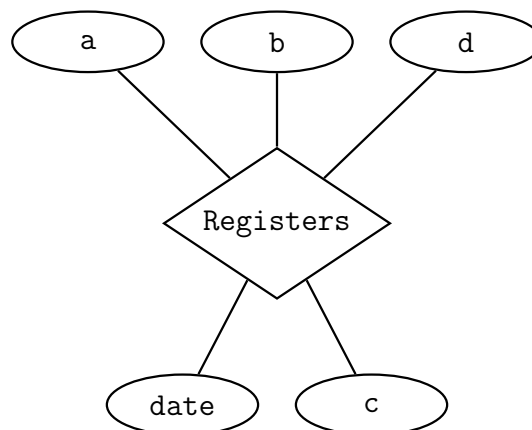
```

from latextool_basic import *
p = Plot()
p += ER.relation(center=(0,0),
                  name='Registers',
                  attribs=['date','a','b','c','d'])
p += Grid(-6, -1, 6, 3)
print(p)

```

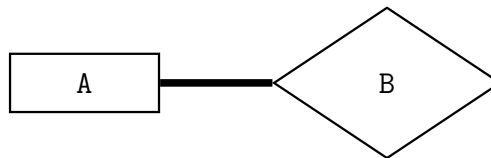


```
from latextool_basic import *
p = Plot()
p += ER.relation(center=(4,0),
                 name='Registers',
                 attribs=['date','a','b','c','d'],
                 anchor={'date':'south', 'c':'south'})
print(p)
```

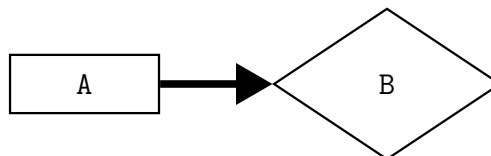


44.4 Entity and relation with different arrows

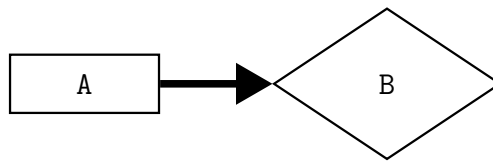
```
from latextool_basic import *
p = Plot()
p += ER.entity(center=(0,0), name='A')
p += ER.relation(center=(4,0), name='B')
p += ER.line(names=['A', 'B'],
             linewidth=ER.boldlinewidth)
print(p)
```



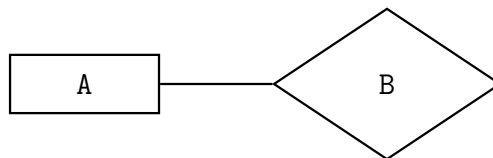
```
from latextool_basic import *
p = Plot()
p += ER.entity(center=(0,0), name='A')
p += ER.relation(center=(4,0), name='B')
p += ER.line(names=['A', 'B'],
             linewidth=ER.boldlinewidth, endstyle='>')
print(p)
```



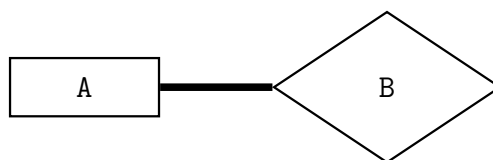
```
from latextool_basic import *
p = Plot()
p += ER.entity(center=(0,0), name='A')
p += ER.relation(center=(4,0), name='B')
p += ER.line(names=['A', 'B'],
             linewidth=ER.boldlinewidth, endstyle='>',
             arrowstyle='triangle')
print(p)
```



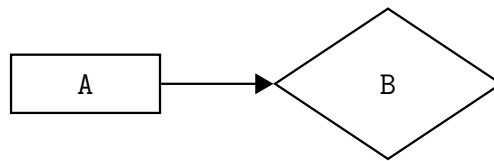
```
from latextool_basic import *
p = Plot()
p += ER.entity(center=(0,0), name='A')
p += ER.relation(center=(4,0), name='B')
p += ER.edge(names=['A', 'B'])
print(p)
```



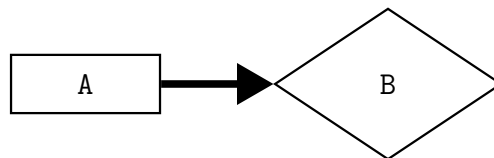
```
from latextool_basic import *
p = Plot()
p += ER.entity(center=(0,0), name='A')
p += ER.relation(center=(4,0), name='B')
p += ER.boldedge(names=['A', 'B'])
print(p)
```



```
from latextool_basic import *
p = Plot()
p += ER.entity(center=(0,0), name='A')
p += ER.relation(center=(4,0), name='B')
p += ER.arc(names=['A', 'B'])
print(p)
```



```
from latextool_basic import *  
p = Plot()  
p += ER.entity(center=(0,0), name='A')  
p += ER.relation(center=(4,0), name='B')  
p += ER.boldarc(names=['A', 'B'])  
print(p)
```

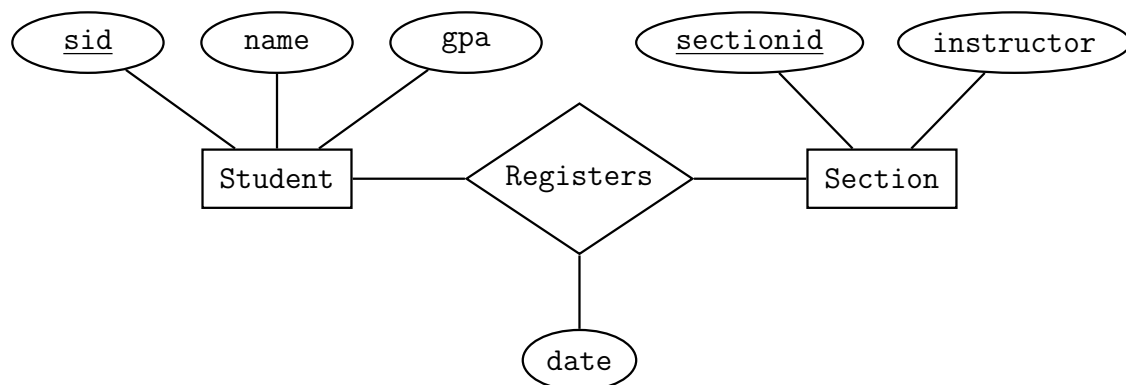


44.5 Entity and relation with attributes

```

from latextool_basic import *
p = Plot()
p += ER.entity(center=(0,0),
               name='Student',
               attrs=['sid', 'name', 'gpa'], keys=['sid'])
ER.attrb_width = 3
p += ER.entity(center=(8,0),
               name='Section',
               attrs=['sectionid', 'instructor'], keys=['sectionid'])
ER.attrb_width = 1.5
p += ER.relation(center=(4,0), name='Registers',
                 attrs=['date'],
                 anchor={'date': 'south'})
p += ER.line(names=['Student', 'Registers'])
p += ER.line(names=['Section', 'Registers'])
print(p)

```



44.6 Weak entity sets and partial keys

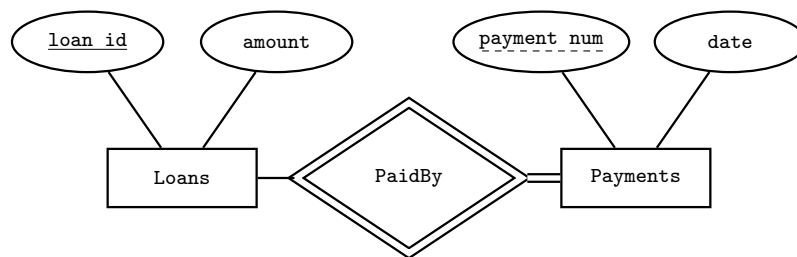
```

from latextool_basic import *
p = Plot()
p += ER.entity(center=(0,0), name='Loans',
               attrs=['loan id', 'amount'], keys=['loan id'])
p += ER.relation(center=(3,0), double=True, name='PaidBy')
p += ER.entity(center=(6,0), name='Payments',
               attrs=['payment num', 'date'],
               dashed=['payment num'])
)

p += ER.line(names=['Loans', 'PaidBy'])
p += ER.line(names=['Payments', 'PaidBy'], linestyle='double', double distance=0.7mm, shorten >= 0.5mm)

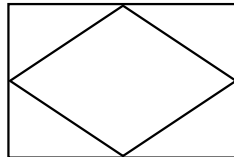
print(r'\scriptsize %s' % p)

```

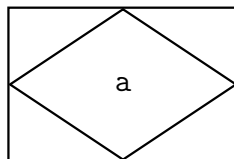


44.7 Associative entity

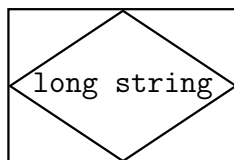
```
from latextool_basic import *
p = Plot()
p += ER.associative_entity_only(center=(0,0), name='a')
print(p)
```



```
from latextool_basic import *
p = Plot()
p += ER.associative_entity_only(center=(0,0), name='a', label='a')
print(p)
```



```
from latextool_basic import *
p = Plot()
p += ER.associative_entity_only(center=(0,0), name='a', label='long string'
)
print(p)
```



44.8 Crow's foot

```

from latextool_basic import *
p = Plot()
ER.attrib_width=0.9
ER.attrib_height=0.5
ER.attrib_dist=0.4
ER.attrib_sep=0.4
ER.entity_width=1.8
ER.entity_height=0.5
ER.relation_width=1.8
ER.relation_height=0.5

p += ER.entity(center=(0,0), name='A', attribs=[], keys=[])
p += ER.relation(center=(3,0), name='R', attribs=[], label=r'\scriptsize R')
p += ER.entity(center=(6,0), name='B', attribs=[], keys=[])
p += ER.line(names=['B', 'R'])
p += ER.line(names=['A', 'R'])
crowfoot(p, x=3 + 0.9, y=0, kind="0..1", direction="west")
crowfoot(p, x=6 - 0.9, y=0, kind="1", direction="east")

# Test "west"
p += Line(points=[(0,-1),(2, -1)])
crowfoot(p, x=0, y=-1, kind="1", direction="west")

p += Line(points=[(0,-1.5),(2, -1.5)])
crowfoot(p, x=0, y=-1.5, kind="0..1", direction="west")

p += Line(points=[(0,-2),(2, -2)])
crowfoot(p, x=0, y=-2, kind="*", direction="west")

p += Line(points=[(0,-2.5),(2, -2.5)])
crowfoot(p, x=0, y=-2.5, kind="1..*", direction="west")

# Test "east"
p += Line(points=[(3,-1),(5, -1)])
crowfoot(p, x=5, y=-1, kind="1", direction="east")

p += Line(points=[(3,-1.5),(5, -1.5)])
crowfoot(p, x=5, y=-1.5, kind="0..1", direction="east")

p += Line(points=[(3,-2),(5, -2)])
crowfoot(p, x=5, y=-2, kind="*", direction="east")

p += Line(points=[(3,-2.5),(5, -2.5)])
crowfoot(p, x=5, y=-2.5, kind="1..*", direction="east")

# Test changing dx
x = 0

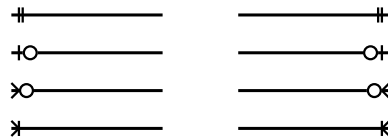
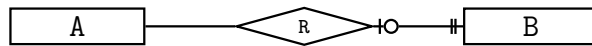
```

```

y = -4
p += Line(points=[(x,y),(x + 2, y)])
crowfoot(p, x=x+2, y=y, dx=0.25, kind="1", direction="east")
x += 3
p += Line(points=[(x, y),(x + 2, y)])
crowfoot(p, x=x+2, y=y, dx=0.25, kind="0..1", direction="east")
x += 3
p += Line(points=[(x, y),(x + 2, y)])
crowfoot(p, x=x+2, y=y, dx=0.25, kind="*", direction="east")
x += 3
p += Line(points=[(x, y),(x + 2, y)])
crowfoot(p, x=x+2, y=y, dx=0.25, kind="1..*", direction="east")

print(p)

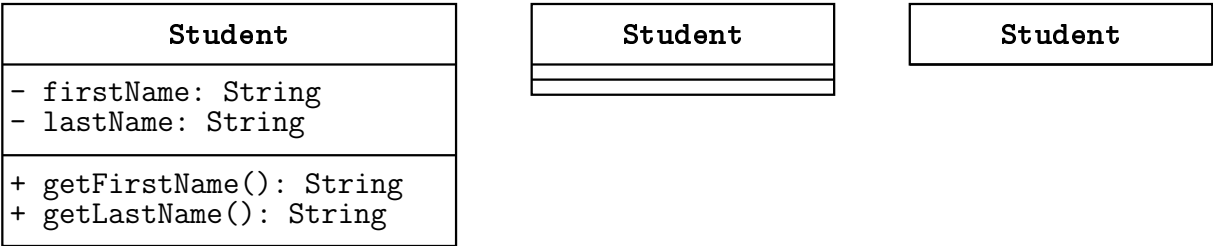
```



45 UML diagrams (chap-uml.tex)

45.1 UML: class diagram (uml-classdiagram.tex)

```
from latextool_basic import *
p = Plot()
uml_class(p, 1, 1,
          classname='Student',
          attributes=['- firstName:\ String',
                     '- lastName:\ String'],
          methods=['+ getFirstName():\ String',
                  '+ getLastName():\ String'])
uml_class(p, 8, 1,
          width=4,
          classname='Student')
uml_class(p, 13, 1,
          width=4,
          classname='Student',
          showempty=False)
print(p)
```



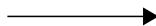
Relevant arrow tips:

```
from latextool_basic import *
p = Plot()

#p += r'\draw[->>] (1,-5) -- (10,-5);'
#p += r'\draw[->] (1,-5.5) -- (10,-5.5);'
#p += r'\draw[-diamond] (1,-6) -- (10,-6);'
#p += r'\draw[-open diamond] (1,-6.5) -- (10,-6.5);'
#p += r'\draw[-open triangle 45] (1,-7) -- (10,-7);'

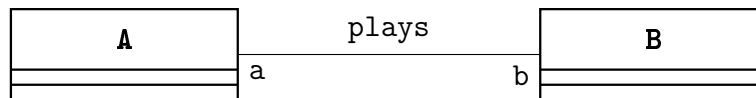
p += Line(points=[(0,0),(2,0)], endstyle='diamond', linewidth='0.02')
p += Line(points=[(0,-1),(2,-1)], endstyle='open diamond', linewidth='0.02')
p += Line(points=[(0,-2),(2,-2)], endstyle='>', linewidth='0.02')
p += Line(points=[(0,-3),(2,-3)], endstyle='>>', linewidth='0.02')
p += Line(points=[(0,-4),(2,-4)], endstyle='open triangle 45', linewidth='0.02')

print(p)
```



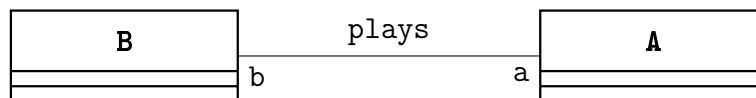
Association. Go east/right:

```
from latextool_basic import *
p = Plot()
A = uml_class(p, 0, 0, width=3, classname='A')
B = uml_class(p, 7, 0, width=3, classname='B')
association(p, p0=A.right(), p1=B.left(),
            s='plays', c0='a', c1='b',
            layout='e')
print(p)
```



Association. Go west/left:

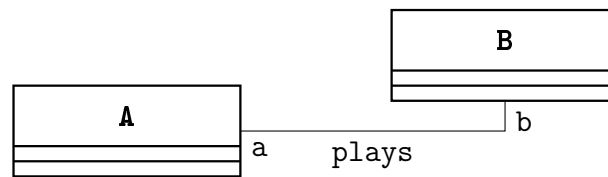
```
from latextool_basic import *
p = Plot()
A = uml_class(p, 7, 0, width=3, classname='A')
B = uml_class(p, 0, 0, width=3, classname='B')
association(p, p0=A.left(), p1=B.right(),
            s='plays', c0='a', c1='b',
            layout='w')
print(p)
```



Association. Go south/down:

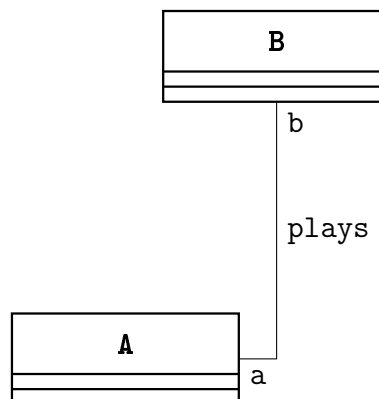
Go right-up (long-short):

```
from latextool_basic import *
p = Plot()
A = uml_class(p, 0, 0, width=3, classname='A')
B = uml_class(p, 5, 1, width=3, classname='B')
p0 = A.right()
p1 = B.bottom()
association(p, p0=p0, p1=p1, s='plays', c0='a', c1='b',
            moves=[('x',p1),('y',p1)])
print(p)
```



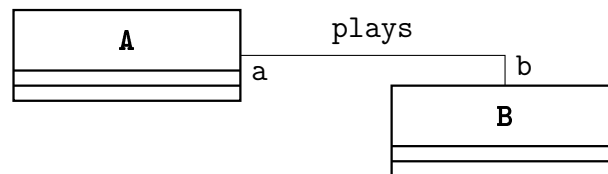
Go right-up (short-long):

```
from latextool_basic import *
p = Plot()
A = uml_class(p, 0, 0, width=3, classname='A')
B = uml_class(p, 2, 4, width=3, classname='B')
p0 = A.right()
p1 = B.bottom()
association(p, p0=p0, p1=p1, s='plays', c0='a', c1='b',
            moves=[('x',p1),('y',p1)])
print(p)
```



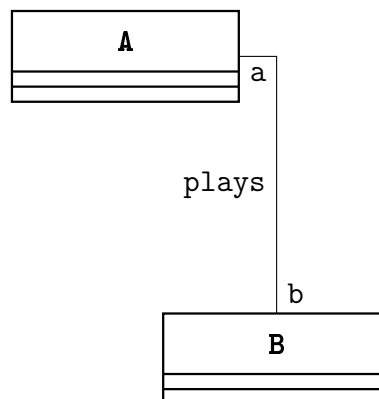
Go right-down (long-short):

```
from latextool_basic import *
p = Plot()
A = uml_class(p, 0, 0, width=3, classname='A')
B = uml_class(p, 5, -1, width=3, classname='B')
p0 = A.right()
p1 = B.top()
association(p, p0=p0, p1=p1, s='plays', c0='a', c1='b',
            moves=[('x',p1),('y',p1)])
print(p)
```



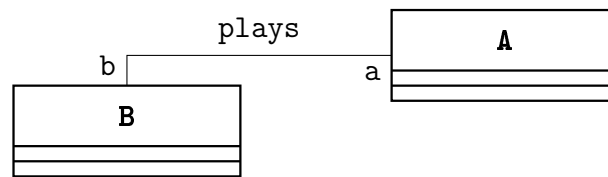
Go right-down (short-long):

```
from latextool_basic import *
p = Plot()
A = uml_class(p, 0, 0, width=3, classname='A')
B = uml_class(p, 2, -4, width=3, classname='B')
p0 = A.right()
p1 = B.top()
association(p, p0=p0, p1=p1, s='plays', c0='a', c1='b',
            moves=[('x',p1),('y',p1)])
print(p)
```



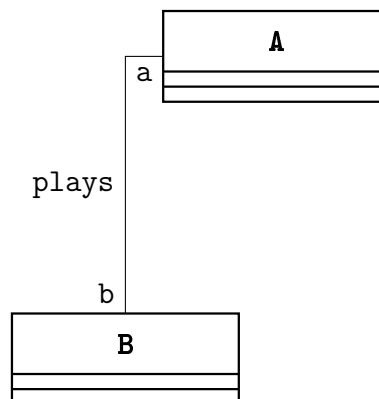
Go left-down (long-short):

```
from latextool_basic import *
p = Plot()
A = uml_class(p, 0, 0, width=3, classname='A')
B = uml_class(p, -5, -1, width=3, classname='B')
p0 = A.left()
p1 = B.top()
association(p, p0=p0, p1=p1, s='plays', c0='a', c1='b',
            moves=[('x',p1),('y',p1)])
print(p)
```



Go left-down (short-long):

```
from latextool_basic import *
p = Plot()
A = uml_class(p, 0, 0, width=3, classname='A')
B = uml_class(p, -2, -4, width=3, classname='B')
p0 = A.left()
p1 = B.top()
association(p, p0=p0, p1=p1, s='plays', c0='a', c1='b',
            moves=[('x',p1),('y',p1)])
print(p)
```

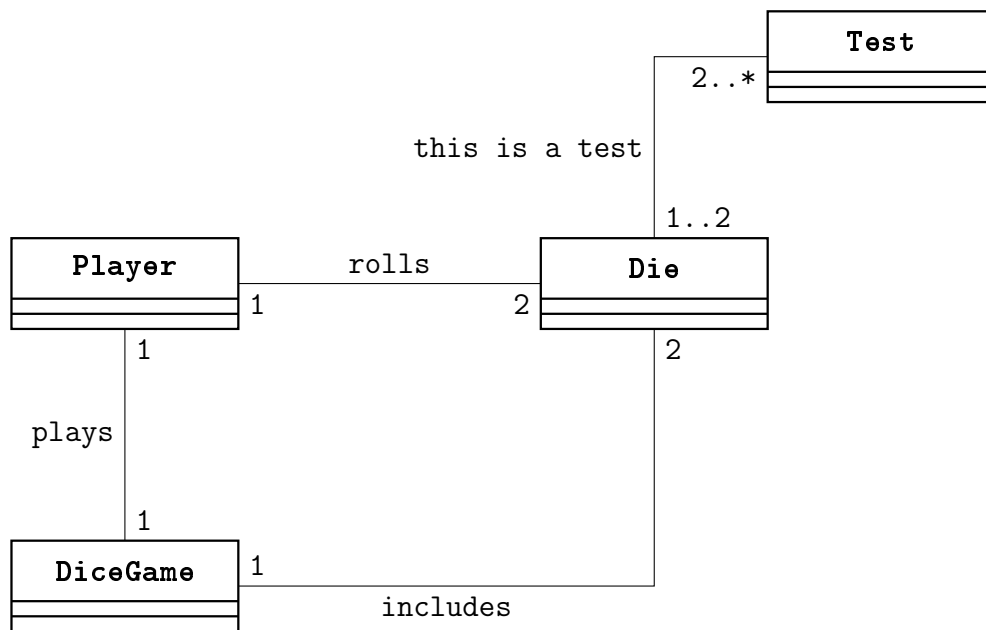


```

from latextool_basic import *
p = Plot()
player = uml_class(p, 0, 0, width=3, classname='Player')
die = uml_class(p, 7, 0, width=3, classname='Die')
dicegame = uml_class(p, 0, -4, width=3, classname='DiceGame')
test = uml_class(p, 10, 3, width=3, classname='Test')

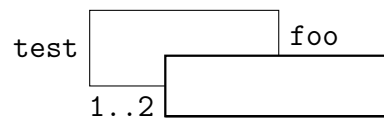
association(p, p0=player.bottom(), p1=dicegame.top(), s='plays',
            c0='1', c1='1', layout='n')
association(p, p0=player.right(), p1=die.left(), s='rolls',
            c0='1', c1='2', layout='e')
association(p, p0=dicegame.right(), p1=die.bottom(), s='includes',
            c0='1', c1='2', layout='en')
association(p, p0=die.top(), p1=test.left(), s='this is a test',
            c0='1..2', c1='2..*', layout='ne')
print(p)

```

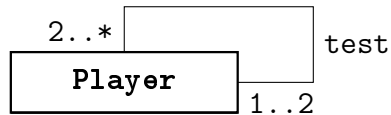


Reflexive associations:

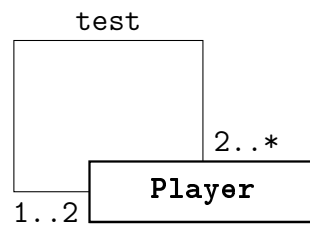
```
from latextool_basic import *
p = Plot()
player = uml_class(p, 0, 0, width=3, classname='', showempty=False)
p0 = player.left()
p1 = player.top()
association(p, p0=p0, p1=p1, s='test',
            c0='1..2', c1='foo', moves=['w', 'n', ('x', p1), ('y', p1)])
print(p)
```



```
from latextool_basic import *
p = Plot()
player = uml_class(p, 0, 0, width=3, classname='Player', showempty=False)
p0 = player.right()
p1 = player.top()
association(p, p0=p0, p1=p1, s='test',
            c0='1..2', c1='2..*', moves=['e', 'n', ('x', p1), ('y', p1)])
print(p)
```



```
from latextool_basic import *
p = Plot()
player = uml_class(p, 0, 0, width=3, classname='Player', showempty=False)
association(p, p0=player.left(), p1=player.top(), s='test',
            c0='1..2', c1='2..*', layout='wnes',
            dn=2)
print(p)
```



```

from latextool_basic import *
p = Plot()
person = uml_class(p, 0, 0, width=3, classname='Person', showempty=False)
room = uml_class(p, 5, 0, width=3, classname='Room', showempty=False)
knife = uml_class(p, 2.5, -2, width=3, classname='Knife', showempty=False)

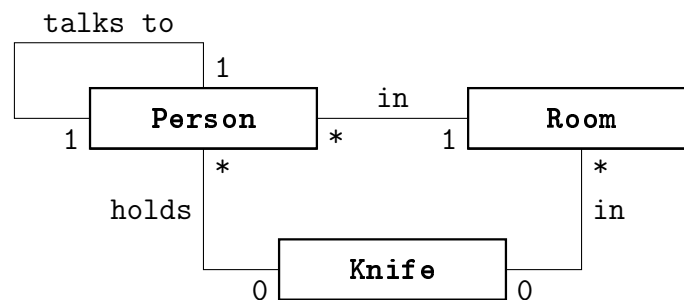
association(p, p0=person.right(), p1=room.left(), s='in',
            c0='*', c1='1', layout='e')
association(p, p0=person.left(), p1=person.top(), s='talks to',
            c0='1', c1='1', layout='wnes')

p0 = knife.right()
p1 = room.bottom()
association(p, p0=p0, p1=p1, s='in',
            c0='0', c1='*', moves=[('x',p1),('y',p1)])

p0 = person.bottom()
p1 = knife.left()
association(p, p0=p0, p1=p1, s='holds',
            c0='*', c1='0', moves=[('y', p1), ('x', p1)])

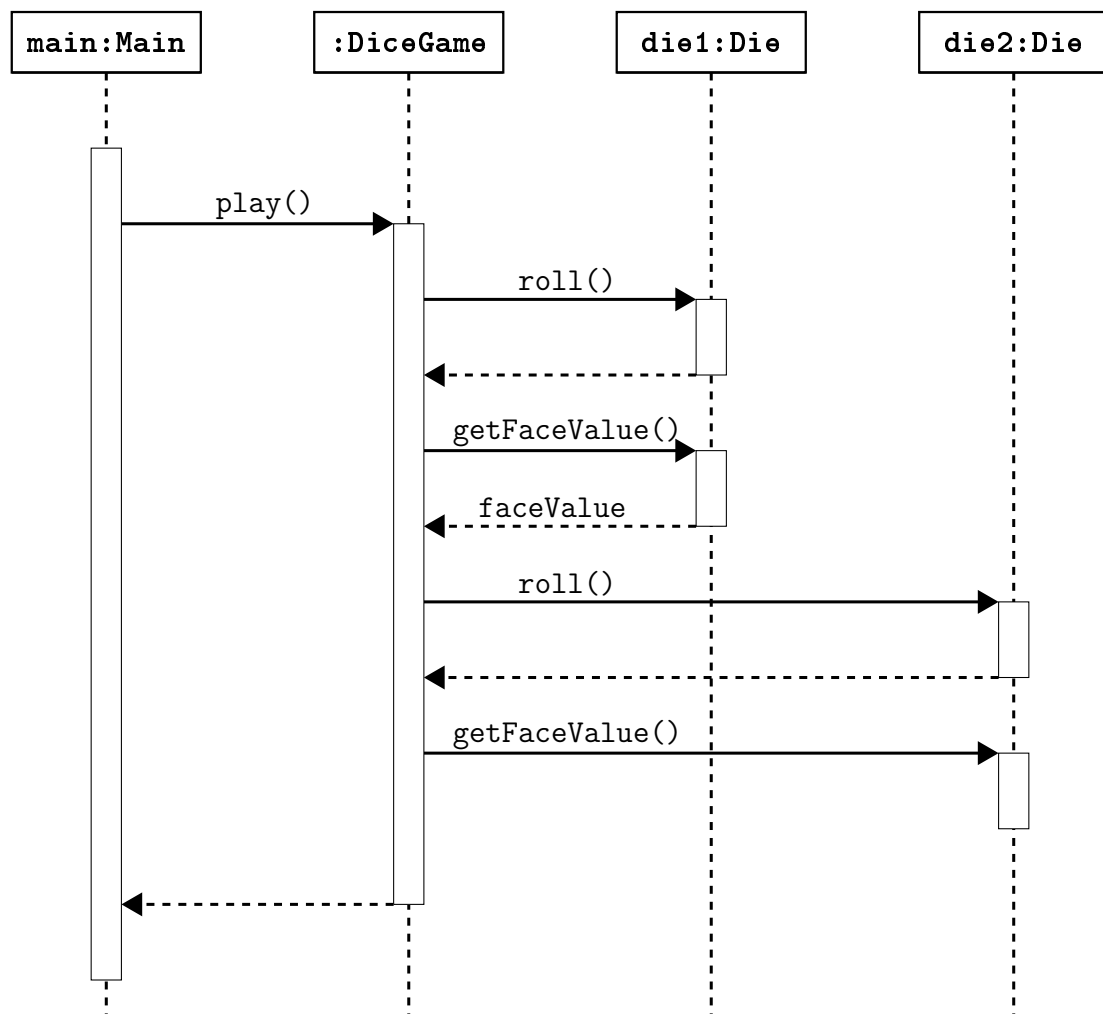
print(p)

```



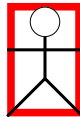
45.3 UML: sequence diagram

DR. YIHSIANG LIOW yliow@ccis.edu 328 OF 517 JUNE 26, 2026

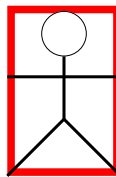


45.4 UML: use case diagram (uml-usecasediagram.tex)

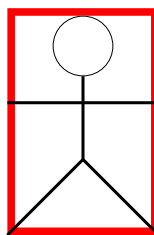
```
from latextool_basic import *
p = Plot()
uml_usecase_man(p)
print(p)
```



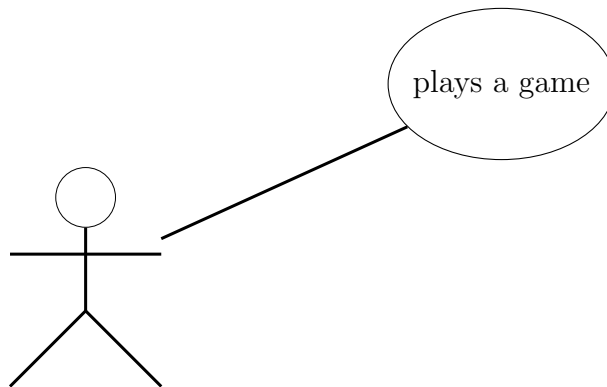
```
from latextool_basic import *
p = Plot()
uml_usecase_man(p, x0=0, y0=0, w=1.5)
print(p)
```



```
from latextool_basic import *
p = Plot()
uml_usecase_man(p, x0=0, y0=0, w=2)
print(p)
```



```
from latextool_basic import *
p = Plot()
rect = uml_usecase_man(p, x0=0, y0=0, w=2, name='a', linecolor='white')
p += ellipse(x0=5, y0=3, x1=8, y1=5, name='b')
p += Rect(x0=5, y0=3, x1=8, y1=5, label='plays a game',
          linewidth=0)
p += Line(names=['a', 'b'])
print(p)
```



46 Matrix (matrix.tex)

```
from latextool_basic import *
m = Matrix([[1,2],[3,4]])
print(r'\[ %s \]' % latex_bmatrix(m))
```

$$\begin{bmatrix} 1 & 2 \\ 3 & 4 \end{bmatrix}$$

```
from latextool_basic import *

m = Matrix([[1,2],[3,4]])
n = Matrix([[1,-2],[3,4]])
p = m*n

print("\[%s %s = %s \]" % \
(latex_bmatrix(m), latex_bmatrix(n), latex_bmatrix(p)))
```

$$\begin{bmatrix} 1 & 2 \\ 3 & 4 \end{bmatrix} \begin{bmatrix} 1 & -2 \\ 3 & 4 \end{bmatrix} = \begin{bmatrix} 7 & 6 \\ 15 & 10 \end{bmatrix}$$

```
from latextool_basic import *  
  
m = Matrix([[1,2],[3,4]])  
n = Matrix([[1,-2],[3,4]])  
  
print(latex_mult(m,n))
```

$$\begin{aligned} & \begin{bmatrix} 1 & 2 \\ 3 & 4 \end{bmatrix} \begin{bmatrix} 1 & -2 \\ 3 & 4 \end{bmatrix} \\ &= \begin{bmatrix} (1)(1) + (2)(3) & (1)(-2) + (2)(4) \\ (3)(1) + (4)(3) & (3)(-2) + (4)(4) \end{bmatrix} \\ &= \begin{bmatrix} 1+6 & (-2)+8 \\ 3+12 & (-6)+16 \end{bmatrix} \\ &= \begin{bmatrix} 7 & 6 \\ 15 & 10 \end{bmatrix} \end{aligned}$$

```
from latextool_basic import *  
m = Matrix([[1,2],[3,4]])  
print(latex_inverse(m))
```

$$\begin{aligned} \begin{bmatrix} 1 & 2 & 1 & 0 \\ 3 & 4 & 0 & 1 \end{bmatrix} &\xrightarrow{R_2 \rightarrow R_2 + (-3) R_1} \begin{bmatrix} 1 & 2 & 1 & 0 \\ 0 & -2 & -3 & 1 \end{bmatrix} \\ &\xrightarrow{R_2 \rightarrow (-0.5) R_2} \begin{bmatrix} 1 & 2 & 1 & 0 \\ 0 & 1 & 1.5 & -0.5 \end{bmatrix} \\ &\xrightarrow{R_1 \rightarrow R_1 + (-2) R_2} \begin{bmatrix} 1 & 0 & -2 & 1 \\ 0 & 1 & 1.5 & -0.5 \end{bmatrix} \end{aligned}$$

`matrix`, `matrix_row_label` and `matrix_col_label` (uses `RectContainer`):

```
from latextool_basic import *

p = Plot()

xss = [['' for _ in range(7)] for _ in range(7)]
for i in range(7):
    if i not in [1, 4]: xss[i][i] = '1'
    elif i == 1: xss[1][4] = '1'
    elif i == 4: xss[4][1] = '1'

C = matrix(p, xss)
p += matrix_row_label(C, 1, r'$i$')
p += matrix_row_label(C, 4, r'$j$')
p += matrix_col_label(C, 1, r'$i$')
p += matrix_col_label(C, 4, r'$j$')

print(p)
```

$$\begin{array}{cc} & i & & j \\ i & \left[\begin{array}{cccc} 1 & & & \\ & & & 1 \\ & & 1 & \\ & & & 1 \end{array} \right] \\ j & \begin{array}{cccc} & 1 & & \\ & & & \\ & & & 1 \\ & & & & 1 \end{array} \end{array}$$

matrix blocks

```

from latextool_basic import *

p = Plot()

xss = [['' for _ in range(7)] for _ in range(7)]
xss[0][0] = '2';
xss[1][1] = '2';
xss[2][2] = '2'
xss[0][1] = '1'
xss[1][2] = '1'

C = matrix(p, xss)

p += matrix_block(C, 2, 0, 0, 2)
p += matrix_block(C, 3, 3, 3, 3)
p += matrix_block(C, 5, 4, 4, 5)
p += matrix_block(C, 6, 4, 6, 6)

p += matrix_row_label(C, 1, r'$i$')
p += matrix_row_label(C, 4, r'$j$')
p += matrix_col_label(C, 1, r'$i$')
p += matrix_col_label(C, 4, r'$j$')

print(p)

```

$$\begin{array}{c}
 \begin{array}{cc}
 & i & j \\
 i & \begin{bmatrix} 2 & 1 \\ & 2 & 1 \\ & & 2 \end{bmatrix} & \\
 j & & \begin{bmatrix} & & & \\ & & & \\ & & & \\ & & & \end{bmatrix}
 \end{array}
 \end{array}
 \end{array}$$

47 Board games (boardgames.tex)

47.1 Othello (othello.tex)

```
from latextool_basic import *
p = Plot()
m = [['', '', '', ''],
      ['', '@', '0', ''],
      ['', '0', '@', ''],
      ['', '', '', '']]
board = othello(p=p, m=m)
print(p)
```

	0	1	2	3
a				
b		●	○	
c		○	●	
d				

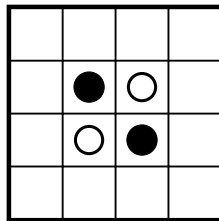
Specify row and column labels:

```
from latextool_basic import *
p = Plot()
m = [['', '', '', ''],
      ['', '@', '0', ''],
      ['', '0', '@', ''],
      ['', '', '', '']]
board = othello(p=p, m=m,
                rownames=range(4),
                colnames=range(4),
                )
print(p)
```

	0	1	2	3
0				
1		●	○	
2		○	●	
3				

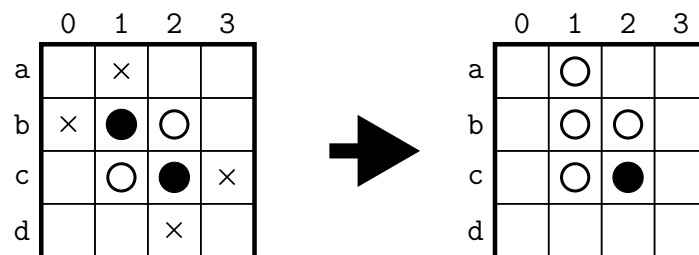
No row, col labels:

```
from latextool_basic import *
p = Plot()
m = [['', '', '', ''],
      ['', '@', '0', ''],
      ['', '0', '@', ''],
      ['', '', '', '']]
board = othello(p=p, m=m, rownames=[], colnames=[])
print(p)
```



Position:

```
from latextool_basic import *
p = Plot()
X = r'$\times$'
m = [['', X, '', ''],
      [X, '@', 'O', ''],
      ['', 'O', '@', X],
      ['', '', X, '']]
board0 = othello(p=p, m=m, x=0, y=0)
m = [['', 'O', '', ''],
      ['', 'O', 'O', ''],
      ['', 'O', '@', ''],
      ['', '', '', '']]
board1 = othello(p=p, m=m, x=6, y=0)
x0,y0 = board0.right(); x0 += 1; p0 = (x0,y0)
x1,y1 = board1.left(); x1 -= 1; p1 = (x1,y1)
p += Line(points=[p0, p1], linewidth=0.2, endstyle='>')
print(p)
```



Highlighting a cell:

```
from latextool_basic import *
p = Plot()
X = r'$\times$'
m = [['', '0', '', ''],
      ['', '0', '0', ''],
      ['', '0', '@', ''],
      ['', ' ', ' ', '']]
board = othello(p=p, m=m, x=6, y=0, do_not_plot=True)

def highlight(p, row, col):
    x0,y0 = board[row][col].bottomleft()
    x1,y1 = board[row][col].topright()
    p += Rect(x0=x0, y0=y0, x1=x1, y1=y1,
              background='black!15',label='', linewidth=0)

highlight(p, 2, 1)
highlight(p, 3, 1)
highlight(p, 2, 2)

board = othello(p=p, m=m, x=6, y=0)
print(p)
```

	0	1	2	3
a		○		
b		○	○	
c		○	●	
d				

47.2 Tic-Tac-Toe (ttt.tex)

```
from latextool_basic import *
p = Plot()
X = 'X'; O = 'O'; s = ''
m = [[X, s, s, s],
      [s, X, O, s],
      [s, O, O, s],
      [s, s, s, s]]
board = ttt(p=p, m=m)
print(p)
```

	0	1	2	3
a	X			
b		X	O	
c		O	O	
d				

no row and column labels:

```
from latextool_basic import *
p = Plot()

X = 'X'; O = 'O'; s = ' '

m = [[X, s, s, s],
      [s, X, O, s],
      [s, O, O, s],
      [s, s, s, s]]
board = ttt(p=p, m=m,
            rownames=['',' ',' ',' '], colnames=['',' ',' ',' '])
print(p)
```

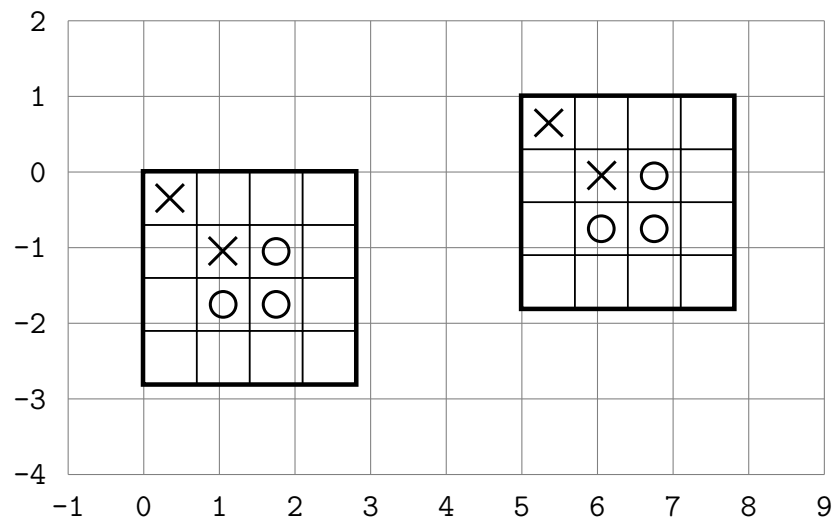
X			
	X	O	
	O	O	

position:

```
from latextool_basic import *
p = Plot()

X = 'X'; O = 'O'; s = ''

m = [[X, s, s, s],
      [s, X, O, s],
      [s, O, O, s],
      [s, s, s, s]]
board = ttt(p=p, x=0, y=0,
            m=m,
            rownames=['',' ',' ',' '], colnames=['',' ',' ',' '])
board = ttt(p=p, x=5, y=1,
            m=m,
            rownames=['',' ',' ',' '], colnames=['',' ',' ',' '])
p += Grid(x0=-1, y0=-4, x1=9, y1=2)
print(p)
```

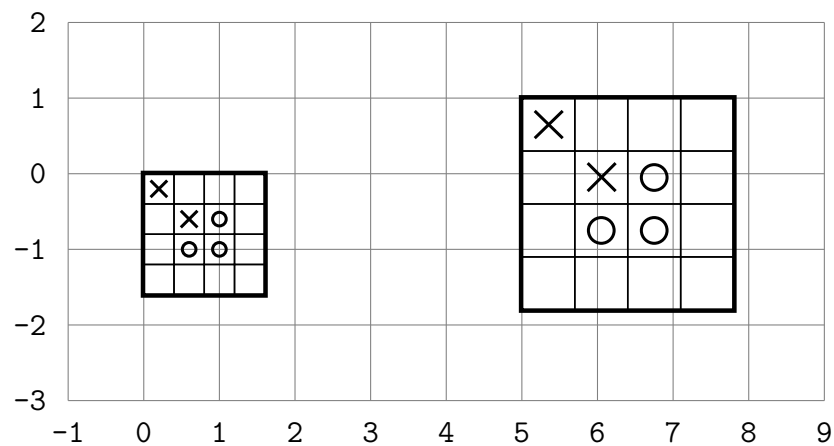


cell size:

```
from latextool_basic import *
p = Plot()

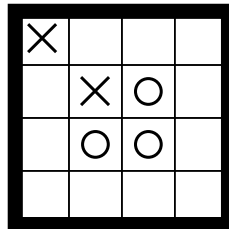
X = 'X'; O = 'O'; s = ' '

m = [[X, s, s, s],
      [s, X, O, s],
      [s, O, O, s],
      [s, s, s, s]]
board = ttt(p=p, x=0, y=0,
            m=m,
            rownames=['',' ',' ',' '], colnames=['',' ',' ',' '],
            width=0.4)
board = ttt(p=p, x=5, y=1,
            m=m,
            rownames=['',' ',' ',' '], colnames=['',' ',' ',' '])
p += Grid(x0=-1, y0=-3, x1=9, y1=2)
print(p)
```



border:

```
from latextool_basic import *
p = Plot()
X = 'X'; O = 'O'; s = ' '
m = [[X, s, s, s],
      [s, X, O, s],
      [s, O, O, s],
      [s, s, s, s]]
board = ttt(p=p, x=0, y=0,
            m=m,
            rownames=['',' ',' ',' '], colnames=['',' ',' ',' '],
            border_linewidth=0.2)
print(p)
```



using the board

```

from latextool_basic import *
from latexcircuit import *

p = Plot()

X = 'X'; O = 'O'; s = ' '

m = [[X, s, s, s],
      [s, X, O, s],
      [s, O, O, s],
      [s, s, s, s]]
board0 = ttt(p=p, x=0, y=0,
             m=m,
             rownames=['',' ',' ',' '], colnames=['',' ',' ',' '])
x,y = board0.topleft()
X = POINT(x=x, y=y, r=0, label='Tic-tac-toe', anchor='south west')
p += str(X)

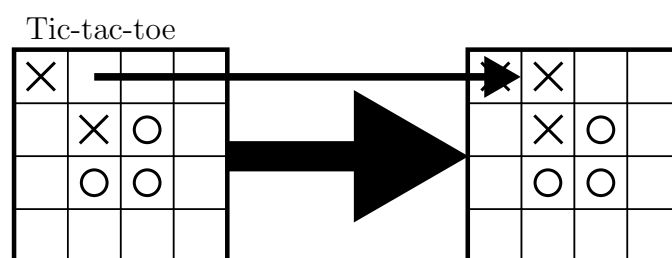
m[0][1] = 'X'
board1 = ttt(p=p, x=6, y=0,
             m=m,
             rownames=['',' ',' ',' '], colnames=['',' ',' ',' '])

p += Line(points=[board0.right(), board1.left()], endstyle='>', linewidth=0.4)

p += Line(points=[board0[0][1].center(),
                  board1[0][1].left()], endstyle='>', linewidth=0.1)

print(p)

```



48 Proof trees (prooftree.tex)

```
from latextool_basic import *
xs = r'\Gamma: 1 + 2 \vdash \texttt{int}'
print(r"""
\[
% s
\]
""") % prooftree(xs))
```

$$\Gamma : 1 + 2 \vdash \texttt{int}$$

```
from latextool_basic import *
xs = {'root': r'\Gamma: 1 + 2 \vdash \texttt{int}',
      'rule': '',
      'children': [],
      }

print(r"""
\[
% s
\]
""") % prooftree(xs))
```

$$\overline{\Gamma : 1 + 2 \vdash \texttt{int}}$$

```

from latextool_basic import *
xs = {'root': r'\Gamma: 1 + 2 \vdash \texttt{int}}',
      'rule': '',
      'children': [
        {'root': r'\Gamma: 1 \vdash \texttt{int}}',
        'rule': r'\textsc{Const}',
        'children': [],
      },
      {'root': r'\Gamma: 2 \vdash \texttt{int}}',
      'rule': r'\textsc{Const}',
      'children': [],
    },
  ],
}

print(r"""
\[
% s
\]
""") % prooftree(xs))

```

$$\frac{\frac{}{\Gamma : 1 \vdash \text{int}} \text{CONST} \quad \frac{}{\Gamma : 2 \vdash \text{int}} \text{CONST}}{\Gamma : 1 + 2 \vdash \text{int}}$$

XXX

$$A$$

YYY

$$\overline{A}$$

ZZZ

$$\frac{B \quad C}{A}$$

49 K-maps (kmap.tex)

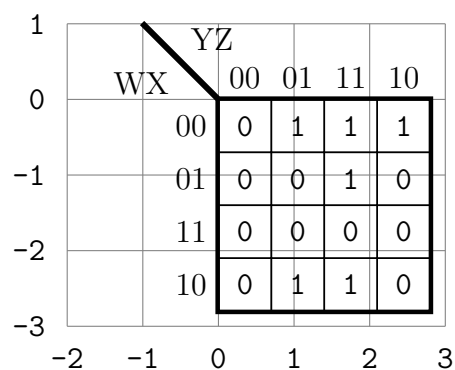
Drawing closed rect groups. Specify 2 rects: the bottom-left rect and the top-right rect.

```
from latextool_basic import *

p = Plot()
p += Grid(-2, -3, 3, 1)
m = [[0,1,1,1],
      [0,0,1,0],
      [0,0,0,0],
      [0,1,1,0],
      ]

C = table2(p, m, width=0.7, height=0.7,
           rownames=['00','01','11','10'],
           colnames=['00','01','11','10'],
           rowlabel='WX', collabel='YZ')

print(p)
```



K-map groups with 1 open side:

```

from latextool_basic import *

p = Plot()
p += Grid(-2, -3, 6, 1)
m = [[0,1,1,0,0,1,1,1],
      [0,1,0,1,1,1,1,1],
      [1,0,1,1,0,0,0,0],
      [0,1,1,1,1,0,1,0],
      ]

C = table2(p, m, width=0.7, height=0.7,
           rownames=['00','01','11','10'],
           colnames=['00','01','11','10'],
           rowlabel='WX', collabel='YZ')

kmap_WW(p, C[0][4], linecolor='red')
kmap_WW(p, C[2][4], C[1][6], linecolor='red')
kmap_WW(p, C[3][4], C[3][6], linecolor='red')

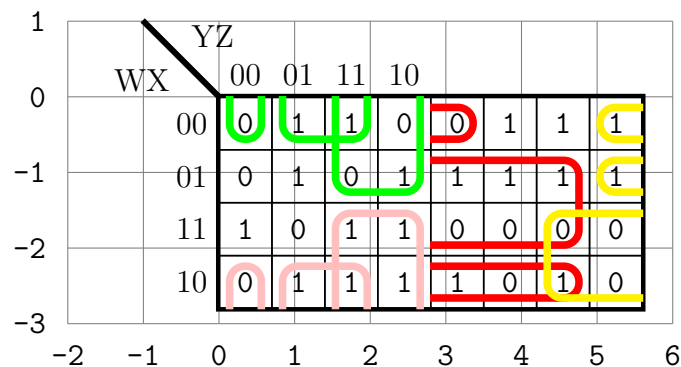
kmap_EE(p, C[0][7], linecolor='yellow')
kmap_EE(p, C[1][7], C[1][7], linecolor='yellow')
kmap_EE(p, C[3][6], C[2][7], linecolor='yellow')

kmap_SS(p, C[3][0], linecolor='pink')
kmap_SS(p, C[3][1], C[3][2], linecolor='pink')
kmap_SS(p, C[3][2], C[2][3], linecolor='pink')

kmap_NN(p, C[0][0], linecolor='green')
kmap_NN(p, C[0][1], C[0][2], linecolor='green')
kmap_NN(p, C[1][2], C[0][3], linecolor='green')

print(p)

```



K-map groups with two open sides:

```
from latextool_basic import *

p = Plot()
p += Grid(-2, -3, 3, 1)
m = [[0,1,1,0],
      [0,1,0,1],
      [1,0,1,1],
      [0,1,1,1],
      ]

C = table2(p, m, width=0.7, height=0.7,
           rownames=['00','01','11','10'],
           colnames=['00','01','11','10'],
           rowlabel='WX', collabel='YZ')

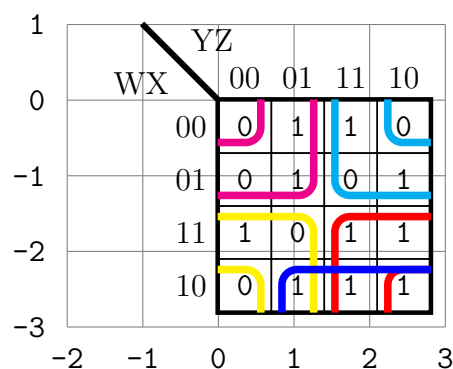
kmap_NW(p, C[0][0], linecolor='magenta')
kmap_NW(p, C[1][0], C[0][1], linecolor='magenta')

kmap_NE(p, C[0][3], linecolor='cyan')
kmap_NE(p, C[1][2], C[0][3], linecolor='cyan')

kmap_SW(p, C[3][0], linecolor='yellow')
kmap_SW(p, C[3][0], C[2][1], linecolor='yellow')

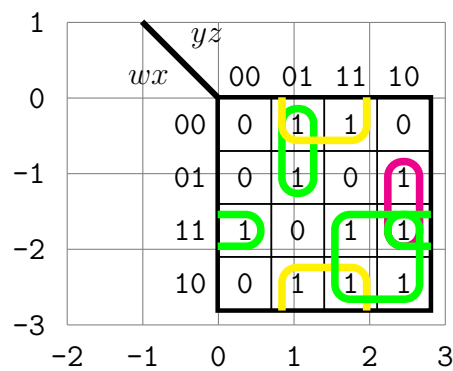
kmap_SE(p, C[3][3], linecolor='red')
kmap_SE(p, C[3][2], C[2][3], linecolor='red')
kmap_SE(p, C[3][1], C[3][3], linecolor='blue')

print(p)
```

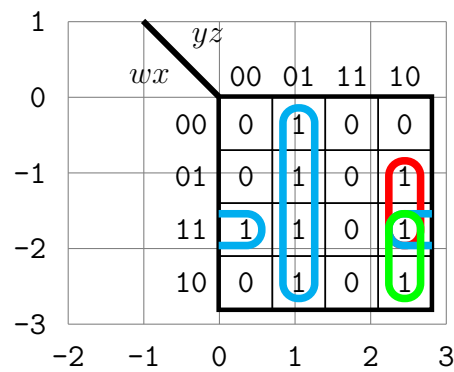


K-maps:

```
from latextool_basic import *
p = Plot()
p += Grid(-2, -3, 3, 1)
m = [[0,1,1,0],
      [0,1,0,1],
      [1,0,1,1],
      [0,1,1,1],
      ]
C = kmap(p, m, width=0.7, height=0.7,
         rowlabel='$wx$', collabel='$yz$')
print(p)
```



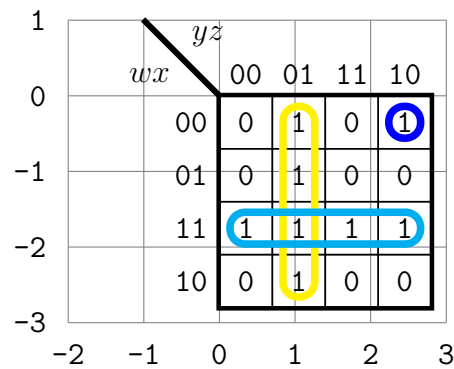
```
from latextool_basic import *
p = Plot()
p += Grid(-2, -3, 3, 1)
m = [[0,1,0,0],
      [0,1,0,1],
      [1,1,0,1],
      [0,1,0,1],
      ]
C = kmap(p, m, width=0.7, height=0.7,
         rowlabel='$wx$', collabel='$yz$')
print(p)
```

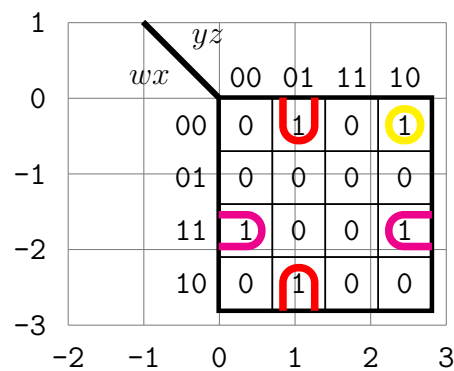


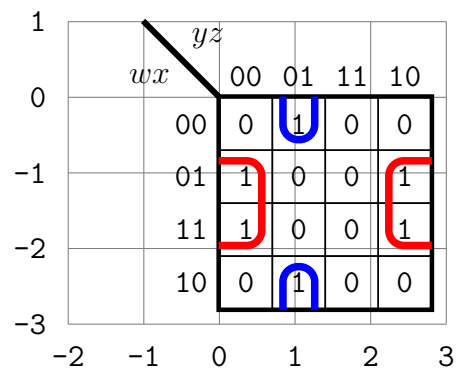
```

from latextool_basic import *
p = Plot()
p += Grid(-2, -3, 3, 1)
m = [[0,1,0,1],
      [0,1,0,0],
      [1,1,1,1],
      [0,1,0,0],
      ]
C = kmap(p, m, width=0.7, height=0.7,
         rowlabel='$wx$', collabel='$yz$')
print(p)

```



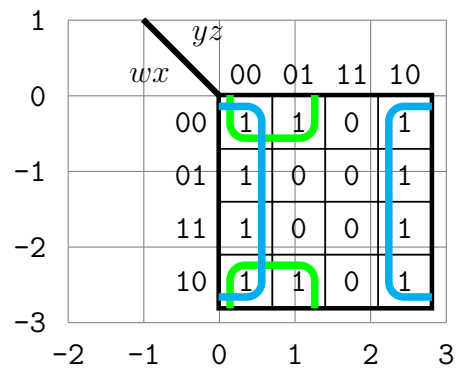




```

from latextool_basic import *
p = Plot()
p += Grid(-2, -3, 3, 1)
m = [[1,1,0,1],
      [1,0,0,1],
      [1,0,0,1],
      [1,1,0,1],
      ]
C = kmap(p, m, width=0.7, height=0.7,
         rowlabel='$wx$', collabel='$yz$')
print(p)

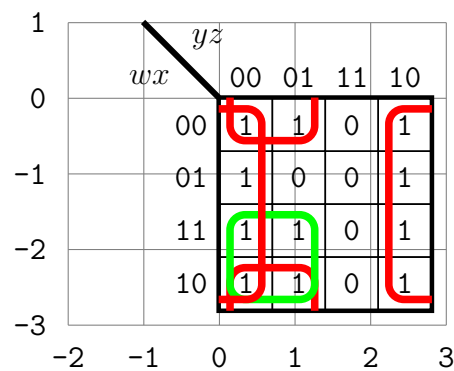
```



```

from latextool_basic import *
p = Plot()
p += Grid(-2, -3, 3, 1)
m = [[1,1,0,1],
      [1,0,0,1],
      [1,1,0,1],
      [1,1,0,1],
      ]
C = kmap(p, m, width=0.7, height=0.7,
          rowlabel='$wx$', collabel='$yz$')
print(p)

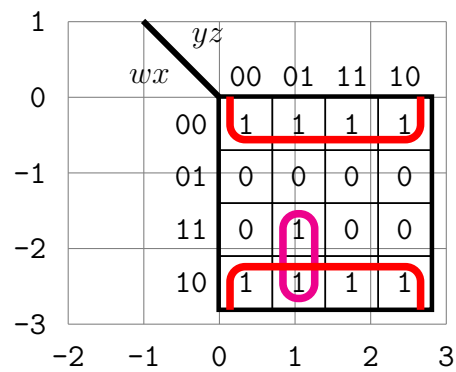
```



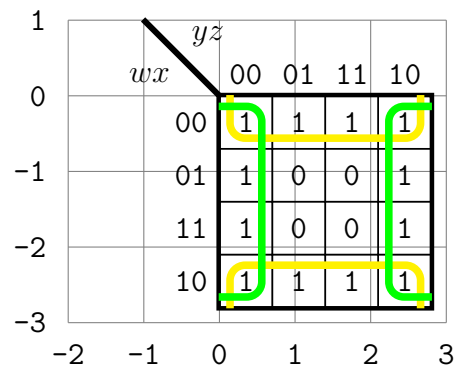
```

from latextool_basic import *
p = Plot()
p += Grid(-2, -3, 3, 1)
m = [[1,1,1,1],
      [0,0,0,0],
      [0,1,0,0],
      [1,1,1,1],
      ]
C = kmap(p, m, width=0.7, height=0.7,
          rowlabel='$wx$', collabel='$yz$')
print(p)

```



```
from latextool_basic import *
p = Plot()
p += Grid(-2, -3, 3, 1)
m = [[1,1,1,1],
      [1,0,0,1],
      [1,0,0,1],
      [1,1,1,1],
      ]
C = kmap(p, m, width=0.7, height=0.7,
         rowlabel='$wx$', collabel='$yz$')
print(p)
```




```

from latextool_basic import *
p = Plot()
m = [[1,0,0,1],
      [0,0,1,0],
      [0,1,0,0],
      [1,0,0,1],
      ]
C = kmap(p, m, width=0.7, height=0.7,
          rowlabel='$wx$', collabel='$yz$')
print(p)

```

	yz	00	01	11	10
wx	00	1	0	0	1
	01	0	0	1	0
	11	0	1	0	0
	10	1	0	0	1

```

from latextool_basic import *
p = Plot()
m = [[1,0,1,1],
      [0,1,1,0],
      [0,1,0,0],
      [1,0,0,1],
      ]
C = kmap(p, m, width=0.7, height=0.7,
          rowlabel='$wx$', collabel='$yz$')
print(p)

```

	yz	00	01	11	10
wx	00	1	0	1	1
	01	0	1	1	0
	11	0	1	0	0
	10	1	0	0	1

```

from latextool_basic import *
p = Plot()
m = [[1,1,1,1],
      [1,1,1,1],
      [1,1,1,1],
      [1,1,1,1],
      ]
C = kmap(p, m, width=0.7, height=0.7,
         rowlabel='$wx$', collabel='$yz$')
print(p)

```

		yz			
wx		00	01	11	10
00		1	1	1	1
01		1	1	1	1
11		1	1	1	1
10		1	1	1	1

```

from latextool_basic import *
p = Plot()
m = [[0,0,1,0],
      [1,1,1,0],
      [0,1,1,1],
      [0,0,1,0],
      ]
C = kmap(p, m, width=0.7, height=0.7,
         rowlabel='$wx$', collabel='$yz$')
print(p)

```

		yz			
wx		00	01	11	10
00		0	0	1	0
01		1	1	1	0
11		0	1	1	1
10		0	0	1	0

```

from latextool_basic import *
p = Plot()
#p += Grid(-2, -3, 3, 1)
m = [[0,1,1,0,1,1,1,1],
      [0,1,0,1,1,0,1,0],
      [0,1,1,0,0,1,0,1],
      [1,0,1,0,1,1,0,1],
      ]
C = kmap(p, m, width=1, height=1,
          rowlabel='$de$', collabel='$abc$')
print(p)

```

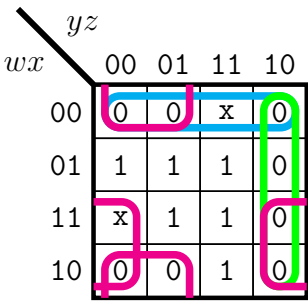
	<i>abc</i>							
<i>de</i>	000	001	011	010	100	101	111	110
00	0	1	1	0	1	1	1	1
01	0	1	0	1	1	0	1	0
11	0	1	1	0	0	1	0	1
10	1	0	1	0	1	1	0	1

49.1 kmap: specify term(s) to circle

```
from latextool_basic import *
p = Plot()
m = [[0,0,'d',0],
      [1,1,1,0],
      ['d',1,1,0],
      [0,0,1,0],
      ]
C = kmap(p, m, width=0.7, height=0.7,
         circle_terms=[1, 'd'],
         rowlabel='$wx$', collabel='$yz$')
print(p)
```

		<i>yz</i>			
	<i>wx</i>	00	01	11	10
00		0	0	d	0
01		1	1	1	0
11		d	1	1	0
10		0	0	1	0

```
from latextool_basic import *
p = Plot()
m = [[0,0,'x',0],
      [1,1,1,0],
      ['x',1,1,0],
      [0,0,1,0],
      ]
C = kmap(p, m, width=0.7, height=0.7,
         circle_terms=[0, 'x'],
         rowlabel='$wx$', collabel='$yz$')
print(p)
```



49.2 kmap: 2-by-4

```
from latextool_basic import *
p = Plot()
m = [[0,0,1,0],
      [0,1,0,1],
      ]
C = kmap(p, m, width=0.7, height=0.7,
          rowlabel='$x$', collabel='$yz$')
print(p)
```

	yz	00	01	11	10
x	0	0	0	1	0
	1	0	1	0	1

```
from latextool_basic import *
p = Plot()
m = [[0,0,0,0],
      [1,1,1,1],
      ]
C = kmap(p, m, width=0.7, height=0.7,
          rowlabel='$x$', collabel='$yz$')
print(p)
```

	yz	00	01	11	10
x	0	0	0	0	0
	1	1	1	1	1

```

from latextool_basic import *
p = Plot()
m = [[1,1,0,0],
      [1,1,0,0],
      ]
C = kmap(p, m, width=0.7, height=0.7,
         rowlabel='$x$', collabel='$yz$')
print(p)

```

	yz	00	01	11	10
x	0	1	1	0	0
	1	1	1	0	0

```

from latextool_basic import *
p = Plot()
m = [[1,0,0,1],
      [0,0,0,0],
      ]
C = kmap(p, m, width=0.7, height=0.7,
         rowlabel='$x$', collabel='$yz$')
print(p)

```

	yz	00	01	11	10
x	0	1	0	0	1
	1	0	0	0	0

```

from latextool_basic import *
p = Plot()
m = [[1,0,0,1],
      [1,0,0,1],
      ]
C = kmap(p, m, width=0.7, height=0.7,
         rowlabel='$x$', collabel='$yz$')
print(p)

```

	yz	00	01	11	10
x	0	1	0	0	1
	1	1	0	0	1

```

from latextool_basic import *
p = Plot()
m = [[1,0,1,1],
      [1,1,0,1],
      ]
C = kmap(p, m, width=0.7, height=0.7,
         rowlabel='$x$', collabel='$yz$')
print(p)

```

	yz	00	01	11	10
x	0	1	0	1	1
	1	1	1	0	1

```

from latextool_basic import *
p = Plot()
m = [[0,0,0,0],
      [1,1,0,1],
      ]
C = kmap(p, m, width=0.7, height=0.7,
         rowlabel='$x$', collabel='$yz$')
print(p)

```

	yz	00	01	11	10
x	0	0	0	0	0
	1	1	1	0	1


```

from latextool_basic import *

p = Plot()
m = [[0,0,1,0],
      [0,1,1,1],
      ]

def f(x):
    ((r0,r1),(c0,c1)) = x
    if ((r0,r1),(c0,c1)) == ((0,1),(2,2)): return 'red'
    elif ((r0,r1),(c0,c1)) == ((1,1),(1,2)): return 'green'
    elif ((r0,r1),(c0,c1)) == ((1,1),(2,3)): return 'blue'
    else:
        return 'black'

def g(t):
    return '0.1'

def h(x):
    ((r0,r1),(c0,c1)) = x
    if ((r0,r1),(c0,c1)) == ((0,1),(2,2)): return 0.05
    elif ((r0,r1),(c0,c1)) == ((1,1),(1,2)): return 0.1
    elif ((r0,r1),(c0,c1)) == ((1,1),(2,3)): return 0.15

C = kmap(p, m, width=0.7, height=0.7,
         style_selector={'linecolor':f, 'linewidth':g, 'd':h},
         rowlabel='$x$', collabel='$yz$')

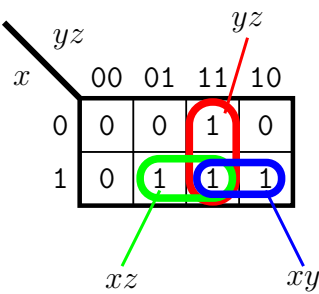
x0,y0 = C[0][2].topright(); x0 += 0.1; y0 += 1
x1,y1 = x0,y0
r = Rect(x0=x0,y0=y0,x1=x1,y1=y1,label=r'$yz$', linewidth=0)
p += r
x0,y0 = r.bottom(); y0 -= 0.2; p0 = (x0,y0)
x1,y1 = C[0][2].topright(); x1 -= 0.2; y1 -= 0.2; p1 = (x1,y1)
p += Line(points=[p0, p1], linecolor='red')

x0,y0 = C[1][1].bottom(); x0 -= 0.5; y0 -= 1
x1,y1 = x0,y0
r = Rect(x0=x0,y0=y0,x1=x1,y1=y1,label=r'$xz$', linewidth=0)
p += r
x0,y0 = r.top(); y0 += 0.2; p0 = (x0,y0)
x1,y1 = C[1][1].bottom(); y1 += 0.2; p1 = (x1,y1)
p += Line(points=[p0, p1], linecolor='green')

x0,y0 = C[1][3].bottom(); x0 += 0.5; y0 -= 1
x1,y1 = x0,y0
r = Rect(x0=x0,y0=y0,x1=x1,y1=y1,label=r'$xy$', linewidth=0)
p += r
x0,y0 = r.top(); y0 += 0.2; p0 = (x0,y0)
x1,y1 = C[1][3].bottom(); y1 += 0.2; p1 = (x1,y1)
p += Line(points=[p0, p1], linecolor='blue')

print(p)

```

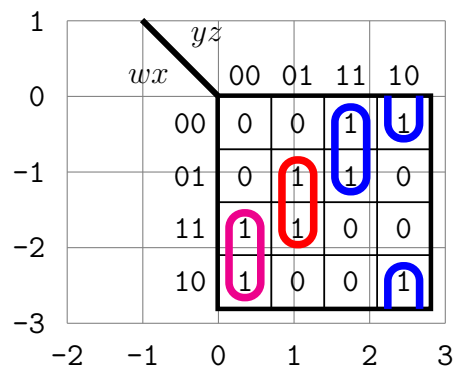


49.3 kmap: force choice of prime implicant

There is more than one way to select essential prime implicants.

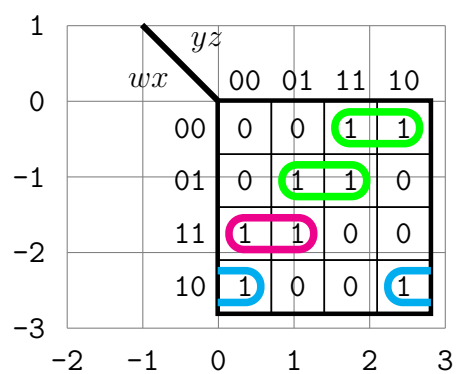
Default:

```
from latextool_basic import *
p = Plot()
p += Grid(-2, -3, 3, 1)
m = [[0,0,1,1],
      [0,1,1,0],
      [1,1,0,0],
      [1,0,0,1],
      ]
C = kmap(p, m, width=0.7, height=0.7,
         rowlabel='$wx$', collabel='$yz$')
print(p)
```



You can force “do not remove $((0,0), (2,3))$ ” using `donotremove` parameter:

```
from latextool_basic import *
p = Plot()
p += Grid(-2, -3, 3, 1)
m = [[0,0,1,1],
      [0,1,1,0],
      [1,1,0,0],
      [1,0,0,1],
      ]
C = kmap(p, m, width=0.7, height=0.7,
         rowlabel='$wx$', collabel='$yz$',
         donotremove=[((0,0), (2,3))])
print(p)
```



49.4 kmap: style selector

You can select color and linewidth for the implicants:

```
from latextool_basic import *

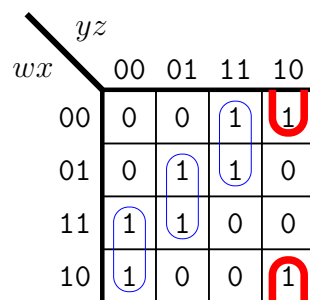
p = Plot()
m = [[0,0,1,1],
      [0,1,1,0],
      [1,1,0,0],
      [1,0,0,1],
      ]

def f(x):
    ((r0,r1),(c0,c1)) = x
    if r0==3 and c0==3:
        return 'red'
    else:
        return 'blue'

def g(x):
    ((r0,r1),(c0,c1)) = x
    if r0==3 and c0==3:
        return 0.1
    else:
        return 0.01

C = kmap(p, m, width=0.7, height=0.7,
         rowlabel='$wx$', collabel='$yz$',
         style_selector={'linecolor':f, 'linewidth':g}
        )

print(p)
```



A 4x4 Karnaugh map for variables wx (rows) and yz (columns). The map contains 1s at positions (00,11), (00,10), (01,01), (01,11), (11,00), (11,01), (10,00), and (10,10). Blue groupings (width 0.7, height 0.01) are shown for the pairs (00,11)-(01,11), (01,01)-(11,01), (11,00)-(10,00), and (10,10)-(11,10). Red groupings (width 0.1, height 0.01) are shown for the pairs (00,11)-(00,10) and (10,00)-(10,10).

		yz			
		00	01	11	10
wx	00	0	0	1	1
	01	0	1	1	0
	11	1	1	0	0
	10	1	0	0	1

49.5 kmap: change distance of implicant boundary to rect

d = 0.1:

```
from latextool_basic import *

p = Plot()
m = [[0,0,1,1],
      [0,1,1,0],
      [1,1,0,0],
      [1,0,0,1],
      ]

def f(x):
    ((r0,r1),(c0,c1)) = x
    if r0==3 and c0==3:
        return 'red'
    else:
        return 'blue'
def g(x):
    ((r0,r1),(c0,c1)) = x
    if r0==3 and c0==3:
        return 0.1
    else:
        return 0.01

C = kmap(p, m, width=1, height=1,
         rowlabel='$wx$', collabel='$yz$',
         d=0.1,
         style_selector={'linecolor':f, 'linewidth':g}
         )
print(p)
```

$wx \backslash yz$		yz			
		00	01	11	10
wx	00	0	0	1	1
	01	0	1	1	0
	11	1	1	0	0
	10	1	0	0	1

d = 0.3

```

from latextool_basic import *

p = Plot()
m = [[0,0,1,1],
      [0,1,1,0],
      [1,1,0,0],
      [1,0,0,1],
      ]

def f(x):
    ((r0,r1),(c0,c1)) = x
    if r0==3 and c0==3:
        return 'red'
    else:
        return 'blue'
def g(x):
    ((r0,r1),(c0,c1))= x
    if r0==3 and c0==3:
        return 0.1
    else:
        return 0.01

C = kmap(p, m, width=1, height=1,
         rowlabel='$wx$', collabel='$yz$',
         d=0.3,
         style_selector={'linecolor':f, 'linewidth':g}
         )
print(p)

```

$wx \backslash yz$		yz			
		00	01	11	10
wx	00	0	0	1	1
	01	0	1	1	0
	11	1	1	0	0
	10	1	0	0	1

49.6 kmap: decimal notation

```
from latextool_basic import *

p = Plot()
m = [[0,0,1,1],
      [0,1,1,0],
      [1,1,0,0],
      [1,0,0,1],
      ]

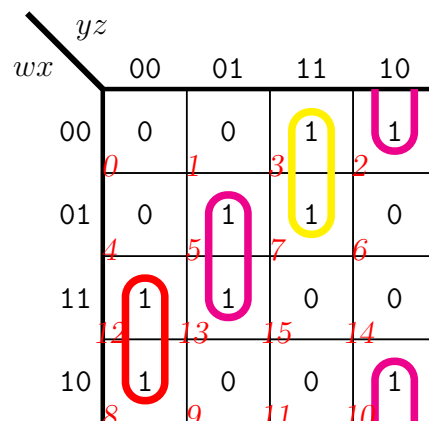
C = kmap(p, m, width=1.1, height=1.1,
          rowlabel='$wx$', collabel='$yz$',
          decimal=True
        )
print(p)
```

		yz			
		00	01	11	10
wx	00	0 0	0 1	1 3	1 2
	01	0 4	1 5	1 7	0 6
	11	1 12	1 13	0 15	0 14
	10	1 8	0 9	0 11	1 10

49.7 kmap: decimal notation 2

You can specify offset of decimal notation and the formatting string:

```
from latextool_basic import *
p = Plot()
m = [[0,0,1,1],
      [0,1,1,0],
      [1,1,0,0],
      [1,0,0,1],
      ]
C = kmap(p, m, width=1.1, height=1.1,
         rowlabel='$wx$', collabel='$yz$',
         d = 0.3,
         decimal=True,
         decimal_offset=0.1,
         decimal_format=r'\textred{\textsl{%s}}}')
print(p)
```



49.8 kmap: using coordinates of rects in the kmap cells

I use a slightly large `d` value so that the implicant boundaries avoid the decimal notation.

```
from latextool_basic import *

p = Plot()
m = [[0,0,1,1],
      [0,1,1,0],
      [1,1,0,0],
      [1,0,0,1],
      ]

def f(a):
    ((r0,r1),(c0,c1)) = a
    if r0==3 and c0==3:
        return 'red'
    else:
        return 'blue'

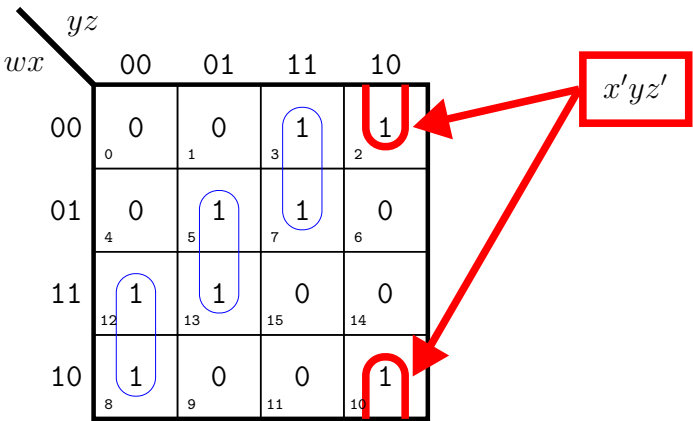
def g(a):
    ((r0,r1),(c0,c1)) = a
    if r0==3 and c0==3:
        return 0.1
    else:
        return 0.01

C = kmap(p, m, width=1.1, height=1.1,
         rowlabel='$wx$', collabel='$yz$',
         d = 0.3,
         style_selector={'linecolor':f, 'linewidth':g},
         decimal=True
        )

x0,y0 = C[0][3].right()
x1,y1 = x0 + 2, y0
r = Rect(x0=x1,y0=y1,x1=x1+1.5,y1=y1+1,label="$x'yz'$", linewidth=0.1, line
color='red')
p += r

x,y = C[0][3].right(); x -= 0.2
p += Line(points=[r.left(),(x,y)], endstyle='>', linecolor='red', linewidth
=0.1)
x,y = C[3][3].right(); x -= 0.2
p += Line(points=[r.left(),(x,y)], endstyle='>', linecolor='red', linewidth
=0.1)

print(p)
```



50 Logic design (logic-design.tex)

50.1 AND gate

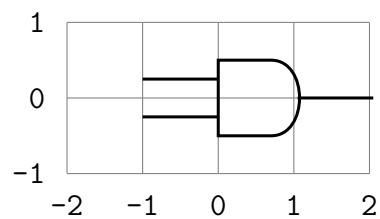
```
from latextool_basic import *
from latexcircuit import *

p = Plot()
g = AND_GATE(x=0, y=0, inputs=2)
p += str(g)

for x,y in g.inputs():
    p += Line(points=[(x - 1, y), (x, y)])

x,y = g.output()
p += Line(points=[(x, y), (x + 1,y)])

p += Grid(x0=-2, y0=-1, x1=2, y1=1)
print(p)
```



50.2 OR gate

```

from latextool_basic import *
from latexcircuit import *

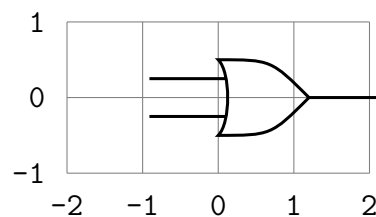
p = Plot()
g = OR_GATE()
p += str(g)

for x,y in g.inputs():
    p += Line(points=[(x - 1, y), (x, y)])

x,y = g.output()
p += Line(points=[(x, y),(x+1,y)])

p += Grid(x0=-2, y0=-1, x1=2, y1=1)
print(p)

```



50.3 NOT gate

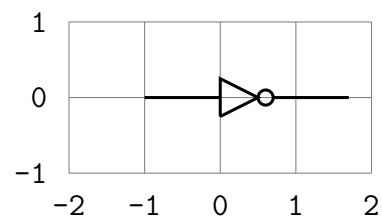
```
from latextool_basic import *
from latexcircuit import *

p = Plot()
g = NOT_GATE()
p += str(g)

x,y = g.input()
p += Line(points=[(x,y),(x - 1,y)])

x,y = g.output()
p += Line(points=[(x,y),(x + 1,y)])

p += Grid(x0=-2, y0=-1, x1=2, y1=1)
print(p)
```



50.4 NAND gate

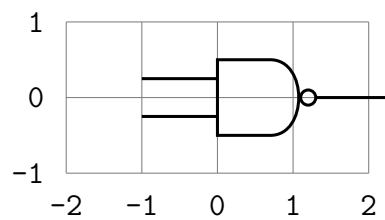
```
from latextool_basic import *
from latexcircuit import *

p = Plot()
g = NAND_GATE()
p += str(g)

for x,y in g.inputs():
    p += Line(points=[(x-1,y),(x,y)])

x,y = g.output()
p += Line(points=[(x,y),(x+1,y)])

p += Grid(x0=-2, y0=-1, x1=2, y1=1)
print(p)
```



50.5 NOR gate

```

from latextool_basic import *
from latexcircuit import *

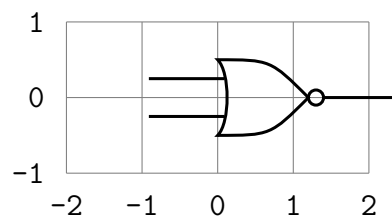
p = Plot()
g = NOR_GATE()
p += str(g)

for x,y in g.inputs():
    p += Line(points=[(x-1,y),(x,y)])

x,y = g.output()
p += Line(points=[(x,y),(x+1,y)])

p += Grid(x0=-2, y0=-1, x1=2, y1=1)
print(p)

```



50.6 XOR gate

```

from latextool_basic import *
from latexcircuit import *

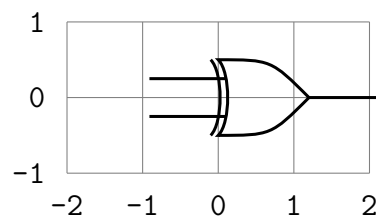
p = Plot()
g = XOR_GATE()
p += str(g)

for x,y in g.inputs():
    p += Line(points=[(x-1,y),(x,y)])

x,y = g.output()
p += Line(points=[(x,y),(x+1,y)])

p += Grid(x0=-2, y0=-1, x1=2, y1=1)
print(p)

```



50.7 Buffer gate

```

from latextool_basic import *
from latexcircuit import *

p = Plot()

g = BUFFER_GATE(x=0, y=0, inputs=1)
p += str(g)

for x,y in g.inputs():
    p += Line(points=[(x - 1, y), (x, y)])

x,y = g.output()
p += Line(points=[(x, y), (x + 1,y)])

g1 = BUFFER_WITH_ENABLE_GATE(x=4, y=0, inputs=1)
p += str(g1)

for i, (x,y) in enumerate(g1.inputs()):
    if i == 0:
        p += Line(points=[(x - 1, y), (x, y)])
    else:
        p += Line(points=[(x, y), (x, y - 1), (x - 1, y - 1)])

x,y = g1.output()
p += Line(points=[(x, y), (x + 1,y)])

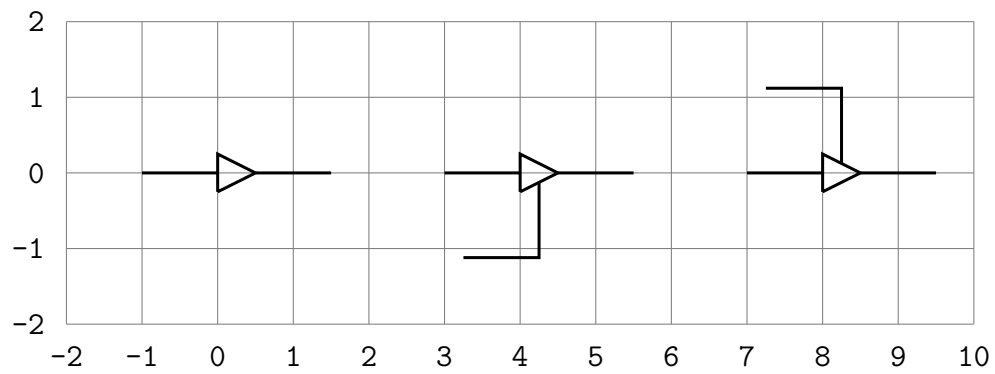
g2 = BUFFER_WITH_ENABLE_GATE(x=8, y=0, inputs=1, enable='top')
p += str(g2)

for i, (x,y) in enumerate(g2.inputs()):
    if i == 0:
        p += Line(points=[(x - 1, y), (x, y)])
    else:
        p += Line(points=[(x, y), (x, y + 1), (x - 1, y + 1)])

x,y = g2.output()
p += Line(points=[(x, y), (x + 1, y)])

p += Grid(x0=-2, y0=-2, x1=10, y1=2)
print(p)

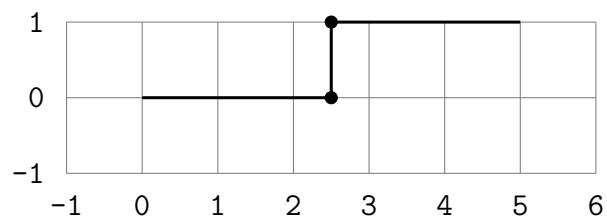
```



50.8 Orthogonal paths

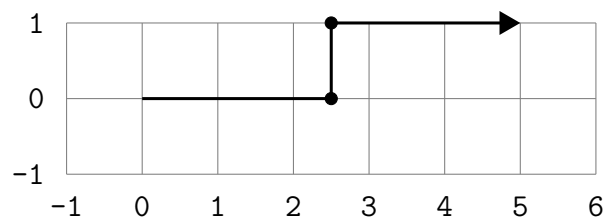
For drawing orthogonal paths between two points.

```
from latextool_basic import *
from latexcircuit import *
p = Plot()
p += str(OrthogonalPath([(0,0), (5,1)]))
p += Grid(x0=-1, y0=-1, x1=6, y1=1)
print(p)
```



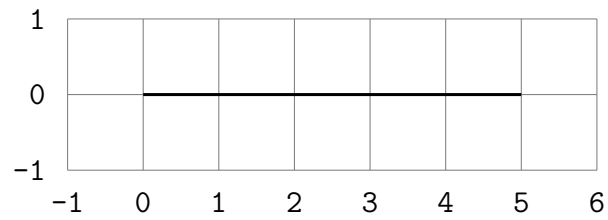
Orthogonal path with arrow head:

```
from latextool_basic import *
from latexcircuit import *
p = Plot()
p += str(OrthogonalPath([(0,0), (5,1)], endstyle='>', arrowstyle='triangle'))
p += Grid(x0=-1, y0=-1, x1=6, y1=1)
print(p)
```



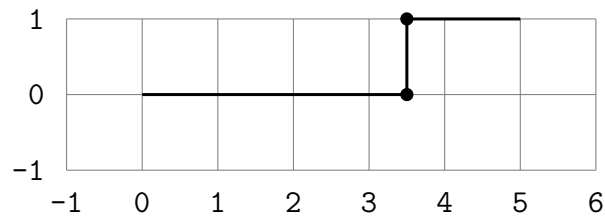
Orthogonal paths – no bend case:

```
from latextool_basic import *
from latexcircuit import *
p = Plot()
p += str(OrthogonalPath([(0,0), (5,0)]))
p += Grid(x0=-1, y0=-1, x1=6, y1=1)
print(p)
```



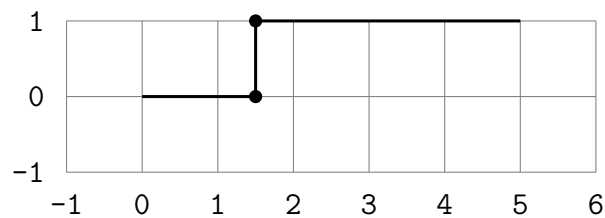
with a positive horizontal shift of the bend:

```
from latextool_basic import *
from latexcircuit import *
p = Plot()
p += str(OrthogonalPath([(0,0), (5,1)], shifts=[1]))
p += Grid(x0=-1, y0=-1, x1=6, y1=1)
print(p)
```



with a negative horizontal shift of the bend:

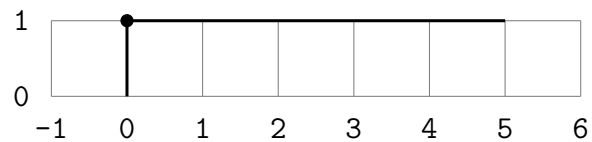
```
from latextool_basic import *
from latexcircuit import *
p = Plot()
p += str(OrthogonalPath([(0,0), (5,1)], shifts=[-1]))
p += Grid(x0=-1, y0=-1, x1=6, y1=1)
print(p)
```



TODO: shifts for vertical case

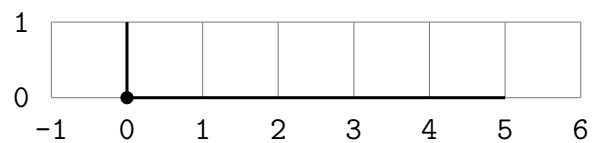
orthogonal path, direction='vh' (vertical-horizontal):

```
from latextool_basic import *
from latexcircuit import *
p = Plot()
p += str(OrthogonalPath(points=[(0,0), (5,1)], direction='vh'))
p += Grid(x0=-1, y0=0, x1=6, y1=1)
print(p)
```



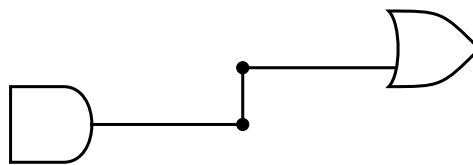
orthogonal path, direction='vh':

```
from latextool_basic import *
from latexcircuit import *
p = Plot()
p += str(OrthogonalPath(points=[(0,1), (5,0)], direction='vh'))
p += Grid(x0=-1, y0=0, x1=6, y1=1)
print(p)
```



50.9 Orthogonal path with gates

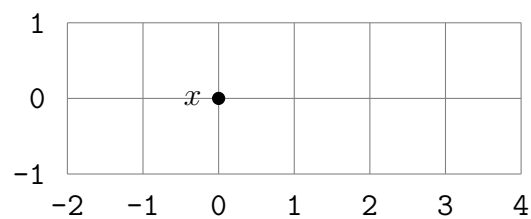
```
from latextool_basic import *
from latexcircuit import *
p = Plot()
AND = AND_GATE(x=0, y=0); p += str(AND)
OR = OR_GATE(x=5, y=1); p += str(OR)
opath = OrthogonalPath(gate0=AND, gate1=OR, input_index=0)
p += str(opath)
print(p)
```



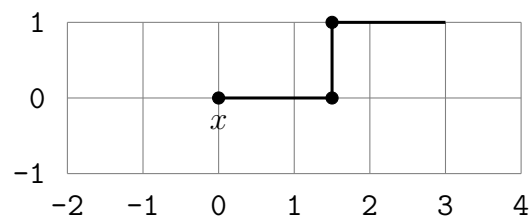
50.10 POINT

POINT:

```
from latextool_basic import *
from latexcircuit import *
p = Plot()
X = POINT(x=0, y=0, label='$x$')
p += str(X)
p += Grid(x0=-2, y0=-1, x1=4, y1=1)
print(p)
```



```
from latextool_basic import *
from latexcircuit import *
p = Plot()
X = POINT(x=0, y=0, label='$x$', anchor='north')
p += str(X)
p += str(OrthogonalPath([X.output(), (3,1)]))
p += Grid(x0=-2, y0=-1, x1=4, y1=1)
print(p)
```



50.11 Linecolor for gate

```
from latextool_basic import *
from latexcircuit import *

p = Plot()
g0 = AND_GATE(linewidth=5, linecolor='blue')
p += str(g0)

print(p)
```

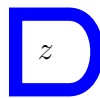


50.12 Label for gate

```
from latextool_basic import *
from latexcircuit import *

p = Plot()
g0 = AND_GATE(linewidth=5, linecolor='blue', label='$z$')
p += str(g0)

print(p)
```



Test 2 inputs and output

```
from latextool_basic import *
from latexcircuit import *

p = Plot()
g0 = AND_GATE(linewidth=5, linecolor='blue'); p += str(g0)

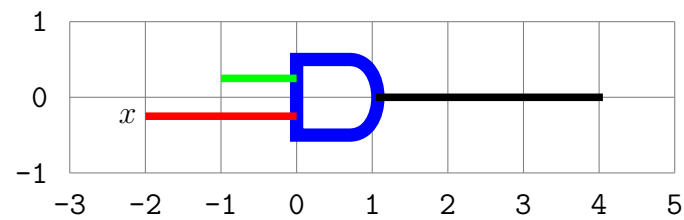
inputs = g0.inputs()

# Test input 0
x0,y0 = inputs[0]; p0 = (x0, y0)
x1,y1 = x0 - 2, y0; p1 = (x1, y1)
p += Line(points=[p0,p1], linewidth=0.1, linecolor='red')
x0 = x1 - 0.3
p += Rect(x0=x0, y0=y0, x1=x0, y1=y0, linewidth=0, s = '$x$')

# test input 1
x0,y0 = inputs[1]; p0 = (x0, y0)
x1,y1 = x0 - 1, y0; p1 = (x1, y1)
p += Line(points=[p0,p1], linewidth=0.1, linecolor='green')

# test output
x0,y0 = g0.output(); p0 = x0,y0
x1,y1 = x0 + 3, y0; p1 = x1,y1
p += Line(points=[p0,p1], linewidth=0.1, linecolor='black')

p += Grid()
print(p)
```



Test 3 inputs and output

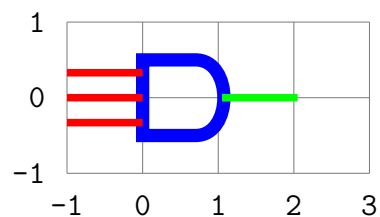
```
from latextool_basic import *
from latexcircuit import *

p = Plot()
g0 = AND_GATE(linewidth=5, linecolor='blue', inputs=3)
p += str(g0)

for x0,y0 in g0.inputs():
    p0 = (x0, y0)
    x1,y1 = x0 - 1, y0
    p1 = (x1, y1)
    p += Line(points=[p0,p1], linewidth=0.1, linecolor='red')

x0,y0 = g0.output()
p0 = (x0, y0)
x1,y1 = x0 + 1, y0
p1 = (x1, y1)
p += Line(points=[p0,p1], linewidth=0.1, linecolor='green')

p += Grid()
print(p)
```



Test different size (height = 2)

```

from latextool_basic import *
from latexcircuit import *

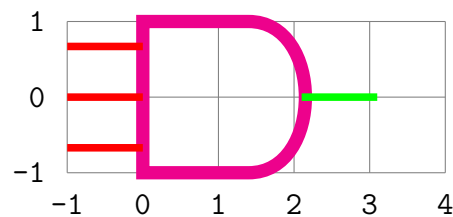
p = Plot()
g0 = AND_GATE(x=0, y=0, h=2,
              linewidth=5, linecolor='magenta', inputs=3)
p += str(g0)

for x0,y0 in g0.inputs():
    p0 = (x0, y0)
    x1,y1 = x0 - 1, y0
    p1 = (x1, y1)
    p += Line(points=[p0,p1], linewidth=0.1, linecolor='red')

x0,y0 = g0.output()
p0 = (x0, y0)
x1,y1 = x0 + 1, y0
p1 = (x1, y1)
p += Line(points=[p0,p1], linewidth=0.1, linecolor='green')

p += Grid()
print(p)

```



Test height = 2, 4 inputs:

```

from latextool_basic import *
from latexcircuit import *

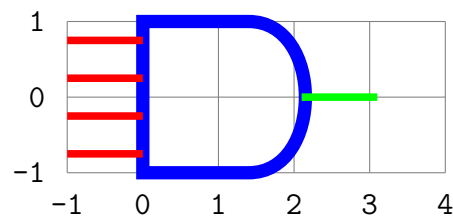
p = Plot()
g0 = AND_GATE(x=0, y=0, h=2,
              linewidth=5, linecolor='blue', inputs=4)
p += str(g0)

for x0,y0 in g0.inputs():
    p0 = (x0, y0)
    x1,y1 = x0 - 1, y0
    p1 = (x1, y1)
    p += Line(points=[p0,p1], linewidth=0.1, linecolor='red')

x0,y0 = g0.output()
p0 = (x0, y0)
x1,y1 = x0 + 1, y0
p1 = (x1, y1)
p += Line(points=[p0,p1], linewidth=0.1, linecolor='green')

p += Grid()
print(p)

```



Test height = 2, 5 inputs:

```

from latextool_basic import *
from latexcircuit import *

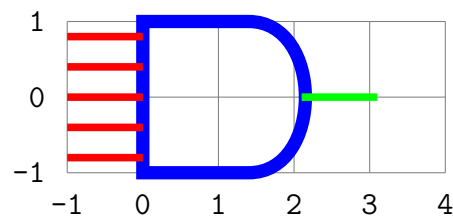
p = Plot()
g0 = AND_GATE(x=0, y=0, h=2,
              linewidth=5, linecolor='blue', inputs=5)
p += str(g0)

for x0,y0 in g0.inputs():
    p0 = (x0, y0)
    x1,y1 = x0 - 1, y0
    p1 = (x1, y1)
    p += Line(points=[p0,p1], linewidth=0.1, linecolor='red')

x0,y0 = g0.output()
p0 = (x0, y0)
x1,y1 = x0 + 1, y0
p1 = (x1, y1)
p += Line(points=[p0,p1], linewidth=0.1, linecolor='green')

p += Grid()
print(p)

```



OR gate:

```

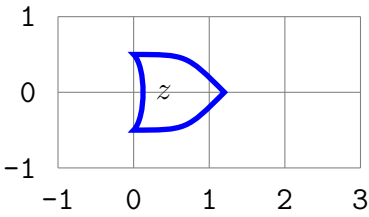
from latextool_basic import *
from latexcircuit import *

p = Plot()
g0 = OR_GATE(linewidth=2, linecolor='blue', label='$z$')
p += str(g0)

p += Grid(x0=-1,y0=-1,x1=3,y1=1)

print(p)

```



OR gate testing 2 inputs

```

from latextool_basic import *
from latexcircuit import *

p = Plot()
g0 = OR_GATE(linewidth=2, linecolor='blue')
p += str(g0)

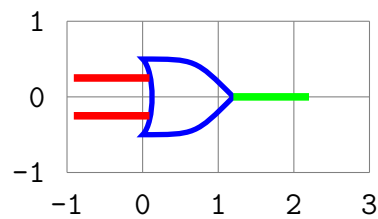
for x0,y0 in g0.inputs():
    p0 = (x0, y0)
    x1,y1 = x0 - 1, y0
    p1 = (x1, y1)
    p += Line(points=[p0,p1], linewidth=0.1, linecolor='red')

x0,y0 = g0.output()
p0 = (x0, y0)
x1,y1 = x0 + 1, y0
p1 = (x1, y1)
p += Line(points=[p0,p1], linewidth=0.1, linecolor='green')

p += Grid(x0=-1,y0=-1,x1=3,y1=1)

print(p)

```



OR gate testing 2 inputs

```

from latextool_basic import *
from latexcircuit import *

p = Plot()
g0 = OR_GATE(linewidth=2, x=0, y=0, linecolor='blue')
p += str(g0)

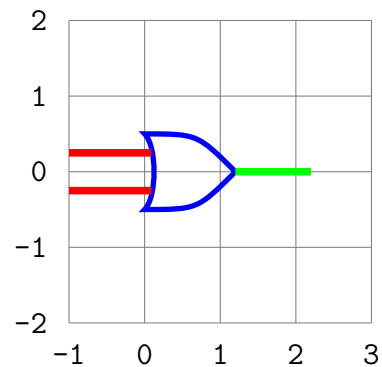
for x0,y0 in g0.inputs():
    p0 = (x0, y0)
    x1, y1 = - 1, y0
    p1 = (x1, y1)
    p += Line(points=[p0,p1], linewidth=0.1, linecolor='red')

x0, y0 = g0.output(); p0 = (x0, y0)
x1, y1 = x0 + 1, y0; p1 = (x1, y1)
p += Line(points=[p0,p1], linewidth=0.1, linecolor='green')

p += Grid(x0=-1,y0=-2,x1=3,y1=2)

print(p)

```



OR gate testing 3 inputs

```

from latextool_basic import *
from latexcircuit import *

p = Plot()
g0 = OR_GATE(linewidth=2, x=0,y=0,h=3, inputs=3, linecolor='blue')
p += str(g0)

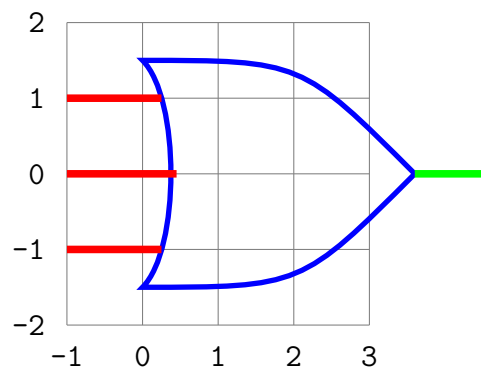
for x0,y0 in g0.inputs():
    p0 = (x0, y0)
    p1 = (-1, y0)
    p += Line(points=[p0,p1], linewidth=0.1, linecolor='red')

x0,y0 = g0.output(); p0 = (x0, y0)
x1,y1 = x0 + 1, y0; p1 = (x1, y1)
p += Line(points=[p0, p1], linewidth=0.1, linecolor='green')

p += Grid(x0=-1,y0=-2,x1=3,y1=2)

print(p)

```



OR gate testing 4 inputs

```

from latextool_basic import *
from latexcircuit import *

p = Plot()
g0 = OR_GATE(linewidth=2, x=0,y=0,h=3, inputs=4,linecolor='blue')
p += str(g0)

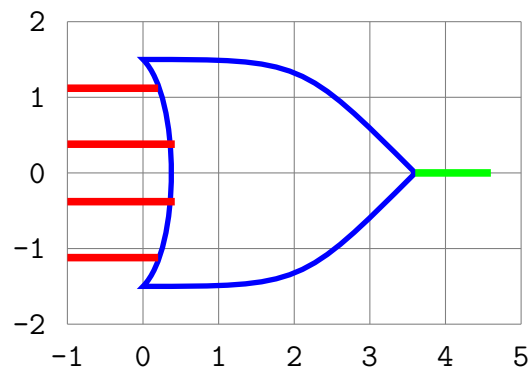
for x0,y0 in g0.inputs():
    p0 = (x0, y0); x1, y1 = -1, y0; p1 = (x1, y1)
    p += Line(points=[p0,p1], linewidth=0.1, linecolor='red')

x0,y0 = g0.output()
p0 = (x0, y0); p1 = (x0 + 1, y0)
p += Line(points=[p0,p1], linewidth=0.1, linecolor='green')

p += Grid(x0=-1, y0=-2, x1=5, y1=2)

print(p)

```



50.13 Rotation

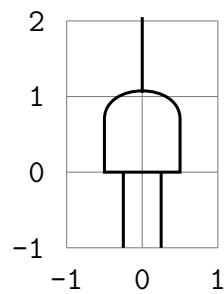
AND gate rotate by $\pi/4$:

```
from math import pi
from latextool_basic import *
from latexcircuit import *
p = Plot()
g = AND_GATE(x=0, y=0, inputs=2, angle=pi/2)
p += str(g)

for x,y in g.inputs():
    p += Line(points=[(x, y - 1), (x, y)])

x,y = g.output()
p += Line(points=[(x, y), (x, y + 1)])

p += Grid(x0=-1, y0=-1, x1=1, y1=2)
print(p)
```



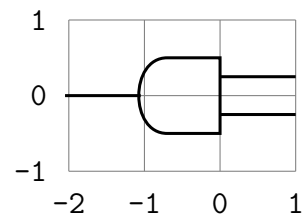
AND gate rotate by π :

```
from math import pi
from latextool_basic import *
from latexcircuit import *
p = Plot()
g = AND_GATE(x=0, y=0, inputs=2, angle=pi)
p += str(g)

for x,y in g.inputs():
    p += Line(points=[(x + 1, y), (x, y)])

x,y = g.output()
p += Line(points=[(x, y),(x - 1, y)])

p += Grid(x0=-2, y0=-1, x1=1, y1=1)
print(p)
```



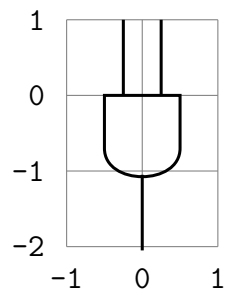
AND gate rotate by $3\pi/2$:

```
from math import pi
from latextool_basic import *
from latexcircuit import *
p = Plot()
g = AND_GATE(x=0, y=0, inputs=2, angle=3*pi/2)
p += str(g)

for x,y in g.inputs():
    p += Line(points=[(x, y + 1), (x, y)])

x,y = g.output()
p += Line(points=[(x, y),(x, y - 1)])

p += Grid(x0=-1, y0=-2, x1=1, y1=1)
print(p)
```



OR gate rotate by $\pi/4$:

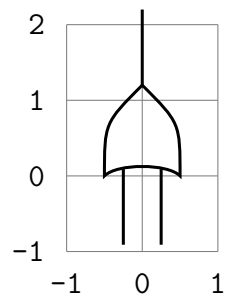
```
from math import pi
from latextool_basic import *
from latexcircuit import *
p = Plot()

g = OR_GATE(x=0, y=0, inputs=2, angle=pi/2)
p += str(g)

for x,y in g.inputs():
    p += Line(points=[(x, y - 1), (x, y)])

x,y = g.output()
p += Line(points=[(x, y), (x, y + 1)])

p += Grid(x0=-1, y0=-1, x1=1, y1=2)
print(p)
```



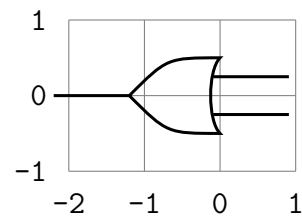
OR gate rotate by π :

```
from math import pi
from latextool_basic import *
from latexcircuit import *
p = Plot()
g = OR_GATE(x=0, y=0, inputs=2, angle=pi)
p += str(g)

for x,y in g.inputs():
    p += Line(points=[(x + 1, y), (x, y)])

x,y = g.output()
p += Line(points=[(x, y),(x - 1, y)])

p += Grid(x0=-2, y0=-1, x1=1, y1=1)
print(p)
```



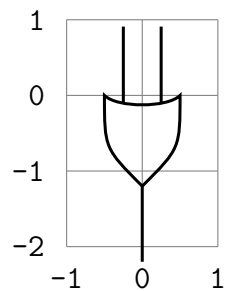
OR gate rotate by $3\pi/2$:

```
from math import pi
from latextool_basic import *
from latexcircuit import *
p = Plot()
g = OR_GATE(x=0, y=0, inputs=2, angle=3*pi/2)
p += str(g)

for x,y in g.inputs():
    p += Line(points=[(x, y + 1), (x, y)])

x,y = g.output()
p += Line(points=[(x, y),(x, y - 1)])

p += Grid(x0=-1, y0=-2, x1=1, y1=1)
print(p)
```



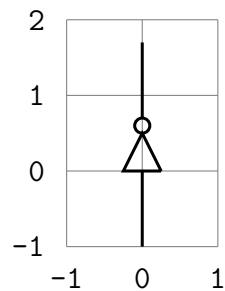
NOT gate rotate by $\pi/4$:

```
from math import pi
from latextool_basic import *
from latexcircuit import *
p = Plot()
g = NOT_GATE(x=0, y=0, inputs=2, angle=pi/2)
p += str(g)

x,y = g.input()
p += Line(points=[(x, y - 1), (x, y)])

x,y = g.output()
p += Line(points=[(x, y), (x, y + 1)])

p += Grid(x0=-1, y0=-1, x1=1, y1=2)
print(p)
```



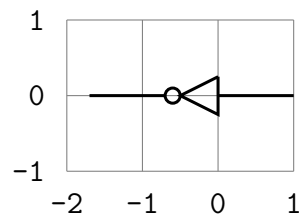
NOT gate rotate by π :

```
from math import pi
from latextool_basic import *
from latexcircuit import *
p = Plot()
g = NOT_GATE(x=0, y=0, inputs=2, angle=pi)
p += str(g)

x,y = g.input()
p += Line(points=[(x + 1, y), (x, y)])

x,y = g.output()
p += Line(points=[(x, y),(x - 1, y)])

p += Grid(x0=-2, y0=-1, x1=1, y1=1)
print(p)
```



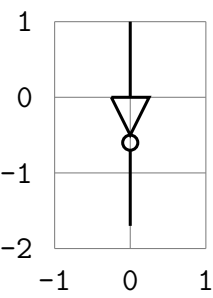
NOT gate rotate by $3\pi/2$:

```
from math import pi
from latextool_basic import *
from latexcircuit import *
p = Plot()
g = NOT_GATE(x=0, y=0, inputs=2, angle=3*pi/2)
p += str(g)

x,y = g.input()
p += Line(points=[(x, y + 1), (x, y)])

x,y = g.output()
p += Line(points=[(x, y),(x, y - 1)])

p += Grid(x0=-1, y0=-2, x1=1, y1=1)
print(p)
```



Test OUTPUT POINT

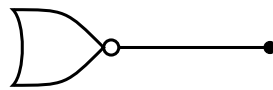
```

from latextool_basic import *
from latexcircuit import *
p = Plot()

NOR = NOR_GATE(x=0, y=3)
P = OUTPUT_POINT(gate=NOR)

p += str(NOR)
p += str(P)
print(p)

```



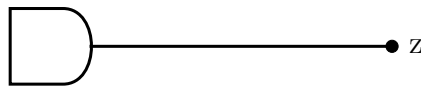
```

from latextool_basic import *
from latexcircuit import *
p = Plot()

AND = AND_GATE(x=0, y=3)
P = OUTPUT_POINT(gate=AND, output_length=4, label='z', anchor='west')

p += str(AND)
p += str(P)
print(p)

```



Test INPUT POINT

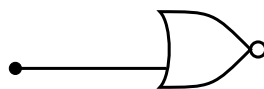
```

from latextool_basic import *
from latexcircuit import *
p = Plot()

NOR = NOR_GATE(x=0, y=3)
P = INPUT_POINT(gate=NOR, input_index=0)

p += str(NOR)
p += str(P)
print(p)

```



Example:

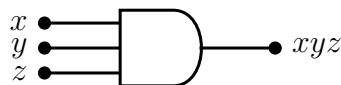
```

from latextool_basic import *
from latexcircuit import *
p = Plot()

AND = AND_GATE(x=0, y=3, inputs=3)
P0 = INPUT_POINT(gate=AND, input_index=2, label='$x$', input_length=1)
P1 = INPUT_POINT(gate=AND, input_index=1, label='$y$', input_length=1)
P2 = INPUT_POINT(gate=AND, input_index=0, label='$z$', input_length=1)
P3 = OUTPUT_POINT(gate=AND, label='$xyz$', anchor='west', output_length=1)

p += str(AND)
p += str(P0)
p += str(P1)
p += str(P2)
p += str(P3)
print(p)

```



Example:

```

from latextool_basic import *
from latexcircuit import *
p = Plot()

and0 = AND_GATE(x=0, y=0, h=1, inputs=3)

x, y = and0.inputs()[2]; x -= 1
X = POINT(x=x, y=y, label='$x$')

x, y = and0.inputs()[1]; x -= 1
Y = POINT(x=x, y=y, label='$y$')

x, y = and0.inputs()[0]; x -= 1
Z = POINT(x=x, y=y, label='$z$')

x, y = and0.output(); x += 1
XYZ = POINT(x=x, y=y, label='$xyz$', anchor='west')

p += str(and0)
p += str(X); p += str(Y); p += str(Z)
p += str(XYZ)

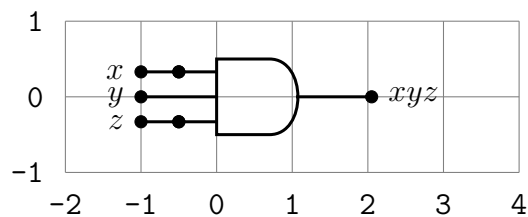
p += '%s' % OrthogonalPath([X.output(), and0.inputs()[2]])

p += '%s' % OrthogonalPath([Y.output(), and0.inputs()[1]])
p += '%s' % OrthogonalPath([Z.output(), and0.inputs()[0]])

p += '%s' % OrthogonalPath([XYZ.output(), and0.output()])

p += Grid(x0=-2, y0=-1, x1=4, y1=1)
print(p)

```



Example:

```

from latextool_basic import *
from latexcircuit import *
p = Plot()

and0 = AND_GATE(x=0,y=0,h=1, inputs=3)
and1 = AND_GATE(x=0,y=2,h=1, inputs=3)
and2 = AND_GATE(x=0,y=4,h=1, inputs=3)

x,y = and1.output(); x += 4
or0 = OR_GATE(x=x, y=y,h=1, inputs=3)

x, y = and1.inputs()[1]; x -= 4
A = POINT(x=x, y=y + 1, label='$A$')
B = POINT(x=x, y=y + 0, label='$B$')
C = POINT(x=x, y=y + -1, label='$C$')
notA = NOT_GATE(x=A.x()+1, y=A.y()-0.5)
notB = NOT_GATE(x=B.x()+1, y=B.y()-0.5)
notC = NOT_GATE(x=C.x()+1, y=C.y()-0.5)

x0,y0 = or0.output()
Z = POINT(x=x0+1, y=y0, label='$z$', anchor='west')

p += str(A); p += str(B); p += str(C)
p += str(notA); p += str(notB); p += str(notC)
p += str(and0); p += str(and1); p += str(and2)
p += str(or0)

p += str(Z)

OP = OrthogonalPath
# Join A to notA, ...
p += '%s' % OP([A.output(), notA.input()])
p += '%s' % OP([B.output(), notB.input()])
p += '%s' % OP([C.output(), notC.input()])

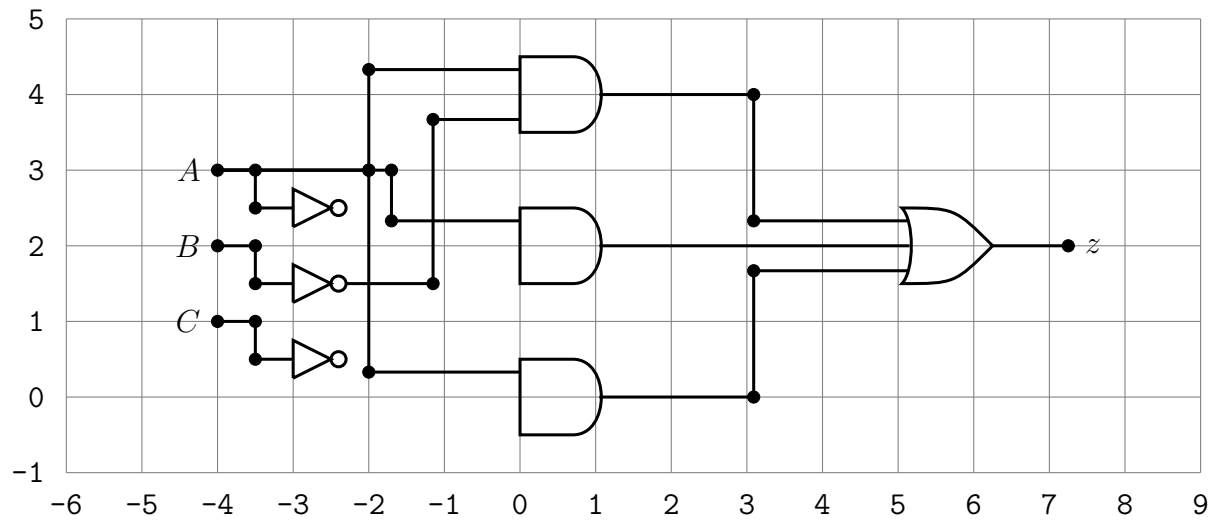
# Join inputs to AND gates
p += '%s' % OP([A.output(), and0.inputs()[2]])
p += '%s' % OP([A.output(), and1.inputs()[2]], shifts=[0.3])
p += '%s' % OP([A.output(), and2.inputs()[2]])
p += '%s' % OP([notB.output(), and2.inputs()[0]])

# Join AND gates to OR
p += '%s' % OP([and0.output(), or0.inputs()[0]])
p += '%s' % OP([and1.output(), or0.inputs()[1]])
p += '%s' % OP([and2.output(), or0.inputs()[2]])

# Join OR to z
p += '%s' % OP([or0.output(), Z.input()])

p += Grid(x0=-6, y0=-1, x1=9, y1=5)
print(p)

```



50.14 Example: SR latch

SR latch:

```

from latextool_basic import *
from latexcircuit import *
p = Plot()

NOR1 = NOR_GATE(x=0, y=3)
R = INPUT_POINT(gate=NOR1, r=0, input_index=1, label='$R$', input_length=1)
R0 = INPUT_POINT(gate=NOR1, r=0, input_index=0, input_length=1)
OUT1 = OUTPUT_POINT(gate=NOR1, output_length=1)
x,y = OUT1.output()
OUT11 = POINT(x=x+2, y=y, r=0, label='$Q$', anchor='west')

NOR2 = NOR_GATE(x=0, y=0)
S = INPUT_POINT(gate=NOR2, r=0, input_index=0, label='$S$', input_length=1)
S0 = INPUT_POINT(gate=NOR2, r=0, input_index=1, input_length=1)
OUT2 = OUTPUT_POINT(gate=NOR2, output_length=1)
x,y = OUT2.output()
OUT22 = POINT(x=x+2, y=y, r=0, label=r'$\overline{Q}$', anchor='west')

OP = OrthogonalPath

p += str(NOR1)
p += str(R)
p += str(R0)
p += str(OUT1); p += str(OUT11)

p += str(NOR2)
p += str(S)
p += str(S0)
p += str(OUT2); p += str(OUT22)

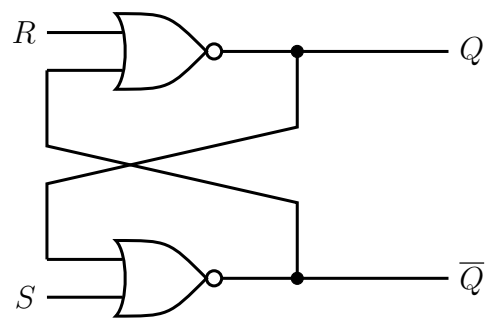
p += str(OP([OUT1.output(), OUT11.input()]))
p += str(OP([OUT2.output(), OUT22.input()]))

# Cross
x0,y0 = R0.input()
x1,y1 = OUT2.input()
p += Line(points=[(x0,y0),(x0,y0-1),(x1,y1+1),(x1,y1)])
p += str(POINT(x=x0,y=y0-1,r=0))
p += str(POINT(x=x1,y=y1+1,r=0))

x0,y0 = S0.input()
x1,y1 = OUT1.input()
p += Line(points=[(x0,y0),(x0,y0+1),(x1,y1-1),(x1,y1)])
p += str(POINT(x=x0,y=y0+1,r=0))
p += str(POINT(x=x1,y=y1-1,r=0))

print(p)

```



Test angle = $3.14159/4$:

```

from latextool_basic import *
from latexcircuit import *
p = Plot()

NOR1 = NOR_GATE(x=0, y=3, angle=3.14159/4)
R = INPUT_POINT(gate=NOR1, input_index=1, label='$R$', input_length=1)
R0 = INPUT_POINT(gate=NOR1, input_index=0, input_length=1)
OUT1 = OUTPUT_POINT(gate=NOR1, output_length=1)
x,y = OUT1.output()
OUT11 = POINT(x=x+2, y=y, label='$Q$', anchor='west')

NOR2 = NOR_GATE(x=0, y=0)
S = INPUT_POINT(gate=NOR2, input_index=0, label='$S$', input_length=1)
S0 = INPUT_POINT(gate=NOR2, input_index=1, input_length=1)
OUT2 = OUTPUT_POINT(gate=NOR2, output_length=1)
x,y = OUT2.output()
OUT22 = POINT(x=x+2, y=y, label='$\overline{Q}$', anchor='west')

OP = OrthogonalPath

p += str(NOR1)
p += str(R)
p += str(R0)
p += str(OUT1); p += str(OUT11)

p += str(NOR2)
p += str(S)
p += str(S0)
p += str(OUT2); p += str(OUT22)

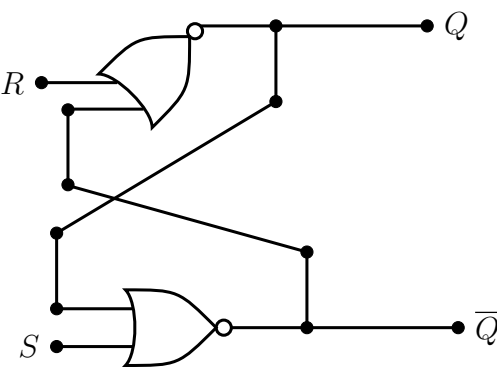
p += str(OP([OUT1.output(), OUT11.input()]))
p += str(OP([OUT2.output(), OUT22.input()]))

# Cross
x0,y0 = R0.input()
x1,y1 = OUT2.input()
p += Line(points=[(x0,y0),(x0,y0-1),(x1,y1+1),(x1,y1)])
p += str(POINT(x=x0,y=y0-1))
p += str(POINT(x=x1,y=y1+1))

x0,y0 = S0.input()
x1,y1 = OUT1.input()
p += Line(points=[(x0,y0),(x0,y0+1),(x1,y1-1),(x1,y1)])
p += str(POINT(x=x0,y=y0+1))
p += str(POINT(x=x1,y=y1-1))

print(p)

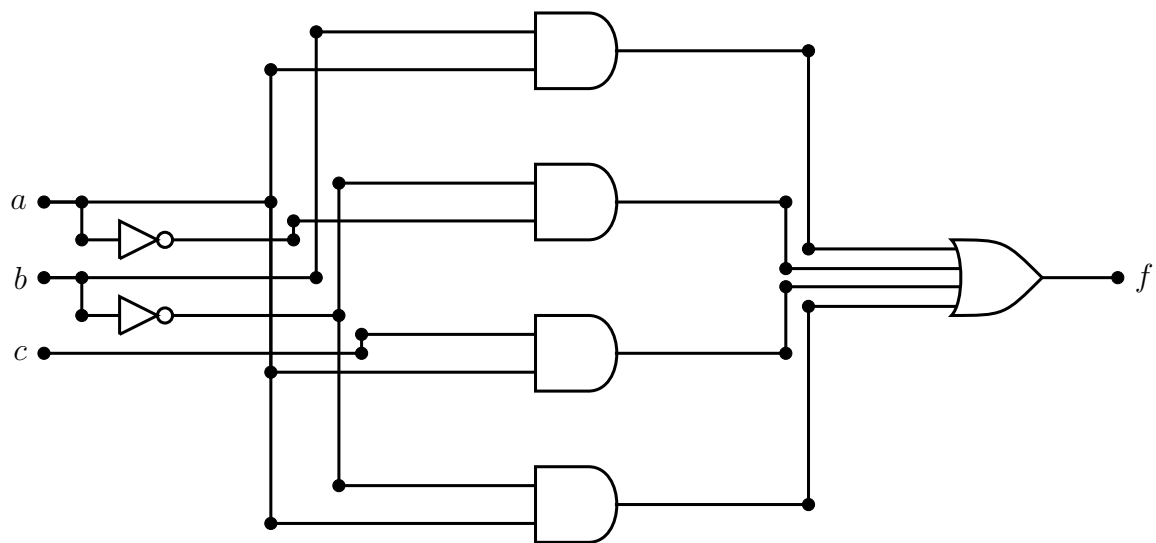
```



50.15 layout function

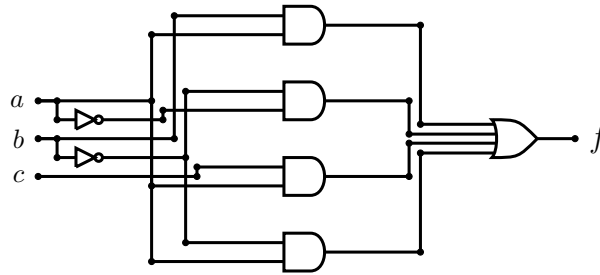
SOP example:

```
from latextool_basic import *
from latexcircuit import *
p = Plot()
expr = [["a", "b"], ["a'", "b'"], ["a", "c"], ["a", "b'"]]
layout(p, expr, AND_GATE, OR_GATE)
print(p)
```



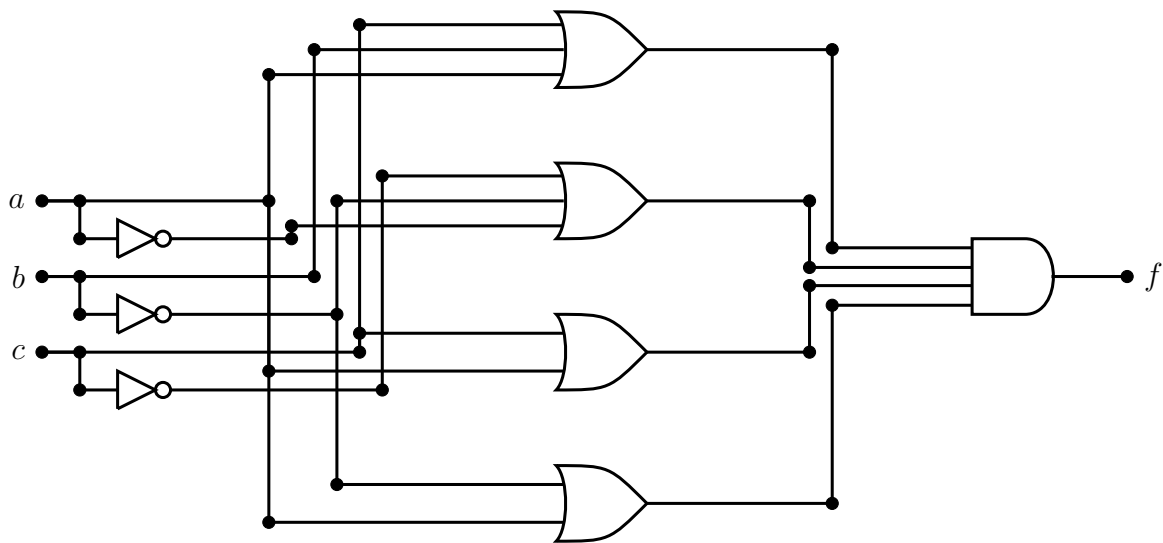
layout with scale and font for label:

```
from latextool_basic import *
from latexcircuit import *
p = Plot(scale=0.5)
expr = [["a", "b"], ["a'", "b'"], ["a", "c"], ["a", "b'"]]
layout(p, expr, AND_GATE, OR_GATE, font='footnotesize')
print(p)
```



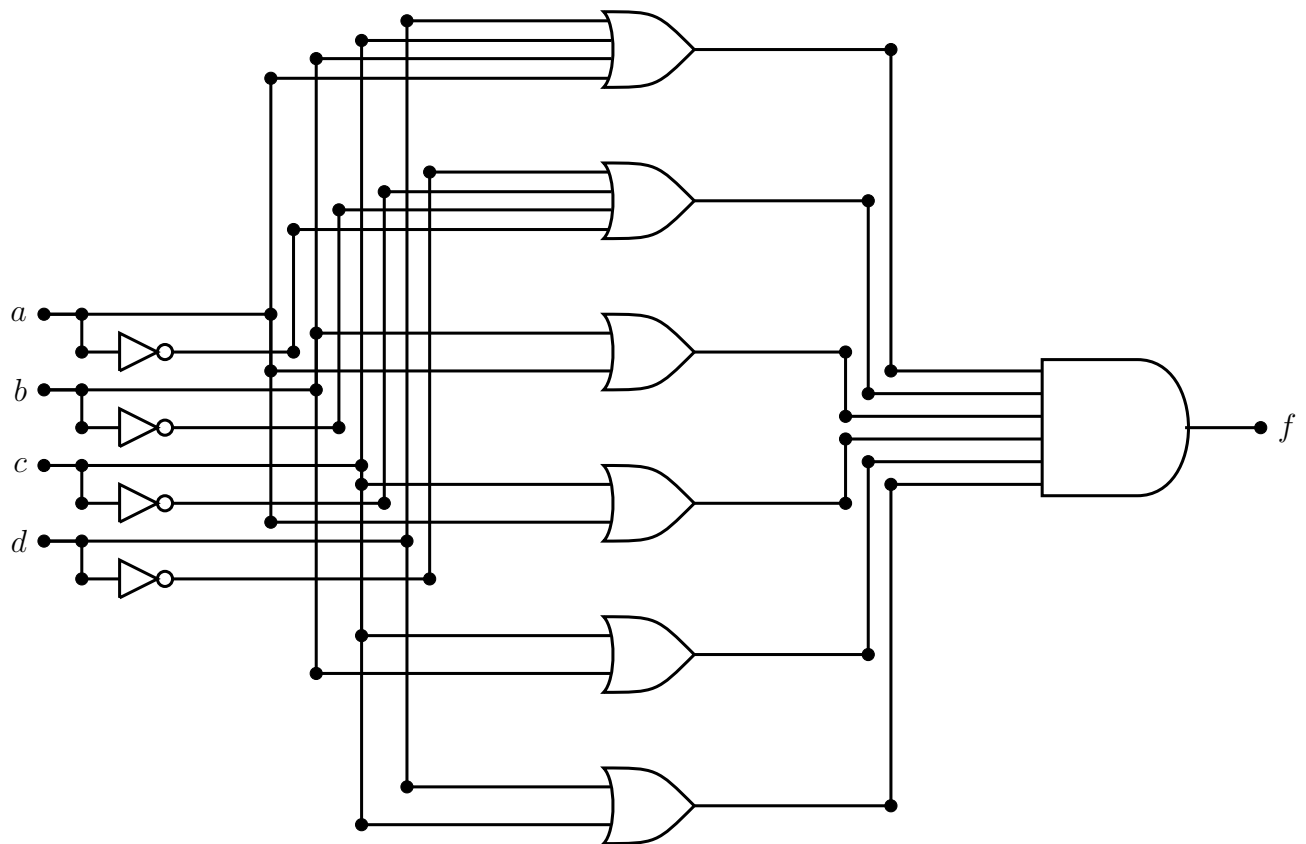
POS example:

```
from latextool_basic import *
from latexcircuit import *
p = Plot()
expr = [["a", "b", "c"], ["a'", "b'", "c'"], ["a", "c"], ["a", "b'"]]
layout(p, expr, OR_GATE, AND_GATE)
print(p)
```



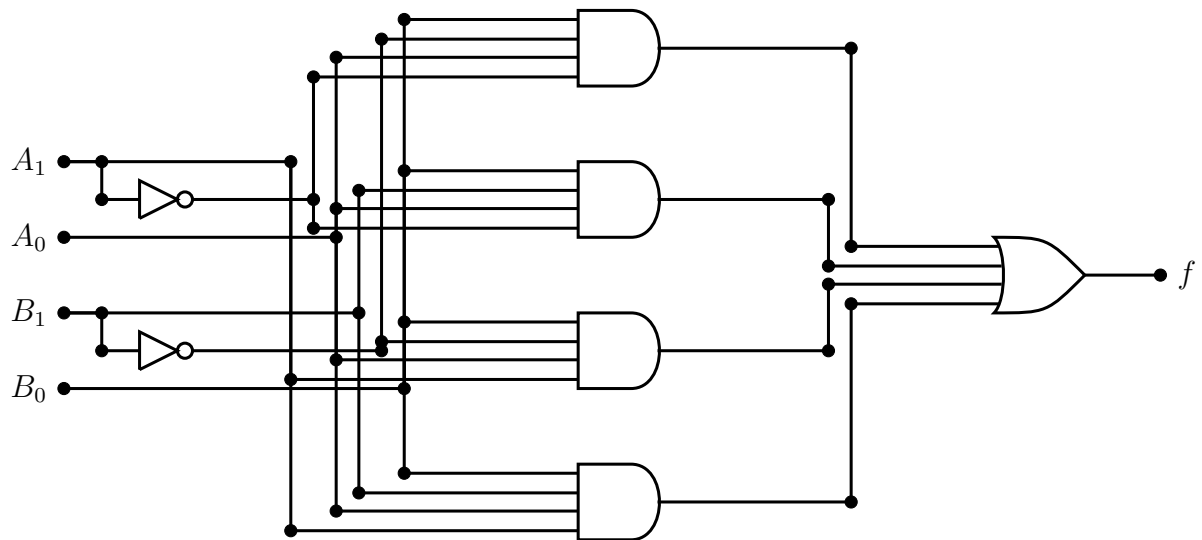
POS example:

```
from latextool_basic import *
from latexcircuit import *
p = Plot()
expr = [["a", "b", "c", "d"], ["a'", "b'", "c'", "d'"],
        ["a", "b"], ["a", "c"], ["b", "c"], ["c", "d"]]
layout(p, expr, OR_GATE, AND_GATE)
print(p)
```



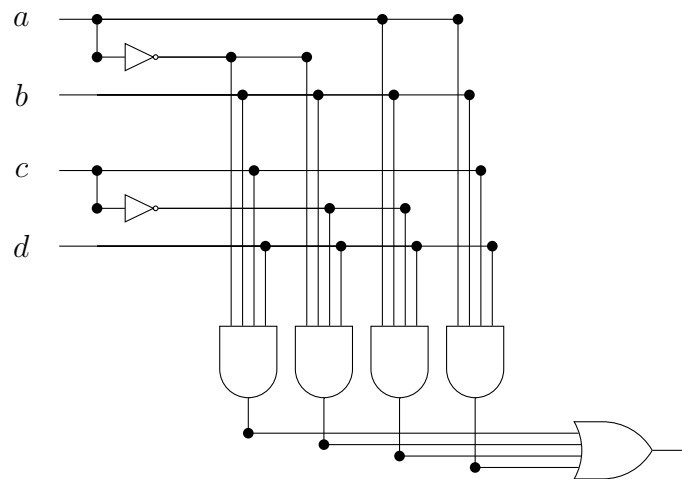
POS example:

```
from latextool_basic import *
from latexcircuit import *
p = Plot()
expr = [
    ["A_1'", "A_0", "B_1'", "B_0"],
    ["A_1'", "A_0", "B_1", "B_0"],
    ["A_1", "A_0", "B_1'", "B_0"],
    ["A_1", "A_0", "B_1", "B_0"],
]
layout(p, expr, AND_GATE, OR_GATE)
print(p)
```



SOP example:

```
from latextool_basic import console
from latexcircuit import SOP2
s = SOP2("a'bcd + a'bc'd + abc'd + abcd")
print(s)
```



50.16 Logic blocks

`points` argument is a dictionary.

`points[k]` is (label, coordinates). `label` is used to label the point in the block. `coordinate` is the coordinates of the point related to the block.

You can get the coordinates of a point by calling the `block.point(k)`.

```
from latextool_basic import *
from latexcircuit import *
p = Plot()
g = LOGIC_BLOCK(x=0, y=0, w=2, h=3, s='half adder',
                points={'s':('$s$', (2,2.5)),
                       'c':('$c$', (2,0.5)),
                       'x':('', (0,2.5)),
                       'y':('', (0,0.5))})
p += str(g)

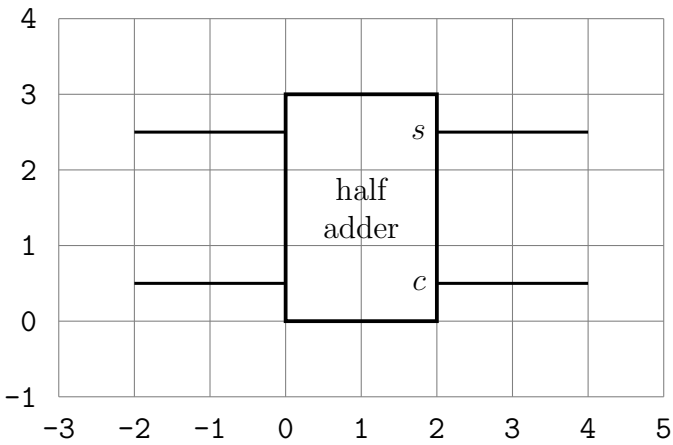
x,y = g.point('x'); P0 = POINT(x=x-2, y=y, label='$x$', anchor='east')
p += str(OrthogonalPath([P0.output(), (x,y)]))

x,y = g.point('y'); P1 = POINT(x=x-2, y=y, label='$y$', anchor='east')
p += str(OrthogonalPath([P1.output(), (x,y)]))

x,y = g.point('s'); P2 = POINT(x=x+2, y=y, label='$s$', anchor='east')
p += str(OrthogonalPath([P2.output(), (x,y)]))

x,y = g.point('c'); P2 = POINT(x=x+2, y=y, label='$c$', anchor='east')
p += str(OrthogonalPath([P2.output(), (x,y)]))

p += Grid(-3, -1, 5, 4)
print(p)
```




```

from latextool_basic import *
from latexcircuit import *
p = Plot()
g = LOGIC_BLOCK(x=0, y=0, w=2, h=3, s='half adder',
                points={'s':('$s$', (2,2.5)),
                       'c':('$c$', (2,0.5)),
                       'x':('x', (0,2.5)),
                       'y':('y', (0,0.5))})

p += str(g)

x,y = g.point('x'); P0 = POINT(x=x-2, y=y, label='$x$', anchor='east')
p += str(OrthogonalPath([P0.output(), (x,y)]))

x,y = g.point('y'); P1 = POINT(x=x-2, y=y, label='$y$', anchor='east')
p += str(OrthogonalPath([P1.output(), (x,y)]))

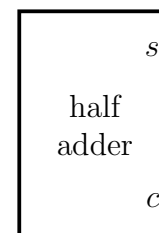
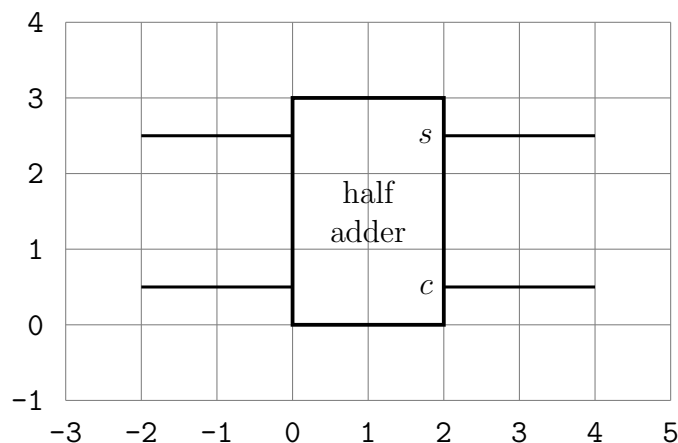
x,y = g.point('s'); P2 = POINT(x=x+2, y=y, label='$s$', anchor='east')
p += str(OrthogonalPath([P2.output(), (x,y)]))

x,y = g.point('c'); P2 = POINT(x=x+2, y=y, label='$c$', anchor='east')
p += str(OrthogonalPath([P2.output(), (x,y)]))

p += '%s' % LOGIC_BLOCK(x=6, y=1, h=3, s='half adder',
                        points={'s':('$s$', (2,2.5)),
                               'c':('$c$', (2,0.5)),
                               'x':('x', (0,2.5)),
                               'y':('y', (0,0.5))})

p += Grid(-3, -1, 5, 4)
print(p)

```



50.17 Logic blocks: full adder

```

from latextool_basic import *
from latexcircuit import *
p = Plot()

g0 = HALF_ADDER(x=0, y=-1.5)
g0_in0 = g0.point('in0')
g0_in1 = g0.point('in1')
g0_out0 = g0.point('out0')
g0_out1 = g0.point('out1')

g1 = HALF_ADDER(x=4, y=0)
g1_in0 = g1.point('in0')
g1_in1 = g1.point('in1')
g1_out0 = g1.point('out0')
g1_out1 = g1.point('out1')

g2 = OR_GATE(x=8, y=-0.5)
g2_in0 = g2.inputs()[1]
g2_in1 = g2.inputs()[0]
g2_out = g2.output()

p += str(g0); p += str(g1); p += str(g2)

# x0,y0 = position for A
x,y = g0_in0; x0 = x - 2; y0 = y
A = POINT(x=x0, y=y0, label='$A$'); p += str(A)
B = POINT(x=A.input()[0], y=A.input()[1] - 1.5, label='$B$'); p += str(B)
C0 = POINT(x=A.input()[0], y=A.input()[1] + 1.5, label='$C_0$'); p += str(C0)

x,y = g2_out; x2 = x + 1
C1 = POINT(x=x2, y=y, label='$C_1$', anchor='west'); p += str(C1)

x,y = C1.input()[0], g1_out0[1]
S = POINT(x=x, y=y, label='$S$', anchor='west'); p += str(S)

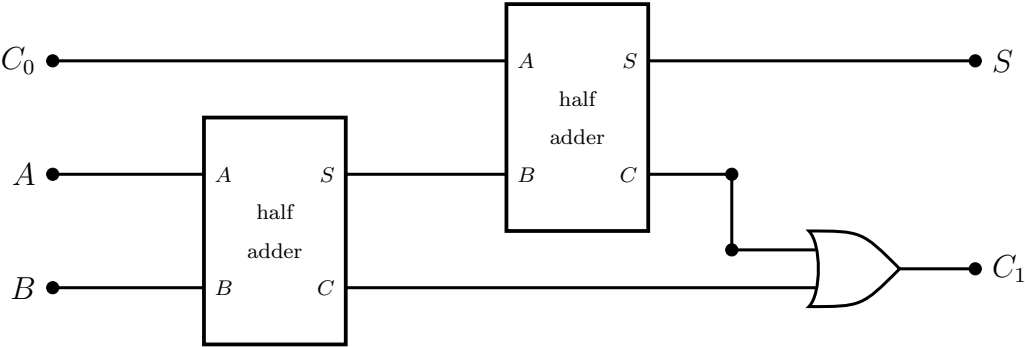
opath = OrthogonalPath

p += '%s' % opath(points=[A.output(), g0_in0]) # A -> g0
p += '%s' % opath(points=[B.output(), g0_in1], shifts=[-0.5]) # B -> g0
p += '%s' % opath(points=[C0.output(), g1_in0]) # C0 -> g1

p += '%s' % opath(points=[g0_out0, g1_in1]) # g0 -> g1
p += '%s' % opath(points=[g0_out1, g2_in1]) # g0 -> g2
p += '%s' % opath(points=[g1_out1, g2_in0]) # g1 -> g2
p += '%s' % opath(points=[g1_out0, S.input()]) # g1 -> S
p += '%s' % opath(points=[g2_out, C1.input()]) # g2 -> C1

print(p)

```



50.18 Two NOTs

```

from latextool_basic import *
from latexcircuit import *
p = Plot()

g0 = NOT_GATE(x=0, y=0)
x,y = g0.output()
g1 = NOT_GATE(x=2, y=y)

p += str(g0)
p += str(g1)

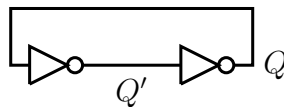
p += Line(points=[g0.output(), g1.input()], label="$Q'$", anchor='below')

x0, y0 = g1.output()
x5, y5 = g0.input()

dx = 0.25
dy = 0.75

x1, y1 = x0+dx, y0
x2, y2 = x1, y1+dy
x4, y4 = x5-dx, y5
x3, y3 = x4, y2
p += Line(points=[(x0, y0),
                  (x1, y1),
                  (x2, y2),
                  (x3, y3),
                  (x4, y4),
                  (x5, y5)])
X = POINT(x=x1, y=y1, r=0, label='$Q$', anchor='west')
p += str(X)
print(p)

```



51 Data structures (chap-data-structures.tex)

51.1 Singly linked list (singlylinkedlist.tex)

```
from latextool_basic import *
p = Plot()

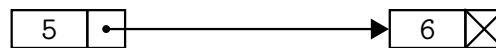
SLLNode = SinglyLinkedListNode

n5 = SLLNode(x=0, y=0, label=5)
p += n5

n6 = SLLNode(x=5, y=0, label=6, next=None)
p += n6

p += Pointer(points = [n5[1].center(), n6[0].left()])

print(p)
```



51.2 Doubly linked list (doublylinkedlist.tex)

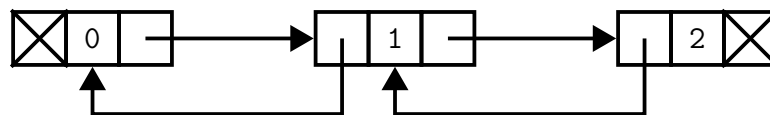
```

from latextool_basic import *

c = DLNode(x0=1, y0=1, key=r'\texttt 0')
d = DLNode(x0=5, y0=1, key=r'\texttt 1')
e = DLNode(x0=9, y0=1, key=r'\texttt 2')
c.next = d; d.prev = c
d.next = e; e.prev = d

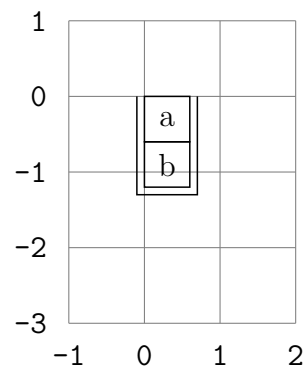
p = Plot()
p += str(c); p += str(d); p += str(e)
print(p)

```



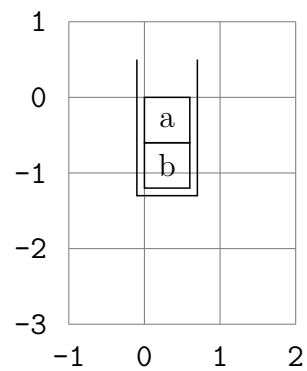
51.3 Stack (stack.tex)

```
from latextool_basic import *
p = Plot()
drawstack(p,
    xs=['a', 'b'],
)
p += Grid(-1, -3, 2, 1)
print(p)
```



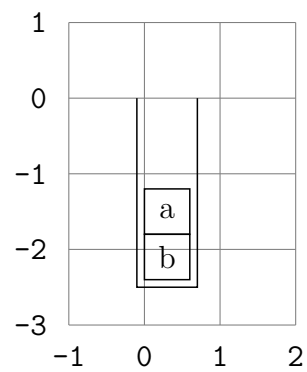
Extra height for container:

```
from latextool_basic import *
p = Plot()
drawstack(p,
  xs=['a', 'b'],
  extra_h=0.5
)
p += Grid(-1, -3, 2, 1)
print(p)
```



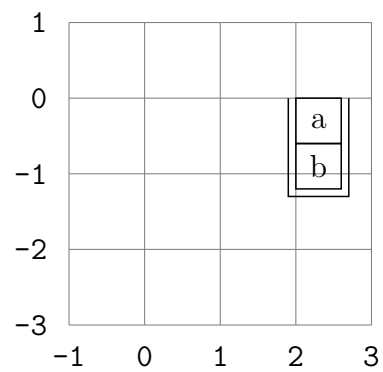
Minimum number of values:

```
from latextool_basic import *
p = Plot()
drawstack(p,
  xs=['a', 'b'],
  min_size=4
)
p += Grid(-1, -3, 2, 1)
print(p)
```



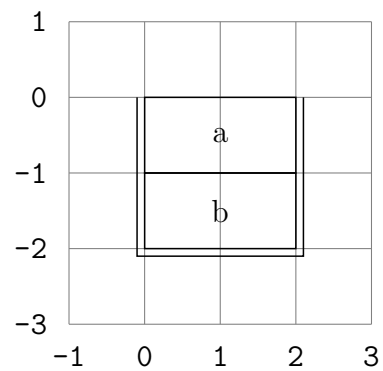
Position:

```
from latextool_basic import *
p = Plot()
drawstack(p,
    x=2, y=0,
    xs=['a', 'b'],
)
p += Grid(-1, -3, 3, 1)
print(p)
```



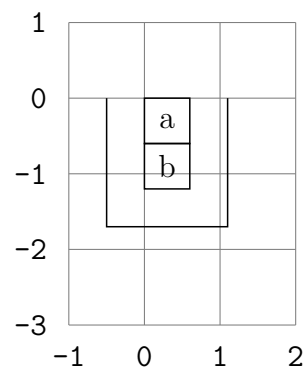
Width and height:

```
from latextool_basic import *
p = Plot()
drawstack(p,
    w=2, h=1,
    xs=['a', 'b'],
)
p += Grid(-1, -3, 3, 1)
print(p)
```



Distance of walls to values:

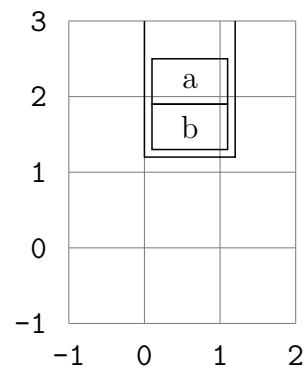
```
from latextool_basic import *  
p = Plot()  
drawstack(p,  
    xs=['a', 'b'],  
    sep=0.5,  
    )  
p += Grid(-1, -3, 2, 1)  
print(p)
```



To make bottom right of container at (0,0):

```
from latextool_basic import *
p = Plot()

sep = 0.1
w = 1; h = 0.6
x = sep; y = sep + 4*h
drawstack(p,
  x=x, y=y, w=w, h=h, extra_h=0.5,
  xs=['a', 'b'],
  sep=sep,
)
p += Grid(-1, -1, 2, 3)
print(p)
```



Return value is a RectContainer:

```
from latextool_basic import *
p = Plot()

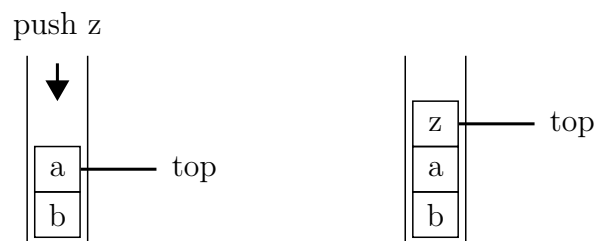
r0 = drawstack(p,
    x=0, y=0,
    xs=['a', 'b'],
    min_size=4
)
x0,y0 = r0[2].right()
x1,y1 = x0 + 1, y0
p += Line(points=[(x0,y0),(x1,y1)])
p += Rect(x0=x1+0.5,y0=y0,x1=x1+0.5,y1=y0,label='top', linewidth=0)

x0,y0 = r0[1].top(); y0 + 0.5
x1,y1 = x0,y0+1
p += Rect(x0=x1, y0=y1, x1=x1,y1=y1,label='push z', linewidth=0)

y1 -= 0.5
p += Line(points=[(x1,y1),(x0,y0)], endstyle='>')

r1 = drawstack(p,
    x=5, y=0,
    xs=['z', 'a', 'b'],
    min_size=4
)
x0,y0 = r1[1].right()
x1,y1 = x0 + 1, y0
p += Line(points=[(x0,y0),(x1,y1)])
p += Rect(x0=x1+0.5,y0=y0,x1=x1+0.5,y1=y0,label='top', linewidth=0)

print(p)
```



51.4 Adjacency list (adjlist.tex)

Even number of linked lists:

```
from latextool_basic import *
p = Plot()

r, crosses, nodes, lines \
= adjlist(xs=[[0],[4,5],[1,2,3],[1,4], [5],[7,7,7]],
          nodevspace=0.5, nodehspace=0.7,
          nodewidth=0.7, nodeheight=0.4)

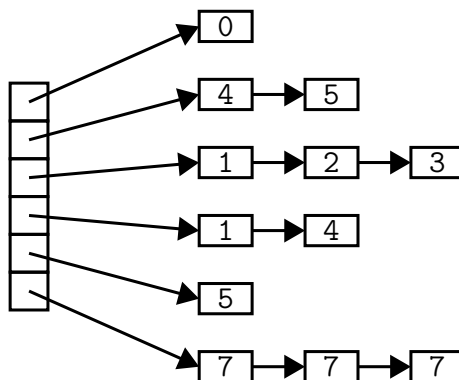
p += r

for k,v in crosses.items():
    for _ in v: p += _

for k,v in nodes.items():
    for _ in v: p += _

for k,v in lines.items():
    for _ in v: p += _

print(p)
```



```

from latextool_basic import *
p = Plot()

r, crosses, nodes, lines = adjlist(xs=[[0],[4,5],[1,2,3],[1,4], [5],[7,7,7]
],
                                nodevspace=0.5, nodehspace=1,
                                nodewidth=0.7, nodeheight=0.4)

p += r

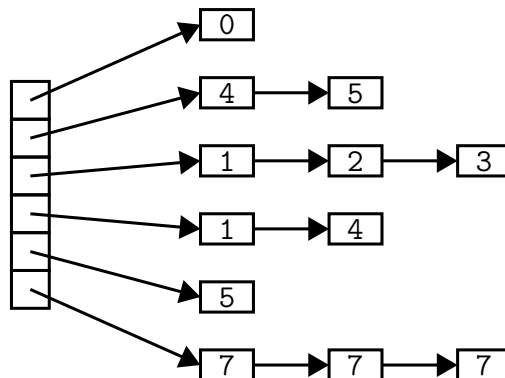
for k,v in crosses.items():
    for _ in v: p += _

for k,v in nodes.items():
    for _ in v: p += _

for k,v in lines.items():
    for _ in v: p += _

print(p)

```



```

from latextool_basic import *
p = Plot()

r, crosses, nodes, lines = adjlist(xs=[[1,2,3],[1,4],],
                                   nodevspace=0.2, nodehspace=0.3,
                                   nodewidth=1, nodeheight=1)

p += r

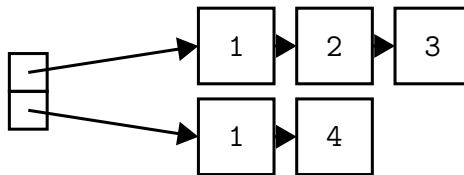
for k,v in crosses.items():
    for _ in v: p += _

for k,v in nodes.items():
    for _ in v: p += _

for k,v in lines.items():
    for _ in v: p += _

print(p)

```



```

from latextool_basic import *
p = Plot()

r, crosses, nodes, lines = adjlist(xs=[[0],[4,5],[], [1,4], [5],[]])
p += r

r0 = r[1] # redraw r[1] with 'blue!20'
r0.background = 'blue!20'
r0.linecolor = 'blue'
p += r0

for k,v in crosses.items():
    for _ in v: p += _

for k,v in nodes.items():
    for _ in v: p += _

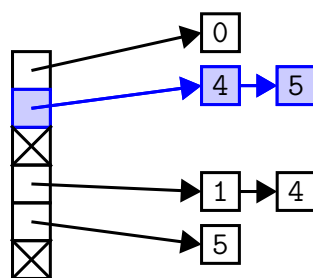
# redraw nodes[1]
for _ in nodes[1]:
    _[0].background = 'blue!20'
    _[0].linecolor = 'blue'
    p += _[0]

for k,v in lines.items():
    for _ in v: p += _

# redraw lines[1]
for _ in lines[1]:
    _[0].linecolor = 'blue'
    p += _

print(p)

```



Test odd number of pointers:

```

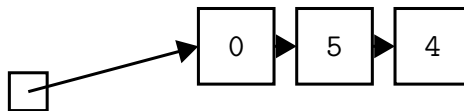
from latextool_basic import *
p = Plot()

r, crosses, nodes, lines = adjlist(xs=[[0,5,4]],
                                   nodevspace=0.2, nodehspace=0.3,
                                   nodewidth=1, nodeheight=1)

p += r
for k,v in crosses.items():
    for _ in v: p += _
for k,v in nodes.items():
    for _ in v: p += _
for k,v in lines.items():
    for _ in v: p += _

print(p)

```



```

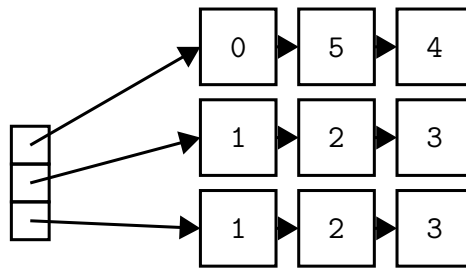
from latextool_basic import *
p = Plot()

r, crosses, nodes, lines = adjlist(xs=[[0,5,4],[1,2,3],[1,2,3]],
                                   nodevspace=0.2, nodehspace=0.3,
                                   nodewidth=1, nodeheight=1)

p += r
for k,v in crosses.items():
    for _ in v: p += _
for k,v in nodes.items():
    for _ in v: p += _
for k,v in lines.items():
    for _ in v: p += _

print(p)

```



```

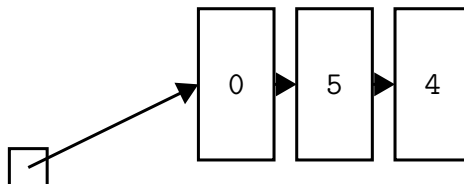
from latextool_basic import *
p = Plot()

r, crosses, nodes, lines = adjlist(xs=[[0,5,4]],
                                   nodevspace=0.2, nodehspace=0.3,
                                   nodewidth=1, nodeheight=2)

p += r
for k,v in crosses.items():
    for _ in v: p += _
for k,v in nodes.items():
    for _ in v: p += _
for k,v in lines.items():
    for _ in v: p += _

print(p)

```



PYTHON ERROR. See source, stderr, stdout below

```

from latextool_basic import *
p = Plot()

r, crosses, nodes, lines = adjlist(xs=[[0,5,4]],
                                   nodevspace=0.2, nodehspace=0.3,
                                   nodewidth=1, nodeheight=1)
)
p += r
for k,v in crosses.items():
    for _ in v: p += _
for k,v in nodes.items():
    for _ in v: p += _
for k,v in lines.items():
    for _ in v: p += _

print(p)

```

```

File "zsqvztmt.tmp.py", line 7
    )
    ^
SyntaxError: invalid syntax

```

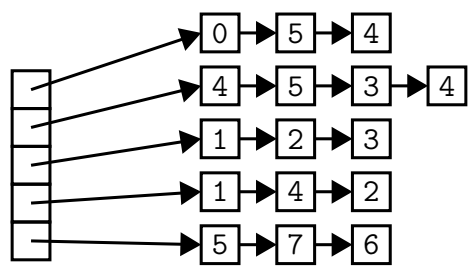
```

from latextool_basic import *
p = Plot()

r, crosses, nodes, lines = adjlist(xs=[[0,5,4],[4,5,3,4],[1, 2, 3], [1,4,2]
, [5,7,6]])
p += r
for k,v in crosses.items():
    for _ in v: p += _
for k,v in nodes.items():
    for _ in v: p += _
for k,v in lines.items():
    for _ in v: p += _

print(p)

```



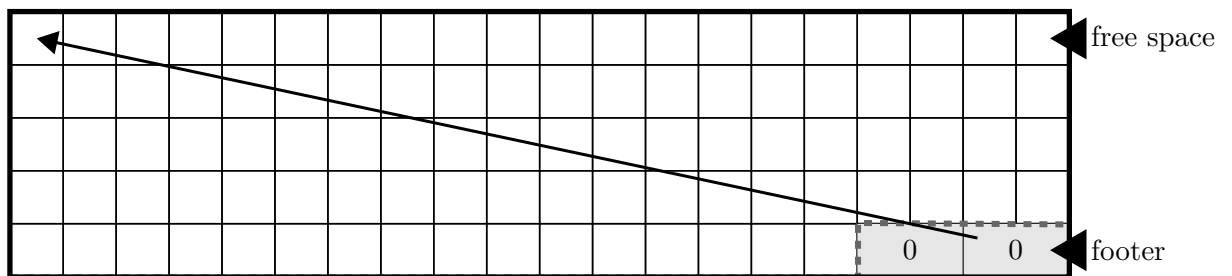
51.5 Heapfile (heapfile.tex)

```
from latextool_basic import *

p = Plot()

nrows = 5
ncols = 20
slots = []

heapfilepage(p, nrows, ncols, slots)
print(p)
```

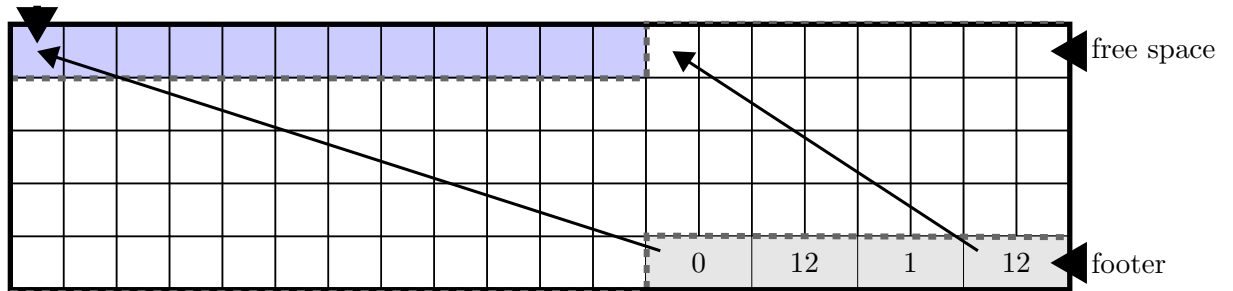


```
from latextool_basic import *
p = Plot()

nrows = 5
ncols = 20
slots = [(0,12)]

heapfilepage(p, nrows, ncols, slots)
print(p)
```

record space



```

from latextool_basic import *

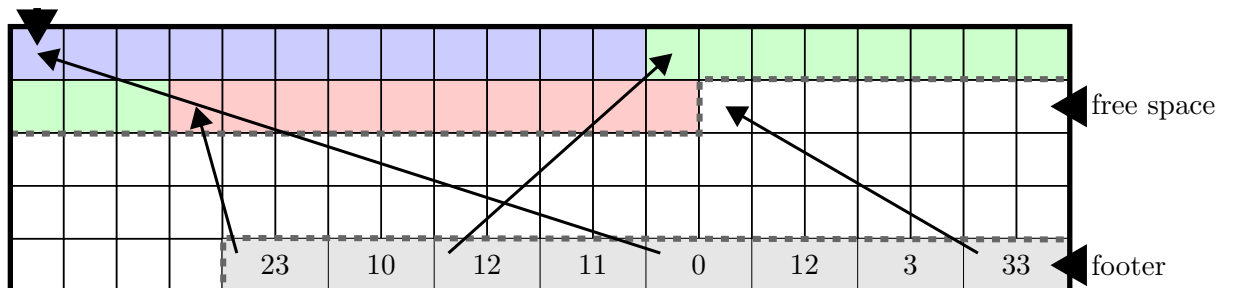
p = Plot()

nrows = 5
ncols = 20
slots = [(0, 12), (12, 11), (23, 10)]

heapfilepage(p, nrows, ncols, slots)
print(p)

```

record space



```

from latextool_basic import *

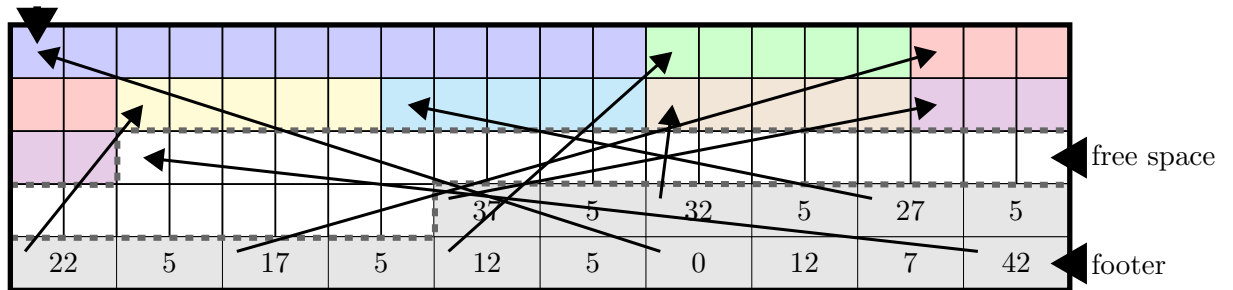
p = Plot()

nrows = 5
ncols = 20
slots = [(0,12), (12,5), (17,5), (22,5), (27,5), (32,5), (37,5)]

heapfilepage(p, nrows, ncols, slots)
print(p)

```

record space



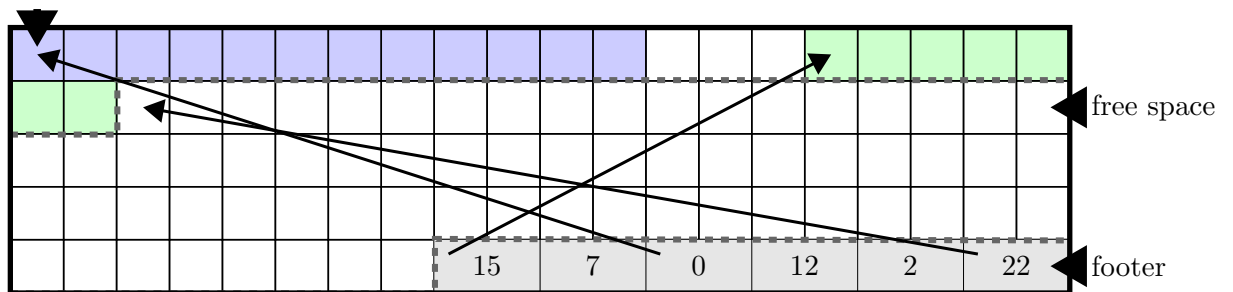
Test with holes:

```
from latextool_basic import *
p = Plot()

nrows = 5
ncols = 20
slots = [(0,12), (15, 7)]

heapfilepage(p, nrows, ncols, slots)
print(p)
```

record space



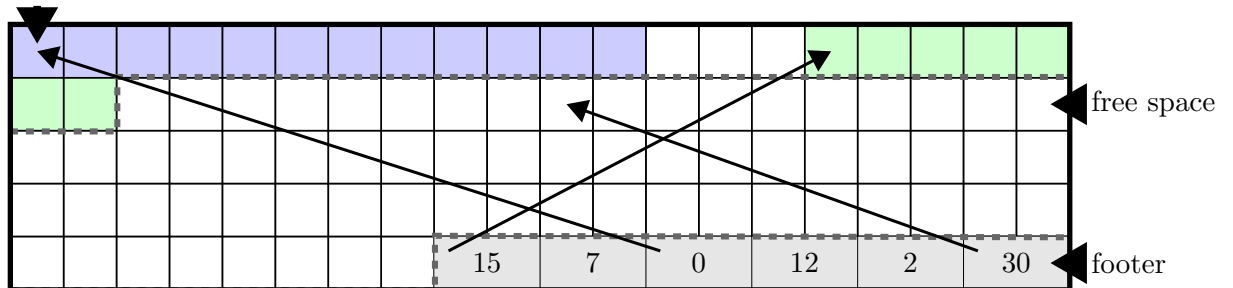
free_offset:

```
from latextool_basic import *
p = Plot()

nrows = 5
ncols = 20
slots = [(0,12), (15, 7)]
free_offset = 30

heapfilepage(p, nrows, ncols, slots, free_offset)
print(p)
```

record space



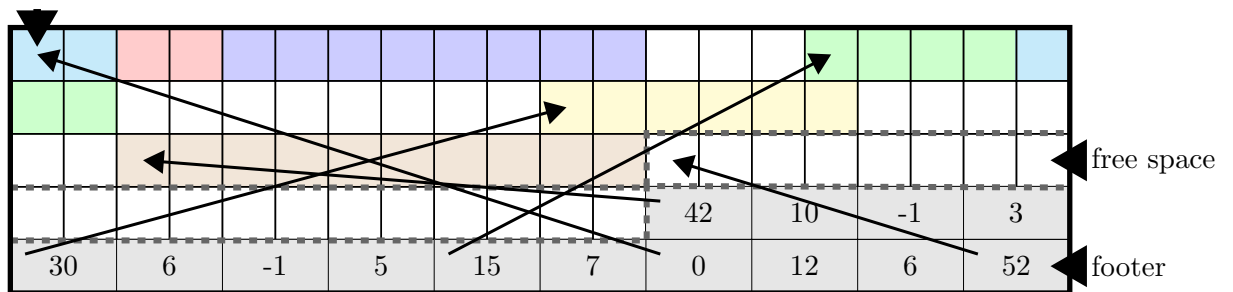
deleted records:

```
from latextool_basic import *
p = Plot()

nrows = 5
ncols = 20
slots = [(0,12), (15, 7), (-1, 5), (30, 6), (-1, 3), (42, 10)]

heapfilepage(p, nrows, ncols, slots)
print(p)
```

record space



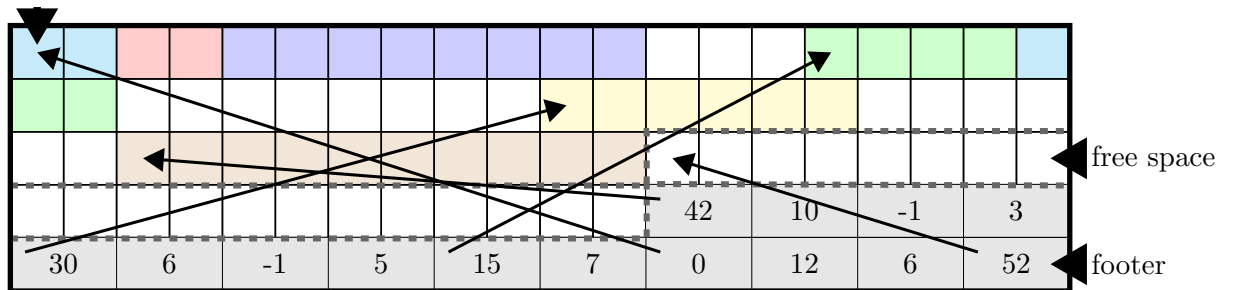
deleted records:

```
from latextool_basic import *
p = Plot()

nrows = 5
ncols = 20
slots = [(0,12), (15, 7), (-1, 5), (30, 6), (-1, 3), (42, 10)]

heapfilepage(p, nrows, ncols, slots)
print(p)
```

record space



51.6 B+ tree (bptree.tex)

```

from latextool_basic import *
p = Plot()

widths= [0.15, 0.4]; height = 0.4

def f(M):
    return [r'{{\scriptsize\texttt{s}}}' % _ for _ in M]

M0 = f([5, 10])

Y = 0
N0 = bpt_node(x=4.5, y=Y, M=M0, widths=widths, height=height)

p += N0
print(p)

```

5	10
---	----

Key width:

```
from latextool_basic import *
p = Plot()

widths= [0.15, 2]; height = 0.4

def f(M):
    return [r'\scriptsize\texttt{%s}'] % _ for _ in M

M0 = f([5, 10])

Y = 0
N0 = bpt_node(x=4.5, y=Y, M=M0, widths=widths, height=height)

p += N0
print(p)
```

	5	10	
--	---	----	--

51.7 Directed edge

```

from latextool_basic import *
p = Plot()
widths= [0.15, 0.4]; height = 0.4

def f(M):
    return [r'{{\scriptsize\texttt{s}}}' % _ for _ in M]

M0, M1, M2, M3 = f([5, 10]), f([0, 1]), f([5, 6]), f([7, 8])

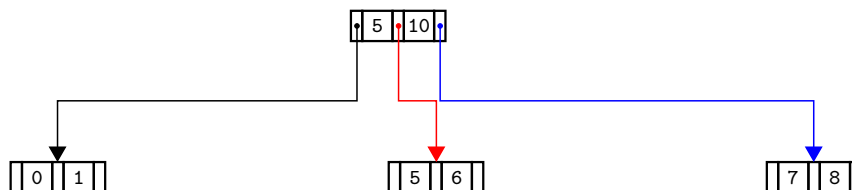
N0 = bpt_node(x=4.5, y=0, M=M0, widths=widths, height=height)
N1 = bpt_node(x=0, y=-2, M=M1, widths=widths, height=height)
N2 = bpt_node(x=5, y=-2, M=M2, widths=widths, height=height)
N3 = bpt_node(x=10, y=-2, M=M3, widths=widths, height=height)

arc0 = bpt_arc(N0, N1, 0);
arc1 = bpt_arc(N0, N2, 1, linecolor='red')
arc2 = bpt_arc(N0, N3, 2, linecolor='blue')

p += N0; p += N1; p += N2; p += N3
p += arc0; p += arc1; p += arc2

print(p)

```



Changing height of the bend:

```
from latextool_basic import *
p = Plot()
widths= [0.15, 0.4]; height = 0.4

def f(M):
    return [r'{{\scriptsize\texttt{s}}}' % _ for _ in M]

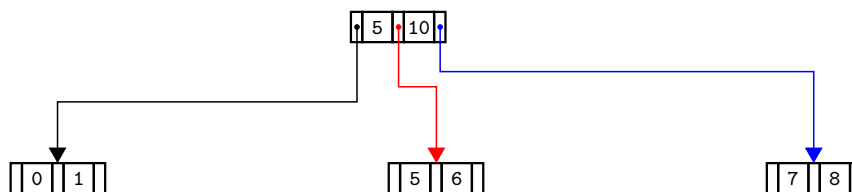
M0, M1, M2, M3 = f([5, 10]), f([0, 1]), f([5, 6]), f([7, 8])

N0 = bpt_node(x=4.5, y=0, M=M0, widths=widths, height=height)
N1 = bpt_node(x=0, y=-2, M=M1, widths=widths, height=height)
N2 = bpt_node(x=5, y=-2, M=M2, widths=widths, height=height)
N3 = bpt_node(x=10, y=-2, M=M3, widths=widths, height=height)

arc0 = bpt_arc(N0, N1, 0);
arc1 = bpt_arc(N0, N2, 1, linecolor='red', delta=0.2)
arc2 = bpt_arc(N0, N3, 2, linecolor='blue', delta=0.4)

p += N0; p += N1; p += N2; p += N3
p += arc0; p += arc1; p += arc2

print(p)
```



Labels:

```

from latextool_basic import *
p = Plot()
widths= [0.15, 0.4]; height = 0.4

def f(M):
    return [r'{{\scriptsize\textttt{%s}}}' % _ for _ in M]

M0 = f([5, 10]); M1 = f([0, 1]); M2 = f([5, 6])

Y = 0
N0 = bpt_node(x=4.5, y=Y, M=M0, widths=widths, height=height)

Y = -2
N1 = bpt_node(x=0, y=Y, M=M1, widths=widths, height=height)
N2 = bpt_node(x=5, y=Y, M=M2, widths=widths, height=height)

arc0 = bpt_arc(N0, N1, 0); arc1 = bpt_arc(N0, N2, 1, linecolor='red')

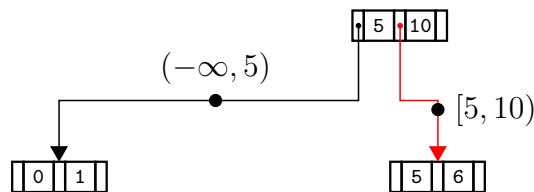
p += N0; p += N1; p += N2
p += arc0; p += arc1

from latexcircuit import *
x,y = arc0.midpoint()
p += Circle(x=x, y=y, r=0.05, background='red')
X = POINT(x=x, y=y, label='$(-\infty,5)$', anchor='south')
p += str(X)

x,y = arc1.midpoint(ratio=0.7)
p += Circle(x=x, y=y, r=0.05, background='blue')
X = POINT(x=x, y=y, label='$[5, 10)$', anchor='west')
p += str(X)

print(p)

```



XXX (why is font not changed for first node):

```

from latextool_basic import *
from latexcircuit import *

def f(M): return tuple(r'\scriptsize\texttt{%s}}' % _ for _ in M)

p = Plot()
widths = [0.1, 0.4]
height = 0.4
node_sep = 0.1
vsep = 1.5 # vertical separation -- from bottom of root to top of child

keyss = [(10, 30, 65, 80), (1, 5, 7, 9), (10, 14, 15, 28),
          (35, 42, 45, 57), (68, 72, 75, 79), (80, 89, 90, 92)]
Ms = [f(keys) for keys in keyss]

Y = -(height + vsep)
children = bptree_get_siblings(0, Y, Ms[1:], widths, height, node_sep)
root = bptree_get_root(0, keyss[0], children, widths, height)

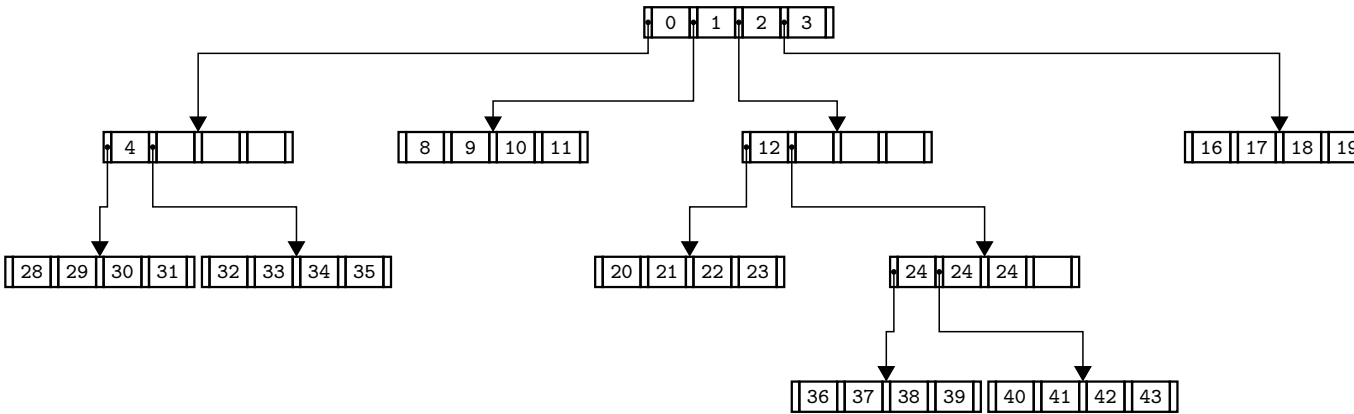
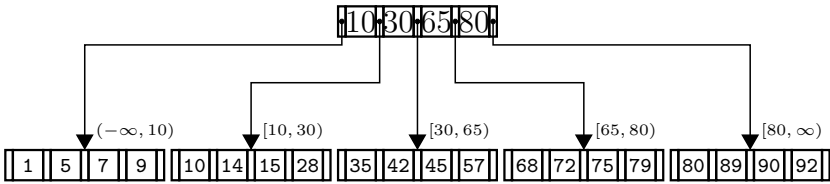
for _ in [root] + children: p += _

arcs = bptree_get_arcs(root, children, vsep, height)
for _ in arcs: p += _

labels = bptree_get_labels(keyss, arcs)
for label in labels:
    p += label

"""
      A
    B  C  D  E
   H I   F G
           J K
"""
edges = {'A': ['B', 'C', 'D', 'E'],
         'D': ['F', 'G'],
         'B': ['H', 'I'],
         'G': ['J', 'K']}
nodes = {'A': f([0,1,2,3]),
         'B': f([4,'',' ','']),
         'C': f([8,9,10,11]),
         'D': f([12,'',' ','']),
         'E': f([16,17,18,19]),
         'F': f([20,21,22,23]),
         'G': f([24,24,24, '']),
         'H': f([28,29,30,31]),
         'I': f([32,33,34,35]),
         'J': f([36,37,38,39]),
         'K': f([40,41,42,43])}
bptree(p, edges=edges, nodes=nodes)
print(p)

```



Width is 0 for pointers:

```

from latextool_basic import *
p = Plot()

widths= [0, 0.4]
height = 0.4

def f(M):
    return [r'\scriptsize\texttt{%s}'] % _ for _ in M

M0 = f([5, 10, 17, 22])
M1 = f([0, 1, 3, 4])
M2 = f([6, 7, 8, 9])
M3 = f([12, 13, 14, 15])
M4 = f([17, 18, 19, 20])
M5 = f([22, 30, 50, 60])

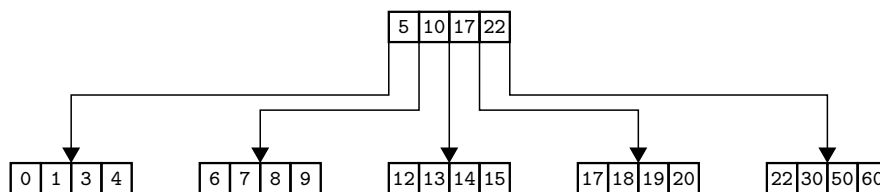
N0 = bpt_node(x=5, y=0, M=M0, widths=widths, height=height)

N1 = bpt_node(x=0, y=-2, M=M1, widths=widths, height=height)
N2 = bpt_node(x=2.5, y=-2, M=M2, widths=widths, height=height)
N3 = bpt_node(x=5, y=-2, M=M3, widths=widths, height=height)
N4 = bpt_node(x=7.5, y=-2, M=M4, widths=widths, height=height)
N5 = bpt_node(x=10, y=-2, M=M5, widths=widths, height=height)
p += N0; p += N1; p += N2; p += N3; p += N4; p += N5

p += bpt_arc(N0, N1, 0, delta=0.2, r=0)
p += bpt_arc(N0, N2, 1, r=0)
p += bpt_arc(N0, N3, 2, r=0)
p += bpt_arc(N0, N4, 3, r=0)
p += bpt_arc(N0, N5, 4, delta=0.2, r=0)

print(p)

```



Width for keys is 0. Might be easier to read (when width of pointer is 0) if the linewidth of rects is thicker:

```
from latextool_basic import *
p = Plot()

widths= [0, 0.4]
height = 0.4

def f(M):
    return [r'\scriptsize\texttt{s}'] % _ for _ in M

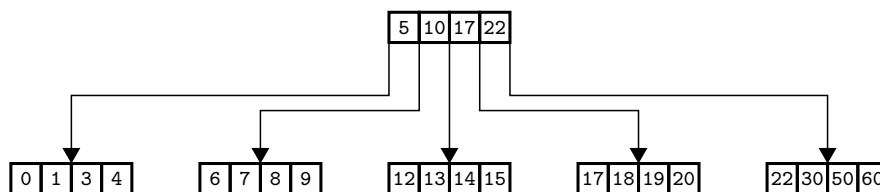
M0,M1,M2,M3,M4,M5 = (f([5, 10, 17, 22]), f([0, 1, 3, 4]), f([6, 7, 8, 9]),
                    f([12, 13, 14, 15]), f([17, 18, 19, 20]), f([22, 30, 50, 60]))

def bpt_node_(x, y, M):
    return bpt_node(x=x, y=y, M=M, widths=widths, height=height,
                    linewidth=0.04)

N0 = bpt_node_(x=5, y=0, M=M0)
N1 = bpt_node_(x=0, y=-2, M=M1)
N2 = bpt_node_(x=2.5, y=-2, M=M2)
N3 = bpt_node_(x=5, y=-2, M=M3)
N4 = bpt_node_(x=7.5, y=-2, M=M4)
N5 = bpt_node_(x=10, y=-2, M=M5)
p += N0; p += N1; p += N2; p += N3; p += N4; p += N5

p += bpt_arc(N0, N1, 0, delta=0.2, r=0)
p += bpt_arc(N0, N2, 1, r=0)
p += bpt_arc(N0, N3, 2, r=0)
p += bpt_arc(N0, N4, 3, r=0)
p += bpt_arc(N0, N5, 4, delta=0.2, r=0)

print(p)
```

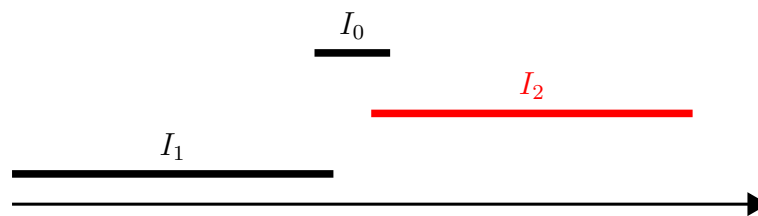


51.8 Intervals (intervals.tex)

For interval scheduling problems.

```
from latextool_basic import *
p = Plot()

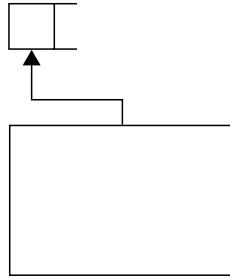
intervals(p,
    dy=0.8,
    axislength=10,
    xss=[[ (0, 4.25, '$I_1$'), (4.75, 9, '$I_2$', 'red') ],
        [(4, 5, '$I_0$')]],
    )
print(p)
```



52 Automata (chap-automata.tex)

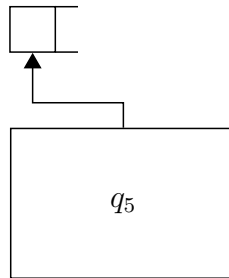
52.1 DFA (dfa.tex)

```
from latextool_basic import *  
p = Plot()  
dfa(p)  
print(p)
```



52.2 State

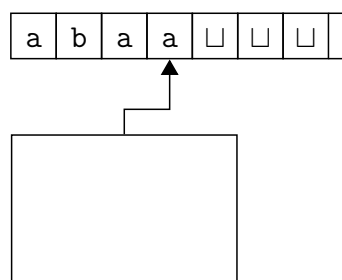
```
from latextool_basic import *  
p = Plot()  
dfa(p, state=r'$q_5$')  
print(p)
```



52.3 Tape and head

```
from latextool_basic import *
p = Plot()

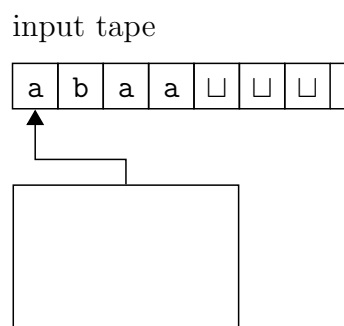
dfa(p,
    tape=['a','b','a','a',r'\SPACE',r'\SPACE',r'\SPACE'],
    head_index=3,
)
print(p)
```



52.4 Label for tape

```
from latextool_basic import *
p = Plot()

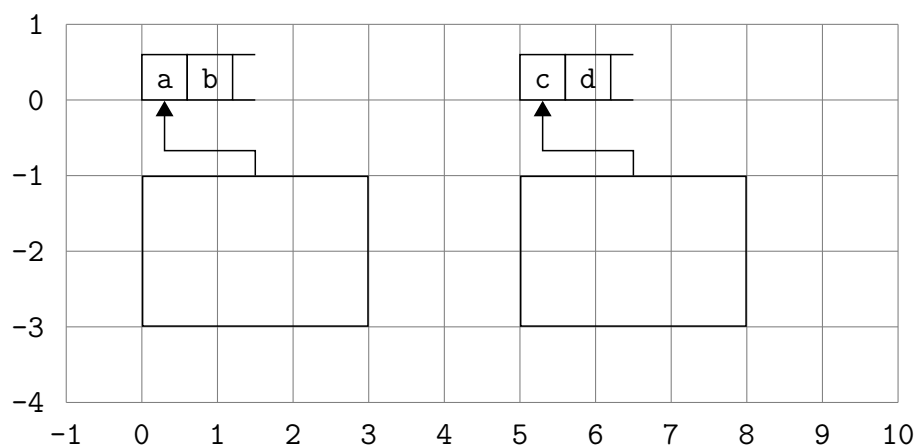
dfa(p,
    tape=['a','b','a','a',r'\SPACE',r'\SPACE',r'\SPACE'],
    input_tape_str=True,
)
print(p)
```



52.5 Position

```
from latextool_basic import *
p = Plot()

dfa(p, x0=0, y0=0,
    tape=['a','b'],
    )
dfa(p, x0=5, y0=0,
    tape=['c','d'],
    )
p += Grid(x0=-1, y0=-4, x1=10, y1=1)
print(p)
```

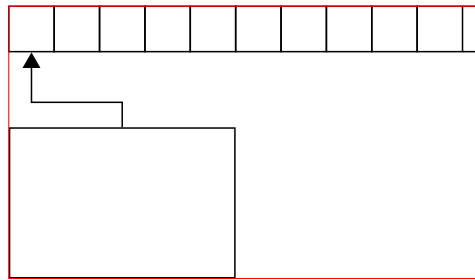


52.6 Bounding rect

The return value of `dfa` is a `Rectangle` object:

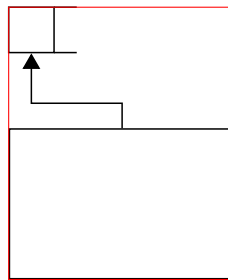
```
from latextool_basic import *
p = Plot()

r = dfa(p, tape=['' for i in range(10)],)
x0,y0 = r.bottomleft()
x1,y1 = r.topright()
p += Rect(x0=x0,y0=y0,x1=x1,y1=y1,linecolor='red')
print(p)
```



```
from latextool_basic import *
p = Plot()

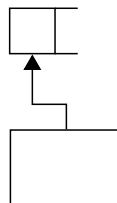
r = dfa(p, tape='')
x0,y0 = r.bottomleft()
x1,y1 = r.topright()
p += Rect(x0=x0,y0=y0,x1=x1,y1=y1,linecolor='red')
print(p)
```



52.7 Dimensions of automata

```
from latextool_basic import *
p = Plot()

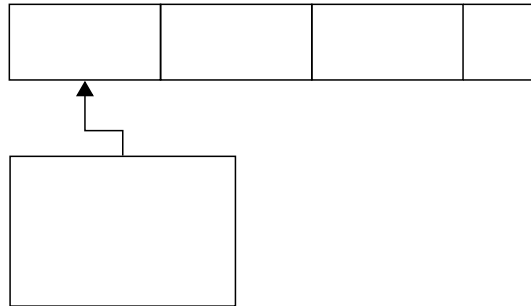
r = dfa(p,
        body_w = 1.5,
        body_h = 1,
        )
print(p)
```



Cell width and height

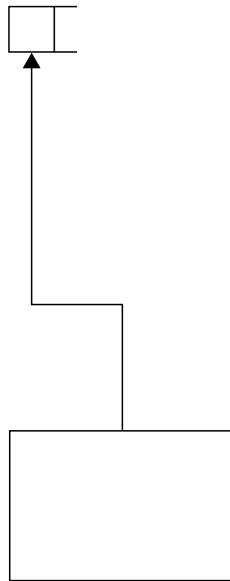
```
from latextool_basic import *
p = Plot()

dfa(p,
    tape=['',' ',''],
    w=2, h=1,
)
print(p)
```



52.8 Body-tape distance

```
from latextool_basic import *  
p = Plot()  
  
dfa(p, vsep=5)  
print(p)
```

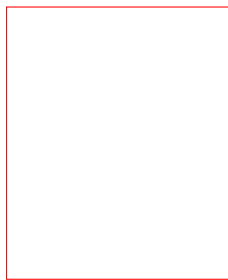


52.9 No drawing

No drawing (just to compute bounding rect):

```
from latextool_basic import *
p = Plot()

r = dfa(p, no_draw=True)
x0,y0 = r.bottomleft()
x1,y1 = r.topright()
p += Rect(x0=x0, y0=y0, x1=x1, y1=y1, linecolor='red')
print(p)
```



52.10 Sequence of automatas

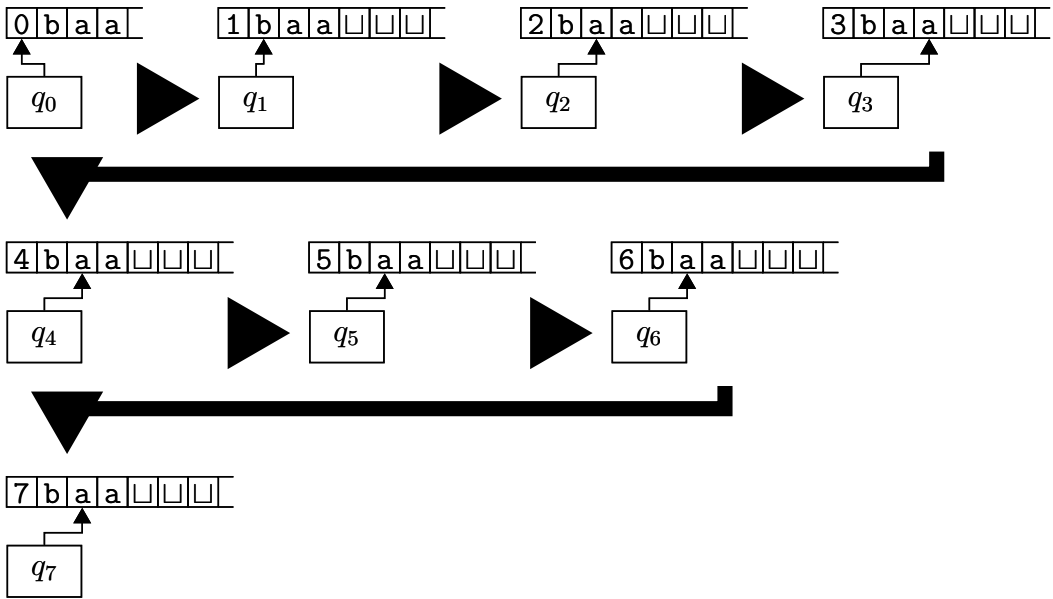
```

from latextool_basic import *
p = Plot()

def rect(index, p, x0, y0, no_draw=False):
    data = [(r'$q_0$',
             ['0','b','a','a'],
             0),
            (r'$q_1$',
             ['1','b','a','a',r'\SPACE',r'\SPACE',r'\SPACE'],
             1),
            (r'$q_2$',
             ['2','b','a','a',r'\SPACE',r'\SPACE',r'\SPACE'],
             2),
            (r'$q_3$',
             ['3','b','a','a',r'\SPACE',r'\SPACE',r'\SPACE'],
             3),
            (r'$q_4$',
             ['4','b','a','a',r'\SPACE',r'\SPACE',r'\SPACE'],
             2),
            (r'$q_5$',
             ['5','b','a','a',r'\SPACE',r'\SPACE',r'\SPACE'],
             2),
            (r'$q_6$',
             ['6','b','a','a',r'\SPACE',r'\SPACE',r'\SPACE'],
             2),
            (r'$q_7$',
             ['7','b','a','a',r'\SPACE',r'\SPACE',r'\SPACE'],
             2),
            ]
    if index < len(data):
        state, tape, head_index = data[index]
        return dfa(p,
                   x0=x0,y0=y0,
                   tape=tape,
                   state=state,
                   head_index=head_index,
                   vsep=0.5, hsep=0.25,
                   body_w=1, body_h=0.7, w=0.4,
                   no_draw=no_draw,
                   )
    else:
        return None

sequence(p, rect=rect)
print(p)

```



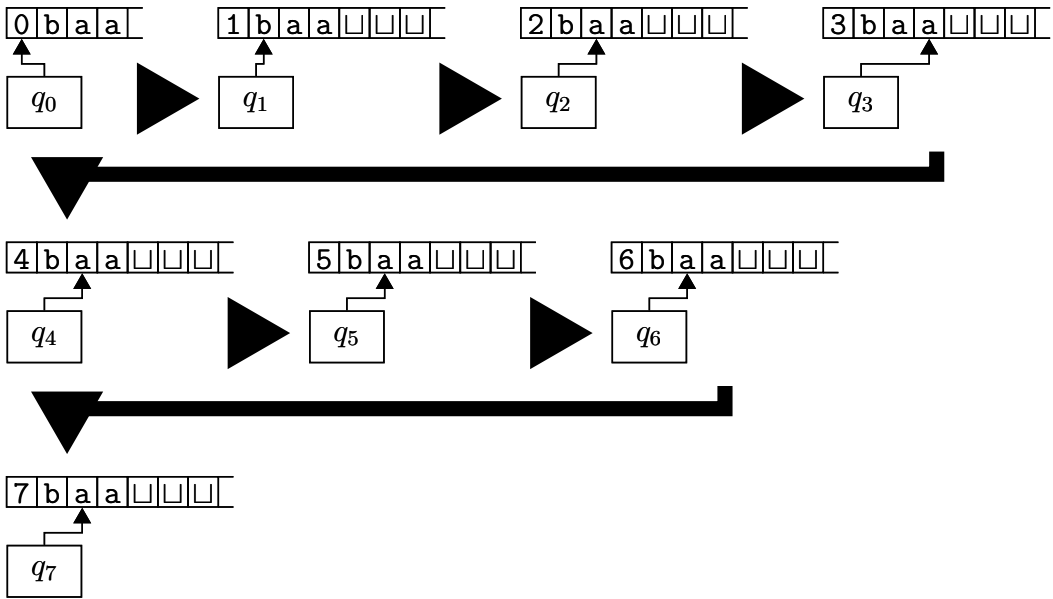
```

from latextool_basic import *
p = Plot()

def rect(index, p, x0, y0, no_draw=False):
    data = [(r'$q_0$',
              ['0', 'b', 'a', 'a'],
              0),
             (r'$q_1$',
              ['1', 'b', 'a', 'a', r'\SPACE', r'\SPACE', r'\SPACE'],
              1),
             (r'$q_2$',
              ['2', 'b', 'a', 'a', r'\SPACE', r'\SPACE', r'\SPACE'],
              2),
             (r'$q_3$',
              ['3', 'b', 'a', 'a', r'\SPACE', r'\SPACE', r'\SPACE'],
              3),
             (r'$q_4$',
              ['4', 'b', 'a', 'a', r'\SPACE', r'\SPACE', r'\SPACE'],
              2),
             (r'$q_5$',
              ['5', 'b', 'a', 'a', r'\SPACE', r'\SPACE', r'\SPACE'],
              2),
             (r'$q_6$',
              ['6', 'b', 'a', 'a', r'\SPACE', r'\SPACE', r'\SPACE'],
              2),
             (r'$q_7$',
              ['7', 'b', 'a', 'a', r'\SPACE', r'\SPACE', r'\SPACE'],
              2),
            ]
    if index < len(data):
        state, tape, head_index, = data[index]
        return dfa(p,
                    x0=x0, y0=y0,
                    tape=tape,
                    state=state,
                    head_index=head_index,
                    vsep=0.5, hsep=0.25,
                    body_w=1, body_h=0.7, w=0.4,
                    no_draw=no_draw,
                    )
    else:
        return None

sequence(p, rect=rect)
print(p)

```



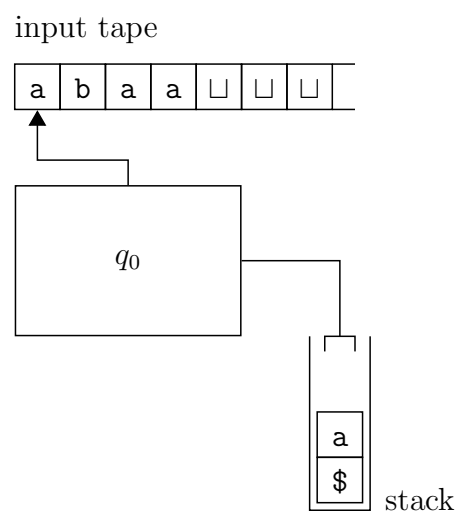
52.11 PDA (pda.tex)

```

from latextool_basic import *
p = Plot()

pda(p,
    stackvalues = ['a','\$',],
    tape = ['a','b','a','a',r'\SPACE',r'\SPACE',r'\SPACE'],
    state = r'$q_0$',
    head_index = 0,
    body_w = 3,
    body_h = 2,
    vsep = 1,
    hsep = 1,
    input_tape_str=True,
    stack_str=True,
)
print(p)

```



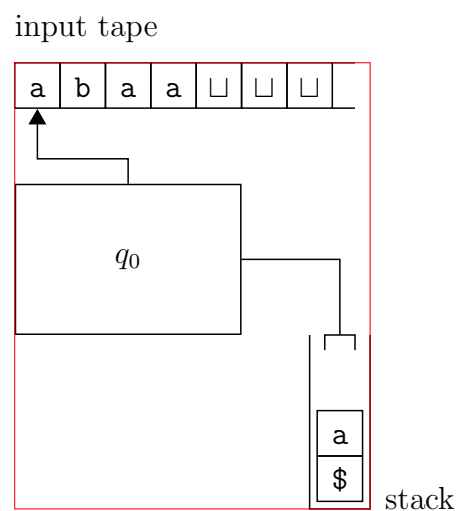
52.12 PDA: bounding rect

```

from latextool_basic import *
p = Plot()

r = pda(p,
    stackvalues = ['a','\$',],
    tape = ['a','b','a','a',r'\SPACE',r'\SPACE',r'\SPACE'],
    state = r'$q_0$',
    head_index = 0,
    body_w = 3,
    body_h = 2,
    vsep = 1,
    hsep = 1,
    input_tape_str=True,
    stack_str=True,
)
x0,y0 = r.bottomleft()
x1,y1 = r.topright()
p += Rect(x0=x0,y0=y0,x1=x1,y1=y1,linewidth='red')
print(p)

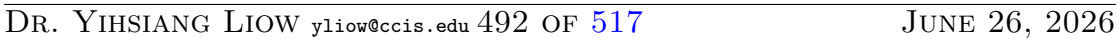
```



```

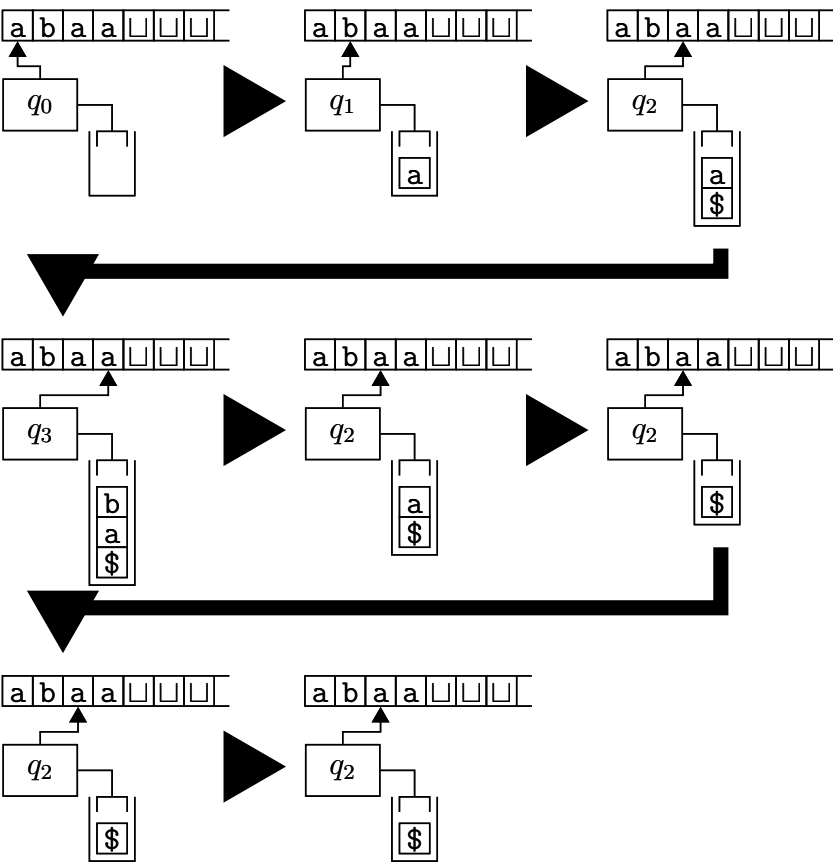
from latextool_basic import *
p = Plot()
pda_computation(p,
    data = [(r'$q_0$',
        ['a','b','a','a',r'\SPACE',r'\SPACE',r'\SPACE'],
        0,
        []),
        (r'$q_1$',
        ['a','b','a','a',r'\SPACE',r'\SPACE',r'\SPACE'],
        1,
        ['a',]), # 'b','c', 'd', 'e', r'\$'],
        (r'$q_2$',
        ['a','b','a','a',r'\SPACE',r'\SPACE',r'\SPACE'],
        2,
        ['a', r'\$']),
        (r'$q_3$',
        ['a','b','a','a',r'\SPACE',r'\SPACE',r'\SPACE'],
        3,
        ['b', 'a', r'\$']),
        (r'$q_2$',
        ['a','b','a','a',r'\SPACE',r'\SPACE',r'\SPACE'],
        2,
        ['a', r'\$']),
        (r'$q_2$',
        ['a','b','a','a',r'\SPACE',r'\SPACE',r'\SPACE'],
        2,
        [r'\$']),
        (r'$q_2$',
        ['a','b','a','a',r'\SPACE',r'\SPACE',r'\SPACE'],
        2,
        [r'\$']),
        (r'$q_2$',
        ['a','b','a','a',r'\SPACE',r'\SPACE',r'\SPACE'],
        2,
        [r'\$'])],
    )
print(p)

```




```
DR. YIHSIANG LIOW yliow@ccis.edu 494 OF 517 JUNE 26, 2026
    else:
        return None

sequence(p, rect=rect)
print(p)
```



52.14 CYK (cyk.tex)

```
from latextool_basic import *
m = [['?', '$A$', '$B$', '', '', ''],
      ['', '', '', '', '', ''],
      ['', '', '', '', '', ''],
      ['', '', '', '', '', ''],
      ['', '', '', '', '', ''],
      ['', '', '', '', '', ''],
      ['', '', '', '', '', ''],
      []]
p = Plot()
cyk(p, m)
print(p)
```

i j		1	2	3	4	5	6
1	?	A	B				
2							
3							
4							
5							
6							

52.14.1 Word being parsed

```
from latextool_basic import *
m = [['?', '$A$', '$B$', '', '', ''],
      ['', '', '', '', '', ''],
      ['', '', '', '', '', ''],
      ['', '', '', '', '', ''],
      ['', '', '', '', '', ''],
      ['', '', '', '', '', ''],
      ['', '', '', '', '', ''],
      []]
p = Plot()
cyk(p, m, w='baaaab')
print(p)
```


		i					
j		b	a	a	a	a	b
		1	2	3	4	5	6
1	?	A	B				
2							
3							
4							
5							
6							

52.14.2 Background

You can set the background using the (row, col) index values. If it's set, the cell is also drawn with a larger linewidth.

```
from latextool_basic import *
m = [['?', '$A$', '$B$', '', '', ''],
      ['', '', '', '', '', ''],
      ['', '', '', '', '', ''],
      ['', '', '', '', '', ''],
      ['', '', '', '', '', ''],
      ['', '', '', '', '', ''],
      ['', '', '', '', '', ''],
      []]
p = Plot()
cyk(p, m, background={(0,0):'blue!10',
                      (0,1):'red!10',})
print(p)
```

i j		1	2	3	4	5	6
1	?	A	B				
2							
3							
4							
5							
6							

52.14.3 Height and width

```
from latextool_basic import *
m = [['?', '$A$', '$B$', '$C$', '$D$', '$E$', '$F$', '$G$', '$H$', '$I$', '$J$', '$K$', '$L$', '$B$', '', '', '']
,
    ['', '', '', '', '', ''],
    ['', '', '', '', '', ''],
    ['', '', '', '', '', ''],
    ['', '', '', '', '', ''],
    ['', '', '', '', '', ''],
    ]
p = Plot()
cyk(p, m, height=2)
print(p)
```

i j		1	2	3	4	5	6
	1	?	A,B,C,D,E,B,G,H,I,J,K,L				
	2						
	3						
	4						
	5						
	6						

53 Automata (automata.tex)

The `automata` function (in the `latextool_basic` module) spits out latex code for state diagrams which can then be modified by hand. Here are some examples.

If you execute the following python code:

```
print(automata(layout="""
    A
    """,
    A="initial|label=$q''_0$",
    ))
```

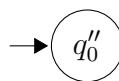
you will get this latex code as output:

```
\begin{center}
\begin{tikzpicture}[shorten >=1pt,node distance=2cm,auto,initial text=]
\node[state,initial] (A) at ( 4, 0) {$q''_0$};

\path[->]

;
\end{tikzpicture}
\end{center}
```

When you paste the above output into a latex document, you will get this picture:

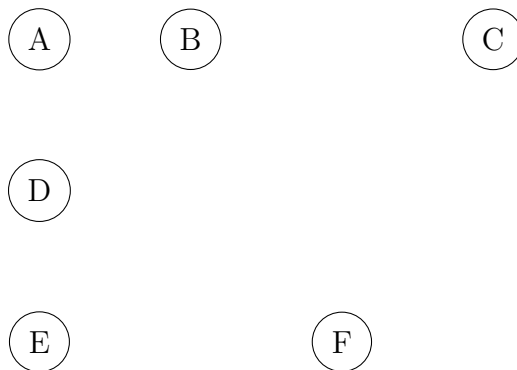


The `layout` parameter is a string for placing the nodes. When you execute this:

```
from latextool_basic import *
automata(layout="""
A B   C
D
E   F
""")
)
```

you get this output:

```
from latextool_basic import *
print(automata(layout="""
A B   C
D
E   F
"""))
```



You can then decorate each node:

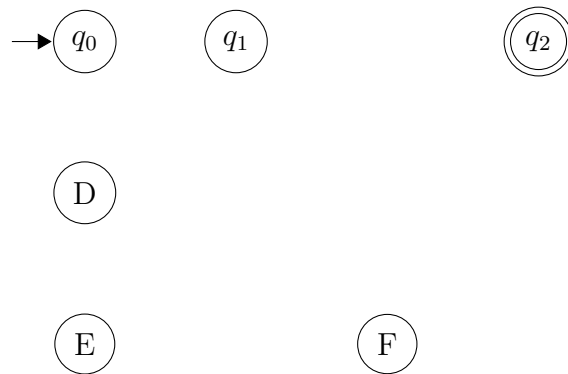
```
from latextool_basic import *
print(automata(layout="""
A B   C
D
E   F
""",
A = "initial|label=$q_0$",
B = "label=$q_1$",
C = "accept|label=$q_2$"
))
```

gives this picture

```

from latextool_basic import *
print(automata(layout="""
A B   C
D
E   F
""",
A = "initial|label=$q_0$",
B = "label=$q_1$",
C = "accept|label=$q_2$"
))

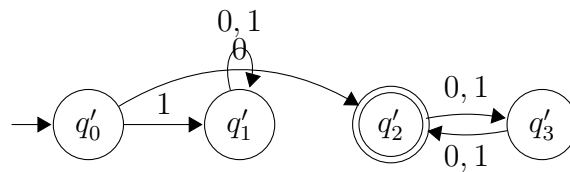
```



For the rest of the examples, I will just give you the python code and the corresponding picture.

DFA for “starts with 0 and has odd length”:

```
from latextool_basic import *
print(automata(layout="""
A D B C
""",
edges="A,$0$,B|B,$0,1$,C|C,$0,1$,B|A,$1$,D|D,$0,1$,D",
A="initial|label=$q'_0$",
B="accept|label=$q'_2$",
C="label=$q'_3$",
D="label=$q'_1$",
))
```



Sipser 1.7c

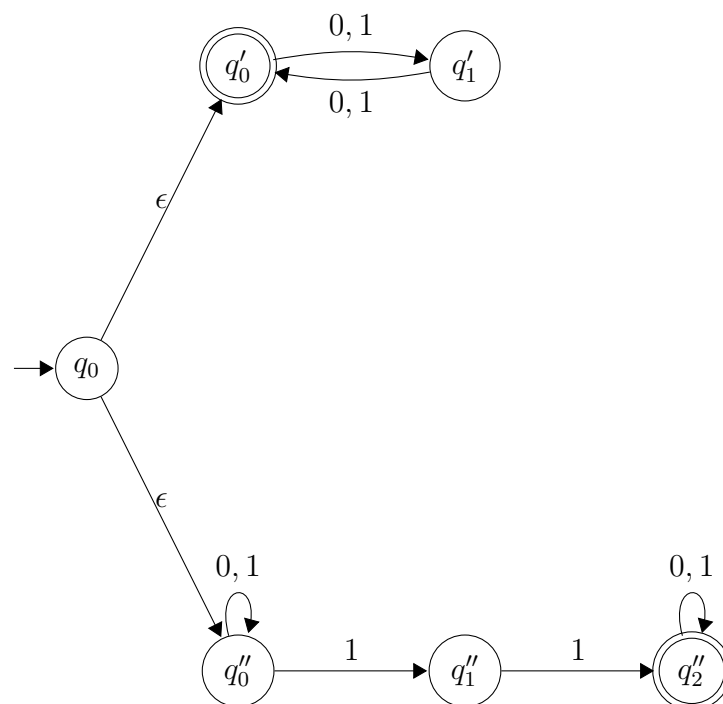
```

from latextool_basic import *
print(automata(layout="""
    B D

A

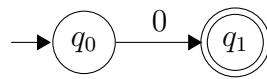
    C E F
    """,
edges="A,$\ep$,B|B,$0,1$,D|D,$0,1$,B|A,$\ep$,C|C,$1$,E|E,$1$,F|C,$0,1$,C|F,
$0,1$,F",
A="initial|label=$q_0$",
B="accept|label=$q'_0$",
C="label=$q''_0$",
D="label=$q'_1$",
E="label=$q''_1$",
F="label=$q''_2$|accept",
))

```



Sipser 1.7d

```
from latextool_basic import *  
  
print(automata(layout="""  
A B  
""",  
edges="A,$0$,B",  
A="initial|label=$q_0$",  
B="accept|label=$q_1$",  
))
```

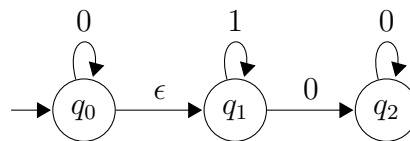


Sipser 1.7e

```

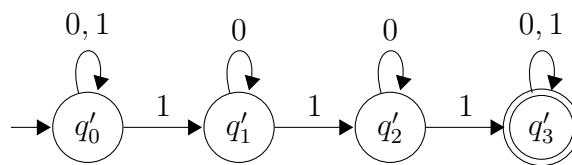
from latextool_basic import *
print(automata(layout="""
A B C
""",
edges="A,$\epsilon$,B|A,$0$,A|B,$1$,B|B,$0$,C|C,$0$,C",
A="initial|label=$q_0$",
B="label=$q_1$",
C="label=$q_2$",
))

```



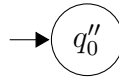
NFA for at least three 1s:

```
from latextool_basic import *
print(automata(layout="""
A B C D
""",
edges="A,1,B|B,1,C|C,1,D|A,$0,1$,A|D,$0,1$,D|B,0,B|C,0,C",
A="initial|label=$q'_0$",
B="label=$q'_1$",
C="label=$q'_2$",
D="accept|label=$q'_3$",
))
```



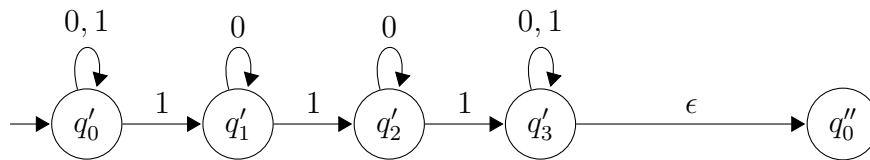
NFA for $\emptyset$:

```
from latextool_basic import *
print(automata(layout="""
A
""",
A="initial|label=$q''_0$",
))
```



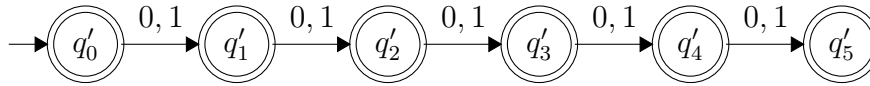
NFA for concat of “contains at least 3 1s” and $\emptyset$:

```
from latextool_basic import *
print(automata(layout="""
A B C D   E
""",
edges="A,1,B|B,1,C|C,1,D|A,$0,1$,A|D,$0,1$,D|B,0,B|C,0,C|D,$\epsilon$,E",
A="initial|label=$q'_0$",
B="label=$q'_1$",
C="label=$q'_2$",
D="label=$q'_3$",
E="label=$q''_0$"
))
```



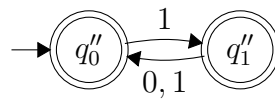
NFA for concat of “length ≤ 5 ” and “every odd position is a 1”:

```
from latextool_basic import *
print(automata(layout="""
A B C D E F
""",
edges="A,$0,1$,B|B,$0,1$,C|C,$0,1$,D|D,$0,1$,E|E,$0,1$,F",
A="accept|initial|label=$q'_0$",
B="accept|label=$q'_1$",
C="accept|label=$q'_2$",
D="accept|label=$q'_3$",
E="accept|label=$q'_4$",
F="accept|label=$q'_5$",
))
```



NFA for “every odd position is a 1”:

```
from latextool_basic import *
print(automata(layout="""
A B
""",
edges="A,$1$,B|B,$0,1$,A",
A="accept|initial|label=$q''_0$",
B="accept|label=$q''_1$",
))
```

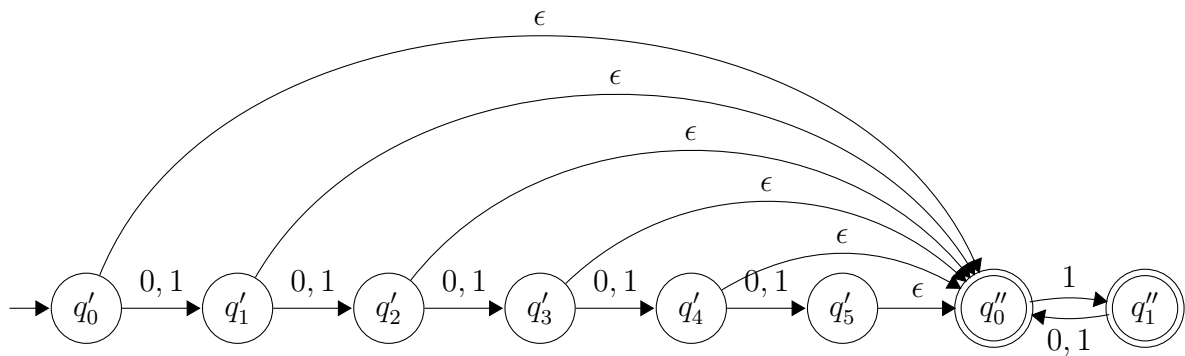


NFA for concat of “length ≤ 5 ” and “every odd position is a 1”:

```

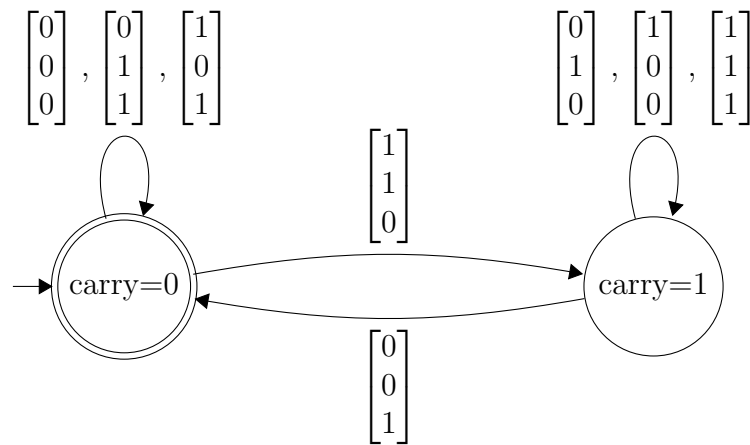
from latextool_basic import *
print(automata(layout="""
A B C D E F G H
""",
edges="A,$0,1$,B|B,$0,1$,C|C,$0,1$,D|D,$0,1$,E|E,$0,1$,F"+\
"|F,$\epsilon$,G"+\
"|A,$\epsilon$,G"+\
"|B,$\epsilon$,G"+\
"|C,$\epsilon$,G"+\
"|D,$\epsilon$,G"+\
"|E,$\epsilon$,G"+\
"|G,$1$,H|H,$0,1$,G",
A="initial|label=$q'_0$",
B="label=$q'_1$",
C="label=$q'_2$",
D="label=$q'_3$",
E="label=$q'_4$",
F="label=$q'_5$",
G="accept|label=$q''_0$",
H="accept|label=$q''_1$",
))

```



NFA with vectors as Σ :

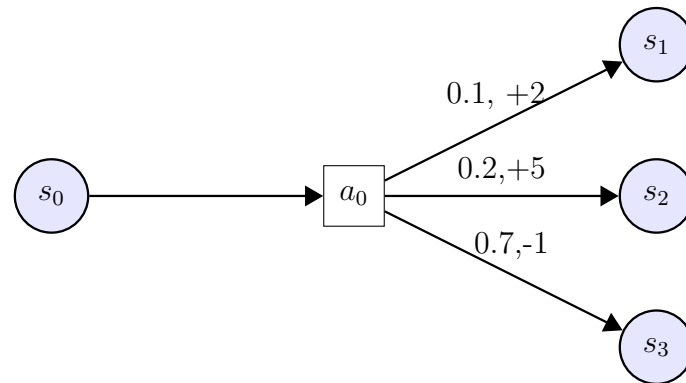
```
from latextool_basic import *
print(automata(layout=""
A      B
""",
edges=r"A,$\begin{bmatrix}0\\0\\0\end{bmatrix},\begin{bmatrix}0\\1\\1\end{bmatrix},\begin{bmatrix}1\\0\\1\end{bmatrix}$,A"+\
r"|A,$\begin{bmatrix}1\\1\\0\end{bmatrix}$,B"+\
r"|B,$\begin{bmatrix}0\\1\\0\end{bmatrix},\begin{bmatrix}1\\0\\0\end{bmatrix},\begin{bmatrix}1\\1\\1\end{bmatrix}$,B"+\
r"|B,$\begin{bmatrix}0\\0\\1\end{bmatrix}$,A"+\
r"",
A="initial|accept|label=carry=0",
B="label=carry=1",
))
```



54 Longtable of minipages (longtable-minipage.tex)

abc	def
v.size()	return size of v etc. etc. etc. etc. etc. etc. etc. etc. etc. etc. etc. etc. etc. etc. etc. etc. etc. etc.
v.size()	return size of v abc def
abc def def def def def def def def def	return size of v abc def

55 MDP – Markov decision process (mdp.tex)



56 Solutions (solutions.tex)

This is not part of latextool.

See `projects/solutions`

Put symlink `/usr/lib64/python3.7/site-packages/solutions.py` to point to `projects/solutions/solutions.py`

How to use: In a section (or chapter) of latex notes with exercises put

```

\section{Natural numbers $\mathbb{N}$}
\begin{python0}
from solutions import *; clear()
\end{python0}

\begin{ex}
  \label{ex:succ-onto-nonzero}
  What is  $1 + 1$ ?
  \solutionlink{sol:succ-onto-nonzero}
  \qed
\end{ex}
\begin{python0}
label = "succ-onto-nonzero"
s = r'''
\proof
 $1 + 1 = 2$ 
\qed
'''
from solutions import *; add(label, s)
\end{python0}

\begin{prop}
  \label{prop:N-is-commutative-monoid}
  What is  $1 + 1$ ?
\end{prop}

\proof
Exercise.
\solutionlink{sol:N-is-commutative-monoid}
\qed
\begin{python0}
from solutions import *
add(label="prop:N-is-commutative-monoid",
    srcfilename='proof-N-is-commutative-monoid.tex')
\end{python0}

\newpage
\subsection{Solutions}

\input{solutions.tex}

```